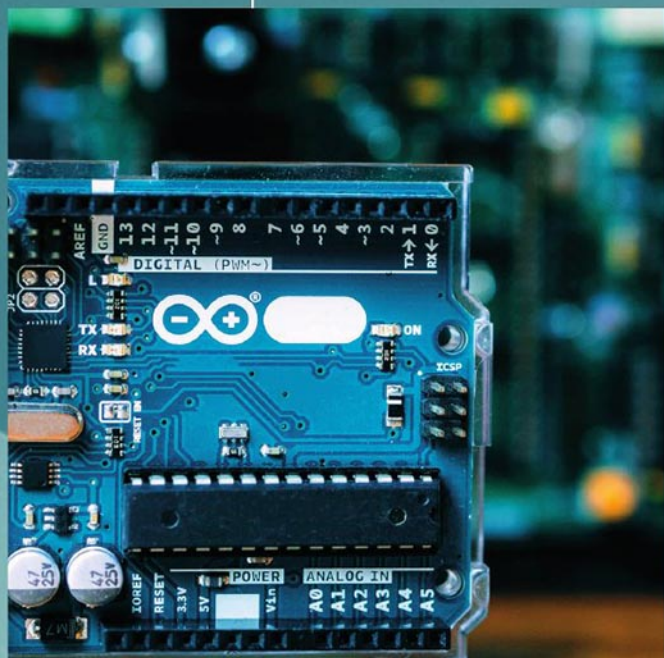




В. В. Кангин, Е. М. Кангин

КОНТРОЛЛЕРЫ ARDUINO

В МОБИЛЬНЫХ РОБОТАХ
И ДРОНАХ

В. В. КАНГИН, Е. М. КАНГИН

КОНТРОЛЛЕРЫ ARDUINO

В МОБИЛЬНЫХ РОБОТАХ И ДРОНАХ

Учебное пособие

Москва Вологда
«Инфра-Инженерия»
2025

УДК 621.3.049.77

ББК 32.844.1

K19

Рецензент:

д. т. н., профессор Нижегородского государственного
технического университета им. Р. Е. Алексеева

Моругин Станислав Львович

Кангин, В. В.

K19 Контроллеры Arduino в мобильных роботах и дронах : учебное пособие / В. В. Кангин, Е. М. Кангин. – Москва ; Вологда : Инфра-Инженерия, 2025. – 324 с. : ил., табл.
ISBN 978-5-9729-2451-6

Рассмотрены основные аспекты проектирования МР: электромеханика приводов; измерительная, силовая и управляющая электроника на базе контроллеров семейства Arduino; программирование; инфокоммуникационные технологии. Большое внимание уделено вопросам управления МР с использованием инфракрасного канала (ИК) связи, SMS-сообщений (мобильная связь). Предложены варианты управления роботами с помощью технологии Bluetooth без использования смартфонов или планшетов, что в значительной степени расширяет диапазон функций, выполняемых роботом. Освещены вопросы стабилизации МР и их компонентов в пространстве с помощью гироскопов, акселерометров. Предложен новый способ стабилизации робота по курсу с помощью модуля гироскопа и акселерометра GY-521. Рассмотрены новые инфракрасные дальномеры SHARP GP2Y0A21YK0F, SHARP GP2Y0A02YK0F и E18-D80NK и способы их использования в задачах управления МР. Показано, как организовать управление дроном самолетного типа с пульта управления по каналу Bluetooth. Большинство представленных проектов иллюстрировано алгоритмами скетчей и схемами электрическими принципиальными.

Для студентов вузов, занимающихся изучением роботов, робототехнических комплексов, контроллерной техники, технического программирования, инфокоммуникационных технологий в промышленной среде. Может быть использовано при курсовом и дипломном проектировании для разработки цеховой и заводской транспортной структуры и различных безлюдных логистических систем. Будет полезно для специалистов-практиков, занимающихся разработкой беспилотных транспортных средств различного назначения, любителей, занимающихся проектированием МР и дронов, в кружках робототехники.

УДК 621.3.049.77

ББК 32.844.1

ISBN 978-5-9729-2451-6

© Кангин В. В., Кангин Е. М., 2025

© Издательство «Инфра-Инженерия», 2025

© Оформление. Издательство «Инфра-Инженерия», 2025

ОГЛАВЛЕНИЕ

Введение	5
1. МОБИЛЬНЫЕ РОБОТЫ.....	7
1.1. Ходовая часть и управление ею.....	7
1.2. Драйверы двигателей и модули драйверов	8
1.3. Драйвер L293D	9
1.4. Модуль драйвера L293D	11
1.5. Управление скоростью вращения вала ДПТ	12
1.6. Борьба с помехами	13
1.7. Модуль драйвера L298N	14
1.8. Модуль драйвера HG7881	15
1.9. Подключение ДПТ правого и левого колёс к контроллеру	17
1.10. ИК-пульта и ИК-приёмники	19
1.11. Примеры.....	22
1.12. Управление скоростью движения робота.....	29
1.13. Организация многопрограммного режима работы робота	32
1.14. Роботы-танки.....	42
1.14.1. Аппаратная часть.....	42
1.14.2. Варианты компоновок роботов-танков.....	45
1.14.3. Примеры.....	48
1.14.4. Роботы с сервоприводом	88
2. GSM-МОДУЛЬ SIM800L В МОБИЛЬНЫХ РОБОТАХ.....	104
2.1. Описание модуля, питание, технические характеристики	104
2.2. Назначение выводов модуля SIM800L и подключение его к плате Arduino	109
2.3. Описание библиотеки SoftwareSerial	110
2.4. AT-команды для модуля SIM800L	121
2.5. Примеры.....	127
2.6. Работа со строками в скетчах	143
2.7. Управление роботом с помощью SMS-сообщений.....	148
3. УПРАВЛЕНИЕ МОБИЛЬНЫМИ РОБОТАМИ ПО BLUETOOTH.....	160
3.1. Введение	160
3.2. Проблемы в приобретении модуля HC-05	162
3.3. Подготовка модуля HC-05 к настройке	162
3.4. Подключение преобразователя к ПК и модулю HC-05 при настройке модуля.....	166
3.5. Настройка slave-модуля Bluetooth.....	167
3.6. Настройка master-модуля Bluetooth	171
3.7. AT-команды модуля HC-05.....	174
3.8. Пульт для передачи информации по Bluetooth	186

3.9. Передача информации с двухосевого джойстика по Bluetooth	188
3.10. Управление роботом с кнопочного пульта по Bluetooth.....	193
3.11. Управление поворотом ротора шагового двигателя с помощью потенциометра по Bluetooth.....	202
4. ГИРОСКОПЫ И АКСЕЛЕРОМЕТРЫ В СИСТЕМАХ УПРАВЛЕНИЯ МОБИЛЬНЫМИ РОБОТАМИ.....	206
4.1. Общие сведения	206
4.2. Модуль GY-521с гироскопом, акселерометром и термометром на базе микросхемы MPU-6050	207
4.3. Шина I2C.....	209
4.4. Библиотека Wire.....	217
4.5. Библиотека Servo	224
4.6. Примеры использования модуля GY-521	229
4.7. Модуль GY-521 в задачах стабилизации курса	232
5. ИНФРАКРАСНЫЕ ДАТЧИКИ В МОБИЛЬНЫХ РОБОТАХ.....	242
5.1. Общие сведения	242
5.2. Простейший ИК-модуль.....	243
5.3. Инфракрасные дальномеры SHARP GP2Y0A21YK0F и SHARP GP2Y0A02YK0F.....	244
5.4. Описание мобильного робота и его схема.....	246
5.5. Движение робота вдоль стенки. Примеры	249
5.6. Движение робота внутри круглого вольера. Примеры	255
5.7. Инфракрасный дальномер (датчик препятствия) E18-D80NK.....	264
5.8. Колесный робот с поворачивающимся задним колесом.....	276
5.9. Движение робота вокруг круглого вольера. Примеры.....	280
5.10. Робот, объезжающий предметы.....	293
6. СИСТЕМА УПРАВЛЕНИЯ ДРОНОМ САМОЛЕТНОГО ТИПА НА БАЗЕ КОНТРОЛЛЕРА ARDUINO	299
6.1. Общие сведения	299
6.2. Подключение бесколлекторного электродвигателя постоянного тока к контроллеру ARDUINO.....	304
6.2.1. Принцип действия БД.....	304
6.2.2. Преимущества БД.....	306
6.2.3. Контроллер скорости ESC.....	306
6.2.4. Схема подключения БД к контроллеру Arduino	308
6.3. Разработка передатчика для дрона.....	311
6.4. Разработка аппаратуры для дрона (минимальная комплектация)	314
6.5. Что дальше?	319
Заключение.....	320
Список литературы и интернет-ресурсов.....	321

ВВЕДЕНИЕ

События последних лет показали, в каком направлении эволюционирует научно-техническая мысль развитых государств. Это – развитие мобильных (движущихся) роботов (МР) и дронов. Это, пожалуй, самая динамически развивающаяся область современного роботостроения.

Как и в других отраслях науки и техники, в этой отрасли существует масса проблем, которые требуют эффективного, быстрого и экономически оправданного решения. Платформа Arduino предоставляет такие возможности.

Контроллеры семейства Arduino представляют собой великолепный строительный материал для систем управления МР. Этому есть ряд причин:

- широкий ассортимент контроллеров семейства Arduino и плат расширения (шилдов);
- большое количество в продаже учебных наборов Arduino;
- бесплатная и простая среда программирования Arduino IDE на русском языке и удобный язык программирования на базе языка C++;
- большое количество датчиков и исполнительных механизмов, которые специально созданы для платформы Arduino;
- огромное число описаний действующих проектов, примеров программирования, библиотек, советов и рекомендаций, справочного материала, фото- и видеоматериалов, блогов по данной теме, имеющихся в Интернете;
- очень низкая цена самих контроллеров, модулей датчиков, шилдов и исполнительных механизмов.

В первой главе рассмотрены следующие аспекты конструирования МР:

- механика и электромеханика – привод колёс и гусениц;
- электроника;
- программирование;
- инфокоммуникационные технологии;
- организация приводов постоянного тока (ППТ) и способы управления ими с помощью модулей драйверов L293D, L298N и HГ7881;
- управление скоростью движения МР;
- борьба с помехами;
- организация многопрограммного режима работы МР.

В этой же главе рассмотрены особенности компоновок роботов-танков и использование в них сервоприводов.

Использованию GSM-модуля SIM800L в МР посвящена вторая глава. Здесь открываются перспективы управления МР на огромных расстояниях с использованием SMS-сообщений.

Третья глава посвящена решению проблемы управления роботами с помощью технологии Bluetooth. Но здесь мы полностью уходим от использования смартфонов и планшетов для управления роботом, а создаем свой собственный пульт управления с практически неограниченным количеством команд, которые передаются с пульта на робот по каналу Bluetooth. Резко возрастают функциональные возможности робота. Пользователь становится практически независим от сторонних приложений, которые еще необходимо найти и разместить на смартфоне или планшете. Есть еще огромное количество плюсов, о которых и говорится в этой главе. Рассмотрены несколько проектов с алгоритмами и схемами электрическими принципиальными (далее – просто схемами).

Четвертая глава посвящена проблемам использования гироскопа и акселерометра (модуль GY-521) в задачах управления роботами. Предложен новый способ стабилизации робота по курсу, используя показания этого модуля. Рассмотрены все необходимые для работы библиотеки плюс алгоритмы и схемы.

В пятой главе рассмотрены новые инфракрасные дальномеры SHARP GP2Y0A21YK0F, SHARP GP2Y0A02YK0F и E18-D80NK. Приведено большое число примеров организации движения робота в пространстве: вдоль стенки, внутри замкнутого вольера, снаружи его. Рассмотрен проект, когда, используя указанные дальномеры, робот объезжает предметы. Все проекты иллюстрируются алгоритмами и схемами.

В шестой главе показано, как организовать управление дроном самолетного типа со стандартного пульта управления по каналу Bluetooth. При этом вся «радио-начинка» стандартного пульта удаляется и внутрь помещается контроллер Arduino с модулем Bluetooth HC-05, работающим в режиме master. Данные, с потенциометров каналов руля высоты, руля курса и газа передаются на дрон, где отрабатываются. На борту дрона находится контроллер с модулем Bluetooth HC-05, работающий в режиме slave. Не нужны автопилот, полетный контроллер. Все их функции великолепно выполнит контроллер Arduino. На борту можно использовать любые виды модулей датчиков: гироскопы и акселерометры, магнетометры, высотомеры и т. д. И еще. Можно организовать управление не по трем – четырем каналам, а, практически, по любому числу каналов. Пульт можно оснастить различными кнопками, которые, задают профили полета, включают/отключают те или иные датчики, инициируют сброс грузов, включают/выключают транспондеры и многое другое. Также можно использовать управление с использованием средств мобильной связи (SMS-сообщения), радиоаппаратуры дальнего радиуса действия, например, модулей NRF24L01.

1. МОБИЛЬНЫЕ РОБОТЫ

Здесь и далее под мобильным роботом мы будем понимать движущийся объект, который может управляться тремя способами:

- диспетчером через дистанционный пульт управления – директорный режим;
- автоматическое управление – автономный режим;
- смешанный, когда можно переключаться из директорного режима в автономный и наоборот.

1.1. Ходовая часть и управление ею

Нужно ли говорить, что число конструктивных решений ходовой части весьма разнообразно. Однако имеются решения, которые массово применяются в учебной и спортивной робототехнике из-за их дешевизны и простоты монтажа. На рис. 1.1, 1.2 приведены комплектующие – колёса и привод постоянного тока (ППТ) одного из популярнейших решений ходовой части.



Рис. 1.1



Рис. 1.2

Колесо легко надевается на ось ППТ. Необходимо отметить, что ППТ представляет собой двигатель постоянного тока (ДПТ), конструктивно объединенный с редуктором. На рис. 1.3 и 1.4 показано крепление двух ППТ левого и правого колёс к корпусу робота. Для крепления ППТ к корпусу робота в ППТ имеются два сквозных отверстия, в которые проходит винт диаметром

2 мм. Кроме этого ППТ может крепиться к корпусу с помощью клея или двустороннего скотча. Можно отметить также, что ППТ могут быть расположены не снаружи корпуса робота, а внутри. Тогда колея робота станет уже. ППТ может располагаться не сбоку, как на приведенных рисунках, а снизу, что тоже приводит к сужению ширины колеи, но увеличивает высоту робота.

Выводы ДПТ желательно зашунтировать конденсатором в 100 000 пФ (пикофарад). Таким образом, мы избавляемся от значительной части электрических помех, создаваемых щётками ДПТ. Итак, мы имеем два ведущих колеса. Сзади можно поставить только одно колесо. Это обеспечивает хорошую маневренность робота. В продаже имеется значительный выбор задних колес, в которых используется шарик. Они относительно дороги, но имеют один существенный недостаток: не обеспечивают прямолинейности движения робота. Шарик «гуляет», что приводит к искажению траектории движения робота. Более хорошие результаты дают обычные дешевые мебельные колёсики. Именно такой показан на рис. 1.4.



Рис. 1.3. Вариант крепления ППТ колёс к корпусу робота (вид сверху)



Рис. 1.4. Вариант крепления ППТ колёс к корпусу робота (вид снизу)

К каждому ДПТ идут два провода. Откуда? Конечно, не с платы Arduino. Выводы контроллера не обеспечивают нужного тока для работы ДПТ и выйдут из строя, если мы попытаемся напрямую подключить двигатель к пинам контроллера. Для сопряжения контроллера с двигателями необходимо использовать специальные электронные устройства, называемые драйверами.

1.2. Драйверы двигателей и модули драйверов

Драйвер двигателя выполняет крайне важную роль в проектах Arduino, использующих ДПТ или шаговые двигатели (ШД). Драйвер выполнен в виде микросхемы или модуля Motor Drive Shield, в котором содержится эта микро-

микросхема. Использование модулей более удобно, т. к. в них оформлены выводы для подключения к контроллеру и к двигателям.

Так для чего нужен драйвер двигателя? Как известно, плата Arduino имеет существенные ограничения по силе тока присоединенной к ней нагрузки. Для платы это 800 мА, а для каждого отдельного вывода (пина) – всего 40 мА. Таким образом, мы не можем подключить напрямую к плате Arduino даже самый маленький ДПТ. Любой из этих двигателей в момент запуска или остановки создаст пиковые броски тока, превышающие этот предел.

Как же тогда подключить двигатель к контроллеру? Есть несколько вариантов.

Использовать реле. Мы включаем двигатель в отдельную электрическую сеть, никак не связанную с платой Arduino. Реле по команде контроллера замыкает или размыкает контакты и тем самым включает или выключает ток. Соответственно, двигатель включается или выключается. Главным преимуществом этой схемы является ее простота. Главным недостатком данной схемы является то, что мы не можем управлять скоростью и направлением вращения двигателя.

Использовать силовой транзистор. В данном случае мы можем управлять током, проходящим через двигатель, а значит, можем управлять скоростью вращения вала. Но для смены направления вращения этот способ не подойдет.

Использовать Н-мост. Использовать специальную схему подключения, называемую Н-мостом, с помощью которой мы можем изменять направление движения вала двигателя и скорость его вращения. Сегодня можно без проблем найти как микросхемы, содержащие два или более Н-мостов, так и отдельные модули и платы расширения, построенные на этих микросхемах.

1.3. Драйвер L293D

L293D является самой простой микросхемой для работы с двигателями. L293D содержит два Н-моста, которые позволяют управлять двумя ДПТ. Рабочее напряжение микросхемы – до 36 В, рабочий ток достигает на один ДПТ – 600 мА. Предельно максимальный ток – 1,2 А.

В микросхеме (рис. 1.5) имеется 16 выводов:

- +V – плюс напряжения питания микросхемы – +5 В;
- +Vmotor – плюс напряжения питания для мотора – до +36 В;
- 0V – земля;
- En1, En2 – управление скоростью вращения 1-го и 2-го ДПТ;
- In1, In2 – управление включением/выключением/реверсом 1-го ДПТ;
- Out1, Out2 – подключение 1-го ДПТ;
- In3, In4 – управление включением/выключением/реверсом 2-го ДПТ;
- Out3, Out4 – подключение 2-го ДПТ.

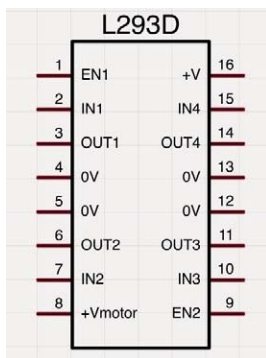


Рис. 1.5

Пример подключения одного ДПТ к контроллеру показан на рис. 1.6.

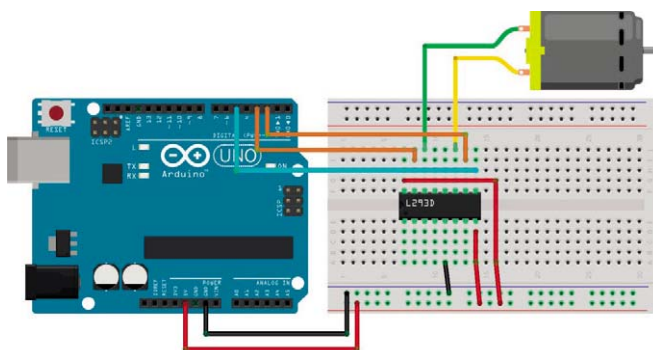


Рис. 1.6

Ниже приведена таблица соединений контроллера, драйвера L293D и ДПТ в соответствии с рис. 1.6.

Таблица соединений контроллера, драйвера L293D и ДПТ

Вывод драйвера	Пин контроллера	ДПТ
10 – In3	2	
15 – In4	3	
9 – En2	5	
16 – +V	5B	
8 – +Vmotor	5B	
14 – Out3		1
11 – Out4		2
5 – 0B	GND(земля)	

На рис. 1.6 драйвер L293D, а следовательно, и ДПТ получают питание непосредственно от контроллера. На практике этого делать не рекомендуется, поскольку при мощной нагрузке (ДПТ) контроллер может выйти из строя.

Питание на контроллер и на драйвер можно подать от одного аккумулятора – аккумуляторной батареи (АКБ) типа литий-полимер (Li-Po). Для обычной робототехники наиболее приемлемым является использование двухбаночной (2S) АКБ номиналом +7,4...+8,4 В. Такое напряжение вполне приемлемо для питания и контроллера и большинства используемых ДПТ.

Из рис. 1.6 видно, что для соединения контроллера и микросхемы драйвера необходимо использовать соединительную плату, что очень неудобно и подходит только при проведении экспериментальных и отладочных действий.

1.4. Модуль драйвера L293D

На практике микросхема драйвера используется в модулях Motor Drive Shield, которые имеют необходимые удобные контакты, перемычки, внутренние источники питания (рис. 1.7).

Обозначение и назначение выводов модуля драйвера L293D приведено в таблице:

**Таблица обозначений и назначение выводов
модуля драйвера L293D**

Вывод модуля драйвера	Назначение
In1, In2	Управление включением, выключением и реверсом 1-го ДПТ
In3, In4	Управление включением, выключением и реверсом 2-го ДПТ
En1	Управление скоростью вращения вала 1-го ДПТ
En2	Управление скоростью вращения вала 2-го ДПТ
VIN	Плюс источника питания (АКБ)
GND	Минус источника питания (АКБ)
A-, A+	Выводы для подключения 1-го ДПТ
B-, B+	Выводы для подключения 2-го ДПТ

Что интересно, подавать на модуль драйвера напряжения +5 В, как показано на рис. 1.7, не нужно. Оно формируется из напряжения, поступающего с АКБ для питания мотора и драйвера. Напряжение с АКБ подаётся на выводы VIN и GND. Из рис. 1.7 видно, что вывод VIN иногда маркируется как VCC.

На рис. 1.8 (а) показана схема электрическая принципиальная подключения ДПТ к контроллеру. Питание контроллера и модуля драйвера осуществляется от двухбаночной АКБ с номинальным напряжением 7,4 В. Выводы

ДПТ зашунтированы керамическим конденсатором С емкостью 100 000 пФ. Это делается для устранения (фильтрации) помех, создаваемых искрением щёток ДПТ.

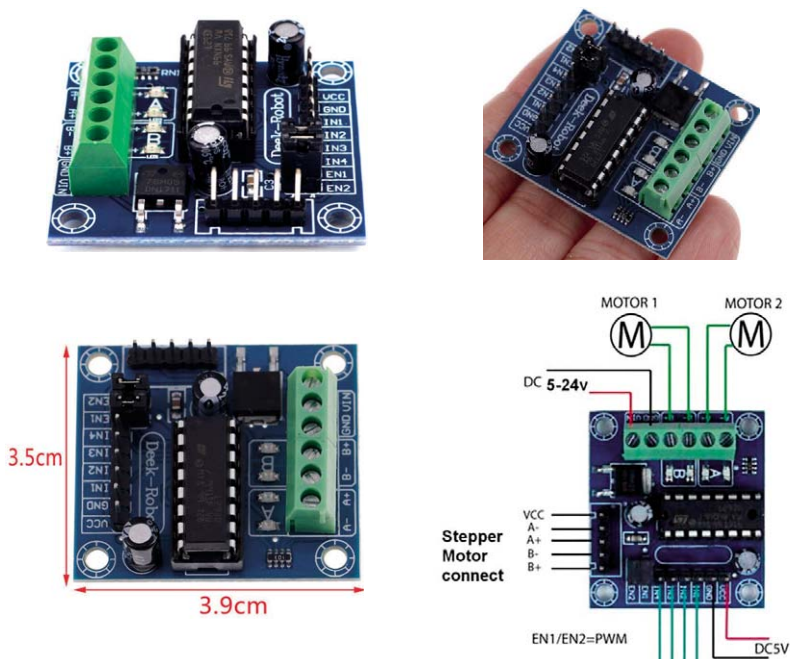


Рис. 1.7

1.5. Управление скоростью вращения вала ДПТ

Из таблицы, приведенной выше, видно, что имеются два вывода – En1 и En2, с помощью которых можно управлять скоростью вращения вала ДПТ. Если у вас нет необходимости управлять этой скоростью, то необходимо оставить переключки, как это показано на рис. 1.7. Скорость вращения вала при этом будет максимальной. Если имеется потребность регулирования скорости вращения вала, то нужно снять соответствующую переключку и соединить этот вывод с одним из пинов контроллера. Но, не с любым, а только с одним из тех, возле которых стоит символ ~ «тильда». Это пины с номерами 3, 5, 6, 9, 10, 11. Только на эти пины подаются импульсы широтно-импульсной модуляции (ШИМ), управляющие скоростью вращения вала ДПТ. Для этого на нужный пин с ШИМ подается целое число в диапазоне 0...255: 0 – скорость 0, 255 – максимальная скорость.

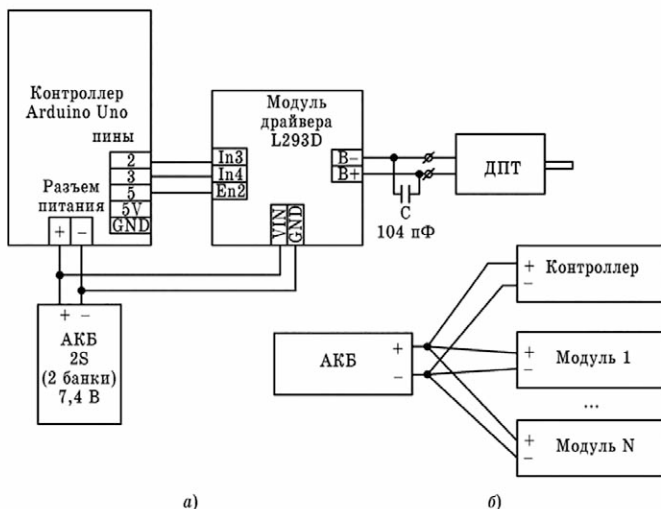


Рис. 1.8

Для примера, показанного на рис. 1.8 (а), пин с номером 5 контроллера используется для регулирования скорости вращения вала ДПТ. Сразу оговоримся, что при использовании некоторых библиотек некоторые пины с ШИМ не работают. Надежно работают только пины 5 и 6. Во всех примерах, как вы увидите дальше, мы будем использовать только эти пины.

Если вы включили ДПТ, а вал вращается не в нужном направлении, то существует три способа исправить ситуацию:

1. Переключить выводы мотора – трудоёмко.
2. Переключить выводы управления, например In3 и In4 – менее трудоёмко.
3. Программно – быстро и просто.

1.6. Борьба с помехами

При работе ДПТ возникают электрические помехи, которые лишь частично гасятся простейшим фильтром в виде конденсатора С. Однако в ряде случаев помеха всё равно проникает в цепи питания контроллера и сбивает его работу. Как поступить в таком случае? Существует несколько приёмов по уменьшению влияния помех, вызванных работой ДПТ, на работу контроллера:

1. Питание от АКБ на контроллер и модули драйверов необходимо разводить отдельными шлейфами – парами проводов (рис. 1.8(б)).
2. Максимально минимизировать длину этих шлейфов.

3. Стараться, чтобы шлейфы меньше пересекались и не шли параллельно друг другу.
4. Шлейфы желательно по возможности максимально разнести в пространстве.
5. Для питания использовать проводники в металлической оплётке.

Если указанные выше меры не приводят к желаемому эффекту (контроллер всё равно сбоят), то необходимо питание контроллера и двигателей осуществлять от разных АКБ.

Все эти меры весьма актуальны, поскольку в работе, как правило, рабочее пространство весьма ограничено и отсутствует реальное заземление электроники и электрических компонентов.

1.7. Модуль драйвера L298N

Модуль используется для управления одним шаговым двигателем или двумя ДПТ с напряжением от 5 до 35 В. Наибольшая нагрузка, которую обеспечивает микросхема, достигает 2 А на каждый двигатель (канал). Внешний вид модуля приведен на рис. 1.9.



Рис. 1.9

Модуль L293D имеет номинальный ток в 0,6 А на один канал, в то время, как модуль L298N имеет ток в 2 А на один канал. Модуль L293D обладает меньшим КПД и быстро греется во время работы. Однако модули L293D являются самыми распространенными и стоят недорого. На рис. 1.10 приведена схема подключения двух ДПТ к модулю L298N.

Модуль L298N получает питание от 9-вольтовой батарейки. Цепи и источник питания контроллера на рис. 1.10 не показаны. Выводы IN1, IN2, IN3 и IN4 модуля L298N подключаются к пинам 7, 6, 5 и 4 контроллера соответственно. Подключение всех остальных контактов представлено на схеме.

Пины 3 и 9 используются здесь для организации регулирования скоростью вращения двигателей.

Таблица обозначений и назначение выводов модуля драйвера L298N

Вывод модуля драйвера	Назначение
In1, In2	Управление включением, выключением и реверсом 1-го ДПТ
In3, In4	Управление включением, выключением и реверсом 2-го ДПТ
En1	Управление скоростью вращения вала 1-го ДПТ
En2	Управление скоростью вращения вала 2-го ДПТ
VMS или VCC	Плюс источника питания (АКБ)
GND	Минус источника питания (АКБ)
OUT1, OUT2 или MOTOR A	Выводы для подключения 1-го ДПТ
OUT3, OUT4 или MOTOR B	Выводы для подключения 2-го ДПТ

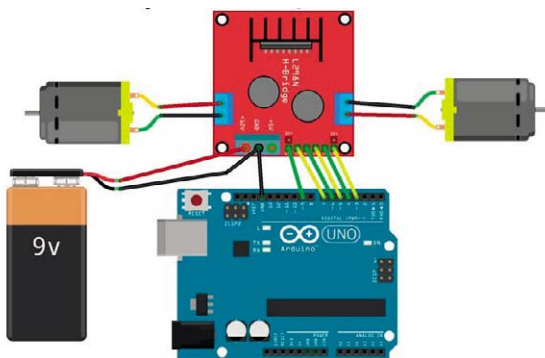


Рис. 1.10

1.8. Модуль драйвера HG7881

Модуль HG7881 (рис. 1.11) позволяет управлять только направлением вращения. Скорость менять он не может. HG7881 – самый дешевый и самый малогабаритный модуль.

HG7881 – двухканальный драйвер, к которому можно подключить два ДПТ или четырехпроводный двухфазный шаговый двигатель (ШД).

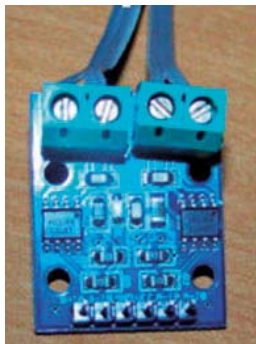


Рис. 1.11

Плата содержит 2 схемы L9110S, работающие как Н-мост.

Характеристики:

- 2 ДПТ;
- Питание – 2,5...12 В;
- Потребляемый ток – 800 мА;
- Малые габариты, небольшой вес.

Выводы:

- GND – земля;
- Vcc – напряжение питания 2,5–12 В;
- A–IA – вход А для двигателя А;
- A–IB – вход В для двигателя А;
- B–IA – вход А для двигателя В;
- B–IB – вход В для двигателя В.

Возможные комбинации сигналов для одного из ДПТ приведены в таблице.

Вывод IA	Вывод IB	Состояние мотора
0	0	Останов
1	0	Вращение вперед
0	1	Вращение назад
1	1	Отключение

Подключение одного ДПТ к контроллеру показано на рис. 1.12.

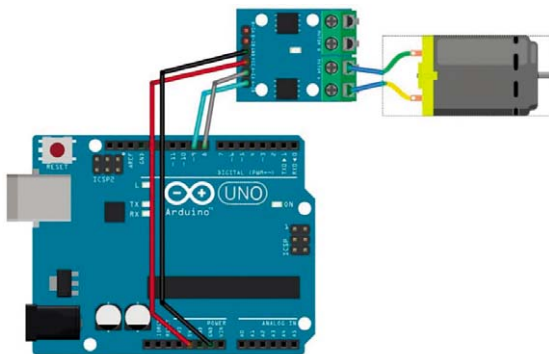


Рис. 1.12

1.9. Подключение ДПТ правого и левого колёс к контроллеру

Для подключения ДПТ приводов колёс, показанных на рис. 1.1...1.4, оказывается вполне достаточным использование одного двухканального модуля драйвера L293D. Схема подключения ДПТ приведена на рис. 1.13. На этом рисунке также показано подключение к контроллеру модулей датчиков:

- KY-022 – ИК-приёмника сигналов с ИК-пультов,
- KY-033 – ИК-датчика, отслеживающего линию,
- HC-SR04 – ультразвукового датчика расстояния.

Кроме этого, на этом рисунке показано подключение к контроллеру сервопривода. Из рис. 1.13 видно, что питание +5 В все датчики и сервопривод получают непосредственно от контроллера. Минус источника питания на модулях датчиков обозначается, как «-», но может использоваться обозначение GND, G или какое-то другое. Плюс источника питания обозначается, как «+», но может использоваться обозначение VCC, V, 5 V, +5 V или какое-то другое.

Выходы датчиков обозначаются, как S, но может использоваться обозначение OUT, Out или какое-то другое. Что касается ультразвукового дальномера HC-SR04, то он имеет один выход Trig и один вход Echo.

После того, как мы через модуль драйвера подсоединили ДПТ колёс к контроллеру, можно смело написать простейший скетч.

Конечно, вы понимаете, что на рис. 1.13 пины, к которым подключены датчики и ДПТ, выбраны произвольно. В реальных устройствах может быть использовано совершенно другое подключение, которое определяется удобством совместного монтажа контроллера и модулей на работе.

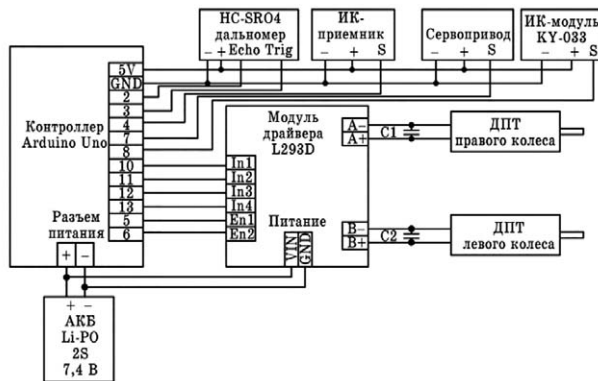


Рис. 1.13

Пример 1.1. Робот должен двигаться 2 секунды назад, затем 2 секунды вперед и т. д. Скетч этого примера (robot1.ino) имеет вид.

```
int a1 = 9; //правое колесо вперед
int a2 = 10; // правое колесо назад
int b1 = 7; //левое колесо вперед
int b2 = 8; // левое колесо назад
void setup()
{
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
}

void loop()
{
    //включаем оба колеса на 2 секунды назад
    digitalWrite(b1, LOW);
    digitalWrite(a1, LOW);
    digitalWrite(b2, HIGH);
    digitalWrite(a2, HIGH);
    delay(2000);

    //включаем оба колеса на 2 секунды вперед
    digitalWrite(b2, LOW);
    digitalWrite(a2, LOW);
    digitalWrite(b1, HIGH);
    digitalWrite(a1, HIGH);
    delay(2000);
}
```

В этом примере не используется изменение скорости вращения вала двигателей, поэтому для управления ими используется всего четыре пина контроллера: 7, 8, 9 и 10. Перемычки на модуле драйвера установлены. Пины 9 и 10 используются для управления включением, выключением и реверсом правого колеса. Пины 7 и 8 используются для управления включением, выключением и реверсом левого колеса. Все эти пины определены в программе, как переменные a1, a2, b1 и b2 соответственно. Переменные a1 и a2 отвечают за вращение правого колеса, а переменные b1 и b2 – за вращение левого колеса в соответствии с таблицей:

Вращение колеса	a1 или b1	a2 или b2
Вперёд	HIGH	LOW
Назад	LOW	HIGH
Останов	LOW	LOW
Недопустимая комбинация	HIGH	HIGH

Из скетча легко угадываются действия, какие необходимо предпринять, если колесо вращается в неправильном направлении. Покажем это на примере.

Было:

```
int a1 = 9; //правое колесо вперёд
```

```
int a2 = 10; // правое колесо назад
```

Переделываем:

```
int a2 = 9; //правое колесо вперёд
```

```
int a1= 10; // правое колесо назад
```

И всё. Колесо начинает вращаться в правильном направлении.

1.10. ИК-пульта и ИК-приёмники

Вполне понятно, что любой робот должен снабжаться средствами, позволяющими управлять им в директорном режиме работы. Самым простейшим является использование для этих целей инфракрасных (ИК) пультов дистанционного управления телевизорами. На рис. 1.14 и 1.15 приведены фото некоторых ИК-пультов и ИК-приёмников (рис. 1.14) и модулей ИК-приёмников (рис. 1.15).

ИК-приёмники замечательно работают без какой-либо дополнительной «обвязки» в виде светодиодов, сопротивлений, которые имеются в модулях. Модули ИК-приёмников, как правило, содержатся в Arduino-наборах.

В работе модули ИК-приёмников (рис. 1.15) ничем не отличаются от просто ИК-приёмников (рис. 1.14), поэтому все эти устройства мы в дальней-

шем будем именовать ИК-приёмниками. Есть только один важный аспект: ИК-приёмник, совместимый с контроллером Arduino, работает не со всеми телевизионными пультами, а лишь с некоторыми из них. На рис. 1.14 и 1.15 показаны три таких пульта. Слева черный «фирменный ардуиновский» пульт Car mp3. Он всем хорош, но имеет небольшую мощность излучения и, как следствие, небольшую дальность действия. По центру расположен пульт от телевизора PHILIPS. Он еще имеет обозначение RC1683701/01. Его дальность действия значительно больше. И, наконец, справа расположен универсальный пульт Qilive (обозначение 863852/RT-003 (E-812)). Этот пульт можно настроить на работу любого телевизионного пульта. Его дальность действия примерно равна дальности действия пульта RC1683701/01. Все эти пульты работоспособны в пределах большой комнаты или небольшого зала.



Рис. 1.14



Рис. 1.15

ИК-приёмник располагается на работе и необходимо стремиться попасть излучением в перекрестье на его передней стороне. Весьма эффективно ИК-приёмники работают на прием отраженного сигнала от стен, пола и потолка. Но необходимо помнить, что это средство управления небольшого радиуса действия – до 12...15 метров.

Так что же принимает ИК-приёмник от ИК-пульта? Он принимает цифровой код. Каждой кнопке пульта соответствует свой код. Программно анализируя принятые коды, робот может «понять», чего от него хочет оператор. В таблице, приведенной ниже, показаны коды кнопок пульта Car mp3.

Пульт RC1683701/01 может работать в трех режимах: TV, DVD, AUX. Наиболее подходящими являются режимы TV и AUX. В режиме DVD большинство кнопок ИК-приёмник «не видит» и выдает число 0. Кстати, некоторые кнопки и в режимах TV и AUX «не видны».

Коды кнопок пульта Car mp3

Кнопка	Код
CH-	16753245
CH	16736925
CH+	16769565
◀◀ (PREV)	16720605
▶▶ (NEXT)	16712445
▶▶ (PLAY/VOL)	16761405
-(VOL-)	16769055
+(VOL+)	16754775
0	16738455
100+	16750695
200+	16756815
1	16724175
2	16718055
3	16743045
4	16716015
5	16726215
6	16734885
7	16728765
8	16730805
9	16732845

В таблице, приведенной ниже, показаны коды некоторых кнопок пульта RC1683701/ 01 в режиме TV и AUX.

Коды некоторых кнопок пульта RC1683701/01 в режимах TV и AUX

Кнопка	Код в режиме TV	Код в режиме AUX
1	1, 2049	3137, 1089
2	2, 2050	3138, 1090
3	3, 2051	3139, 1091
4	4, 2052	3140, 1092
5	5, 2053	3141, 1093
6	6, 2054	3142, 1094
7	7, 2055	3143, 1095
8	8, 2056	3144, 1096
9	9, 2057	3145, 1097
0	0, 2048	3136, 1088
i+	15, 2063	-----
p<p	34, 2082	3147, 1099
vol+	16, 2064	3088, 1040
vol-	17, 2065	3089, 1041
p+	32, 2080	3168, 1120
p-	33, 2081	3169, 1121

Из этой таблицы видно, что при нажатии кнопки пульт в один раз отправляет первый код, а во второй раз – второй код и т. д. Необходимо в скетче прописать обработку обоих кодов, иначе кнопка будет «срабатывать» через раз. Видно также, что в режиме AUX кнопка i+ не работает.

В таблице, приведенной ниже, показаны коды некоторых кнопок пульта Qilive в режимах tv1 и dvd.

Коды некоторых кнопок пульта Qilive в режимах tv1 и dvd

Кнопка	Код в режиме tv1	Код в режиме dvd
1	16748655	284111565
2	16758855	284101365
3	16775175	284134005
4	16756815	284160015
5	16750695	284109525
6	16767015	284142165
7	16746615	284149815
8	16754775	284157975
9	16771095	284105445
0	16730295	284125335
AV	-----	284106975
last	16769565	-----
vol+	16760895	284144205
vol-	16728255	284138085
ch+	16734375	284121765
ch-	16740495	284107485

В этих таблицах приведены коды не всех кнопок пультов. Да и большего количества вряд ли понадобится. В примере 1.2 приведён простенький скетч, который позволяет просмотреть коды кнопок ИК-пультов.

1.11. Примеры

Пример 1.2. Просмотр кодов ИК-пультов. Скетч примера (kody.ino) приведен ниже.

```
#include "IRremote.h" //подключаем библиотеку
int receiver = 12; //к пину 12 подключен ИК-приемник
IRrecv irrecv(receiver);
decode_results results;

void setup()
{
```

```

    irrecv.enableIRIn();
    pinMode(receiver, INPUT); //пин 12 – на ввод
    Serial.begin(9600); // инициализация монитора порта
}

void loop()
{
    if (irrecv.decode(&results))
    {
        translateIR();
        irrecv.resume();
    }
}

void translateIR()
{
    //выводим принятый код в монитор порта
    Serial.println(results.value);
}

```

Кнопки на ИК-пультах могут выполнять следующие действия:

1. Непосредственно управлять исполнительными механизмами (ИМ), расположенными на роботе;
2. Задавать режимы работы робота, т. е. выбирать нужные функции, определяющие характер поведения робота;
3. Задавать различные параметры работы робота: скорость движения, углы поворота, время задержки и т. д.

Ниже приведена таблица использования кнопок в скетчах для непосредственного управления ИМ робота.

Таблица управления ИМ кнопками ИК-пульта

Кнопка	Действие
1	Движение робота вперед влево
2	Движение робота вперед
3	Движение робота вперед вправо
4	Добавить скорости движения
5	Останов
6	Убавить скорости движения
7	Движение робота назад влево
8	Движение робота назад
9	Движение робота назад вправо
0	Дополнительная функция
AV, i+	Выстрел из пушки

Кнопка	Действие
last, p<p	Выстрел из пушки
vol+	Поворот башни танка влево
vol-	Поворот башни танка вправо
ch+, p+	Дополнительная функция
ch-, p-	Дополнительная функция

Для увеличения надежности приема сигнала с пульта с разных ракурсов допускается использование нескольких ИК-приёмников, включенных параллельно. В этом случае ИК-приёмники должны быть развернуты в разные стороны. Даже использование двух ИК-приёмников, «смотрящих» в разные стороны, резко повышает вероятность приема сигналов от ИК-пульта. Рассмотрим ещё несколько скетчей на эту тему.

Пример 1.3. Просмотр кодов пульта Car mp3 в шестнадцатеричной форме с расшифровкой названия кнопки. Скетч примера (robot7.ino) приведен ниже.

```
#include "IRremote.h" // подключаем библиотеку
// пин 12 для подключения ИК-приёмника
int receiver = 12;
//создаём экземпляр класса IRecv; в качестве параметра
//передаём пин подключения ИК-приёмника
IRecv irrecv(receiver);
// переменная для принятого кода
decode_results results;

void setup()
{
    Serial.begin(9600); //устанавливаем скорость монитора
    irrecv.enableIRIn(); // включаем ИК-приёмник
}

void loop()
{
    //проверка буфера приёмника
    if (irrecv.decode(&results))
    {
        //кнопка была нажата
        //выведем в монитор порта её HEX-код
        Serial.println(results.value,HEX);
        translateIR(); //процедура обработки кода кнопки
        irrecv.resume(); // читаем следующий код
    }
}
```

```

void translateIR()
{
    switch(results.value)
    {
        //вывод в монитор порта названия нажатой кнопки
        case 0xFFA25D:
            Serial.println(" CH-      ");
            break;

        case 0xFF629D:
            Serial.println(" CH      ");
            break;

        case 0xFFE21D:
            Serial.println(" CH+      ");
            break;

        case 0xFF22DD:
            Serial.println(" PREV      ");
            break;

        case 0xFF02FD:
            Serial.println(" NEXT      ");
            break;

        case 0xFFC23D:
            Serial.println(" PLAY/PAUSE  ");
            break;

        case 0xFFE01F:
            Serial.println(" VOL-      ");
            break;

        case 0xFFA857:
            Serial.println(" VOL+      ");
            break;

        case 0xFF906F:
            Serial.println(" EQ      ");
            break;

        case 0xFF6897:
            Serial.println(" 0      ");
            break;

        case 0xFF9867:

```

```

        Serial.println(" 100+ ");
        break;

    case 0xFFB04F:
        Serial.println(" 200+ ");
        break;

    case 0xFF30CF:
        Serial.println(" 1 ");
        break;

    case 0xFF18E7:
        Serial.println(" 2 ");
        break;

    case 0xFF7A85:
        Serial.println(" 3 ");
        break;

    case 0xFF10EF:
        Serial.println(" 4 ");
        break;

    case 0xFF38C7:
        Serial.println(" 5 ");
        break;

    case 0xFF5AA5:
        Serial.println(" 6 ");
        break;

    case 0xFF42BD:
        Serial.println(" 7 ");
        break;

    case 0xFF4AB5:
        Serial.println(" 8 ");
        break;

    case 0xFF52AD:
        Serial.println(" 9 ");
        break;

    default:
        Serial.println("Другая клавиша");
}

```

```
delay(500);
```

```
}
```

Пример 1.4. Управление движением робота от пульта Саг mр3: вперед, назад, вбок, останов. Скетч примера (robot8.ino) приведен ниже. ДПТ колёс подключены к контроллеру в соответствии со скетчем. Некоторые операторы скетча закомментированы и использовались лишь при отладке скетча.

```
#include "IRremote.h"
```

```
int receiver = 12; //пин ИК-приёмника
IRecv irrecv(receiver);
decode_results results;
//пины колёс
int a1 = 9; //правое колесо вперёд
int a2 = 10; //правое колесо назад
int b1 = 7; //левое колесо вперёд
int b2 = 8; //левое колесо назад
```

```
void setup()
{
    irrecv.enableIRIn();
    //настройка пинов на вывод
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    //настройка пина на ввод
    pinMode(receiver, INPUT);
}
```

```
void loop()
{
    if (irrecv.decode(&results))
    {
        translateIR();
        irrecv.resume();
    }
}
```

```
void translateIR()
```

```

{
  //анализ кода нажатой клавиши
  switch(results.value)
  {

    case 0xFF30CF:
      //Serial.println(" 1          ");
      //правое колесо вперед
      digitalWrite(b1, LOW);
      digitalWrite(a1, HIGH);
      digitalWrite(b2, LOW );
      digitalWrite(a2, LOW);
      break;

    case 0xFF18E7:
      //Serial.println(" 2          ");
      //оба колеса вперед
      digitalWrite(b1, HIGH);
      digitalWrite(a1, HIGH);
      digitalWrite(b2, LOW );
      digitalWrite(a2, LOW);
      break;

    case 0xFF7A85:
      //Serial.println(" 3          ");
      //левое колесо вперед
      digitalWrite(b1, HIGH);
      digitalWrite(a1, LOW );
      digitalWrite(b2, LOW );
      digitalWrite(a2, LOW);
      break;

    case 0xFF38C7:
      //Serial.println(" 5          ");
      //оба колеса – стоп
      digitalWrite(b1, LOW);
      digitalWrite(a1, LOW);
      digitalWrite(b2, LOW );
      digitalWrite(a2, LOW);
      break;

    case 0xFF42BD:
      //Serial.println(" 7          ");
      //правое колесо назад
      digitalWrite(b1, LOW);
      digitalWrite(a1, LOW);
      digitalWrite(b2, LOW );

```

```

        digitalWrite(a2, HIGH);
        break;

    case 0xFF4AB5:
        //Serial.println(" 8      ");
        //оба колеса назад
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(b2, HIGH);
        digitalWrite(a2, HIGH);
        break;

    case 0xFF52AD:
        //Serial.println(" 9      ");
        //левое колесо назад
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(b2, HIGH );
        digitalWrite(a2, LOW);
        break;

    }
}

```

1.12. Управление скоростью движения робота

В предыдущем примере мы управляли роботом, но он будет двигаться с максимальной скоростью. Это в большинстве случаев неприемлемо и вообще не обеспечивает плавность хода робота. Для изменения скорости вращения колес модули драйверов снабжаются входами (En1, En2 или EnA, EnB), на которые с контроллера подаются ШИМ-сигналы. ШИМ – это широтно-импульсная модуляция. ШИМ – сигнал – это импульсы, длительность которых можно регулировать от 0 до максимального значения. Значение ШИМ – сигнала задается целым числом в диапазоне 0...255. В предыдущий скетч введем возможность изменения скорости вращения правого и левого колес.

Пример 1.5. Управление движением робота от пульта Car mp3: вперед, назад, вбок, останов. Скетч примера (robot9.ino) приведен ниже. ДПТ колёс подключены к контроллеру в соответствии со скетчем. Некоторые операторы скетча закомментированы и использовались лишь при отладке скетча.

```

#include "IRremote.h"

int receiver = 12; //пин ИК-приёмника
IRrecv irrecv(receiver);
decode_results results;
//пины колёс
int a1 = 9; //правое колесо вперёд
int a2 = 10; //правое колесо назад
int b1 = 7; //левое колесо вперёд
int b2 = 8; //левое колесо назад
int enA = 6; // правый скорость
int enB = 5; // левый скорость
//значение скорости
int k=100;

void setup()
{
    irrecv.enableIRIn();
    //настройка пинов на вывод
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    //настройка пина на ввод
    pinMode(receiver, INPUT);
    analogWrite(enA, k);
    analogWrite(enB, k);

}

void loop()
{
    if (irrecv.decode(&results))
    {
        translateIR();
        irrecv.resume();
    }
}

void translateIR()
{
    //анализ кода нажатой клавиши
    switch(results.value)

```

```

{

case 0xFF30CF:
    //Serial.println(" 1          ");
    //правое колесо вперед со скоростью 190
    analogWrite(enA, 190);
    digitalWrite(b1, LOW);
    digitalWrite(a1, HIGH);
    digitalWrite(b2, LOW );
    digitalWrite(a2, LOW);
    break;

case 0xFF18E7:
    //Serial.println(" 2          ");
    //оба колеса вперед с разной скоростью
    analogWrite(enA, k);
    analogWrite(enB, k+20);
    digitalWrite(b1, HIGH);
    digitalWrite(a1, HIGH);
    digitalWrite(b2, LOW );
    digitalWrite(a2, LOW);
    break;

case 0xFF7A85:
    //Serial.println(" 3          ");
    //левое колесо вперед со скоростью 212
    analogWrite(enB, 212);
    digitalWrite(b1, HIGH);
    digitalWrite(a1, LOW );
    digitalWrite(b2, LOW );
    digitalWrite(a2, LOW);
    break;

case 0xFF38C7:
    //Serial.println(" 5          ");
    //оба колеса – стоп
    digitalWrite(b1, LOW);
    digitalWrite(a1, LOW);
    digitalWrite(b2, LOW );
    digitalWrite(a2, LOW);
    break;

case 0xFF42BD:
    //Serial.println(" 7          ");
    //правое колесо назад со скоростью 95
    analogWrite(enA, k-5);
    digitalWrite(b1, LOW);

```



```

        digitalWrite(a1, LOW);
        digitalWrite(b2, LOW );
        digitalWrite(a2, HIGH);
        break;

    case 0xFF4AB5:
        //Serial.println(" 8");
        //оба колеса назад со скоростью 190
        analogWrite(enA, 190);
        analogWrite(enA, k+90);
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(b2, HIGH);
        digitalWrite(a2, HIGH);
        break;

    case 0xFF52AD:
        //Serial.println(" 9");
        //левое колесо назад со скоростью 176
        analogWrite(enB, k*2-24);
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(b2, HIGH);
        digitalWrite(a2, LOW);
        break;

}
}

```

1.13. Организация многопрограммного режима работы робота

Пример 1.6. Скетч позволяет запускать на роботе 11 программ. Выбор программы осуществляется нажатием одной из верхних кнопок пульта Car mp3. Скетч примера (tank14-3.ino) приведен ниже. ДПТ колёс подключены к контроллеру в соответствии со скетчем. Некоторые операторы скетча закомментированы и использовались лишь при отладке скетча.

```

//скетч позволяет запускать на роботе 11 программ.
//выбор программы осуществляется нажатием одной из верхних
//кнопок пульта Car mp3:
//CH-   – останов робота
//CH     – вперед-назад по 1 секунде
//CH+   – ручное управление роботом кнопками 1–9

```

//PREV – перемещение по квадрату влево
//NEXT – перемещение по квадрату вправо
//PLAY/PAUSE – разгон-торможение, вперед-назад
//VOL– – катается по кругу против часовой стрелки
//VOL+ – катается по кругу по часовой стрелке
//0 – катается по змейке
//100+ – катается по восьмёрке
//200+ – резерв

```
#include "IRremote.h"  
//#include <Wire.h>  
//#include <MPU6050.h>  
int receiver = 8; // пин ИК-приёмника  
IRrecv irrecv(receiver);  
decode_results results;  
decode_results results1;  
//пины
```

```
// подключите пины контроллера к цифровым пинам Arduino  
// правый двигатель  
int enA = 6; //правый скорость  
int in1 = 13; //правый вперед  
int in2 = 12; //правый назад  
// левый двигатель  
int enB = 5; //левый скорость  
int in3 = 11; //левый вперед  
int in4 = 7; //левый назад
```

```
//переменные  
//начальная скорость  
int k=150;  
//номер режима  
int regim=0;
```

```
void setup()  
{  
  pinMode(enA, OUTPUT);  
  pinMode(enB, OUTPUT);  
  pinMode(in1, OUTPUT);  
  pinMode(in2, OUTPUT);  
  pinMode(in3, OUTPUT);  
  pinMode(in4, OUTPUT);  
  irrecv.enableIRIn();  
  pinMode(receiver, INPUT);  
  // инициализация монитора порта  
  Serial.begin(9600);
```

```

    delay(1000);    //ждем
}

void loop()
{
    if (irrecv.decode(&results))
    {
        //Serial.println(results.value);
        switch(results.value)
        {
            //нажата CH-
            case 16753245:
                regim =0;
                break;

            //нажата CH
            case 16736925:
                regim =1;
                break;

            //нажата CH+
            case 16769565:
                regim =2;
                break;

            //нажата PREV
            case 16720605:
                regim =3;
                break;

            //нажата NEXT
            case 16712445:
                regim =4;
                break;

            //нажата PLAY/PAUSE
            case 16761405:
                regim =5;
                break;

            //нажата VOL-
            case 16769055:
                regim =6;
                break;

            //нажата VOL+

```

```

        case 16754775:
            regim =7;
            break;

            //нажата 0
        case 16738455:
            regim =8;
            break;

            //нажата 100+
        case 16750695:
            regim =9;
            break;

            //нажата 200+
        case 16756815:
            regim =10;
            break;

    }
    irrecv.resume();
}
//Serial.println(regim);
switch (regim)
{
    case 0:
        reg0();
        break;

    case 1:
        reg1();
        break;

    case 2:
        reg2();
        break;

    case 3:
        reg3();
        break;

    case 4:
        reg4();
        break;

    case 5:
        reg5();

```

```

        break;

    case 6:
        reg6();
        break;

    case 7:
        reg7();
        break;

    case 8:
        reg8();
        break;

    case 9:
        reg9();
        break;

    }
}

//останов работа
void reg0()
{
    analogWrite(enA, k);
    analogWrite(enB, k);
    digitalWrite(in2, LOW);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
}

//движение вперед-назад по 1 секунде
void reg1()
{
    analogWrite(enA, k);
    analogWrite(enB, k);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(1000);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW );
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH) ;
    delay(1000);
}

```

```

}

//квадрат влево
void reg3()
{
    analogWrite(enA, k);
    analogWrite(enB, k);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(700);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    delay(500);

}

//квадрат вправо
void reg4()
{
    analogWrite(enA, k);
    analogWrite(enB, k);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(700);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    delay(500);

}

//разгон-торможение
void reg5()
{
    for (int i=0; i<=255; i=i+10)
    {
        analogWrite(enA, i);
        analogWrite(enB, i);
        digitalWrite(in2, LOW);
        digitalWrite(in1, HIGH);
    }
}

```

```

        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
        delay(100);
    }

    for (int i=255; i>=0; i=i-10)
    {
        analogWrite(enA, i);
        analogWrite(enB, i);
        digitalWrite(in2, HIGH );
        digitalWrite(in1,LOW );
        digitalWrite(in3,LOW );
        digitalWrite(in4, HIGH );
        delay(100);
    }
}

//движение по кругу влево
void reg6()
{
    analogWrite(enA, 255);
    analogWrite(enB, 70);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

//движение по кругу вправо
void reg7()
{
    analogWrite(enA,70 );
    analogWrite(enB, 255 );
    digitalWrite(in2,LOW );
    digitalWrite(in1,HIGH );
    digitalWrite(in3,HIGH );
    digitalWrite(in4, LOW );
}

//движение змейкой
void reg8()
{
    analogWrite(enA, k+100);
    analogWrite(enB, k);
    digitalWrite(in2, LOW );
    digitalWrite(in1,HIGH );

```

```

        digitalWrite(in3,HIGH );
        digitalWrite(in4, LOW );
        delay (800);
        analogWrite(enA, k);
        analogWrite(enB, k+100);
        digitalWrite(in2, LOW );
        digitalWrite(in1,HIGH );
        digitalWrite(in3,HIGH );
        digitalWrite(in4, LOW );
        delay (800);
    }

//движение по восьмерке
void reg9()
{
    analogWrite(enA, 255);
    analogWrite(enB, 70);
    digitalWrite(in2, LOW );
    digitalWrite(in1,HIGH );
    digitalWrite(in3,HIGH );
    digitalWrite(in4, LOW );
    delay (1800);
    analogWrite(enA, 70);
    analogWrite(enB, 255);
    digitalWrite(in2, LOW );
    digitalWrite(in1,HIGH );
    digitalWrite(in3,HIGH );
    digitalWrite(in4, LOW );
    delay (1800);
}

//ручной режим
//управление роботом кнопками 1 – 9
void reg2()
{
    if (irrecv.decode(&results1))
    {
        //Serial.println(results1.value);
        komanda();
    }
}

void komanda()
{

```



```

        switch(results1.value)
    {
        //нажата 1: вперед – влево
        case 16724175:
            //Serial.println(" 1          ");
            digitalWrite(in2, LOW);
            digitalWrite(in1, HIGH);
            digitalWrite(in3, LOW);
            digitalWrite(in4, HIGH);
            analogWrite(enA, k);
            analogWrite(enB, k);
            break;

            //нажата 2: вперед
        case 16718055:
            //Serial.println(" 2          ");
            digitalWrite(in1, HIGH);
            digitalWrite(in2, LOW);
            digitalWrite(in3, HIGH);
            digitalWrite(in4, LOW);
            analogWrite(enA, k);
            analogWrite(enB, k);
            break;

            //нажата 3: вперед -вправо
        case 16743045:
            //Serial.println(" 3          ");
            digitalWrite(in3, HIGH);
            digitalWrite(in1, LOW);
            digitalWrite(in4, LOW);
            digitalWrite(in2, HIGH);
            analogWrite(enA, k);
            analogWrite(enB, k);
            break;

            //нажата 5: останов
        case 16726215:
            //Serial.println(" 5          ");
            digitalWrite(in1, LOW);
            digitalWrite(in2, LOW);
            digitalWrite(in3, LOW);
            digitalWrite(in4, LOW);
            break;

            //нажата 7: назад -влево
        case 16728765:
            //Serial.println(" 7          ");

```

```

digitalWrite(in4, LOW);
digitalWrite(in1, LOW);
digitalWrite(in3, HIGH);
digitalWrite(in2, HIGH);
//digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

//нажата 8: назад
case 16730805:
    //Serial.println(" 8      ");
    digitalWrite(in4, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in2, HIGH);
    //digitalWrite(c1, LOW );
    analogWrite(enA, k);
    analogWrite(enB, k);
    break;

//нажата 9: назад -вправо
case 16732845:
    //Serial.println(" 9      ");
    digitalWrite(in4, HIGH);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in2, LOW);
    //digitalWrite(c1, LOW );
    analogWrite(enA, k);
    analogWrite(enB, k);
    break;

//нажата 4: добавим скорость на 10
case 16716015:
    //Serial.println(" 4      ");
    if (k<230)
    {
        k=k+10;
    }
    break;

//нажата 6: убавим скорость на 10
case 16734885:
    //Serial.println(" 6      ");
    if (k>40)
    {

```

```

        k=k-10;
    }
    break;
}
}

```

Хотя всё хорошо закомментировано, дадим несколько пояснений. Кнопки CH-, CH, CH+, PREV, NEXT, PLAY/PAUSE, VOL-, VOL+, 0, 100+, 200+ используются для выбора профиля поведения робота – режима его работы. При нажатии одной из этих кнопок переменной `regim` присваивается номер режима. Затем по этому номеру вызывается соответствующая процедура `reg0()...reg9()`. При нажатии кнопки CH+ вызывается процедура `reg2()`, которая обеспечивает ручное управление роботом кнопками 1...9. Здесь возможны увеличение и уменьшение скорости движения робота: кнопки 4 и 6 соответственно. Скетч при выполнении некоторых процедур «тормозит», т. е. не обеспечивает мгновенной реакции на нажатие новой кнопки, поэтому кнопку иногда нужно нажимать несколько раз.

1.14. Роботы-танки

https://t.me/it_boooks/2

1.14.1. Аппаратная часть

Все знают, что роботы получили широкое применение в военном деле. Всё это хорошо видно из материалов различных военных выставок, военных шоу и обзоров. Робототехника на основе контроллеров Arduino не осталась в стороне от этого направления развития современной техники. Стали выпускаться достаточно качественные гусеничные шасси (рис. 1.16 и 1.17) и пушки (рис. 1.18).

Гусеничное шасси. На рис. 1.16 показано мощное, но медленное гусеничное шасси с поворотной платформой наверху. На борту этого агрегата находятся три ДПТ: два в приводе гусениц и один в приводе поворотной платформы. Кроме этого, в продаже имеются высокоскоростные гусеничные шасси (рис. 1.17). Они на борту имеют два ДПТ в приводах гусениц. В обоих этих случаях для управления ДПТ в приводе гусениц необходимо использовать модуль драйвера L298N из-за значительных токов, потребляемых ДПТ. Для управления приводом гусениц желательно использовать регулирование скорости (ШИМ).

Поворотная платформа. На поворотную платформу легко монтируется башня танка, а также датчики и вся электроника. В приводе поворотной платформы можно использовать маломощный L293D. Для управления поворотной платформой можно использовать регулирование скорости или не использовать его.

Всё зависит от заданного качества управления поворотной платформой.



Рис. 1.16

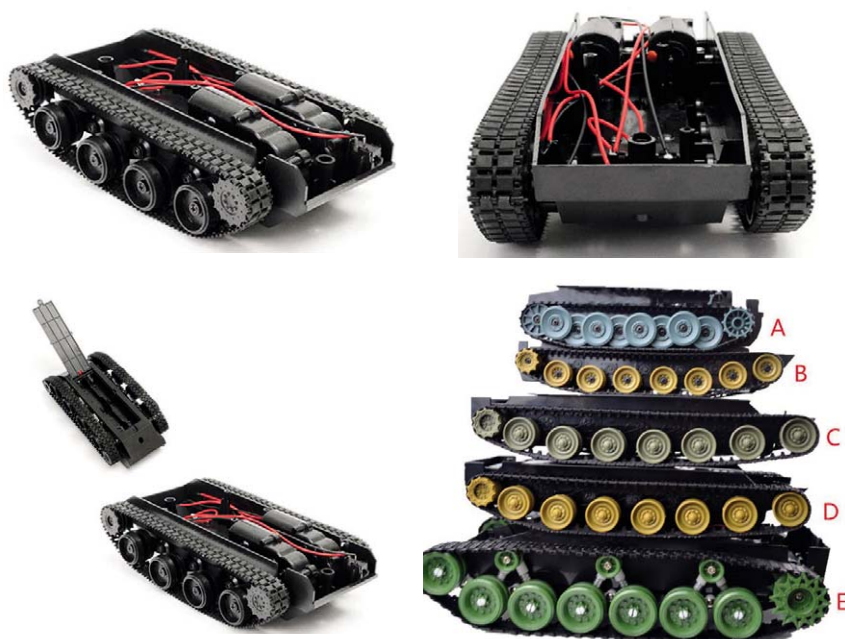


Рис. 1.17

Пушка. Пушка, конечно, – это то, что делает танк танком. К сожалению, в отличие от гусеничных шасси, ассортимент танковых башен в продаже не очень обширен. Здесь мы приводим самый дешевый вариант, имеющийся в продаже (рис. 1.18). Эта легко разбираемая танковая башня содержит:

- ДПТ, совмещенный с редуктором, приводящим в движение механизм, выстреливающий пластмассовые шарики;
- ствол;
- бункер для загрузки пластмассовых шариков;
- корпус башни, который сверху накрывает все элементы конструкции.

Все эти детали легко стыкуются между собой, образуя единую конструкцию. Редуктор легко разбирается, так как состоит из двух половин, скрепленных маленькими саморезами. С точки зрения управления стрельбой, мы имеем один ДПТ, который может включаться в систему через модуль драйвера L293D без регулирования скорости, т. е. без ШИМ.

Становится понятным, что для управления танком понадобится один модуль L298N и один модуль L293D. Подсчитаем минимальное количество необходимых пинов контроллера для управления ходовой частью, поворотной платформой и пушкой: $2 \cdot 3 + 1 \cdot 1 + 1 \cdot 2 = 9$. Первое слагаемое – приводы ходовой части: два привода с реверсом и с регулированием скоростью. Второе слагаемое – один привод пушки без реверса и регулирования скорости. Третье слагаемое: один привод поворотной платформы с реверсом без регулирования скорости.



Рис. 1.18

Шарик из бункера поступает в выстреливающий механизм и выстреливается через ствол пушки. Его место занимает следующий шарик и т. д. После включения ДПТ привода пушки шарики очень быстро будут выстреливаться и боекомплект быстро закончится. Но противника при этом можно эффективно «залить свинцом». Однако ресурс времени работы пластиковых шестеренок в редукторе привода пушки весьма ограничен – несколько часов. После выработки этого ресурса пушка перестает стрелять. Выходит из строя (стачивается) всего одна пара шестерёнок, но запасных нет. Поэтому для экономии ресурса пушки программно необходимо реализовать режим стрельбы одиночными выстрелами.

Датчики. На танк можно навешивать любые датчики от ультразвукового дальномера до модулей датчиков обнаружения газа. Под датчики и другие исполнительные механизмы (сервоприводы, лазерные излучатели, светодиоды, соленоиды и т. д.) остается не так уж и много пинов. Для контроллера Arduino Uno это будет: $14 - 9 = 5$ пинов. Если учесть, что пины с номерами 0 и 1 стараются не использовать, то остается всего три цифровых пина. Этого мало. Но тут можно использовать ещё шесть аналоговых пинов A0...A5. Всего получается $3 + 6 = 9$ пинов. Это уже вполне приемлемая величина для робота средней конфигурации. Теперь поясним, почему нежелательно использовать пины с номерами 0 и 1 в наших проектах. Ну, во-первых, если вы используете Bluetooth-модули, то они как-раз и подключаются только к пинам с номерами 0 и 1. Кроме того, имеется ещё ряд модулей, которые подключаются к этим пинам. И ещё. Если вы все-таки используете эти пины, то при каждой загрузке скетча в контроллер необходимо отключать оборудование от этих пинов.

Теперь несколько слов о подключении датчиков к контроллеру. Сигнальные линии (провода) в значительной степени подвержены влиянию помех, создаваемых ДПТ, поэтому желательно, по возможности, выполнять следующие правила:

1. Сигнальные и силовые провода желательно разнести в пространстве.
2. Желательно, чтобы они никогда не проходили параллельно.
3. Желательно, чтобы они нигде не пересекались и находились на максимальном удалении друг от друга.
4. Желательно, по возможности, экранировать и сигнальные и силовые провода, соединяя экран с «землей» устройства.
5. ДПТ необходимо шунтировать конденсаторами в 100 000 пФ.

Эти простые правила позволят вам избавиться от многих «непоняток» в работе скетча и многочасовых поисков несуществующих ошибок в нем.

1.14.2. Варианты компоновок роботов-танков

Компоновка на основе мощного гусеничного шасси с поворотной платформой.

Эта компоновка приведена на рис. 1.19.



Рис. 1.19

Здесь на поворотной платформе смонтированы пушка и несколько датчиков:

- ультразвуковой дальномер HC-SR04 (спереди в центре);
- модуль датчика пламени ИК KY-026 (спереди слева);
- модуль с ИК-датчиком обнаружения препятствий KY-032 (спереди справа);
- двоянный ИК-приёмник для управления роботом с помощью ИК-пульта (справа); к нему подводится шлейф из трех проводов: сигнальный, питание +5 В и «земля» -GND;
- Bluetooth-модуль HC-06; он хорошо просматривается на виде сзади;
- телекамера EAXINE TX02 (на крыше башни), с помощью которой легко управлять танком и вести стрельбу даже не видя танка;
- герконовый датчик.

Герконовый датчик (рис. 1.20, слева) представляет собой обычный геркон, прикрепленный к поворотной платформе. Справа и слева по корпусу робота прикреплены магниты (рис. 1.20, справа). При повороте башни на угол, близкий к 90 градусам, геркон попадает в поле действия магнита и его контакт замыкается. В систему управления поступает сигнал, что башня повернулась на предельный угол. Система, получив этот сигнал, игнорирует дальнейший поворот башни в эту сторону.

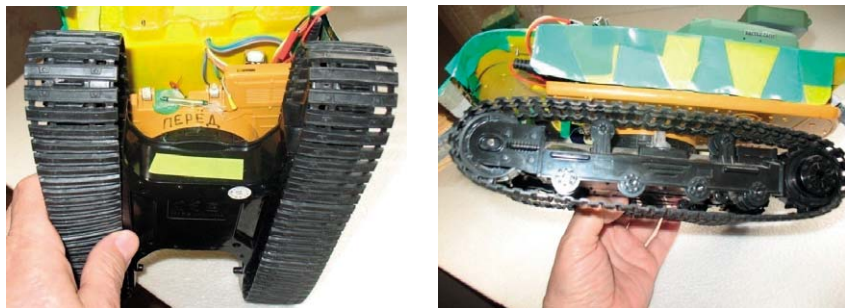


Рис. 1.20

Компоновка на основе высокоскоростного гусеничного шасси.
Эта компоновка приведена на рис. 1.21.

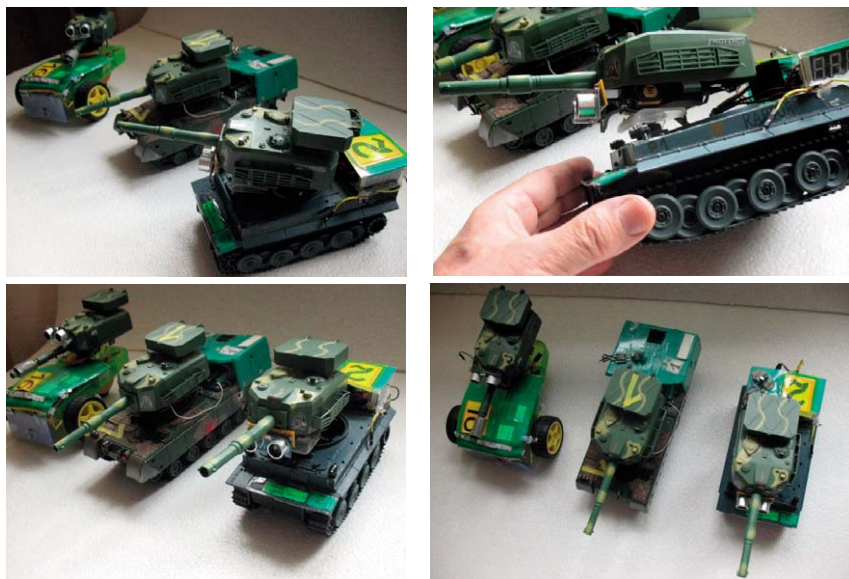


Рис. 1.21

Из рис. 1.21 следует, что:

- башня пушки устанавливается на поворотную платформу, закрепленную на валу сервопривода;
- башня пушки может устанавливаться и на колёсном шасси;
- робот можно оснащать любыми датчиками, число которых ограничено числом свободных пинов в контроллере.

1.14.3. Примеры

Сначала рассмотрим несколько простых примеров, которые позволят нам затем перейти к примерам реального управления роботами.

Пример 1.7. Циклический поворот вала серводвигателя (сервомашинки). Скетч примера (robot2.ino) приведен ниже.

```
//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo
void setup() //процедура setup
{
    servo.attach(6); //привязываем привод к пину 6
    servo.write(0); //ставим вал под 0
    delay(2000);    //ждем 2 секунды
    servo.write(77); //ставим вал по центру
    delay(2000);    //ждем 2 секунды
}
void loop()
{
    servo.write(154); //ставим вал под 154
    delay(2000);      //ждем 2 секунды
    servo.write(0);   //ставим вал под 0
    delay(2000);      //ждем 2 секунды
}
```

Пример 1.8. Вывод в монитор порта показаний ультразвукового даль-
номера. Скетч примера (robot3.ino) приведен ниже.

```
const int trigPin = 10;
const int echoPin = 11;
void setup()
{
    pinMode(trigPin, OUTPUT); // пин для Trig
    pinMode(echoPin, INPUT);  // пин для Echo
    Serial.begin(9600); // инициализация монитора порта
}
```

```

void loop()
{
    // получаем дистанцию с датчика
    long distance = getDistance();
    if (distance <= 400) // убираем помехи
    {
        // выводим результат в монитор порта
        Serial.println(distance);
    }
    delay(100);
}

// функция определения дистанции до объекта в см
long getDistance()
{
    long distacne_cm = getEchoTiming() * 1.7 * 0.01;
    return distacne_cm;
}

// функция определения времени задержки
long getEchoTiming()
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // генерируем импульс запуска
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // определение на пине echoPin длительности
    // уровня HIGH в мсек
    long duration = pulseIn(echoPin, HIGH);
    return duration;
}

```

Пример 1.9. Включение светодиода при обнаружении препятствия ИК-датчиком обнаружения препятствий KY-032.

Скетч примера (robot4.ino) приведен ниже.

```

//KY-032 ИК-датчик обнаружения препятствий
int led = 9 ; // пин светодиода
int sensorObstaclesPin = 10; // пин датчика
int value ; // значение датчика
void setup ()
{
    pinMode (led, OUTPUT) ;
    pinMode (sensorObstaclesPin, INPUT);
}

```

```

    }
void loop ()
{
    // чтение датчика
    value = digitalRead (sensorObstaclesPin) ;
    if (value == LOW)
    {
        // если датчик обнаружил препятствие,
        //светодиод горит
        digitalWrite (led, HIGH);
    }
    else
    {
        digitalWrite (led, LOW);
    }
}

```

Пример 1.10. Робот едет вперёд и останавливается при появлении препятствия на расстоянии менее 60 см.

Скетч примера (robot5.ino) приведен ниже.

```

//робот едет вперёд и останавливается
//при появлении препятствия на расстоянии
//менее 60 см
int a1 = 2; //правое колесо вперёд
int a2 = 3; // правое колесо назад
int b1 = 4; //левое колесо вперёд
int b2 = 5; // левое колесо назад
//пины для датчика
const int trigPin = 10;
const int echoPin = 11;

void setup()
{
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    pinMode(trigPin, OUTPUT); // пин для Trig
    pinMode(echoPin, INPUT);  // пин для Echo
    Serial.begin(9600); // инициализация послед. порта
}

void loop()
{

```

```

        long distance = getDistance();
        if (distance < 255)
        {

                //Serial.println(distance);
                if (distance >= 60)
                {
                        digitalWrite(b1, HIGH);
                        digitalWrite(a1, HIGH);
                }
                else
                {
                        digitalWrite(b1, LOW);
                        digitalWrite(a1, LOW);
                }
        }
}

// функция определения дистанции до объекта в см
long getDistance()
{
        long distacne_cm = getEchoTiming() * 1.7 * 0.01;
        return distacne_cm;
}

// функция определения времени задержки
long getEchoTiming()
{
        digitalWrite(trigPin, LOW);
        delayMicroseconds(2);
        // генерируем импульс запуска
        digitalWrite(trigPin, HIGH);
        delayMicroseconds(10);
        digitalWrite(trigPin, LOW);
        // определение на пине echoPin длительности
        // уровня HIGH в мсек
        long duration = pulseIn(echoPin, HIGH);
        return duration;
}

```

В следующих примерах рассматривается робот, который оснащен ультразвуковым дальномером, установленным на сервомашинке (рис. 1.22). Таким образом, можно просматривать пространство не только впереди робота, но и в секторе $-90^{\circ} \dots +90^{\circ}$ относительно продольной его оси без поворота самого робота.

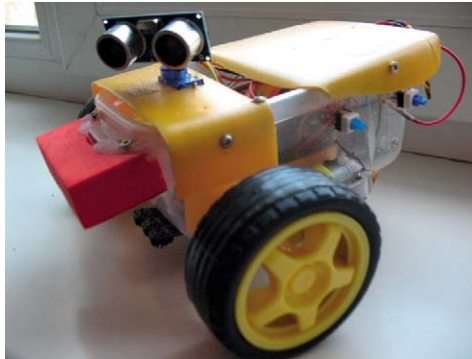


Рис. 1.22

Пример 1.11. Робот едет вперёд и при появлении препятствия стремится отвернуть влево.

Скетч примера (robot1.ino) приведен ниже.

```
//робот едет вперёд и при появлении препятствия
// стремится повернуть влево
```

```
//подключаем библиотеку Servo.h для
//работы с сервоприводом
#include <Servo.h>
```

```
//объявляем переменную servo типа Servo
Servo servo;
```

```
//пины
int a1 = 9; //правое колесо вперёд
int a2 = 10; // правое колесо назад
int b1 = 7; //левое колесо вперёд
int b2 = 8; // левое колесо назад
int enA =6; // правый скорость
int enB =5; // левый скорость
const int trigPin = 2; //дальномер
const int echoPin = 3; //дальномер
```

```
int exatmojno=0; //признак возможности движения
int k=150; //скорость
```

```
void setup()
{
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
```

```

pinMode(a1, OUTPUT);
pinMode(a2, OUTPUT);
pinMode(enA, OUTPUT);
pinMode(enB, OUTPUT);
pinMode(trigPin, OUTPUT); // пин для Trig
pinMode(echoPin, INPUT); // пин для Echo
Serial.begin(9600); // инициализация послед. порта

//вертим сервомашинку влево-вправо
servo.attach(13); //привязываем привод к пину 13
servo.write(0); //ставим вал под 0
delay(300); //ждем
servo.write(77); //ставим вал по центру
delay(300); //ждем
servo.write(154); //ставим вал под 154
delay(300); //ждем
servo.write(77); //ставим вал под по центру
delay(3000); //ждем

//начальная скорость правого колеса
analogWrite(enA, k);
//начальная скорость левого колеса
analogWrite(enB, k);
}

void loop()
{
    long distance = getDistance();
    //Serial.println(distance);
    //дальномером сканируем пространство впереди
    if (distance >= 30)
    {
        //впереди "чисто", едем вперед
        analogWrite(enA, k);
        analogWrite(enB, k);
        digitalWrite(b1, HIGH);
        digitalWrite(a1, HIGH);
    }
    else
    {
        //впереди что-то есть, стоп
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        //поворачиваем дальномер влево от центра (77)
        //до крайне левого положения (154) через 10 градусов
        for ( int ugol=77; ugol<154; ugol=ugol+10)

```

```

{
    servo.write(ugol);
    delay(400);
    //дальномером сканируем пространство
    //влево от продольной оси робота
    distance = getDistance();
    if ((distance >= 40))
    {
        //влево ехать можно
        exatmojno=1; //установим признак в 1
        break;
    }
    else
    {
        //влево ехать нельзя
        exatmojno=0; //установим признак в 0
        //отъедем назад
        analogWrite(enA, k-50);
        analogWrite(enB, k-50);
        digitalWrite(a2, HIGH);
        digitalWrite(b2, HIGH);
        delay(400);
        digitalWrite(a2, LOW);
        digitalWrite(b2, LOW);
    }
}
//сканирование пространства закончено
servo.write(77); //дальномер – в центр
delay(300);
}
if (exatmojno==1)
{
    //разворачиваем робот влево
    //до появления возможности двигаться
    do
    {
        //ищем свободное направление
        distance = getDistance();
        analogWrite(enA, k-50);
        analogWrite(enB, k-50);
        digitalWrite(a1, HIGH);
        digitalWrite(b1, LOW);
        delay (100);
    }
    while (distance <= 70);
    exatmojno=0;
}

```

```

}

// функция определения дистанции до объекта в см
long getDistance()
{
    long distacne_cm = getEchoTiming() * 1.7 * 0.01;
    return distacne_cm;
}

// функция определения времени задержки
long getEchoTiming()
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // генерируем импульс запуска
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // определение на пине echoPin длительности
    // уровня HIGH в мсек
    long duration = pulseIn(echoPin, HIGH);
    return duration;
}

```

В этом примере при появлении препятствия впереди робота он останавливается и начинает сканировать пространство влево от продольной оси робота. Сканирование осуществляется дальномером путем поворота его с шагом в 10°. При нахождении свободного для движения направления устанавливается в 1 признак `exatmojno` и робот начинает разворачиваться влево. При повороте влево производится сканирование пространства впереди робота. При каждом неудачном сканировании робот немного отъезжает назад.

Если при первичном сканировании робот не находит нужного направления, то это может привести к блокированию его работы. От этого недостатка свободен скетч, приведенный в примере 1.12.

Пример 1.12. Робот едет вперёд и при появлении препятствия стремится отвернуть влево.

Скетч примера (`robot10.ino`) приведен ниже. Здесь приведена только функция `loop()`, которой отличаются примеры 1.11 и 1.12. Отличий от скетча, приведенного в примере 1.11, два:

- перед разворотом робот немного отъезжает назад;
- если робот не находит свободного для движения направления, он немного отъезжает назад и снова сканирует пространство.

Этот скетч работает значительно лучше предыдущего.


```

void loop()
{
    exatmojno=2; //произвольное значение
    long distance = getDistance();
    //Serial.println(distance);
    if (distance >= 30)
    {
        //ехать можно, едем
        analogWrite(enA, k);
        analogWrite(enB, k);
        digitalWrite(b1, HIGH);
        digitalWrite(a1, HIGH);
        digitalWrite(a2, LOW);
        digitalWrite(b2, LOW);
    }
    else
    {
        //ехать нельзя, стоим
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(a2, LOW);
        digitalWrite(b2, LOW);
        //смотрим по сторонам – влево
        for ( int ugol=77; ugol<154; ugol=ugol+10)
        {
            servo.write(ugol);
            delay(400);
            distance = getDistance();
            if ((distance >= 40))
            {
                //влево ехать можно
                exatmojno=1;
                break;
            }
            else
            {
                //влево ехать нельзя
                exatmojno=0;
            }
        }
        servo.write(77); //дальномер – в центр
        delay(300);
    }
    if (exatmojno==1)
    {
        //ехать можно, отъедем чуть назад
        digitalWrite(a1, LOW);
    }
}

```

```

digitalWrite(b1, LOW);
digitalWrite(a2, HIGH);
digitalWrite(b2, HIGH);
delay(400);
digitalWrite(a2, LOW);
digitalWrite(b2, LOW);

do
{
    //разворачиваем робот влево и ищем
    //направление, куда можно ехать
    distance = getDistance();
    delay (100);
    analogWrite(enA, k-40);
    analogWrite(enB, k-40);
    digitalWrite(a1, HIGH);
    digitalWrite(b1, LOW);
    digitalWrite(a2, LOW);
    digitalWrite(b2, LOW);
}
while (distance <= 70);
}
if (exatmojno==0)
{
    //ехать влево нельзя, просто отъезжаем назад
    analogWrite(enA, k-40);
    analogWrite(enB, k-40);
    digitalWrite(a1, LOW);
    digitalWrite(b1, LOW);
    digitalWrite(a2, HIGH);
    digitalWrite(b2, HIGH);
    delay(400);
    digitalWrite(a2, LOW);
    digitalWrite(b2, LOW);
}
}
}

```

Пример 1.13. Робот-танк (рис. 1.19) с помощью пульта RC1683701/01 ездит, стреляет, вращает башней с контролем её положения и автоматически обстреливает всё, что попало в зону видимости. Имеется режим циклического поворота башни влево – вправо, запускаемый кнопкой р<р пульта.

Скетч примера (tank97.ino) приведен ниже.

```

//с помощью пульта RC1683701/01танк 9 ездит,
//стреляет, вертит башней с контролем её положения
//и обстреливает всё, что попало в зону видимости
//имеется режим циклического поворота башни влево – вправо

```

```

#include "IRremote.h"
int receiver = 11;      //ИК-приемник
IRrecv irrecv(receiver);
decode_results results; //переменная для кода из пульта

//пины
// подключите пины контроллера к цифровым пинам Arduino
// правый двигатель
int enA = 5; //правый скорость
int in1 = 10; //правый вперед
int in2 = 7;  //правый назад

// левый двигатель
int enB = 6; //левый скорость
int in3 = 8;  //левый вперед
int in4 = 9;  //левый назад

//пушка
int c1=4;    //выстрел

//башня
int b1 = 3; //башня влево
int b2 = 2; //башня вправо
int d=14;   //герконовый датчик положения башни

//ультразвуковой дальномер
int trigPin = 12;
int echoPin = 13;

//переменные
//датчик башни
int z=0;
//признак режима поворота башни; 0 – выключен, 1 – включен
int pov=0;
// признак поворота башни влево; 0 – выключен, 1 – включен
int levo=0;
//признак поворота башни вправо; 0 – выключен, 1 – включен
int prav=0;
//начальная скорость движения танка (диапазон 0... 255)
int k=150;

void setup()
{
    //настройка пинов на вывод или на ввод
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);

```

```

pinMode(in1, OUTPUT);
pinMode(in2, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);
pinMode(b1, OUTPUT);
pinMode(c1, OUTPUT);
pinMode(b2, OUTPUT);
pinMode(trigPin, OUTPUT); // пин для Trig
pinMode(echoPin, INPUT); // пин для Echo
irrecv.enableIRIn();
pinMode(receiver, INPUT);
pinMode(d, INPUT);
Serial.begin(9600); // инициализация послед. порта

delay(5000); //ждем

}

void loop()
{
    //читаем герконовый датчик башни
    z=digitalRead(d);
    //Serial.println (z);
    //проверяем признак поворота башни
    if (pov==1)
    {
        //башня поворачивается
        //проверяем герконовый датчик башни
        if (z==1)
        {
            // герконовый датчик башни сработал
            // дальше поворачивать в этом направлении нельзя
            if (levo==1)
            {
                // дальше поворачивать влево нельзя
                levo=0; //запрет поворота влево
                prav=1; //разрешение поворота вправо
                //съедем с датчика вправо
                digitalWrite(b1, LOW );
                digitalWrite(b2, HIGH );
                delay(500);
            }
            else
            {
                //дальше поворачивать вправо нельзя
                levo=1; //разрешение поворота влево
            }
        }
    }
}

```

```

        prav=0; //запрет поворота вправо
        //съедем с датчика влево
        digitalWrite(b1, HIGH );
        digitalWrite(b2,LOW );
        delay(500);
    }
}
else
{
    //герконовый датчик не сработал, ничего не делаем
}
}
else
{
    //башня не поворачивается
    if (z==1)
    {
        //датчик башни не сработал
        digitalWrite(b1, LOW );
        digitalWrite(b2,LOW );
    }
}

if (irrecv.decode(&results))
{
    //Serial.println(results.value);
    translateIR(); //вызываем функцию
    irrecv.resume(); // считываем следующий код с пульта
}
//читаем дальномер
long distance = getDistance();
//Serial.println(distance);
if (distance <10 )
{
    //если дистанция <10 см, открываем огонь
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
    //delay(300);
}
}
}

```

```
// основная функция. она анализирует коды,  
//принятые с пульта, и выполняет необходимые действия  
void translateIR()
```

```
{  
    //анализируем код, поступивший с пульта  
    switch(results.value)  
    {  
  
        case 1:  
            //нажата кнопка 1 – едем вперед влево  
            //Serial.println(" 1          ");  
            digitalWrite(in2, LOW);  
            digitalWrite(in1, HIGH);  
            digitalWrite(in3, LOW);  
            digitalWrite(in4, HIGH);  
            analogWrite(enA, k);  
            analogWrite(enB, k);  
            break;  
  
        case 2049:  
            //нажата кнопка 1 – едем вперед влево  
            //Serial.println(" 1          ");  
            digitalWrite(in2, LOW);  
            digitalWrite(in1, HIGH);  
            digitalWrite(in3, LOW);  
            digitalWrite(in4, HIGH);  
            analogWrite(enA, k);  
            analogWrite(enB, k);  
            break;  
  
        case 2:  
            //нажата кнопка 2 – едем прямо  
            //Serial.println(" 2          ");  
            digitalWrite(in1, HIGH);  
            digitalWrite(in2, LOW);  
            digitalWrite(in3, HIGH);  
            digitalWrite(in4, LOW);  
            analogWrite(enA, k);  
            analogWrite(enB, k);  
            break;  
  
        case 2050:  
            //нажата кнопка 2 – едем прямо  
            //Serial.println(" 2          ");  
            digitalWrite(in1, HIGH);  
            digitalWrite(in2, LOW);
```

```
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
analogWrite(enA, k);
analogWrite(enB, k);
break;
```

case 3:

```
//нажата кнопка 3 – едем вправо вперед
//Serial.println(" 3 ");
digitalWrite(in3, HIGH);
digitalWrite(in1, LOW);
digitalWrite(in4, LOW);
digitalWrite(in2, HIGH);
analogWrite(enA, k);
analogWrite(enB, k);
break;
```

case 2051:

```
//нажата кнопка 3 – едем вправо вперед
//Serial.println(" 3 ");
digitalWrite(in3, HIGH);
digitalWrite(in1, LOW);
digitalWrite(in4, LOW);
digitalWrite(in2, HIGH);
analogWrite(enA, k);
analogWrite(enB, k);
break;
```

case 16:

```
//нажата кнопка vol+
//поворачиваем башню влево
//Serial.println(" Башня влево ");
digitalWrite(b1, HIGH);
digitalWrite(b2, LOW);
delay(300);
digitalWrite(b1, LOW);
digitalWrite(b2, LOW);
//проверим геркон башни
z=digitalRead(d);
//Serial.println (z);
if(z==1)
{
    //всё! дальше поворачивать влево нельзя!
    //съедем с датчика вправо
    digitalWrite(b1,LOW );
    digitalWrite(b2, HIGH );
}
```

```

        delay(300);
        digitalWrite(b1, LOW);
        digitalWrite(b2, LOW);
    }

```

```

break;

```

```

case 2064:

```

```

    //здесь всё то же, что и при коде 16
    //Serial.println(" Башня влево ");
    digitalWrite(b1, HIGH);
    digitalWrite(b2, LOW);
    delay(300);
    digitalWrite(b1, LOW);
    digitalWrite(b2, LOW);
    z=digitalRead(d);
    //Serial.println (z);
    if(z==1)
    {
        digitalWrite(b1,LOW );
        digitalWrite(b2, HIGH );
        delay(300);
        digitalWrite(b1, LOW);
        digitalWrite(b2, LOW);
    }

```

```

break;

```

```

case 17:

```

```

    //нажата кнопка vol-
    //поворачиваем башню вправо
    //Serial.println(" Башня вправо ");
    digitalWrite(b2, HIGH);
    digitalWrite(b1, LOW);
    delay(300);
    digitalWrite(b2, LOW);
    digitalWrite(b1, LOW);
    z=digitalRead(d);
    //Serial.println (z);
    if(z==1)
    {
        //всё! дальше поворачивать вправо нельзя!
        //съедем с датчика влево
        digitalWrite(b1, HIGH );
        digitalWrite(b2,LOW );
        delay(300);
        digitalWrite(b1, LOW);
    }

```



```

        digitalWrite(b2, LOW);
    }
    break;

case 2065:
    //здесь всё то же, что и при коде 17
    //Serial.println(" Башня вправо ");
    digitalWrite(b2, HIGH);
    digitalWrite(b1, LOW);
    delay(300);
    digitalWrite(b2, LOW);
    digitalWrite(b1, LOW);
    z=digitalRead(d);
    //Serial.println (z);
    if(z==1)
    {
        digitalWrite(b1, HIGH );
        digitalWrite(b2,LOW );
        delay(300);
        digitalWrite(b1, LOW);
        digitalWrite(b2, LOW);
    }
    break;

case 15:
    //нажата кнопка i+ – выстрел из пушки
    //Serial.println(" Выстрел ");
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
    delay(300);
    break;

case 2063:
    //нажата кнопка i+ – выстрел из пушки
    //Serial.println(" Выстрел ");
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
    delay(300);
    break;

case 5:
    //нажата кнопка 5 – стоп

    //останавливаем всё, обнуляем все режимы
    //Serial.println(" 5 ");

```

```
digitalWrite(in1, LOW);  
digitalWrite(in2, LOW);  
digitalWrite(in3, LOW);  
digitalWrite(in4, LOW);  
digitalWrite(c1, LOW );  
digitalWrite(b1, LOW );  
digitalWrite(b2, LOW );  
pov=0;  
levo=0;  
prav=0;  
break;
```

case 2053:

```
//нажата кнопка 5 – стоп  
//останавливаем всё, обнуляем все режимы  
//Serial.println(" 5 ");  
digitalWrite(in1, LOW);  
digitalWrite(in2, LOW);  
digitalWrite(in3, LOW);  
digitalWrite(in4, LOW);  
digitalWrite(c1, LOW );  
digitalWrite(b1, LOW );  
digitalWrite(b2, LOW );  
pov=0;  
levo=0;  
prav=0;  
break;
```

case 7:

```
//нажата кнопка 7 – влево назад  
//Serial.println(" 7 ");  
digitalWrite(in4, LOW);  
digitalWrite(in1, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in2, HIGH);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

case 2055:

```
//нажата кнопка 7 – влево назад  
//Serial.println(" 7 ");  
digitalWrite(in4, LOW);  
digitalWrite(in1, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in2, HIGH);
```

```
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```
case 8:  
  //нажата кнопка 8 – назад  
  //Serial.println(" 8      ");  
  digitalWrite(in4, HIGH);  
  digitalWrite(in1, LOW);  
  digitalWrite(in3, LOW);  
  digitalWrite(in2, HIGH);  
  digitalWrite(c1, LOW );  
  analogWrite(enA, k);  
  analogWrite(enB, k);  
  break;
```

```
case 2056:  
  //нажата кнопка 8 – назад  
  //Serial.println(" 8      ");  
  digitalWrite(in4, HIGH);  
  digitalWrite(in1, LOW);  
  digitalWrite(in3, LOW);  
  digitalWrite(in2, HIGH);  
  digitalWrite(c1, LOW );  
  analogWrite(enA, k);  
  analogWrite(enB, k);  
  break;
```

```
case 9:  
  //нажата кнопка 9 – назад вправо  
  //Serial.println(" 9      ");  
  digitalWrite(in4, HIGH);  
  digitalWrite(in1, HIGH);  
  digitalWrite(in3, LOW);  
  digitalWrite(in2, LOW);  
  digitalWrite(c1, LOW );  
  analogWrite(enA, k);  
  analogWrite(enB, k);  
  break;
```

```
case 2057:  
  //нажата кнопка 9 – назад вправо  
  //Serial.println(" 9      ");  
  digitalWrite(in4, HIGH);  
  digitalWrite(in1, HIGH);  
  digitalWrite(in3, LOW);
```

```
digitalWrite(in2, LOW);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```
case 4:  
  //нажата кнопка 4 – добавим скорость на 10  
  //Serial.println(" 4 ");  
  if (k<230)  
  {  
      k=k+10;  
  }  
  break;
```

```
case 2052:  
  //нажата кнопка 4 – добавим скорость на 10  
  //Serial.println(" 4 ");  
  if (k<230)  
  {  
      k=k+10;  
  }  
  break;
```

```
case 6:  
  //нажата кнопка 4 – убавим скорость на 10  
  //Serial.println(" 6 ");  
  if (k>40)  
  {  
      k=k-10;  
  }  
  break;
```

```
case 2054:  
  //нажата кнопка 4 – убавим скорость на 10  
  //Serial.println(" 6 ");  
  if (k>40)  
  {  
      k=k-10;  
  }  
  break;
```

```
case 34:  
  //нажата кнопка p<p – циклический порот башни  
  //Serial.println(" Циклический порот башни ");  
  if (pov==0)  
  {
```

```

        pov=1;
        levo=1;
        prav=0;
        digitalWrite(b1, HIGH );
        digitalWrite(b2, LOW );
    }
    else
    {
        pov=0;
        levo=0;
        prav=0;
        digitalWrite(b1, LOW );
        digitalWrite(b2, LOW );
    }
    break;

case 2082:
//нажата кнопка p<p – циклический порот башни
//Serial.println(" Циклический порот башни ");
if (pov==0)
{
    pov=1;
    levo=1;
    prav=0;
    digitalWrite(b1, HIGH );
    digitalWrite(b2, LOW );
}
else
{
    pov=0;
    levo=0;
    prav=0;
    digitalWrite(b1, LOW );
    digitalWrite(b2, LOW );
}
    break;
}
}

// функция определения дистанции до объекта в см
long getDistance()
{
    long distacne_cm = getEchoTiming() * 1.7 * 0.01;
    return distacne_cm;
}

```

```
// функция определения времени задержки
long getEchoTiming()
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // генерируем импульс запуска
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // определение на пине echoPin длительности
    // уровня HIGH в мксек
    long duration = pulseIn(echoPin, HIGH);
    return duration;
}
```

Скетч хорошо закомментирован. Напомним лишь, что пульт RC168370/01 при нажатии какой-либо кнопки сначала выдает один код, затем – второй и так – попеременно. В скетче это учтено. Имеется возможность автоматического обстрела обнаруженных дальномером целей. В этом примере установлена дальность обнаружения в 10 см. Естественно, она может быть увеличена до 255 см.

Можно предусмотреть возможность включения и выключения режима автоматического обстрела целей. Будем для этого использовать кнопки р+ и р- пульта.

Пример 1.14. Робот-танк (рис. 1.19) с помощью пульта RC168370/01 (режим TV) ездит, стреляет, вращает башней с контролем её положения и автоматически обстреливает всё, что попало в зону видимости. Имеется режим циклического поворота башни влево – вправо, запускаемый кнопкой р<р пульта. Имеется возможность включения и выключения режима автоматического обстрела целей с помощью кнопок р+ и р- пульта.

Скетч примера (tank95.ino) мало чем отличается от скетча tank97.ino. Поэтому мы приведем лишь фрагменты этого скетча, которые дополняют скетч tank97.ino.

В раздел объявления переменных в начале скетча поместим переменную bum, которая отвечает за разрешение – 1 или запрет – 0 стрельбы из пушки:

```
int bum = 0; //разрешение – 1 или запрет – 0 стрельбы
В функции loop() проведем "лёгкую модернизацию" кода:
    long distance = getDistance();
    //Serial.println(distance);
    if (distance < 10 )
    {
        if (bum==1)
        {
            // стрелять можно, стреляем
```

```

        digitalWrite(c1, HIGH);
        delay(300);
        digitalWrite(c1, LOW);
    }
    if (bum==0)
    {
        //стрелять нельзя, не стреляем
        digitalWrite(c1, LOW);
    }
}
else
{
    //цели не видно, не стреляем
    digitalWrite(c1, LOW);
}
}

```

В функции translateIR() добавим новые строки кода:

```

case 32:
    //нажата кнопка p+ – разрешить стрельбу
    //Serial.println(" включить стрельбу ");
    bum=1;
    break;

case 2080:
    //нажата кнопка p+ – разрешить стрельбу
    //Serial.println(" включить стрельбу ");
    bum=1;
    break;

case 33:
    //нажата кнопка p- – запретить стрельбу
    //Serial.println(" отключить стрельбу ");
    bum=0;
    break;

case 2081:
    //нажата кнопка p- – запретить стрельбу
    //Serial.println(" отключить стрельбу ");
    bum=0;
    break;

```

Пример 1.15. Робот-танк (рис. 1.19) управляется от ИК-пульта RC1683701/01 (режим TV) и смартфона. Кнопки, используемые в пульте, указаны также в комментариях к скетчу:

- кнопки 1...9 – движение робота вперед-назад, повороты робота, останов движения, управление скоростью движения робота;
- кнопка i+ – стрельба из пушки;
- кнопки +vol, -vol – поворот башни с контролем её положения;
- кнопки +p, -p – поворот башни без контроля её положения;
- кнопка p<p – однократный полный поворот башни с реагированием на пламя;
- кнопка AV – однократный полный поворот башни с поиском цели и автоматическим ее обстрелом.

Смартфон обладает меньшими функциональными возможностями, но значительно большей дальностью работы. Для управления роботом используется приложение Arduino Bluetooth Servo Control, работа с которым рассмотрена в главе 4 книги [7]. Назначение кнопок этого приложения в данном скетче следующее:

- кнопка 0 – движение робота влево;
- кнопка 45 – движение робота прямо;
- кнопка 90 – движение робота вправо;
- кнопка 135 – движение робота назад;
- кнопка 180 – останов робота.

Имеются несколько моментов, которые необходимо отметить.

1. Робот может поворачивать башню влево и вправо. Ранее говорилось, что на башне расположен датчик-геркон, который замыкает свои контакты при попадании их в поле постоянного магнита. Два таких магнита расположены снизу робота: один – по левому борту, второй – по правому. Эти магниты далее в комментариях и тексте называют «концевиками» (жаргон от термина «концевой выключатель»). Таким образом, с помощью концевиков мы можем контролировать конечные положения башни, что соответствует допустимому её повороту в диапазоне углов $-90^{\circ} \dots +90^{\circ}$ относительно продольной оси робота.

2. К традиционным ИК-приёмнику, ультразвуковому дальномеру и геркону добавились:

- датчик пламени, подключенный к пину A2, работающему на ввод информации в цифровом режиме;
- ИК-дальномер, подключенный к пину A1, работающему на ввод информации в цифровом режиме;
- Bluetooth-модуль, позволяющий принимать управляющие сигналы со смартфона.

3. ИК-пульт RC 168370/01 обладает «странный» особенностью вырабатывать на нажатие любой кнопки попеременно два кода, что приводит к необходимости программно обрабатывать оба кода, вместо одного. Это, конечно, увеличивает размеры скетча.

4. В скетче реализованы повороты башни с контролем её положения и без контроля положения. В первом случае поворот башни ограничивается кон-

цевиками, о которых говорилось выше. Во втором случае поворот башни ничем не ограничен. Этот режим используется для настройки работы робота.

5. Датчики подключаются к контроллеру стандартным образом – получают питание от контроллера, а выход датчика соединяется с одним из пинов контроллера. Исключение составляют ультразвуковой дальномер и Bluetooth-модуль, для которых необходимы по 2 пина контроллера.

В самом начале функции loop выполняются действия по приему и анализу информации из буфера Bluetooth-модуля, если, конечно, таковая имеется. Для этих целей используются переменные `v` и `val`. Подробно работа этого фрагмента скетча рассмотрена в главе 4 книги [7]. Если в буфере что-то имеется, то после анализа его содержимого будет вызвана одна из функций: `vlevo`, `vravno`, `pravo`, `nazad`, `stop`. Они управляют движением робота и их названия говорят сами за себя.

Далее анализируется содержимое буфера ИК-приёмника, который хранит код последней нажатой кнопки на ИК-пульте.

Если была нажата кнопка 1 (коды 1 и 2049), то вызывается функция `vlevo`, реализующая поворот робота влево.

Если была нажата кнопка 2 (коды 2 и 2050), то вызывается функция `primo`, реализующая движение робота прямо.

Если была нажата кнопка 3 (коды 3 и 2051), то вызывается функция `vravo`, реализующая поворот робота вправо.

Если была нажата кнопка `+vol` (коды 16 и 2064), то на 0,3 секунды включается поворот башни влево. Затем идет проверка, не сработал ли левый концевик. Если он сработал ($z = 1$), то башня повернута в крайнее левое положение. Включаем на 0,3 секунды поворот башни вправо, чтобы датчик-геркон сошел с концевика. Если этого не сделать, то контроллер перестанет понимать, где право, а где лево и перестанет поворачивать башню.

Если была нажата кнопка `-vol` (коды 17 и 2065), то на 0,3 секунды включается поворот башни вправо. Затем идет проверка, не сработал ли правый концевик. Если он сработал ($z=1$), то башня повернута в крайнее правое положение. Включаем на 0,3 секунды поворот башни влево, чтобы датчик-геркон сошел с концевика. Если этого не сделать, то контроллер перестанет понимать, где право, а где лево и перестанет поворачивать башню.

Если была нажата кнопка `+p` (коды 32 и 2080), то на 0,3 секунды включается поворот башни влево. Проверки состояния концевика не выполняется. Этот режим нужен только для наладки.

Если была нажата кнопка `-p` (коды 33 и 2081), то на 0,3 секунды включается поворот башни вправо. Проверки состояния концевика не выполняется. Этот режим нужен только для наладки.

Если была нажата кнопка `i+` (коды 15 и 2063), то на 0,3 секунды включается двигатель пушки. Этого времени достаточно, что бы пушка выполнила один выстрел.

Если была нажата кнопка 5 (коды 5 и 2053), то выполняется функция `stop`, которая выключает все двигатели робота и обнуляет ряд переменных скетча.

Если была нажата кнопка 7 (коды 7 и 2055), то выполняется поворот робота влево назад.

Если была нажата кнопка 8 (коды 8 и 2056), то вызывается функция *nazad*, которая выполняет движение робота назад.

Если была нажата кнопка 9 (коды 9 и 2057), то выполняется поворот робота вправо назад.

Если была нажата кнопка 4 (коды 4 и 2052), то скорость робота *k*, если она не превышает 230, увеличивается на 10 единиц.

Если была нажата кнопка 6 (коды 6 и 2054), то скорость робота *k*, если она превышает 40, уменьшается на 10 единиц.

Если была нажата кнопка *p* (*p* < *r* (коды 34 и 2082), то выполняются действия по однократному повороту башни от концевика до концевика с поиском пламени. Сначала башня поворачивается влево и одновременно анализируется состояние датчика пламени (переменная *a*). Если датчик обнаружил пламя (*a* < 700), то башня останавливается на 1,5 секунды. Здесь могут быть предусмотрены и другие действия, например обстрел огня снарядами с пламегасящим веществом. Поворот башни влево продолжается до срабатывания левого концевика (переменная *z*). После этого башня съезжает с концевика вправо и начинает свой поворот вправо с анализом состояния датчика пламени (переменная *a*) и реакцией на обнаружение пламени. При срабатывании правого концевика башня немного (1,5 секунды) съезжает с него и останавливается. На этом цикл заканчивается.

Если была нажата кнопка *AV* (коды 56 и 2104), то выполняются действия по однократному повороту башни от концевика до концевика с поиском цели с помощью ультразвукового дальномера (переменная *distance*). Сначала башня поворачивается влево и одновременно анализируется состояние дальномера. Если датчик обнаружил цель (*distance* < 30), то сразу же включается двигатель пушки и она производит один выстрел. Обстрел будет продолжаться, пока дальномер будет обнаруживать цель. Поворот башни влево продолжается до срабатывания левого концевика (переменная *z*). После этого башня съезжает с концевика вправо и начинает свой поворот вправо с анализом состояния дальномера и обстрелом целей. При срабатывании правого концевика башня немного (1,5 секунды) съезжает с него и останавливается. На этом цикл заканчивается.

Скетч примера (tank98e.ino) приведен ниже.

```
//танк 9 управляется пультом RC1683701/01 (режим TV)
//и смартфоном
//+vol – поворот башни влево с контролем
//-vol – поворот башни вправо с контролем
//i+ – выстрел
//+p – поворот башни влево без контроля
//-p – поворот башни вправо без контроля
//P<P – однократный полный поворот башни с реагированием на пламя
//AV – однократный полный поворот башни с реагированием на цель
```

```

#include "IRremote.h"
int receiver = 11;    //ИК-приемник
IRecv irrecv(receiver);
decode_results results; //переменная для кода из ИК-пульта

//пины
// правый двигатель
int enA = 5; //правый скорость
int in1 = 10; //правый вперед
int in2 = 7;  //правый назад

// левый двигатель
int enB = 6; //левый скорость
int in3 = 8;  //левый вперед
int in4 = 9;  //левый назад

//пушка
int c1=4;

//башня
int b1 = 3; //башня влево
int b2 = 2; //башня вправо
int d=14;  //герконовый датчик башни

//ультразвуковой дальномер
int trigPin = 12;
int echoPin = 13;

//датчик пламени
int plamia=A2;

//ИК-дальномер
int dalnomerik=A1;

//переменные
int z=0;// датчик башни
//признак режима поворота башни; 0 – выключен, 1 – включен
int pov=0;
// признак поворота башни влево; 0 – выключен, 1 – включен
int levo=0;
//признак поворота башни вправо; 0 – выключен, 1 – включен
int prav=0;
int k=150; //скорость движения танка( 0... 255)
float a;
int val, v; //переменные для работы со смартфоном

```

```

void setup()
{
    //настройка пинов контроллера
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(b1, OUTPUT);
    pinMode(c1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(trigPin, OUTPUT); // пин для Trig
    pinMode(echoPin, INPUT); // пин для Echo
    irrecv.enableIRIn();
    pinMode(receiver, INPUT); // пин для ИК-приемника
    pinMode(d, INPUT);
    Serial.begin(9600); // скорость передачи данных
    v=0;
    val=0;
    delay(3000); //ждем
}

```

```

void loop()
{
    val=0;
    v=0;
    //проверим буфер блютуз-модуля
    while (Serial.available())
    {
        val = Serial.read();
        Serial.println(val);
        switch (val)
        {
            case 44:
                v = Serial.read();
                switch (v)
                {
                    case 48: //поворот влево
                        vlevo();
                        break;

                    case 52: //вперед
                        priamo();
                        break;

```

```

        case 57: //поворот вправо
            vpravo();
            break;
    }
    case 49:
        v = Serial.read();
        switch (v)
        {
            case 51://назад
                nazad();
                break;

            case 56://стоп
                stop();
                break;
        }
    }
    delay (10);
}

if (irrecv.decode(&results))
{
    //Serial.println(results.value);
    translateIR(); //проверяем буфер ИК-приёмника
    irrecv.resume();
}
//принимаем данные с ультразвукового датчика
long distance = getDistance();
//Serial.println(distance);
if (distance <10 )
{
    //стреляем
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
}
}

//эта функция принимает коды с ИК-пульта
//и выполняет необходимые действия
void translateIR()
{
    //проверим буфер ИК-приемника
    switch(results.value)
    {

```

```

case 1:
//нажата кнопка 1 – вперед влево
vlevo();
break;

case 2049:
//нажата кнопка 1 – вперед влево
vlevo();
break;

case 2:
//нажата кнопка 2 – вперед
primo();
break;

case 2050:
//нажата кнопка 2 – вперед
primo();
break;

case 3:
//нажата кнопка 3 – вперед вправо
vravo();
break;

case 2051:
//нажата кнопка 3 – вперед вправо
vravo();
break;

case 16:
//нажата кнопка +vol – башня влево
digitalWrite(b1, HIGH);
digitalWrite(b2, LOW);
delay(300);
digitalWrite(b1, LOW);
digitalWrite(b2, LOW);
z=digitalRead(d);
if(z==1)
{
//наехали на левый концевик,
//чуть-чуть назад вправо
digitalWrite(b1,LOW );
digitalWrite(b2, HIGH );
delay(300);
digitalWrite(b1, LOW);
digitalWrite(b2, LOW);
}

```

```
    }  
    break;
```

case 2064:

```
//нажата кнопка +vol – башня влево  
digitalWrite(b1, HIGH);  
digitalWrite(b2, LOW);  
delay(300);  
digitalWrite(b1, LOW);  
digitalWrite(b2, LOW);  
z=digitalRead(d);  
if(z==1)  
{  
    //наехали на левый концевик,  
    //чуть-чуть назад вправо  
    digitalWrite(b1,LOW );  
    digitalWrite(b2, HIGH );  
    delay(300);  
    digitalWrite(b1, LOW);  
    digitalWrite(b2, LOW);  
}  
break;
```

case 17:

```
//нажата кнопка -vol – башня вправо  
digitalWrite(b2, HIGH);  
digitalWrite(b1, LOW);  
delay(300);  
digitalWrite(b2, LOW);  
digitalWrite(b1, LOW);  
z=digitalRead(d);  
if(z==1)  
{  
    //наехали на правый концевик,  
    //чуть-чуть назад влево  
    digitalWrite(b1, HIGH );  
    digitalWrite(b2,LOW );  
    delay(300);  
    digitalWrite(b1, LOW);  
    digitalWrite(b2, LOW);  
}  
break;
```

case 2065:

```
//нажата кнопка -vol – башня вправо
```

```

digitalWrite(b2, HIGH);
digitalWrite(b1, LOW);
delay(300);
digitalWrite(b2, LOW);
digitalWrite(b1, LOW);
z=digitalRead(d);
if(z==1)
{
    //наехали на правый концевик,
    //чуть-чуть назад влево
    digitalWrite(b1, HIGH );
    digitalWrite(b2,LOW );
    delay(300);
    digitalWrite(b1, LOW);
    digitalWrite(b2, LOW);
}
break;

```

case 2080:

```

//нажата кнопка +p – башня влево без контроля
digitalWrite(b1, HIGH);
digitalWrite(b2, LOW);
delay(300);
digitalWrite(b1, LOW);
digitalWrite(b2, LOW);
break;

```

case 32:

```

//нажата кнопка +p – башня влево без контроля
digitalWrite(b1, HIGH);
digitalWrite(b2, LOW);
delay(300);
digitalWrite(b1, LOW);
digitalWrite(b2, LOW);
break;

```

case 2081:

```

//нажата кнопка -p – башня вправо без контроля
digitalWrite(b2, HIGH);
digitalWrite(b1, LOW);
delay(300);
digitalWrite(b2, LOW);
digitalWrite(b1, LOW);
break;

```


case 33:

```
//нажата кнопка -р – башня вправо без контроля  
digitalWrite(b2, HIGH);  
digitalWrite(b1, LOW);  
delay(300);  
digitalWrite(b2, LOW);  
digitalWrite(b1, LOW);  
break;
```

case 15:

```
//нажата кнопка i+ – выстрел  
digitalWrite(c1, HIGH);  
delay(300);  
digitalWrite(c1, LOW);  
delay(300);  
break;
```

case 2063:

```
//нажата кнопка i+ – выстрел  
digitalWrite(c1, HIGH);  
delay(300);  
digitalWrite(c1, LOW);  
delay(300);  
break;
```

case 5:

```
//нажата кнопка 5 – останов  
stop();  
break;
```

case 2053:

```
//нажата кнопка 5 – останов  
stop();  
break;
```

case 7:

```
//нажата кнопка 7 – назад влево  
digitalWrite(in4, LOW);  
digitalWrite(in1, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in2, HIGH);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

case 2055:

```
//нажата кнопка 7 – назад влево  
digitalWrite(in4, LOW);  
digitalWrite(in1, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in2, HIGH);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

case 8:

```
//нажата кнопка 7 – назад  
nazad();  
break;
```

case 2056:

```
//нажата кнопка 7 – назад  
nazad();  
break;
```

case 9:

```
//нажата кнопка 9 – назад вправо  
digitalWrite(in4, HIGH);  
digitalWrite(in1, HIGH);  
digitalWrite(in3, LOW);  
digitalWrite(in2, LOW);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

case 2057:

```
//нажата кнопка 9 – назад вправо  
digitalWrite(in4, HIGH);  
digitalWrite(in1, HIGH);  
digitalWrite(in3, LOW);  
digitalWrite(in2, LOW);  
digitalWrite(c1, LOW );  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

case 4:

```
//нажата кнопка 4 – добавит скорости  
if (k<230)  
{
```

```

        k=k+10;
    }
    break;

case 2052:
    //нажата кнопка 4 – добавить скорости
    if (k<230)
    {
        k=k+10;
    }
    break;

case 6:
    //нажата кнопка 6 – убавить скорость
    if (k>40)
    {
        k=k-10;
    }
    break;

case 2054:
    //нажата кнопка 6 – убавить скорость
    if (k>40)
    {
        k=k-10;
    }
    break;

case 34:
    //нажата кнопка p<p – однократный поворот
    // башни с поиском огня датчиком пламени
    do
    {
        //поворачиваем башню влево
        //до срабатывания левого концевика
        z=digitalRead(d);
        digitalWrite(b2, LOW);
        digitalWrite(b1, HIGH);
        a=analogRead(plamia);
        if (a<700)
        {
            //если есть пламя, останавливаем
            //поворот на 1,5 секунды
            digitalWrite(b2, LOW);
            digitalWrite(b1, LOW);
            delay(1500);
        }
    }

```

```

    }
    while(z!=1);
    //съезжаем с левого концевика немного вправо
    digitalWrite(b1, LOW);
    digitalWrite(b2, HIGH);
    delay(500);

    do
    {
        //поворачиваем башню вправо
        //до срабатывания правого концевика
        z=digitalRead(d);
        //Serial.println (z);
        digitalWrite(b1, LOW);
        digitalWrite(b2, HIGH);
        a=analogRead(plamia);
        Serial.println(a);
        if (a<700)
        {
            //если есть пламя, останавливаем
            //поворот на 1,5 секунды
            digitalWrite(b2, LOW);
            digitalWrite(b1, LOW);
            delay(1500);
        }
    }
    while(z!=1);

    //съезжаем с правого концевика немного влево
    digitalWrite(b2, LOW);
    digitalWrite(b1, HIGH);
    delay(1500);
    //останавливаем башню
    digitalWrite(b2, LOW);
    digitalWrite(b1, LOW);
    break;

case 2082:
    //здесь всё то же самое, что и в case 34
    do
    {
        z=digitalRead(d);
        //Serial.println (z);
        digitalWrite(b2, LOW);
        digitalWrite(b1, HIGH);
        a=analogRead(plamia);
        Serial.println(a);
    }

```

```

        if (a<700)
        {
            digitalWrite(b2, LOW);
            digitalWrite(b1, LOW);
            delay(1500);
        }
    }
    while(z!=1);
    digitalWrite(b1, LOW);
    digitalWrite(b2, HIGH);
    delay(500);

do
    {
        z=digitalRead(d);
        //Serial.println (z);
        digitalWrite(b1, LOW);
        digitalWrite(b2, HIGH);
        a=analogRead(plamia);
        Serial.println(a);
        if (a<700)
        {
            digitalWrite(b2, LOW);
            digitalWrite(b1, LOW);
            delay(1500);
        }
    }
    while(z!=1);
    digitalWrite(b2, LOW);
    digitalWrite(b1, HIGH);
    delay(1500);
    digitalWrite(b2, LOW);
    digitalWrite(b1, LOW);
    break;

case 56:
    // однократный поворот башни с
    // поиском цели с помощью дальномера
    do
    {
        //поворачиваем башню влево до
        //срабатывания левого концевика
        long distance = getDistance();
        if (distance <30 )
        {
            //цель обнаружена, стреляем

```

```

        digitalWrite(c1, HIGH);
        delay(300);
        digitalWrite(c1, LOW);
    }
    z=digitalRead(d);
    digitalWrite(b2, LOW);
    digitalWrite(b1, HIGH);
}
while(z!=1);
//наехали на левый концевик,
//чуть сдадим вправо
digitalWrite(b1, LOW);
digitalWrite(b2, HIGH);
delay(500);

do
{
    //поворачиваем башню вправо до
    //срабатывания правого концевика
    long distance = getDistance();
    if (distance <30 )
    {
        //дальномер обнаружил цель,
        //стреляем
        digitalWrite(c1, HIGH);
        delay(300);
        digitalWrite(c1, LOW);

    }
    z=digitalRead(d);
    digitalWrite(b2, HIGH );
    digitalWrite(b1,LOW );
}
while(z!=1);
//наехали на правый концевик
//сдадим немного влево
digitalWrite(b2, LOW);
digitalWrite(b1, HIGH);
delay(1500);
//останов поворота башни
digitalWrite(b2, LOW);
digitalWrite(b1, LOW);
break;

```

case 2104:

```

//здесь всё то же самое, что и в case 56
do

```

```

    {
        long distance = getDistance();
        if (distance <30 )
        {
            digitalWrite(c1, HIGH);
            delay(300);
            digitalWrite(c1, LOW);
        }
        z=digitalRead(d);
        digitalWrite(b2, LOW);
        digitalWrite(b1, HIGH);
    }
    while(z!=1);
    digitalWrite(b1, LOW);
    digitalWrite(b2, HIGH);
    delay(500);

do
{
    long distance = getDistance();
    if (distance <30 )
    {
        digitalWrite(c1, HIGH);
        delay(300);
        digitalWrite(c1, LOW);
    }
    z=digitalRead(d);
    digitalWrite(b2, HIGH );
    digitalWrite(b1,LOW );
}
while(z!=1);
digitalWrite(b2, LOW);
digitalWrite(b1, HIGH);
delay(1500);
digitalWrite(b2, LOW);
digitalWrite(b1, LOW);
break;
    }

}

// функция определения дистанции до объекта в см
long getDistance()
{
    long distacne_cm = getEchoTiming() * 1.7 * 0.01;
    return distacne_cm;
}

```

```
// функция определения времени задержки
long getEchoTiming()
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // генерируем импульс запуска
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // определение на пине echoPin длительности
    // уровня HIGH в мксек
    long duration = pulseIn(echoPin, HIGH);
    return duration;
}
```

```
//поворот робота влево вперед
void vleft()
{
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    analogWrite(enA, k);
    analogWrite(enB, k);
}
```

```
//движение робота вперед
void pforward()
{
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    analogWrite(enA, k);
    analogWrite(enB, k);
}
```

```
//поворот робота вправо вперед
void vright()
{
    digitalWrite(in3, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in4, LOW);
    digitalWrite(in2, HIGH);
    analogWrite(enA, k);
    analogWrite(enB, k);
}
```



```

}

//останов работа
void stop()
{
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, LOW);
    digitalWrite(c1, LOW );
    digitalWrite(b1, LOW );
    digitalWrite(b2, LOW );
    pov=0;
    levo=0;
    prav=0;
}

//движение робота назад
void nazad()
{
    digitalWrite(in4, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(c1, LOW );
    analogWrite(enA, k);
    analogWrite(enB, k);
}

```

1.14.4. Роботы с сервоприводом

В предыдущих разделах мы не использовали сервопривод, однако его можно очень эффективно использовать в механизмах поворота башен в роботах-танках и для поворота различных датчиков без поворота всего робота. Надо сказать, что сервопривод – это весьма экономный вариант организации подвижных частей в роботе, обладающий повышенной точностью позиционирования вала. На рис. 1.21 имеется робот-танк с бортовым номером 2. На борту робота расположены ультразвуковой дальномер и сдвоенный ИК-приёмник. Его башня закреплена на валу мощной сервомашинки, которая обеспечивает поворот башни на углы $0^\circ \dots 180^\circ$ с точностью до 1° . Нет необходимости в использовании датчиков-герконов и концевых магнитов для определения положения башни в пространстве. Электроника сервомашинки сама контролирует поворот вала на заданный угол.

Пример 1.16. Робот-танк (рис. 1.21) перемещается в автоматическом режиме по прямоугольной траектории, в углах прямоугольника останавлива-

ется и просматривает пространство с помощью ультразвукового дальномера. При обнаружении цели, обстреливает её из пушки. Скетч примера (tank2-10.ino) приведен ниже.

```
//робот перемещается и с помощью дальномера
//обнаруживает и обстреливает имеющиеся цели
```

```
//подключаем библиотеку Servo.h
//для работы с сервоприводом
#include <Servo.h>
//объявляем переменную servo типа Servo
Servo servo;
//подключаем библиотеку IRemote.h
//для работы с ИК-приемником
#include "IRremote.h"
int receiver = 13; //ИК-приемник
IRrecv irrecv(receiver);
decode_results results;
```

```
//пины
// правый двигатель
int enA = 5; //правый скорость
int in1 = 11; //правый вперед
int in2 = 7; //правый назад
```

```
// левый двигатель
int enB = 6; //левый скорость
int in3 = 8; //левый вперед
int in4 = 9; //левый назад
```

```
//пушка
int c1=4;
```

```
//дальномер
const int trigPin = 3;
const int echoPin = 2;
```

```
//переменные
int ugol = 77; //угол поворота орудия
int k=100; //скорость (0...255)
long distance; //расстояние до цели
```

```
void setup()
{
    //настройка пинов
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
```

```

pinMode(in1, OUTPUT);
pinMode(in2, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);
pinMode(c1, OUTPUT);
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);
pinMode(receiver, INPUT);

irrecv.enableIRIn();
Serial.begin(9600); //инициализация послед. порта
servo.attach(12); //пин сервопривода 12
//задаем скорости вращения двигателей
analogWrite(enA, k);
analogWrite(enB, k);
servo.write(77); //ставим вал по центру
delay(3000); //ждем
}

```

```

void loop()
{
  for (int ugol = 0; ugol<=144; ugol=ugol+5)
  {
    //поворачиваем башню в диапазоне
    //углов 0°...144° с шагом 5°
    servo.write(ugol);
    delay(400); //ждем
    distance = getDistance();
    //Serial.println(distance);
    if (distance >10 && distance <40)
    {
      // цель обнаружена, выстрел
      digitalWrite(c1, HIGH);
      delay(300);
      digitalWrite(c1, LOW);
    }
  }
  //поворот робота влево
  analogWrite(enA, k);
  analogWrite(enB, k);
  digitalWrite(in3, LOW);
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  digitalWrite(in4, LOW);
  delay(800);
}

```

```

//движение робота прямо
digitalWrite(in3, HIGH );
digitalWrite(in1, HIGH);
digitalWrite(in2, LOW);
digitalWrite(in4, LOW);
delay(1300);
//останов робота
digitalWrite(in3, LOW);
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in4, LOW);
//поворачиваем башню в диапазоне
//углов 144°...0° с шагом – 5°

for (int ugol = 144; ugol>=0; ugol=ugol-5)
{
    servo.write(ugol);
    delay(400);    //ждем
    distance = getDistance();
    //Serial.println(distance);
    if (distance >10 && distance <40)
    {
        //цель обнаружена, выстрел
        digitalWrite(c1, HIGH);
        delay(300);
        digitalWrite(c1, LOW);
    }
}

//поворот робота влево
digitalWrite(in3, LOW);
digitalWrite(in1, HIGH);
digitalWrite(in2, LOW);
digitalWrite(in4, LOW);
delay(800);
//движение робота прямо
digitalWrite(in3, HIGH);
digitalWrite(in1, HIGH);
digitalWrite(in2, LOW);
digitalWrite(in4, LOW);
delay(1300);
//останов робота
digitalWrite(in3, LOW);
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in4, LOW);
}

```

```

// функция определения дистанции до объекта в см
long getDistance()
{
    long distacne_cm = getEchoTiming() * 1.7 * 0.01;
    return distacne_cm;
}

// функция определения времени задержки
long getEchoTiming()
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    // генерируем импульс запуска
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    // определение на пине echoPin длительности
    // уровня HIGH в мксек
    long duration = pulseIn(echoPin, HIGH);
    return duration;
}

```

Дадим некоторые пояснения по этому скетчу.

1. Практика показала, что диапазон углов поворота вала сервомашинки во избежание её поломки лучше ограничить числами $0^{\circ} \dots 144^{\circ}$. Тогда центр этого диапазона будет составлять 77° . Именно с ним нужно совмещать ось пушки при настройке.

2. При останове робота пушка вместе с дальномером поворачивается слева направо. При этом дальномером сканируется пространство.

Далее робот поворачивается влево на 180° и с повернутой пушкой движется вдоль стороны прямоугольника.

Затем робот останавливается и пушка вместе с дальномером поворачивается справа налево. При этом дальномером сканируется пространство.

Далее робот поворачивается влево на 180° и с повернутой пушкой движется вдоль стороны прямоугольника.

Этот процесс продолжается бесконечно.

Имеются ещё ряд скетчей, которые содержатся на компакт-диске к книге [7]. Это tank2-11.ino, tank2-12.ino, tank2-13.ino. Здесь же рассмотрим скетч tank2-14.ino.

Пример 1.17. Робот-танк (рис. 1.21) может управляться в соответствии с двумя алгоритмами, выбираемыми кнопками AV и I–II. Скетч примера (tank2-14.ino) приведен ниже. В нем имеются все необходимые комментарии

```

//Пульт RC168370/01, режим AUX.
//Робот 2 работает в двух режимах:

```

```
// 1. Кнопка AV
// Робот стоит на месте, поворачивает башню, и с помощью
// дальномера ищет и обстреливает подвернувшиеся цели
// 2. Кнопка I-II
// Робот управляется кнопками:
// 1..9 – движение, управление скоростью
// p<p – выстрел
// vol+, -vol – поворот башни
// +p – добавляется, -p – убавляется количество выстрелов
```

```
//подключаем библиотеку Servo.h
//для работы с сервоприводом
#include <Servo.h>
//объявляем переменную servo типа Servo
Servo servo;
#include "IRremote.h"
int receiver = 13; //ИК-приемник
IRrecv irrecv(receiver);
decode_results results;
decode_results results1;
//пины
// правый двигатель
int enA = 5; //правый скорость
int in1 = 11; //правый вперед
int in2 = 7; //правый назад
// левый двигатель
int enB = 6; //левый скорость
int in3 = 8; //левый вперед
int in4 = 9; //левый назад

int c1=4; //пушка

int trigPin = 3;
int echoPin = 2;

//переменные
int ugol = 77; //угол поворота орудия
int povorot = 0; // 1-поворот орудия
int k=100;
long distance;
int regim;
int n=0;
int i=0;

void setup()
{
```

```

pinMode(enA, OUTPUT);
pinMode(enB, OUTPUT);
pinMode(in1, OUTPUT);
pinMode(in2, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);
pinMode(c1, OUTPUT);
pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);

irrecv.enableIRIn();
pinMode(receiver, INPUT);
Serial.begin(9600); // инициализация послед. порта
servo.attach(12); //привязываем привод к пину 12

servo.write(77); //ставим вал по центру
delay(3000);    //ждем

analogWrite(enA, k);
analogWrite(enB,k);
}

void loop()
{
  if (irrecv.decode(&results))
  {
    Serial.println(results.value);
    switch(results.value)
    {

      case 2104:
        regim =1;
        break;
      case 56:
        regim =1;
        break;

      case 1103:
        regim =2;
        break;
      case 3151:
        regim =2;
        break;

    }

    irrecv.resume();
  }
}

```

```

    }
    Serial.println(regim);
    switch (regim)
    {

        case 1:
            reg1();
            break;

        case 2:
            reg2();
            break;

    }

    //delay(1000);
}

void reg1()
{
    n=0;
    for (int ugol = 0; ugol<=144; ugol=ugol+3)
    {
        servo.write(ugol); //поворачиваем локатор на 3 град.
        delay(400);        //ждем
        distance = getDistance();
        Serial.println(distance);
        //Serial.println(n);
        if (distance >10 && distance <70)
        {
            n=n+1;
            if (n>3 && n<7)
            {
                digitalWrite(c1, HIGH);
                delay(300);
                digitalWrite(c1, LOW);
            }
        }
        else
        {
            n=0;
        }
    }

    n=0;
    for (int ugol = 144; ugol>=0; ugol=ugol-3)

```



```

{
    servo.write(ugol); //поворачиваем локатор на 3 град.
    delay(400);        //ждем
    distance = getDistance();
    Serial.println(distance);
    if (distance >10 && distance <70)
    {
        n=n+1;
        if (n>3 && n<7)
        {
            digitalWrite(c1, HIGH);
            delay(300);
            digitalWrite(c1, LOW);
        }
    }
    else
    {
        n=0;
    }
}

void reg2()
{
    if (irrecv.decode(&results1))
    {
        Serial.println(results1.value);
        komanda();
    }
}

void komanda()
{
    switch(results1.value)
    {
        case 1089:
            //Serial.println(" 1      ");
            digitalWrite(in2, LOW);
            digitalWrite(in1, HIGH);
            digitalWrite(in3, LOW);
            digitalWrite(in4, HIGH);
            analogWrite(enA, k);

```

```
analogWrite(enB, k);  
break;
```

```
case 3137:
```

```
//Serial.println(" 1      ");  
digitalWrite(in2, LOW);  
digitalWrite(in1, HIGH);  
digitalWrite(in3, LOW);  
digitalWrite(in4, HIGH);  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```
case 1090:
```

```
//Serial.println(" 2      ");  
  
digitalWrite(in1, HIGH);  
digitalWrite(in2, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in4, LOW);  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```
case 3138:
```

```
//Serial.println(" 2      ");  
  
digitalWrite(in1, HIGH);  
digitalWrite(in2, LOW);  
digitalWrite(in3, HIGH);  
digitalWrite(in4, LOW);  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```
case 1091:
```

```
//Serial.println(" 3      ");  
digitalWrite(in3, HIGH);  
digitalWrite(in1, LOW);  
digitalWrite(in4, LOW);  
digitalWrite(in2, HIGH);  
analogWrite(enA, k);  
analogWrite(enB, k);  
break;
```

```

case 3139:
//Serial.println(" 3          ");
digitalWrite(in3, HIGH);
digitalWrite(in1, LOW);
digitalWrite(in4, LOW);
digitalWrite(in2, HIGH);
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 1040:
//Serial.println(" Башня влево          ");

if (ugol<=150)
{
    ugol=ugol+5;
    servo.write(ugol); //поворот
    Serial.println(ugol);
    //delay(400);
}
else
{
    ugol=ugol-5;
    servo.write(ugol); //поворот
}

break;

case 3088:
//Serial.println(" Башня влево          ");
if (ugol<=150)
{
    ugol=ugol+5;
    servo.write(ugol); //поворот
    Serial.println(ugol);
    //delay(400);
}
else
{
    ugol=ugol-5;
    servo.write(ugol); //поворот
}
break;

```

```

case 1041:
//Serial.println(" Башня вправо ");
if (ugol>=10)
{
    ugol=ugol-5;
    servo.write(ugol); //поворот
}
else
{
    ugol=ugol+5;
    servo.write(ugol); //поворот
}

break;

```

```

case 3089:
//Serial.println(" Башня вправо ");
if (ugol>=10)
{
    ugol=ugol-5;
    servo.write(ugol); //поворот
}
else
{
    ugol=ugol+5;
    servo.write(ugol); //поворот
}

```

```

break;

```

```

case 1099:
//Serial.println(" Выстрел ");
for (int l=1; l<=1; l=l+1)
{
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
    delay(700);
}

break;

```

```

case 3147:
//Serial.println(" Выстрел ");
for (int l=1; l<i; l=l+1)
{
    digitalWrite(c1, HIGH);
    delay(300);
    digitalWrite(c1, LOW);
    delay(700);
}
break;

```

```

case 1093:
//Serial.println(" 5 ");
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, LOW);
digitalWrite(c1, LOW );

```

```

break;

```

```

case 3141:
//Serial.println(" 5 ");
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, LOW);
digitalWrite(c1, LOW );

```

```

break;

```

```

case 1095:
//Serial.println(" 7 ");
digitalWrite(in4, LOW);
digitalWrite(in1, LOW);
digitalWrite(in3, HIGH);
digitalWrite(in2, HIGH);
digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

```

```

case 3143:
//Serial.println(" 7 ");
digitalWrite(in4, LOW);

```

```

digitalWrite(in1, LOW);
digitalWrite(in3, HIGH);
digitalWrite(in2, HIGH);
digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 1096:
//Serial.println(" 8      ");
digitalWrite(in4, HIGH);
digitalWrite(in1, LOW);
digitalWrite(in3, LOW);
digitalWrite(in2, HIGH);
digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 3144:
//Serial.println(" 8      ");
digitalWrite(in4, HIGH);
digitalWrite(in1, LOW);
digitalWrite(in3, LOW);
digitalWrite(in2, HIGH);
digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 1097:
//Serial.println(" 9      ");
digitalWrite(in4, HIGH);
digitalWrite(in1, HIGH);
digitalWrite(in3, LOW);
digitalWrite(in2, LOW);
digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 3145:
//Serial.println(" 9      ");
digitalWrite(in4, HIGH);
digitalWrite(in1, HIGH);
digitalWrite(in3, LOW);
digitalWrite(in2, LOW);

```

```

digitalWrite(c1, LOW );
analogWrite(enA, k);
analogWrite(enB, k);
break;

case 1092:
//Serial.println(" 4      ");
if (k<230)
{
    k=k+10;
}
break;

case 3140:
//Serial.println(" 4      ");
if (k<230)
{
    k=k+10;
}
break;

case 1094:
//Serial.println(" 6      ");
if (k>40)
{
    k=k-10;
}
break;

case 3142:
//Serial.println(" 6      ");
if (k>40)
{
    k=k-10;
}
break;

case 1120:
//Serial.println(" Добавление выстрелов      ");
{
    i=i+1;
}
break;

case 3168:
//Serial.println(" Добавление выстрелов      ");
{

```

```
    i=i+1;
  }
  break;
```

```
case 1121:
  //Serial.println(" Убавление выстрелов ");
  {
    i=i-1;
  }
```

```
  break;
```

```
case 3169:
  //Serial.println(" Убавление выстрелов ");
  {
    i=i-1;
  }
  break;
```

```
}
```

```
}
```

```
// функция определения дистанции до объекта в см
long getDistance()
{
  long distacne_cm = getEchoTiming() * 1.7 * 0.01;
  return distacne_cm;
}
```

```
// функция определения времени задержки
long getEchoTiming()
{
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // генерируем импульс запуска
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  // определение на пине echoPin длительности
  // уровня HIGH в мксек
  long duration = pulseIn(echoPin, HIGH);
  return duration;
}
```


2. GSM-МОДУЛЬ SIM800L В МОБИЛЬНЫХ РОБОТАХ

2.1. Описание модуля, питание, технические характеристики

Модуль GSM/GPRS на чипе SIM800L – это миниатюрный GSM-модем, который можно использовать в различных проектах, таких, как управление роботами, системах передачи данных, охранных системах и многих других. Данный модуль по функционалу не чем не уступает обычному сотовому телефону и с его помощью можно отправлять SMS-сообщения, совершать или принимать телефонные звонки, подключаться к Интернету через GPRS, TCP/IP и многое другое. А также модуль поддерживает четырехдиапазонную сеть GSM/ GPRS.

Что такое GSM?

GSM (аббревиатура от Global System for Mobile Communications) – глобальная система для мобильных коммуникаций. GSM является международным стандартом для мобильных телефонов. Иногда его также называют стандартом 2G, поскольку это сотовая сеть второго поколения. Среди всего прочего, GSM позволяет:

- осуществлять входящие и исходящие голосовые звонки,
- отправлять и принимать текстовые сообщения – SMS (Simple Message System),
- обмениваться данными через GPRS.

Что такое GPRS?

GPRS (аббревиатура от General Packet Radio Service) – это технология пакетной передачи данных, которая может обеспечить скорость передачи в 56...114 кбит в секунду.

Ряд других технологий, таких, как SMS, базируются именно на технологии GPRS. Плата расширения Arduino GSM позволяет также использовать пакетную передачу данных для доступа к сети Интернет.

Роль мобильного оператора

Для того, чтобы можно было работать в мобильной сети, необходимо:

- быть абонентом мобильного оператора на prepaid тарифе или по контракту,
- иметь GSM-совместимое устройство – плату расширения Arduino GSM или мобильный телефон,
- SIM-карту (Subscriber Identity Module).

Оператор мобильной сети предоставляет SIM-карту, на которой может храниться такая информация, как мобильный номер, некоторое количество контактов и SMS-сообщений.

Для того, чтобы можно было выходить через GPRS в Интернет для возможности Arduino работать с веб-страницами, необходимо получить у оператора сети имя точки доступа APN (Access Point Name), а также имя пользователя и пароль.

SIM-карты

Помимо платы расширения GSM и самого Arduino, вам также понадобится SIM-карта. Взаимодействие с провайдером связи осуществляется посредством SIM-карты. Провайдер продает вам SIM-карту и обеспечивает GSM-покрытие сети той области, где вы находитесь, или осуществляет роуминг с той компанией, у которой покрытие сети охватывает ваше месторасположение.

Как правило, с каждой SIM-картой ассоциирован специальный защитный код (PIN-код), состоящий из четырех цифр. Не теряйте этот номер, поскольку он необходим для подключения устройства к мобильной сети. Если же вы потеряли свой PIN-код, то для его восстановления необходимо будет связаться с мобильным оператором. Некоторые SIM-карты автоматически блокируются, если набрать неверный PIN-код несколько раз. Поэтому перед его набором лучше еще раз заглянуть в документацию, поставляющуюся вместе с SIM-картой.

Использование PUK-кода (PIN Unlock Code) позволяет сбросить потерявшийся PIN-код с помощью Arduino или платы расширения GSM. PUK-код также указан в документации к вашей SIM-ке.

Стандартом предусмотрено несколько разных размеров SIM-карт (рис. 2.1).

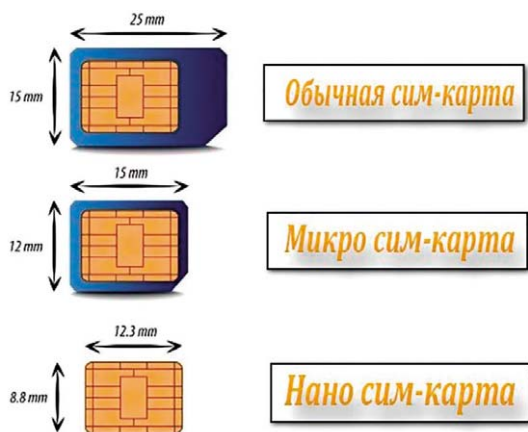


Рис. 2.1

GSM-модуль SIM800L является GSM-модемом. Внешний вид комплекта поставки приведен на рис. 2.2.

Имеется несколько модификаций модуля SIM800. Их характеристики приведены в табл. 2.1. Модуль SIM800L обладает необходимым минимумом для доступа к подавляющему большинству функций, включая голосовые – выводы для подключения микрофона и динамика. Внешний вид модуля со стороны гнезда SIM-карты показан на рис. 2.3. Назначение выводов модуля показано на рис. 2.4. Для начала работы понадобится рабочая SIM-карта формата microSIM.

Питание модуля

Для стабильной работы модуля SIM800L необходим источник питания с выходным напряжением от +3,4 В до +4,4 В (в идеале +4,0 В) с максимальным рабочим током 2 А. В качестве источника питания можно использовать

- литиевый аккумулятор Li-ion (литий-ион) (одна банка);
- литиевого аккумулятора Li-Po (литий-полимер) (одна банка);
- стабилизатор напряжения на LM2596.

Рекомендованное напряжение питания модуля равно +4 В.



Рис. 2.2

Таблица 2.1

Функции	SIM800	SIM800C	SIM800H	SIM800L	SIM800F	SIM868	SIM808
Голосовые вызовы	+	+	+	+	+		
GNSS (GPS)						+	+
Bluetooth 3.0	+	+	+				
Встроенный FM-приёмник: (76-109 МГц) с шагом 50 кГц			+	+			
CSD	+		+	+			+

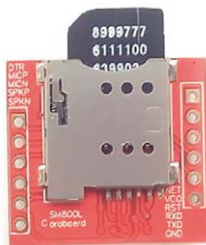


Рис. 2.3

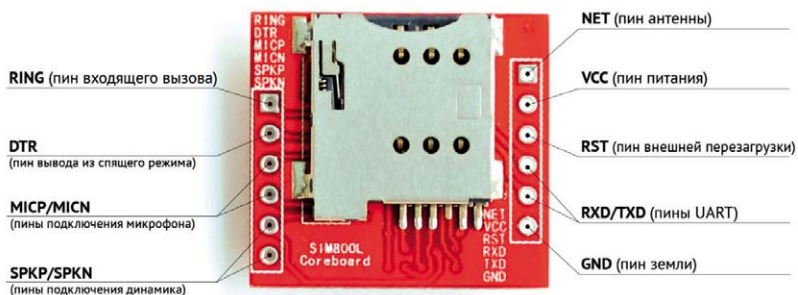


Рис. 2.4

При подаче питания, не соответствующего требуемому, модуль выдает 4 типа сообщений (рис. 2.5):

- если напряжение питания ≤ 3.5 В, модуль выдает предупреждение UNDER-VOLTAGE WARNING;
- если напряжение питания ≥ 4.4 В, модуль выдает предупреждение OVER-VOLTAGE WARNING;
- если напряжение питания ≤ 3.4 В, модуль выдает предупреждение UNDER-VOLTAGE POWER DOWN и выключается;
- если напряжение питания ≥ 4.5 В, выдает предупреждение OVER-VOLTAGE POWER DOWN и выключается.

С температурой дела обстоят так же: при выходе за пороговые значения ($-30...80$ °C) – сначала предупреждение, потом выключение.

Нельзя запитывать модуль SIM800L от платы Arduino.

Технические параметры

- Напряжение питания: +3.7...+4.4 В.
- Потребляемый ток режима ожидания: 0,7 мА.
- Пиковый ток: 2 А.
- Скорость UART: 1200–115200 бод.

- Формат SIM-карты: microSIM.
- Рабочий диапазон: EGSM900, DCS1800, GSM850, PCS1900.
- Мощность передачи DCS1800, PCS1900: 1 Вт.
- Мощность передачи GSM850, EGSM900: 2 Вт.
- Режим сети: 2G.
- Габариты: 25×24×4 мм.

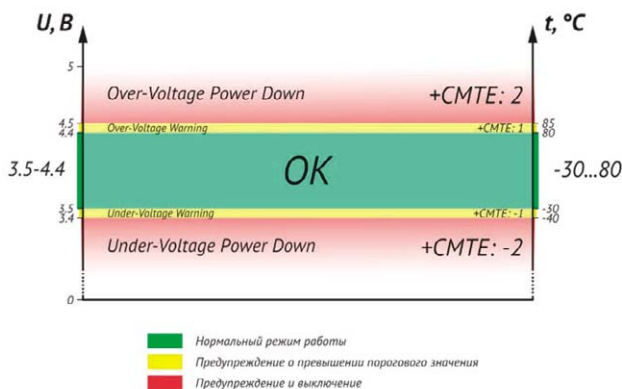


Рис. 2.5

Контакты чипа SIM800L выведены по бокам модуля. Для подключения к сотовой сети нужна антенна, которая идет в комплекте с модулем. Также на плате имеется гнездо для подключения выносной антенны.

На задней панели расположено гнездо для установки SIM-карты. Подойдет любая SIM-карта. Главное, чтобы она была активирована. Устанавливать SIM-карту необходимо так, как показано на рис. 2.3. В верхнем правом углу модуля SIM800L находится светодиод, который показывает состояние сотовой сети.

- Мигает раз в 1 с – модуль работает, но еще не подключился к сотовой сети.
- Мигает раз в 2 с – запрошенное вами соединение для передачи данных GPRS активно.
- Мигает раз в 3 с – модуль установил связь с сотовой сетью и может отправлять / получать голосовые и SMS-сообщения.

2.2. Назначение выводов модуля SIM800L и подключение его к плате Arduino

На модуле SIM800L расположено 12 контактов, которые необходимы для связи его с микроконтроллером и подключения динамика и микрофона (рис. 2.6).



Рис. 2.6

- NET – вывод для припаивания спиральной антенны.
- VCC – питание модуля от +3,4 В до +4,4 В.
- RST (Reset) – вывод сброса модуля.
- RxD (Receiver) – вывод последовательной связи.
- TxD (Transmitter) – вывод последовательной связи.
- GND – вывод заземления, должен быть подключен к выводу GND на Arduino.
- RING – вывод индикатора звонка.
- DTR – вывод активации / деактивации спящего режима.
- MIC ± – выводы для подключения микрофона
- SPK ± – выводы для подключения динамика.

Подключение модуля SIM800L к Arduino

Необходимые детали:

Arduino UNO R3 – 1 шт.

- Макетная плата 400 контактов, breadboard – 1 шт.
- Резисторы 0,125 Вт, 1...10 кОм – 2 шт.
- Модуль GSM, GPRS на чипе SIM800L – 1 шт.
- Провод DuPont, 2,54 мм, 20 см, F-M (Female – Male) – 4 шт.
- Кабель USB 2.0 A-B – 1 шт.

Последовательность действий:

1. Припаять антенну из комплекта или установить выносную внешнюю антенну.
2. Установить SIM-карту в разъем.
3. Подключить вывод Tx модуля к выводу 3 платы Arduino (рис. 2.7).
Можно использовать любой пин.

4. Вывод Rx нельзя подключать напрямую, так как цифровые пины Arduino Uno работают с напряжением +5В. Модуль SIM800L использует +3,3В. Поэтому необходимо сигнал Tx, поступающий от Arduino UNO, понизить до 3,3В. Самый простой способ: воспользоваться делителем напряжения на резисторах (рис. 2.7). Подключаем резистор на 10 кОм между выводом Rx (SIM800L) и выводом 2 (Arduino) и второй резистор на 10 кОм между выводом Rx (SIM800L) и GND. Теперь осталось подключить питание модуля. В примере используется стабилизатор напряжения на LM2596.

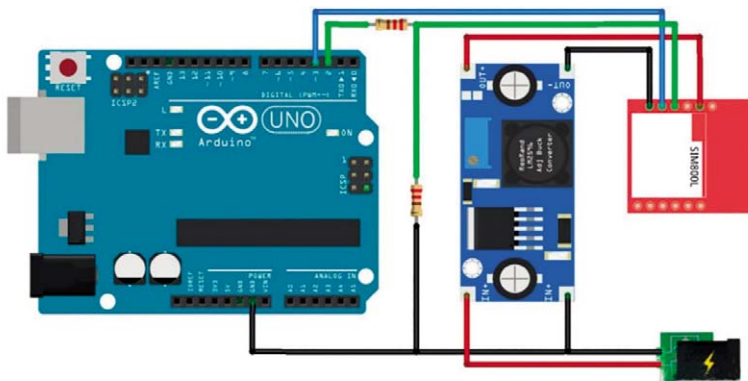


Рис. 2.7

2.3. Описание библиотеки SoftwareSerial

В Arduino реализована аппаратная поддержка интерфейса последовательной передачи данных через выводы 0 и 1, которые также используются для связи с компьютером посредством USB. Аппаратная работа с последовательным интерфейсом осуществляется с помощью встроенного в микроконтроллер специального устройства, называемого приемопередатчиком UART. Он позволяет Arduino обрабатывать поступающие данные даже во время работы над другими задачами.

Библиотека SoftwareSerial позволяет реализовать последовательный интерфейс на любых цифровых выводах Arduino с помощью программных средств, дублирующих функциональность UART. Отсюда и название «SoftwareSerial». Библиотека позволяет программно создавать несколько последовательных портов, работающих на скорости до 115 200 бод. Для устройств, работающих с инвертированным сигналом, в библиотеке предусмотрен соответствующий параметр, включающий инвертирование.

В наших примерах будут постоянно использоваться функции из этой библиотеки.

Ограничения

Среди известных ограничений библиотеки SoftwareSerial можно перечислить следующие:

1. При использовании нескольких последовательных портов в каждый момент времени только один из них может получать данные.
2. На платах Arduino Mega и Mega2560 некоторые выводы не поддерживают прерывания, возникающие при изменении уровня сигнала. В силу этого, на данных платах в качестве вывода RX могут использоваться только следующие выводы: 10, 11, 12, 13, 14, 15, 50, 51, 52, 53, A8 (62), A9 (63), A10 (64), A11 (65), A12 (66), A13 (67), A14 (68), A15 (69).
3. На Arduino Leonardo некоторые выводы не поддерживают прерывания, возникающие при изменении уровня сигнала. Поэтому, на этой плате в качестве вывода RX могут использоваться только следующие выводы: 8, 9, 10, 11, 14 (MISO), 15 (SCK), 16 (MOSI).

Примеры, приведенные в этом разделе, поясняют принцип действия той или иной функции и, естественно, носят демонстрационный характер. Поэтому эти примеры не нумеруются.

Пример

/*

Программа тестирования последовательных портов, создаваемых с помощью библиотеки SoftwareSerial

Данные, получаемые аппаратным портом, отправляются на программный порт. Данные, получаемые программным портом, отправляются на аппаратный порт.

Схема:

RX – цифровой вывод 10 (необходимо соединить с выводом TX другого устройства)

TX – цифровой вывод 11 (необходимо соединить с выводом RX другого устройства)

*/

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial mySerial(10, 11); // RX, TX
```

```
void setup()
```

```
{
```

```
    // Инициализируем последовательный
```

```
    // интерфейс и ждем открытия порта:
```

```
    Serial.begin(57600);
```

```
    while (!Serial)
```

```
{
```

```
    ; // ожидаем подключения к последовательному порту.
```



```

        // Необходимо только для Leonardo
    }
    Serial.println("Amazing!");
    // устанавливаем скорость передачи данных
    // для последовательного порта, созданного
    // библиотекой SoftwareSerial
    mySerial.begin(4800);
    mySerial.println("Privet, mir!");
}

void loop() // выполняется циклически
{
    if (mySerial.available())
        Serial.write(mySerial.read());
    if (Serial.available())
        mySerial.write(Serial.read());
}

```

Функция `SoftwareSerial(rxPin, txPin)`

Функция `SoftwareSerial(rxPin, txPin)` создает новый объект типа `SoftwareSerial` и присваивает его указанной переменной (см. пример ниже). Для начала связи служит функция `SoftwareSerial.begin()`.

Параметры:

- `rxPin`: вывод для приема данных по последовательному интерфейсу.
- `txPin`: вывод для передачи данных по последовательному интерфейсу.

Пример

```

#define rxPin 2
#define txPin 3
// инициализируем новый последовательный порт
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);

```

Функция `available()`

Возвращает количество непрочитанных байт (символов), принятых через программный последовательный порт. Непрочитанные данные накапливаются во входном последовательном буфере.

Синтаксис: `mySerial.available()`

Возвращаемые значения – количество непрочитанных байт.

Пример

```
// подключаем библиотеку SoftwareSerial
//для использования ее функций:
#include <SoftwareSerial.h>

#define rxPin 10
#define txPin 11

// инициализируем новый последовательный порт
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);

void setup()
{
    // задаем режим работы выводов tx, rx:
    pinMode(rxPin, INPUT);
    pinMode(txPin, OUTPUT);
    // устанавливаем скорость передачи
    // данных последовательного порта
    mySerial.begin(9600);
}

void loop()
{
    if (mySerial.available()>0)
    {
        mySerial.read();
    }
}
```

Функция begin(speed)

Задаёт скорость передачи данных (в бодах) последовательного порта. Поддерживаемые значения: 300, 600, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 31250, 38400, 57600 и 115200.

Параметры: speed – скорость передачи данных в бодах (long)

Пример

```
// подключаем библиотеку SoftwareSerial
//для использования ее функций:
#include <SoftwareSerial.h>

#define rxPin 10
#define txPin 11

// инициализируем новый последовательный порт
SoftwareSerial mySerial = SoftwareSerial(rxPin, txPin);
```

```

void setup()
{
    // задаем режим работы выводов tx, rx:
    pinMode(rxPin, INPUT);
    pinMode(txPin, OUTPUT);
    // устанавливаем скорость передачи
    // данных последовательного порта
    mySerial.begin(9600);
}

void loop()
{
    // ...
}

```

Функция `isListening()`

Позволяет проверить, активен ли программный последовательный порт. Функция возвращает `true`, если порт находится в режиме ожидания данных.

Синтаксис: `mySerial.isListening()`

Возвращаемое значение: `boolean`

Пример

```

#include <SoftwareSerial.h>
// программный последовательный порт:
// TX – пин 10, RX пин 11
SoftwareSerial portOne(10,11);

void setup()
{
    // инициализируем аппаратный последовательный порт
    Serial.begin(9600);
    // инициализируем программный последовательный порт
    portOne.begin(9600);
}

void loop()
{
    if (portOne.isListening())
    {
        Serial.println("Порт 1 готов!");
    }
}

```

Функция overflow()

Проверяет входной буфер программного последовательного порта на предмет его переполнения. При вызове этой функции, флаг переполнения буфера сбрасывается. Поэтому при всех последующих вызовах, функция будет возвращать false до тех пор, пока не будет принят (и проигнорирован) очередной байт данных. Входной буфер последовательного порта рассчитан на 64 байта.

Синтаксис: mySerial.overflow()

Возвращаемое значение: boolean

Пример

```
#include <SoftwareSerial.h>
```

```
// программный последовательный порт:
```

```
//TX – пин 10, RX – пин 11
```

```
SoftwareSerial portOne(10,11);
```

```
void setup()
```

```
{  
    // инициализируем аппаратный последовательный порт  
    Serial.begin(9600);  
    // инициализируем программный последовательный порт  
    portOne.begin(9600);  
}
```

```
void loop()
```

```
{  
    if (portOne.overflow())  
    {  
        Serial.println("Программный порт переполнен!");  
    }  
}
```

Функция peek

Возвращает символ, принятый программным последовательным портом через вывод RX. Однако, в отличие от read(), многократный вызов данной функции будет приводить к получению одного и того же значения.

Следует иметь в виду, что в каждый момент времени принимать поступающие данные может только один программный порт. Для выбора принимающего порта (экземпляра класса SoftwareSerial) используйте функцию listen().

Возвращаемое значение: полученный символ, или -1, если такового нет.

Пример

```
SoftwareSerial mySerial(10,11);  
void setup()  
{  
  mySerial.begin(9600);  
}  
  
void loop()  
{  
  char c = mySerial.peek();  
}
```

Функция read

Возвращает символ, принятый программным последовательным портом через вывод RX. Следует иметь в виду, что в каждый момент времени принимать поступающие данные может только один программный порт. Для выбора принимающего порта (экземпляра класса `SoftwareSerial`) используйте функцию `listen()`.

Возвращаемое значение: полученный символ, или -1, если такового нет.

Пример

```
SoftwareSerial mySerial(10,11);  
void setup()  
{  
  mySerial.begin(9600);  
}  
  
void loop()  
{  
  char c = mySerial.read();  
}
```

Функция print(data)

Выводит данные через вывод TX программного последовательного порта. Работа данной функции аналогична функции `Serial.print()`.

Параметры могут варьироваться, смотри описание функции `Serial.print()`.

Возвращаемое значение: `byte`

Функция `print()` возвращает количество отправленных байт. Считывание этого значения не обязательно.

Пример

```
SoftwareSerial serial(10,11);
int analogValue;

void setup()
{
    serial.begin(9600);
}

void loop()
{
    // считываем значение с аналогового входа 0:
    analogValue = analogRead(A0);
    // выводим его в разных форматах:
    // выводим как ASCII-символы в десятичном виде
    serial.print(analogValue);
    // выводим символ табуляции
    serial.print("\t");
    // выводим как ASCII-символы в десятичном виде
    serial.print(analogValue, DEC);
    // выводим символ табуляции
    serial.print("\t");
    // выводим как ASCII-символы в шестнадцатеричном виде
    serial.print(analogValue, HEX);
    // выводим символ табуляции
    serial.print("\t");
    // выводим как ASCII-символы в восьмеричном виде
    serial.print(analogValue, OCT);
    // выводим символ табуляции
    serial.print("\t");
    // выводим как ASCII-символы в двоичном виде
    serial.print(analogValue, BIN);
    // выводим символ табуляции
    serial.print("\t");
    // выводим в необработанном виде, предварительно
    // поделив на 4, т. к. analogRead() возвращает значения
    // в диапазоне от 0 до 1023, а в байте данных может
    // храниться число не больше 255)
    serial.print(analogValue/4, BYTE);
    // выводим символ табуляции
    serial.print("\t");
    // выводим символ перевода строки
    serial.println();
    // задержка 10 миллисекунд перед очередным считыванием:
    delay(10);
}
```

Функция println(data)

Выводит данные через вывод TX программного последовательного порта с последующим символом возврата каретки и перевода строки. Работа данной функции аналогична функции Serial.println().

Параметры могут варьироваться, смотри описание функции Serial.println().

Возвращаемое значение: byte

Функция println() возвращает количество отправленных байт. Считывание этого значения не обязательно.

Пример

```
SoftwareSerial serial(10,11);
```

```
int analogValue;
```

```
void setup()
```

```
{
    serial.begin(9600);
}
```

```
void loop()
```

```
{
    // считываем значение с аналогового входа 0:
    analogValue = analogRead(A0);

    // выводим его в разных форматах:
    // выводим как ASCII-символы в десятичном виде
    serial.print(analogValue);
    serial.print("\t");          // выводим символ табуляции
    // выводим как ASCII-символы в десятичном виде
    serial.print(analogValue, DEC);
    serial.print("\t");          // выводим символ табуляции
    // выводим как ASCII-символы в шестнадцатеричном виде
    serial.print(analogValue, HEX);
    serial.print("\t");          // выводим символ табуляции
    // выводим как ASCII-символы в восьмеричном виде
    serial.print(analogValue, OCT);
    serial.print("\t");          // выводим символ табуляции
    // выводим как ASCII-символы в двоичном виде
    serial.print(analogValue, BIN);
    serial.print("\t");          // выводим символ табуляции
    // выводим в необработанном виде (предварительно
    // поделив на 4, т. к. analogRead() возвращает значения
    // в диапазоне от 0 до 1023, а в байте данных может
    // храниться число не больше 255)
    serial.print(analogValue/4, BYTE);
}
```

```

serial.print("\t");          // выводим символ табуляции
serial.println();           // выводим символ перевода строки
// задержка 10 миллисекунд перед очередным считыванием:
delay(10);
}

```

Функция listen()

Переводит указанный последовательный порт в режим ожидания данных. В каждый момент времени только один программный порт может принимать данные. При этом данные, поступающие другим портам, будут игнорироваться. Если при вызове функции listen() текущий активный порт изменяется на другой, то все принятые ранее данные стираются.

Синтаксис: mySerial.listen()

Параметры: mySerial – имя экземпляра класса SoftwareSerial, который должен принимать данные.

Пример

```
#include <SoftwareSerial.h>
```

```

// первый программный последовательный порт:
// TX – пин 10, RX – пин 11
SoftwareSerial portOne(10, 11);

```

```

// второй программный последовательный порт:
// TX – пин 8, RX – пин 9
SoftwareSerial portTwo(8, 9);

```

```

void setup()
{
    // инициализируем аппаратный последовательный порт
    Serial.begin(9600);
    // инициализируем оба программных порта
    portOne.begin(9600);
    portTwo.begin(9600);
}

```

```

void loop()
{
    portOne.listen();

    if (portOne.isListening())
    {
        Serial.println("Port One is listening!");
    }
    else

```



```

        {
            Serial.println("Port One is not listening!");
        }

    if (portTwo.isListening())
    {
        Serial.println("Port Two is listening!");
    }
    else
    {
        Serial.println("Port Two is not listening!");
    }
}

```

Функция write(data)

Выводит данные в виде последовательности байт через вывод TX программного последовательного порта. Работа данной функции аналогична функции Serial.write().

Возвращаемое значение: byte.

Функция write() возвращает количество записанных байт. Считывание этого значения не обязательно.

Пример

```
SoftwareSerial mySerial(10, 11);
```

```

void setup()
{
    mySerial.begin(9600);
}

void loop()
{
    // отправляем байт данных со значением 45
    mySerial.write(45);
    //отправляем строку "hello", считывая длину этой строки.
    int bytesSent = mySerial.write("hello");
}

```

2.4. АТ-команды для модуля SIM800L

Взаимодействие контроллера и модуля SIM800L организуется с помощью, так называемых, АТ-команд. Их назначение и структура даны в таблицах, приведенных ниже.

Команда	Ответ	Описание
AT+GCAP	+GCAP:+FCLASS,+CGSM OK	Возможности модуля
AT+GMM	SIMCOM_SIM900 OK	Идентификатор модуля
AT+GMR	Revision:1137B09SIM900M64_ST OK	Ревизия модуля и сети
AT+GSN	01322600XXXXXXX OK	IMEI
Команда	Ответ	Описание
AT+COPS?	+COPS: 0,0,»MTS-RUS» OK	Информация об операторе
AT+COPS=?	+COPS: (2,»MTS RUS»,»,»,»25001"), (1,»MOTIV»,»MOTIV»,»25035"), (1,»Utel»,»Utel»,»25039"),, (0,1,4),(0,1,2) OK	Доступные операторы
AT+CPAS	+CPAS: 0 OK	Информация о состоянии модуля 0 – готов к работе 2 – неизвестно 3 – входящий звонок 4 – голосовое соединение
AT+CREG?	+CREG: 0,1 OK	Тип регистрации сети <u>Первый параметр:</u> 0 – нет кода регистрации сети 1 – есть код регистрации сети 2 – есть код регистрации сети + доп. параметры <u>Второй параметр:</u> 0 – не зарегистрирован, поиска сети нет 1 – зарегистрирован, домашняя сеть 2 – не зарегистрирован, идёт поиск новой сети 3 – регистрация отклонена

Команда	Ответ	Описание
		4 – неизвестно 5 – роуминг
AT+CSQ	+CSQ: 17,0 OK	Уровень сигнала: 0–115 дБ и меньше 1–112 дБ 2–110 дБ 30–54 дБ 31–52 дБ 31–52 дБ и сильнее 99 – нет сигнала
AT+CCLK?	+CCLK: «00/01/01,04:21:27+00» OK	Текущая дата и время телефона
AT+CBC	+CBC: 0,95,4134 OK	Монитор напряжения питания модуля <u>Первый параметр:</u> 0 – не заряжается 1 – заряжается 2 – зарядка окончена <u>Второй параметр:</u> 1–100 % – уровень заряда батареи <u>Третий параметр:</u> Напряжение питания модуля (VBAT), мВ
AT+CADC?	+CADC: 1,7 OK	Значение АЦП (до 2,8 В)

Команды настроек вызовов

Команда	Ответ	Описание
AT+CLIP=1	OK	АОН 1 – вкл / 0 – выкл
AT+GSMBUSY=0	OK	Запрет входящих звонков 0 – разрешены 1 – запрещены
ATS0=0	OK	Автоответ 0 – ручной 1-более – автоматический после заданного количества звонков

Команды настроек СМС

Команда	Ответ	Описание
AT+CMGF=1	OK	Текстовый режим 1 – включить 0 – выключить см. примечание
AT+CSCS= «GSM»	OK	Кодировка текстового режима Доступны следующие кодировки: IRA, GSM, UCS2, HEX, PCPP, PCDN, 8859-1 см. примечание
AT+CSCB=0	OK	Приём специальных сообщений 0 – разрешен (по умолчанию) 1 – запрещен

Прочие команды настроек модуля

Команда	Ответ	Описание
ATE0	OK	ЭХО 1 – вкл (по умолчанию) 0 – выкл
ATV1	OK	Формат ответа модуля 0 – только ответ 1 – полный ответ с ЭХО (по умолчанию)
AT+CMEE=0	OK	Информация об ошибках 0 – отключён (по умолчанию) 1 – код ошибки 2 – описание ошибки
AT+CCLK=>13/09/25,13:25:33+05"	OK	Установка часов «yy/mm/dd,hh:mm:ss+zz» Где: год/месяц/дата, часы:минуты:секунды +часовой пояс
AT+CPIN=XXXX		Ввод PIN-кода
ATZ0		Сброс настроек до значений по умолчанию (не до заводских) 0 или 1 – выбор профиля
AT&F		Сброс настроек до заводских

Команда	Ответ	Описание
AT&W	OK	Сохранение настроек для текущего профиля. <u>Параметр</u> 0 или 1 – выбор профиля Параметр указывать сразу за командой (AT&W0)
AT+CPOWD=1	NORMAL POWER DOWN	Выключение модуля 0 – срочное 1 – нормальное
AT+CFUN=1,1		Энергосберегающий режим и перезагрузка <u>Первый параметр:</u> 0 – минимальный функционал 1 – нормальный режим (по умолчанию) 2 – выключения цепей приёма и передачи сигнала <u>Второй параметр:</u> 0 – выполнить без перезагрузки 1 – перезагрузить (доступно только в нормальном режиме, т. е. параметры = 1,1)

Команды для осуществления телефонных звонков

Команда	Ответ	Описание
ATD+380XXXXXXXXX;	OK	Позвонить на номер +380XXXXXXXXX;
	NO DIALTONE BUSY NO CARRIER NO ANSWER	Нет сигнала Если вызов отклонён Повесили трубку Нет ответа
ATDL	OK	Позвонить по последнему исходящему номеру
ATA	OK	Ответить на звонок
ATH0	OK	Повесить трубку/ разорвать соединение
	RING	Входящий звонок
AT+CLIP=1	OK	см. настройки

Команда	Ответ	Описание
	RING +CLIP: «+380XXXXXXXX»,145,»,»,»,0	Входящий звонок с включенным АОН Где: <u>Первый параметр</u> – номер телефона входящего звонка 2 – тип входящего номера 129 – не определен 161 – национальный 145 – интернациональный 177 – сетевой, специальный

Команды для отправки СМС-сообщений

Команда	Ответ	Описание
AT+CMGS= «+380XXXXXXXX» >Test sms.elschemo.ru	> +CMGS: 15 OK	Отправка СМС. Указываем номер получателя в кавычках и отправляем модулю с символом переноса строки (13 в ASCII). После приглашения «>» вводим текст сообщения. Для отправки в конце сообщения отправляем символ SUB (26 в ASCII) или ESC (27) для отмены
AT+CMGF=1 AT+CSCS= «GSM»		Режим и кодировка. см. настройки и примечание
	+CMTI: «SM»,4	Уведомление о приходе СМС. <u>Второй параметр</u> номер пришедшего СМС.
AT+CMGL=»REC UNREAD»	+CMGL: 4,»REC UNREAD»,»+380 XXXXXXXX»,» «,»13/09/24,23:02: 22+24» Test2. OK	Чтение групп СМС. Всего 5 групп: REC UNREAD – входящие непрочитанные REC READ – входящие прочитанные STO UNSENT – пользовательские непрочитанные STO SENT – пользовательские прочитанные ALL – прочитать все сообщения
AT+CMGR=2	+CMGR: «REC READ»,»+380XX XXXXXX»,» «,»13/09/21,11:57: 46+24» Test sms. elschemo.ru OK	Чтение SMS сообщений. Запрос: <u>Первый параметр</u> – номер сообщения. <u>Второй параметр</u> (необязателен): 0 – обычный режим (по умолчанию) 1 – не изменять статус сообщения

Команда	Ответ	Описание
		Ответ: <u>Первый параметр</u> – группа сообщений, см предыдущий пункт. <u>Второй параметр</u> – номер отправителя 3 – дата отправки Далее следует текст сообщения
AT+CMGDA=>DEL SENT»	OK	Удаление групп СМС: DEL READ – прочитанные DEL UNREAD – не прочитанные DEL SENT – отправленные DEL UNSENT – не отправленные DEL INBOX – полученные DEL ALL – всех сообщения
AT+CMGD=4	OK	Удаление СМС. <u>Первый параметр</u> – номер сообщения <u>Второй параметр</u>: 0 – удаление указанного сообщения (по умолчанию) 1 – удаление прочитанных сообщений 2 – удаление прочитанных и отправленных сообщений 3 – удаление прочитанных, отправленных и не отправленных сообщений 4 – удаление всех сообщений
AT+CSCA?	+CSCA: «+380991234567», 145 OK	Возвращает номер сервис центра отправки сообщений

Тоновый набор (DTMF). Тоновые сигналы: 0-9,*,#,A-D

Команда	Ответ	Описание
AT+VTD=3	OK	Длительность тоновых сигналов для AT+VTD. Значение параметра 1...255
AT+VTS=>1,4,#,A,6,7,0"	OK	Отправить последовательность тоновых сигналов (до 20). Длительность задается командой AT+VTS
AT+CLDTMF=7, «1,4,#,A,6,7,0»	OK	Проиграть на модуле (через аудио выход) тоновые сигналы. <u>Первый параметр</u> – длительность 1–100 <u>Второй параметр</u> – строка тоновых сигналов, до 20

Команды USSD. Команды приведены для текстового режима и в GSM кодировке

Команда	Ответ	Описание
AT+CUSD=1,»#100#»	OK +CUSD: 0,»Balance:240,68r «,	USSD запрос <u>Первый параметр</u> – режим обработки операции: 0 – выполнить запрос, ответ проигнорировать 1 – выполнить запрос, вернуть ответ 2 – отменить запрос <u>Второй параметр</u> – запрос в кавычках
ATD#100#;	OK +CUSD: 0,»Balance:280 UAH»,	Упрощенный USSD запрос (работает только при GSM кодировке)

Настройка и установка GPRS-соединения

AT+SAPBR=3,1,«CTYPE»,«GPRS»
 AT+SAPBR=3,1,«APN»,«internet.beeline.ru»
 AT+SAPBR=3,1,«USER»,«beeline»
 AT+SAPBR=3,1,«PWD»,«beeline»
 AT+SAPBR=1,1 – установка GPRS-связи
 AT+SAPBR=2,1 – полученный IP адрес
 +SAPBR: 1,1,«10.229.9.115»
 AT+SAPBR=4,1 – текущие настройки соединения
 AT+SAPBR=0,1 – разорвать GPRS-соединение

2.5. Примеры

Пример 2.1. Тестирование некоторых AT-команд, с помощью которых осуществляется программное взаимодействие модуля с контроллером. Для отправки AT-команд и связи с модулем SIM800L будем использовать монитор порта. В этом примере пин TXD модуля подключен к пину 10 контроллера, а пин RXD подключен через делитель напряжения к пину 11 контроллера. В окне монитора порта установим скорость «19200» и «NL (Новая строка)». Скетч примера (sms2.ino) имеет вид.

```

#include <SoftwareSerial.h>
//Выводы SIM800L TXD и RXD подключены
//к выводам Arduino 10 и 11
SoftwareSerial mySerial(10, 11);
  
```



```

void setup()
{
    // Инициализация монитора порта
    Serial.begin(19200);
    // Инициализация последовательной связи Arduino и SIM800L
    mySerial.begin(19200);
    Serial.println("Initializing..."); // Печать текста
    delay(1000);                      // Пауза 1 с
    // Отправка AT-команд
    mySerial.println("AT");
    updateSerial();
    // Проверка качества сигнала в децибелах,
    // диапазон значений 0-31, 31 – лучший, 5 – допустимый
    mySerial.println("AT+CSQ");
    updateSerial();
    // Чтение информации о SIM-карте
    mySerial.println("AT+CCID");
    updateSerial();
    // Проверка регистрации в сети
    mySerial.println("AT+CREG?");
    updateSerial();
}

void loop()
{
    updateSerial();
}

void updateSerial()
{
    {
        delay(500); // Пауза 500 мс
        while (Serial.available())
        {

            // Переадресация с последовательного порта SIM800L
            //на последовательный порт Arduino IDE
            mySerial.write(Serial.read());
        }
        while(mySerial.available())
        {
            // Переадресация с Arduino IDE на
            //последовательный порт SIM800L
            Serial.write(mySerial.read());
        }
    }
}

```

Результаты работы скетча приведены на рис. 2.8.

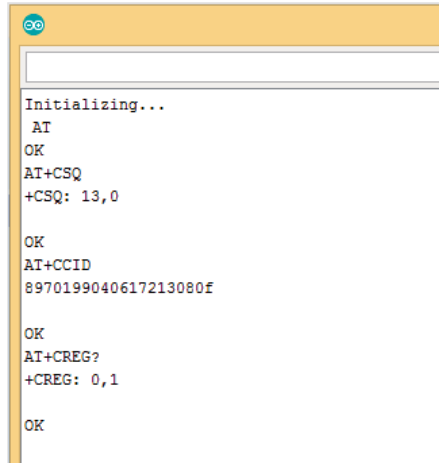


Рис. 2.8

Пример 2.2. Добавим еще несколько команд для тестирования модуля, сети и состояния источника питания. Здесь:

- AT+I – получить название и ревизию модуля и сети;
- AT+COPS? – проверка регистрации в сети (MTS);
- AT+COPS=? – список операторов в сети;
- AT+CBC – команда возвращает состояние батареи.

Для отправки AT-команд и связи с модулем SIM800L будем использовать монитор порта. В этом примере пин TXD модуля подключен к пину 10 контроллера, а пин RXD подключен через делитель напряжения к пину 11 контроллера. В окне монитора порта, установим скорость «19200» и «NL (Новая строка)». Скetch примера (sms3.ino) имеет вид.

```
#include <SoftwareSerial.h>
//Выводы SIM800L TXD и RXD подключены
//к выводам Arduino 10 и 11
SoftwareSerial mySerial(10, 11);

void setup()
{
    // Инициализация монитора порта
    Serial.begin(19200);
    // Инициализация последовательной связи Arduino и SIM800L
    mySerial.begin(19200);
    Serial.println("Initializing..."); // Печать текста
    delay(1000);                       // Пауза 1 с
}
```

```

// Отправка AT-команд
mySerial.println("AT");
updateSerial();

// Проверка качества сигнала,
// диапазон значений 0-31, 31 – лучший, 5 – минимальный
mySerial.println("AT+CSQ");
updateSerial();

// Чтение информации о SIM-карте
mySerial.println("AT+CCID");
updateSerial();

// Проверка регистрации в сети
mySerial.println("AT+CREG?");
updateSerial();
//ATI – получить название модуля и ревизию
//AT+COPS? – проверка регистрации в сети (MTS)
//AT+COPS=? – список операторов в сети.
//AT+CBC – команда возвращает состояние батареи
mySerial.println("ATI");
updateSerial();
mySerial.println("AT+COPS?");
updateSerial();
mySerial.println("AT+CBC");
updateSerial();
mySerial.println("AT+COPS=?");
updateSerial();
}

void loop()
{
    updateSerial();
}

void updateSerial()
{
    delay(500); // Пауза 500 мс
    while (Serial.available())
    {
        //Переадресация с послед-го порта SIM800L
        //на последовательный порт Arduino IDE
        mySerial.write(Serial.read());
    }
    while(mySerial.available())
    {

```

```

// Переадресация с Arduino IDE на
//последовательный порт SIM800L
Serial.write(mySerial.read());
}
}

```

```

Initializing...
AT
OK
AT+CSQ
+CSQ: 10,0

OK
AT+CCID
8970199040617213080f

OK
AT+CREG?
+CREG: 0,1

OK
ATI
SIM800 R14.18

OK
AT+COPS?
+COPS: 0,0,"Bee Line GSM"

OK
AT+CBC
+CBC: 0,65,3922

OK
AT+CGSM=?
+COPS: (2,"Bee Line GSM","BeeLine","25099"),(3,"MOTIV","MOTIV

```

Рис. 2.9

Результаты работы скетча приведены на рис. 2.9. При выполнении команды AT+CBC число «0» указывает, что модуль питается от батареи. Число «65» показывает процент заряда батареи в % (65 %). Число «3922» показывает фактическое напряжением в мВ (3,922 В).

Пример 2.3. Отправка SMS на указанный номер телефона. Прежде чем загрузить пример необходимо ввести номер телефона. Найдите строку +7xxxxxxxxx и замените его на необходимый номер телефона. Скетч примера (sms4.ino) приведен ниже. Некоторые комментарии не показаны, т. к. приводились в предыдущих скетчах. Результат приведен на рис. 2.10.

```

#include <SoftwareSerial.h>
//Выводы SIM800L TXD и RXD подключены
//к выводам платы Arduino 10 и 11
SoftwareSerial mySerial(10, 11);

```

```

void setup()
{
    Serial.begin(19200);
    mySerial.begin(19200);
    Serial.println("Инициализация");    // Печать текста
    delay(1000);                        // Пауза 1 с

    mySerial.println("AT");              // Отправка команды AT
    updateSerial();
    // Выбирает формат SMS – сообщения в виде текста.
    mySerial.println("AT+CMGF=1");
    updateSerial();
    // Отправка СМС на указанный номер
    mySerial.println("AT+CMGS=\"+7xxxxxxxxxx\"");
    updateSerial();
    // Тест сообщения
    mySerial.print("Privet ot deda");
    updateSerial();
    mySerial.write(26);
}

void loop()
{
    updateSerial();
}

void updateSerial()
{
    delay(500);                        // Пауза 500 мс
    while (Serial.available())
    {
        mySerial.write(Serial.read());
    }
    while(mySerial.available())
    {
        Serial.write(mySerial.read());
    }
}

```

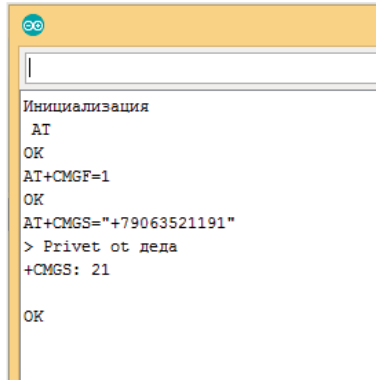


Рис. 2.10

Пример 2.4. Отправка SMS на указанный номер телефона при каждом нажатии кнопки. Кнопка подключается между пином 3 контроллера и землей GND. Прежде чем загрузить пример необходимо ввести номер нужного телефона. Скетч примера (sms1.ino) приведен ниже. Некоторые комментарии не показаны, т. к. приводились в предыдущих скетчах.

```
// Подключить TXD модуля SIM800L к пину платы Arduino
// Подключить RXD модуля SIM800L к пину 11 платы Arduino
// Подключить нормально разомкнутый контакт кнопки
// между пином 3 и землей GND
// При нажатии на этот контакт будет отправлено SMS-сообщение
#include <SoftwareSerial.h>
SoftwareSerial smsSerial(10,11);
#define sense_pin 3
// Замените эти цифры на нужный номер телефона
String number ="9XXXXXXXXX";
```

```
void setup()
{
    Serial.begin(9600);
    smsSerial.begin(9600);
    pinMode(sense_pin,INPUT);
    digitalWrite(sense_pin,HIGH);
}
```

```
void loop()
{
    // Отправлять сообщения всякий раз,
    // когда sense_pin устанавливается в 0
    // Проверка если sense_pin установлен в 0
```

```

if (digitalRead(sense_pin)==LOW)
{
    // Установка текстового режима
    smsSerial.println("AT+CMGF=1");
    delay(150);
    // Укажите номер получателя в
    // международном формате, заменив 7,
    // если это не Россия
    smsSerial.println("AT+CMGS="+7+number+"");
    delay(150);
    // Введите отправляемое сообщение
    smsSerial.print("Privet ot propellera");
    delay(150);
    // Символ окончания сообщения 0x1A
    // эквивалентно Ctrl+z
    smsSerial.write((byte)0x1A);
    delay(50);
    smsSerial.println();
}
}

```

Пример 2.5. Получение SMS. Скetch примера (sms5.ino) приведен ниже. Некоторые комментарии не показаны, т. к. приводились в предыдущих скетчах.

```

#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11);
void setup()
{
    // Настройка скорости монитора порта
    Serial.begin(19200);
    // Настройка скорости порта обмена информацией
    // между SIM800L и контроллером
    mySerial.begin(19200);
    Serial.println("Initializing..."); // Печать текста
    delay(1000);                       // Пауза 1 с
    // Отправка команды AT
    mySerial.println("AT");
    updateSerial();
    // Выбираем формат SMS
    mySerial.println("AT+CMGF=1");
    updateSerial();
    // Обработка вновь поступившие SMS
    mySerial.println("AT+CNMI=1,2,0,0,0");
    updateSerial();
}

```

```

void loop()
{
  updateSerial();
}

void updateSerial()
{
  delay(500);           // Пауза 500 мс
  while (Serial.available())
  {
    //Переадресация с послед-го порта SIM800L
    //на последовательный порт Arduino IDE
    mySerial.write(Serial.read());
  }
  while(mySerial.available())
  {
    // Переадресация с послед-го порта Arduino IDE
    // на последовательный порт SIM800L
    Serial.write(mySerial.read());
  }
}

```

Здесь AT+CNMI=1,2,0,0,0 указывает, как должны обрабатываться вновь поступившие SMS-сообщения. Ответ получим +CMT: все поля разделены запятыми:

- первое поле указывает номер телефона, отправившего SMS;
- второе поле указывает имя человека, отправившего SMS;
- третье поле указывает время получения SMS;
- четвертое поле указывает сообщение.

Пример 2.6. Вызов на указанный номер телефона. Скетч примера (sms6.ino) приведен ниже. Будет осуществлен звонок на заданный номер. Некоторые комментарии не показаны, т. к. приводились в предыдущих скетчах.

```

#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11);
void setup()
{
  Serial.begin(19200);
  mySerial.begin(19200);
  Serial.println("Initializing..."); // Печать текста
  delay(1000);                       // Пауза 1 с
  // Отправка команды AT
  mySerial.println("AT");
  updateSerial();
}

```



```

        // Номер телефона для вызова
        mySerial.println("ATD+ +7xxxxxxxxxx;");
        updateSerial();
        delay(20000);
        // Положить трубку
        mySerial.println("ATH");
        updateSerial();
    }

void loop()
{
    updateSerial();
}

void updateSerial()
{
    delay(500);
    while (Serial.available())
    {
        mySerial.write(Serial.read());
    }
    while(mySerial.available())
    {
        Serial.write(mySerial.read());
    }
}

```

Для осуществления вызова используются следующие AT-команды:

- ATD+ +7xxxxxxxxxx набирает указанный номер.
- ATH – закончить звонок

Пример 2.7. Принятие звонка. Скетч примера (sms2.ino) приведен ниже. Будет осуществлен звонок на заданный номер. Некоторые комментарии не показаны, т. к. приводились в предыдущих скетчах.

```

#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11);
void setup()
{
    Serial.begin(19200);
    mySerial.begin(19200);
    Serial.println("Initializing..."); // Печать текста
    // Отправка команды AT
    mySerial.println("AT");
    updateSerial();
    // Вызов разрешен
}

```

```

        mySerial.println("ATA");
        updateSerial();
        mySerial.println("AT+CLIP=1");
        updateSerial();
    }

void loop()
{
    updateSerial();
}

void updateSerial()
{
    delay(500);
    while (Serial.available())
    {
        mySerial.write(Serial.read());
    }
    while(mySerial.available())
    {
        Serial.write(mySerial.read());
    }
}

```

Входящий вызов обычно обозначается как «RING» на последовательном мониторе, за которым следует номер телефона и идентификатор звонящего. Чтобы принять / повесить звонок, используются следующие AT-команды:

- ATA – принять звонок.
- ATH – отказ от звонка.

Результат работы скетча приведена на рис. 2.11.

Пример 2.8. Управление светодиодом со смартфона. При отправке слова "ON" светодиод включается. При отправке слова "OFF" светодиод выключается. На этот пример имеется ролик. Необходимо обратить внимание, что отправляемое командное слово должно быть в кавычках. Необходимо набирать именно "ON", а не ON, и "OFF", а не OFF. Это касается и других примеров. Команда "OK" нужна для очищения памяти модуля от SMS-сообщений. Схема подключения модуля SIM800L, двух светодиодов и кнопки к контроллеру приведена на рис. 2.12. Слова "ON" и "OFF" можно набирать и в верхнем и в нижнем регистре.

```

Initializing...
AT
OK
ATA
ERROR
AT+CLIP=1
OK

RING

+CLIP: "+79063521191",145,"",0,"",0

RING

+CLIP: "+79063521191",145,"",0,"",0

RING

+CLIP: "+79063521191",145,"",0,"",0

NO CARRIER

```

Рис. 2.11

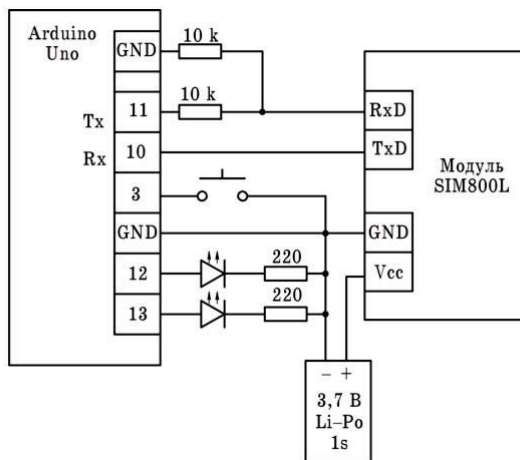


Рис. 2.12

Скетч примера (sms9.ino) приведен ниже.

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10,11 ); // (Rx,Tx > Tx,Rx)
//переменные
//переменная для ввода очередного символа
char incomingByte;
//переменная для хранения принятой строки
String inputString;
// пин для подключения светодиода
int relay = 13;

void setup()
{
    pinMode(relay, OUTPUT);
    //выключаем светодиод
    digitalWrite(relay, LOW);
    //настраиваем порты
    Serial.begin(9600);
    mySerial.begin(9600);

    while(!mySerial.available())
    {
        //пока модуль не готов
        mySerial.println("AT");
        delay(1000);
        Serial.println("Connecting...");
    }
    //модуль готов к обмену
    Serial.println("Connected!");

    //Команды модулю
    // Установка SMS в текстовый режим
    mySerial.println("AT+CMGF=1");
    delay(1000);
    // Процедура обработки вновь полученных сообщений
    mySerial.println("AT+CNMI=1,2,0,0,0");
    delay(1000);
    // Прочитать непрочитанные сообщения
    mySerial.println("AT+CMGL=\"REC UNREAD\"");
}

void loop()
{
    if(mySerial.available())
    {
        //модуль готов к обмену
```

```

delay(100);
// Читаем последовательный буфер
while(mySerial.available())
{
    incomingByte = mySerial.read();
    inputString += incomingByte;
}
delay(10);
//Serial.println(inputString);
//переводим принятую строку к верхнему регистру
inputString.toUpperCase();
// включить или выключить светодиод
if (inputString.indexOf("ON") > -1)
{
    digitalWrite(relay, HIGH);
}
if (inputString.indexOf("OFF") > -1)
{
    digitalWrite(relay, LOW);
}

delay(50);

// Удалить сообщения и сохранить память
if (inputString.indexOf("OK") == -1)
{
    mySerial.println("AT+CMGDA=\"DEL ALL\"");
    delay(1000);
}
//очищаем строку
inputString = "";
}
}

```

Пример 2.9. Управление двумя светодиодами со смартфона (рис. 2.12). При отправке команды "ON1" включается первый светодиод. При отправке команды "OFF1" первый светодиод выключается. Команды "ON2" и "OFF2" управляют вторым светодиодом. Команды "ON" и "OFF" включают и выключают сразу оба светодиода соответственно. На этот пример имеется ролик. Необходимо обратить внимание, что отправляемое командное слово должно быть в кавычках. Необходимо набирать именно "ON1", а не ON1, и "OFF1", а не OFF1. Это касается и других команд. Команда "OK" нужна для очищения памяти модуля от SMS-сообщений. Скетч примера (sms8.ino) приведен ниже.

```

#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11);

```

```

char inc;
String inp;
int n1 = 12; // пин управления светодиодом 1
int n2 = 13; // пин управления светодиодом 2

void setup()
{
    //настройка пинов
    pinMode(n1, OUTPUT); // пин – на вывод
    digitalWrite(n1, LOW); // устанавливаем высокий уровень
    pinMode(n2, OUTPUT); // пин – на вывод
    digitalWrite(n2, LOW); // устанавливаем высокий уровень
    //настройка портов
    Serial.begin(9600);
    mySerial.begin(9600);

    // Ожидаем инициализацию SIM800L
    while(!mySerial.available())
    {
        mySerial.println("AT"); // Отправка команды AT
        delay(1000); // Пауза
        Serial.println("Соединяюсь..."); // Печатаем текст
    }
    Serial.println("Соединился!"); // Печатаем текст
    // Отправка команды AT+CMGF=1
    mySerial.println("AT+CMGF=1");
    delay(1000); // Пауза
    // Отправка команды AT+CNMI=1,2,0,0,0
    mySerial.println("AT+CNMI=1,2,0,0,0");
    delay(1000); // Пауза
    mySerial.println("AT+CMGL=\"REC UNREAD\"");
}

void loop()
{
    if(mySerial.available())
    {
        // Проверяем, если есть доступные данные
        delay(100); // Пауза
        while(mySerial.available())
        {
            // Проверяем, есть ли еще данные
            // Считываем байт и записываем в переменную inc
            inc = mySerial.read();
            // Записываем считанный байт в массив inp
            inp += inc;
        }
    }
}

```

```

delay(10); // Пауза
// Отправка в монитор порта принятые данные
Serial.println(inp);
// Меняем все буквы на заглавные
inp.toUpperCase();

// Проверяем полученные данные
if (inp.indexOf("ON1")>-1)
{
    //Serial.println(inp.indexOf("ON1"));
    // принято "ON1", включаем светодиод 1
    digitalWrite(n1, HIGH);
    // Отправка SMS отправителю
    sms(String("ON1"), String("+79063521191"));
}

// Проверяем полученные данные
if (inp.indexOf("OFF1") > -1)
{
    // принято "OFF1", выключаем светодиод 1
    digitalWrite(n1, LOW);
    // Отправка SMS отправителю
    sms(String("OFF1"), String("+79063521191"));
}

//Проверяем полученные данные
if (inp.indexOf("ON2") > -1)
{
    // принято "ON2", включаем светодиод 2
    digitalWrite(n2, HIGH);
    // Отправка SMS отправителю
    sms(String("ON2"), String("+79063521191"));
}

//Проверяем полученные данные
if (inp.indexOf("OFF2") > -1)
{
    // принято "OFF2", выключаем светодиод 2
    digitalWrite(n2, LOW );
    // Отправка SMS отправителю
    sms(String("OFF2"), String("+79063521191"));
}

if (inp.indexOf("ON") > -1)
{
    // принято "ON", включаем светодиоды 1 и 2
    digitalWrite(n2, HIGH);
    digitalWrite(n1, HIGH);
    // Отправка SMS отправителю
    sms(String("ON"), String("+79063521191"));
}

```

```

//Проверяем полученные данные
if (inp.indexOf("OFF") > -1)
{
    // принято "OFF", выключаем светодиоды 1 и 2
    digitalWrite(n2, LOW );
    digitalWrite(n1, LOW );
    // Отправка SMS отправителю
    sms(String("OFF"), String("+79063521191"));
}
delay(50);
//Проверяем полученные данные
if (inp.indexOf("OK") > -1)
{
    //Принято "OK", стираем все SMS-ки в памяти
    //и обнуляем принятое сообщение
    mySerial.println("AT+CMGDA=\"DEL ALL\"");
    delay(1000);
    inp = "";
}
//обнуляем принятое сообщение
inp = "";
}
}
// функция отправки SMS отправителю
void sms(String text, String phone)
{
    Serial.println("SMS send started");
    mySerial.println("AT+CMGS=\"" + phone + "\"");
    delay(500);
    mySerial.print(text); //отправка текста
    delay(500);
    mySerial.print((char)26); //конец сообщения
    delay(500);
    Serial.println("SMS send complete");
    delay(2000);
}

```

2.6. Работа со строками в скетчах

Как мог уже заметить пытливым читатель, при работе с модулем SIM800L информация между смартфоном и контроллером передается в строковом виде. Но нам для управления роботом информация нужна и в виде символов, и в виде чисел. Символы могут выполнять роль команд, а числа – параметры команд, например, угол поворота башни танка, скорость движения ро-

бота, время останова и т. д. Отсюда и возникает потребность в изучении строк и работы с ними в скетчах.

Arduino String – основная библиотека для работы со строками. С ее помощью существенно упрощается использование массивов символов и строк в скетче. Объект типа String содержит множество полезных функций для создания и объединения строк, преобразований string to int (парсинг чисел) и int to string (форматирование чисел). Строки используются практически в любых проектах, поэтому и вероятность встретить String в скетче очень высока. Здесь мы рассмотрим основные методы этого класса и наиболее часто возникающие ситуации.

Для чего нужен String в C Wiring

Стандартным способом работы со строками в языке C является использование массива символов. Это все означало необходимость работы с указателями и понимания адресной арифметики. В C Wiring и C++ у программистов появилось гораздо больше возможностей. Все «низкоуровневые» операции по работе со строкой выделены в отдельный класс, а для основных операций даже переопределены операторы. Например, для объединения строк мы просто используем хорошо знакомый знак «+», а не зубодробильные функции типа malloc и strcpy. С помощью String мы работаем со строкой как с целым объектом, а не рассматриваем его как массив символов. Это позволяет сосредоточиться на логике скетча, а не деталях реализации хранения символов в памяти (рис. 2.13).

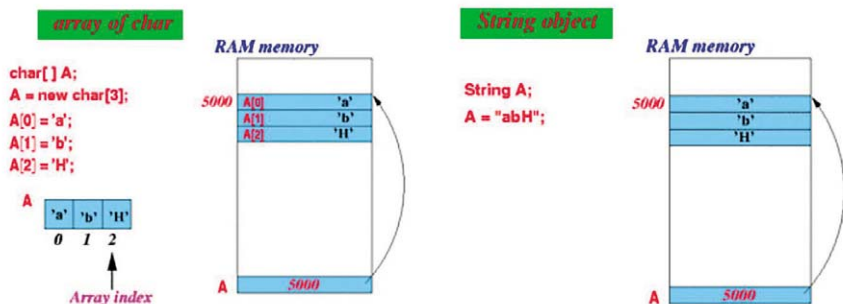


Рис. 2.13

Естественно, у любого упрощения всегда есть свои подводные камни. String всегда использует больше оперативной памяти и в некоторых случаях функции класса могут медленнее обрабатываться. Поэтому в реальных больших проектах придется тщательно взвешивать все плюсы и минусы и не забывать, что никто не мешает нам работать со строками в стиле C. Все обычные функции обработки массивов char остаются в нашем арсенале и в C Wiring.

Создание строк с помощью String

В Arduino у нас есть несколько способов создать строку, приведем основные:

- `char myCharStr [ ] = "Start";` – массив типа `char` с завершающим пустым символом;
- `String myStr = "Start";` – объявляем переменную, создаем экземпляр класса `String` и записываем в него константу-строку;
- `String myStr = String("Start");` – аналогичен предыдущему: создаем строку из константы;
- `String myStr(myCharStr);` – создаем объект класса `String` с помощью конструктора, принимающего на вход массив типа `char` и создающего из `char` `String`;
- `String myStr = String(50);` – создаем строку из целого числа (преобразование `int` to `string`);
- `String myStr = String(30, H);` – создаем строку – представление числа в 16-чной системе (`HEX` to `String`);
- `String myStr = String(16, B);` – создаем строку – представление числа в двоичной системе (`Byte` to `String`).

Каждый раз, когда мы объявляем в коде строку с использованием двойных кавычек, мы создаем неявный объект класса `String`, являющийся константой. При этом обязательно использование именно двойных кавычек: `"String"` – это строка. Одинарные кавычки нужны для обозначения отдельных символов. `'S'` – это символ.

Функции и методы класса String

Для работы со строками в `String` предусмотрено множество полезных функций. Приведем краткое описание каждой из них:

- **`String()`** – конструктор, создает элемент класса данных `string`. Возвращаемого значения нет. Есть множество вариантов, позволяющих создавать `String` из строк, символов, чисел разных форматов.
- **`charAt()`** возвращает указанный в строке элемент. Возвращаемое значение – `n`-ный символ строки.
- **`compareTo()`** – функция нужна для проверки двух строк на равенство и позволяет выявить, какая из них идет раньше по алфавиту. Возвращаемые значения: отрицательное число, если строка 1 идет раньше строки 2 по алфавиту; 0 – при эквивалентности двух строк; положительное число, если вторая строка идет раньше первой в алфавитном порядке.
- **`concat()`** – функция, которая объединяет две строки в одну. Итог сложения строк объединяется в новый объект `String`.
- **`startsWith()`** – функция показывает, начинается ли строка с символа, указанного во второй строке. Возвращаемое значение: `true`, если строка начинается с символа из второй строки, в ином случае `false`.

- **endsWith()** – работает так же, как и **startsWith()**, но проверяет уже окончание строки. Также возвращает значения **true** и **false**.
- **equals()** – сравнивает две строки с учетом регистра, т. е. строки «start» и «START» не будут считаться эквивалентными. Возвращаемые значения: **true** при эквивалентности, **false** в ином случае.
- **equalsIgnoreCase()** – похожа на **equals**, только эта функция не чувствительна к регистру символов.
- **getBytes()** – позволяет скопировать символы указанной строки в буфер.
- **indexOf()** – выполняет поиск символа в строке с начала. Возвращает значение индекса подстроки **val** или **-1**, если подстрока не обнаружена.
- **lastIndexOf()** – выполняет поиск символа в строке с конца.
- **length()** – указывает длину строки в символах без учета завершающего нулевого символа.
- **replace()** – заменяет в строке вхождения определенного символа на другой.
- **setCharAt()** – изменяет нужный символ в строке.
- **substring()** – возвращает подстроку. Может принимать два значения – начальный и конечный индексы. Первый является включительным, т. е. соответствующий ему элемент будет включаться в строку, второй – не является им.
- **toArray()** – копирует элементы строки в буфер.
- **toLowerCase()** – возвращает строку, которая записана в нижнем регистре.
- **toUpperCase()** – возвращает записанную в верхнем регистре строку.
- **toInt()** – позволяет преобразовать строку в число (целое). При наличии в строке не целочисленных значений функция прерывает преобразование.
- **trim()** – отбрасывает ненужные пробелы в начале и в конце строки.

Объединение строк Arduino

Объединить две строки в одну можно различными способами. Эта операция также называется конкатенацией. В ее результате получается новый объект **String**, состоящий из двух соединенных строк. Добавить к строке можно различные символы:

- **String3 = string1 + 111;** – позволяет прибавить к строке числовую константу. Число должно быть целым.
- **String3 = string1 + 111111111;** – добавляет к строке длинное целое число.
- **String3 = string1 + 'A';** – добавляет символ к строке.
- **String3 = string1 + "aaa";** – добавляет строковую постоянную.
- **String3 = string1 + string2;** – объединяет две строки вместе.

Важно осторожно объединять две строки из разных типов данных, так как это может привести к ошибке или неправильному результату.

Преобразование строк в целые и вещественные числа

Для конвертации строк в целые числа используется функция `toInt()`:

```
String MyStr = "111";  
int x = MyStr.toInt();
```

Если нужно конвертировать строку в число с плавающей запятой, применяется функция `atof()`.

```
String MyStr = "11.111";  
char myStr1[10];  
// копируется String в массив myStr1  
MyStr.toCharArray(myStr1, MyStr.length());  
float x = atof(myStr1); // преобразование в float
```

Преобразование целых чисел в строки

Для создания строки из целого числа не требуется делать особых усилий. Мы можем просто объединить строку и число:

```
int i = 50;  
String str = "Строка номер "+ i;  
  
Можем создать объект, используя конструктор:  
String str = String(50);  
Можем объединить оба способа:  
String str = "Строка номер"+ String(50);
```

Объявление строк в виде массива символов

Тип данных `Char` позволяет объявлять текстовые строки несколькими способами:

- `char myStr1[10];` – в данном случае объявлен массив определенного размера.
- `char myStr2 [6] = {'a', 'b', 'c', 'd', 'e'};` – объявлен сам массив. Конечный символ не записан явно, его прибавит сам компилятор.
- `char myStr3[6] = {'a', 'b', 'c', 'd', 'e', '/0'};` – объявлен массив, при этом в конце прописан признак окончания строки.
- `char myStr4 [ ] = "abcde";` – инициализация массива строковой постоянной. Размер и завершающий символ добавляются автоматически компилятором.
- `char myStr5 [6 ] = "abcde";` – инициализация массива с точным указанием его размера.
- `char myStr 6[30 ] = "abcde";` – аналогично, но размер указан больше для возможности использования строк большей длины.

Еще раз напомним, что в типе данных `char` строковые константы нужно записывать в двойные кавычки: `"ABCDE"`, а одиночные символы – в одинарные: `'a'`.

Преобразование строки в массив символов

Это возможно при помощи следующего кода:

```
String stringVar = "111";  
char charBufVar[20];  
stringVar.toCharArray(charBufVar, 20);
```

Преобразование массива символов в строку

Это возможно при помощи следующего кода:

```
char[] chArray = "start";  
String str(chArray);
```

Как показывают примеры, ничего страшного и сложного в классе `String` нет. Более того, часто мы можем даже не догадываться, что работаем с классом `String`. Мы просто создаем переменную нужного типа, присваиваем ей строку в двойных кавычках. Создав строку, мы используем все возможности библиотеки `String`: можем без проблем модифицировать строку, объединять строки, преобразовывать `string` в `int` и обратно, а также делать множество других операций с помощью методов класса.

В ситуациях, когда скетч большой и возникает дефицит памяти, использовать `String` нужно осторожно, по возможности заменяя на `char*`. Впрочем, в большинстве начальных проектах рекомендуется использовать `String` без опаски – это предотвратит появление ошибок адресной арифметики, возникающих при работе с массивами `char`.

2.7. Управление роботом с помощью SMS-сообщений

Сейчас нам необходимо разработать систему команд для управления роботом. Мы знаем, что, кроме набранного строкового сообщения, смартфон пересылает еще и служебную информацию: номер телефона отправителя, имя отправителя, дату и время отправки сообщения. Кроме этого в этих строках находятся и ряд служебных символов. Для того, чтобы наш робот не спутал нужную ему информацию со служебной, будем использовать команды с нижним регистром набора, поскольку служебная информация передается символами с верхним регистром набора. Кроме этого, будем использовать уникальный набор символов, вероятность появления которого в служебной информации равна нулю.

Будем использовать короткие строки на нижнем регистре:

- "VpXXX" – вперед, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "NzXXX" – назад, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "VlXXX" – вперед влево, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "VrXXX" – вперед вправо, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "St" – стоп,
- "NlXXX" – назад влево, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "NrXXX" – назад вправо, XXX – трехзначное целое число в диапазоне 000...999 – время выполнения в секундах.
- "SkXXX" – скорость перемещения робота, XXX – трехзначное целое число в диапазоне 000...255;
- "UgXXX" – угол поворота вала сервомашинки, XXX – трехзначное целое число в диапазоне 000...180.

Пример 2.10. Управление мобильным роботом. На роботе расположен модуль SIM800L, подключенный к пинам 8 и 11 контроллера. Питание модуля производится от отдельного аккумулятора Li-Po (1 банка), земля (минус) которого соединена с землей контроллера. Из датчиков на борту имеется двоянный ИК-датчик пульта, два ИК-датчика препятствия, ультразвуковой дальномер.

На борту также расположены следующие исполнительные механизмы: два двигателя постоянного тока приводов правого и левого колес и сервомашинка, которая используется для размещения на ней одного, либо нескольких датчиков с целью поворота их на углы в диапазоне 0° ... 180° . Обычно на ней размещается ультразвуковой дальномер.

Пины, к которым подключены датчики и исполнительные механизмы легко определить из комментариев к скетчу (sms11.ino). Скетч примера приведен ниже.

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(11, 8); //пины для модуля SIM800L
//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo
#include "IRremote.h"
int receiver = 13; //ик-датчик
IRrecv irrecv(receiver);
decode_results results;
char inc;
```

```

String inp,in;
int a1 = 3; // правое колесо вперёд
int a2 = 2; // правое колесо назад
int b1 = 7; // левое колесо вперёд
int b2 = 4; // левое колесо назад
int enA =5; // правый скорость
int enB =6; // левый скорость
int c2=10; // первый ИК-датчик препятствия
int c3=12; // второй ИК-датчик препятствия
int trigPin = A0; // пин для Trig
int echoPin = A1; // пин для Echo
int k=255; // начальная скорость робота
int s; // скорость движения робота
int z; // время движения робота
int ugol; // угол поворота сервомашинки

void setup()
{
    //настройка пинов
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(c2, INPUT);
    pinMode(c3, INPUT);
    Serial.begin(9600);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    irrecv.enableIRIn();
    pinMode(receiver, INPUT);

    servo.attach(9); //привязываем сервопривод к пину 9
    servo.write(90); //ставим вал по центру

    digitalWrite(b2, LOW);
    digitalWrite(a2, LOW);
    digitalWrite(b1, LOW);
    digitalWrite(a1, LOW);
    //настройка портов
    Serial.begin(9600);
    mySerial.begin(9600);

    // Ожидаем инициализацию SIM800L
    while(!mySerial.available())

```

```

{
    mySerial.println("AT");           // Отправка команды AT
    delay(1000);                      // Пауза
    Serial.println("Connecting...");  // Печатаем текст
}
Serial.println("Connected!");        // Печатаем текст
// Отправка команды AT+CMGF=1
mySerial.println("AT+CMGF=1");
delay(1000);                        // Пауза
// Отправка команды AT+CNMI=1,2,0,0,0
mySerial.println("AT+CNMI=1,2,0,0,0");
delay(1000);                        // Пауза
mySerial.println("AT+CMGL=\"REC UNREAD\"");
}

void loop()
{
    if(mySerial.available())
    {
        // Проверяем, если есть доступные данные
        delay(100);                  // Пауза
        while(mySerial.available())
        {
            // Проверяем, есть ли еще данные
            // Считываем байт и записываем в переменную inc
            inc = mySerial.read();
            // Записываем считанный байт в массив inp
            inp += inc;

        }
        delay(10);                   // Пауза
        // Отправка в монитор порта принятых данных
        //Serial.println(inp);

        // Проверяем полученные данные
        if (inp.indexOf("Vp")>-1)
        {
            // если принято "Vp", то едем вперед
            s=inp.indexOf("Vp"); //индекс символа "V" в строке
            in=inp.substring(s+2,s+5); //выделяем подстроку с цифрами
            z=in.toInt(); //преобразуем подстроку в число
            //Serial.println(z);
            z=z*1000; //секунды превращаем в миллисекунды
            // Отправка SMS
            //sms(String("Vp"), String("+79063521191"));
            vpered(z,k); //вызываем функцию движения вперед
        }
    }
}

```



```

        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\"DEL ALL\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("Vl")>-1)
    {
        // если принято "Vl", то едем вперед влево
        s=inp.indexOf("Vl");
        in=inp.substring(s+2,s+5);
        z=in.toInt();
        //Serial.println(z);
        z=z*1000;
        // Отправка SMS
        //sms(String("Vl"), String("+79063521191"));
        //вызываем функцию движения вперед влево
        vlevo(z,k);
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\"DEL ALL\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("Vr")>-1)
    {
        // если принято "Vr", то едем вперед вправо
        s=inp.indexOf("Vr");
        in=inp.substring(s+2,s+5);
        z=in.toInt();
        Serial.println(z);
        z=z*1000;
        // Отправка SMS
        //sms(String("Vr"), String("+79063521191"));
        //вызываем функцию движения вперед вправо
        vpravo(z,k);
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\"DEL ALL\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("St")>-1)
    {
        // если принято "St", то останов
        s=inp.indexOf("St");
        in=inp.substring(s+2,s+5);
        z=in.toInt();
        //Serial.println(z);
        z=z*1000;
    }

```

```

        // Отправка SMS
        //sms(String("St"), String("+79063521191"));
        //вызываем функцию останова работа
        stope();
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\\"DEL ALL\\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("Nz")>-1)
    {
        // если принято "Nz", то назад
        s=inp.indexOf("Nz");
        in=inp.substring(s+2,s+5);
        z=in.toInt();
        //Serial.println(z);
        z=z*1000;
        // Отправка SMS
        //sms(String("Nz"), String("+79063521191"));
        //вызываем функцию движения назад
        nazad(z,k);
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\\"DEL ALL\\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("NI")>-1)
    {
        // если принято "NI", то назад влево
        s=inp.indexOf("NI");
        in=inp.substring(s+2,s+5);
        z=in.toInt();
        //Serial.println(z);
        z=z*1000;
        // Отправка SMS
        //sms(String("NI"), String("+79063521191"));
        //вызываем функцию движения назад влево
        nazlev(z,k);
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\\"DEL ALL\\"");
    }

    // Проверяем полученные данные
    if (inp.indexOf("Nr")>-1)
    {
        // если принято "Nr", то назад вправо
        s=inp.indexOf("Nr");

```

```

in=inp.substring(s+2,s+5);
z=in.toInt();
//Serial.println(z);
z=z*1000;
// Отправка SMS
//sms(String("Nr"), String("+79063521191"));
//вызываем функцию движения назад вправо
nazprav(z,k);
//очищаем память модуля от SMS-ок
mySerial.println("AT+CMGDA=\"DEL ALL\"");
}

// Проверяем полученные данные
if (inp.indexOf("Sk")>-1)
{
    // если принято "Sk", то меняем скорость
    s=inp.indexOf("Sk");
    in=inp.substring(s+2,s+5);
    z=in.toInt();
    //Serial.println(z);
    k=z;
    // Отправка SMS
    //sms(String("Sk"), String("+79063521191"));
    //очищаем память модуля от SMS-ок
    mySerial.println("AT+CMGDA=\"DEL ALL\"");
}

// Проверяем полученные данные
if (inp.indexOf("Ug")>-1)
{
    // если принято "Ug", то меняем угол поворота
    // вала сервомашинки
    s=inp.indexOf("Ug");
    in=inp.substring(s+2,s+5);
    z=in.toInt();
    //Serial.println(z);
    servo.write(z); //ставим вал на заданный угол
    //Отправка SMS
    //sms(String("Ug"), String("+79063521191"));
    //очищаем память модуля от SMS-ок
    mySerial.println("AT+CMGDA=\"DEL ALL\"");
}

delay(50);
// Проверяем полученные данные
if (inp.indexOf("OK") > -1)

```

```

    {
        //очищаем память модуля от SMS-ок
        mySerial.println("AT+CMGDA=\"DEL ALL\"");
        delay(1000);
        inp = "";
    }
    delay(1000);
    inp = "";
}
}

```

// функция, реализующая движение робота вперед;
 // параметр x – время выполнения в секундах (000...999),
 // параметр y – скорость движения (0...255)
 void vpered(int x, int y)

```

{
    analogWrite(enA, y);
    analogWrite(enB, y );
    digitalWrite(b1, HIGH );
    digitalWrite(a1, HIGH );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);
    delay(x);
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);

}

```

// функция, реализующая движение робота вперед влево;
 // параметр x – время выполнения в секундах (000...999),
 // параметр y – скорость движения (0...255)
 void vlevo(int x, int y)

```

{
    analogWrite(enA, y);
    analogWrite(enB, y );
    digitalWrite(b1, LOW );
    digitalWrite(a1, HIGH );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);
    delay(x);
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);
}

```

```

}

// функция, реализующая движение робота вперед вправо;
// параметр x – время выполнения в секундах (000...999),
// параметр y – скорость движения (0...255)
void vpravo(int x, int y)
{
    analogWrite(enA, y);
    analogWrite(enB, y );
    digitalWrite(b1, HIGH );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);
    delay(x);
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);

}

// функция, реализующая останов робота;
void stop()
{
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
    digitalWrite(a2, LOW);

}

// функция, реализующая движение робота назад;
// параметр x – время выполнения в секундах (000...999),
// параметр y – скорость движения (0...255)
void nazad(int x, int y)
{
    analogWrite(enA, y);
    analogWrite(enB, y );
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,HIGH );
    digitalWrite(a2, HIGH);
    delay(x);
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW );
    digitalWrite(b2,LOW );
}

```

```

        digitalWrite(a2, LOW);

    }

    // функция, реализующая движение робота назад влево;
    // параметр x – время выполнения в секундах (000...999),
    // параметр y – скорость движения (0...255)
    void nazlev(int x, int y)
    {
        analogWrite(enA, y);
        analogWrite(enB, y );
        digitalWrite(b1, LOW );
        digitalWrite(a1, LOW );
        digitalWrite(b2,LOW );
        digitalWrite(a2, HIGH);
        delay(x);
        digitalWrite(b1, LOW );
        digitalWrite(a1, LOW );
        digitalWrite(b2,LOW );
        digitalWrite(a2, LOW);

    }

    // функция, реализующая движение робота назад вправо;
    // параметр x – время выполнения в секундах (000...999),
    // параметр y – скорость движения (0...255)
    void nazprav(int x, int y)
    {
        analogWrite(enA, y);
        analogWrite(enB, y );
        digitalWrite(b1, LOW );
        digitalWrite(a1, LOW );
        digitalWrite(b2,HIGH );
        digitalWrite(a2, LOW);
        delay(x);
        digitalWrite(b1, LOW );
        digitalWrite(a1, LOW );
        digitalWrite(b2,LOW );
        digitalWrite(a2, LOW);

    }

    // функция отправки SMS
    void sms(String text, String phone)
    {
        Serial.println("SMS send started");
        mySerial.println("AT+CMGS=\"" + phone + "\"");
    }

```

```

delay(500);
mySerial.print(text);
delay(500);
mySerial.print((char)26);
delay(500);
Serial.println("SMS send complete");
delay(2000);
}

```

Рассмотрим этот скетч подробнее. Поскольку в работе используется сервомашинка и мы собираемся ею управлять (поворачивать на заданный угол), то мы подключаем библиотеку Servo.h. В нашем примере используются два ДПТ (колёса), одна сервомашинка и модуль SIM800L для приема SMS-сообщений. Контроллер выполняет следующие действия:

- принимает сообщение символ за символом и формирует строку сообщения;
- выделяет из принятой строки нужную подстроку – команду;
- выполняет эту команду;
- при необходимости отправляет SMS-сообщение отправителю.

В функции setup() помимо многого другого мы привязываем сервомашинку к пину 9. Реально сервомашинка должна быть подключена именно к пину 9. Далее с помощью AT-команд идет настройка модуля SIM800L для нормального приема SMS-сообщений. О назначении каждой из этих команд говорилось выше.

В функции loop() выполняются следующие действия.

Если пришло SMS-сообщение, то в буфере модуля SIM800L появляются данные. В цикле while(myserial.available()) мы считываем символ за символом эти данные в строковую переменную inp и отправляем её в монитор порта для визуализации. Вся визуализация нужна только при отладке скетча, поэтому оператор визуализации Serial.println(inp) закомментирован. И все подобные операторы в скетче также закомментированы с помощью двойного слэша //.

Далее выполняется поиск в принятой строке подстроки "Vp". Если таковая имеется, то находится и записывается в переменную s индекс (адрес) символа 'V': оператор s=inp.indexOf("Vp"). Далее в переменную in записываем часть сообщения, содержащую цифровую информацию: оператор in=inp.Substring(s+2,s+5). Поскольку цифры записаны в виде строки, то её необходимо преобразовать в целое десятичное число: оператор z=in.toInt(). Далее число секунд превращаем в миллисекунды: оператор z=z*1000. Затем вызывается функция vpered(z,k) с двумя параметрами z и k. В параметре z находится время перемещения робота в миллисекундах, а в параметре k – текущая скорость перемещения. Скорость перемещения робота может изменяться в пределах от 000 до 255 единиц.

Функция vpered(z,k) организует включение обоих моторов по направлению "вперед" в течении z секунд со скоростью k единиц. Вызовом функции

`sms(String("Vp"),String("+79063521191"))` робот отправляет сообщение с содержанием "Vp" на телефон +79063521191. Понятно, что и SMS-сообщение и номер телефона могут быть произвольными. Поскольку отправка SMS-сообщения от робота требует финансовых затрат в соответствии с тарифным планом SIM-карты робота, то мы используем эту функцию только при отладке скетча. Кроме того, баланс SIM-карты робота довольно трудно проверить.

При поступлении нескольких SMS-сообщений память модуля SIM800L может быть переполнена, поэтому мы её очищаем, отправив в модуль соответствующую AT-команду: оператор `mySerial.println("AT+CMGDA=\"DEL ALL\"")`. Аналогичным образом выполняются все остальные команды, реализующие перемещение и останов робота.

Отличия имею лишь команды "Sk", "UG" и "OK". Команда "Sk" меняет скорость движения робота. При этом числовой параметр из SMS-сообщения напрямую передается в переменную `k`. Команда "UG" меняет угол поворота вала сервомашинки. Принятое в SMS-сообщении число используется в операторе `servo.write(z)`, который устанавливает вал сервомашинки на заданный угол. Каких-либо функций эти команды, как и команда "OK", не вызывают. Команда "OK" предназначена для освобождения памяти модуля SIM800L от SMS-ок.

И еще один важный момент. После выполнения действий, предписанных командой, содержащейся в SMS-сообщении, строка, содержащая текст "отработанного" сообщения" обнуляется – стирается. Это делается с целью предотвращения многократной реакции скетча на одно SMS-сообщение.

3. УПРАВЛЕНИЕ МОБИЛЬНЫМИ РОБОТАМИ ПО BLUETOOTH

https://t.me/it_boooks/2

3.1. Введение

В книге «Контроллеры Arduino в мобильных роботах» управляемый робот оснащался Bluetooth-модулем HC-05 или HC-06, работающем в режиме slave – ведомый. Через этот модуль осуществлялся прием информации, передаваемой со смартфона (рис. 3.1).

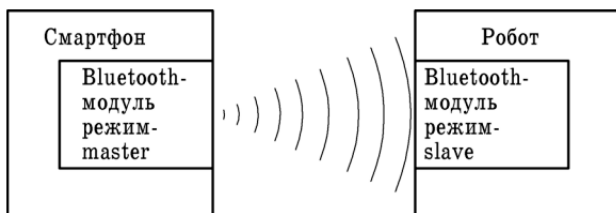


Рис. 3.1

Передача информации из смартфона происходит через внутренний Bluetooth-модуль. Этот модуль работает в режиме master – ведущий. Так что здесь не так? Что не нравится? Давайте разбираться.

1. Практика показала, что не каждый смартфон обнаруживает все Bluetooth-устройства, находящиеся в его радиусе действия. Так, например, старенькая NOKIA обнаруживает практически всё. А новенькие «навороченные» айфоны многих Bluetooth-устройств не обнаруживают. Следовательно, для задач управления они не подходят. Какова причина этого? Не известно. Ведь характеристики Bluetooth-модулей в айфонах, смартфонах и способы их настройки для нас неведомы.

3. Для управления роботом со смартфона необходимо написать мобильное приложение и установить его на смартфон. Всё очень просто, если не считать процесса написания этого самого приложения. Нужно осваивать технологию написания мобильных приложений для смартфонов. А это очень многотрудный процесс, который требует напряженного труда в качестве программиста – разработчика мобильных приложений. Вы готовы? Тогда, вперед! Когда вы научитесь проектировать эти приложения, то вы уже забудете, для чего вам это было нужно.

Как мы вышли из этого положения в книге «Контроллеры Arduino в мобильных роботах»? Мы нашли в магазине GOOGLE PLAY приложение

Arduino Bluetooth Servo Control, изначально предназначенное для управления серво-машинками. Это приложение содержит пять сенсорных кнопок. Щелчок по каждой из этих кнопок вызывает отправление определенного числового кода в выбранное Bluetooth-устройство – модуль на роботе. Мы аккуратно приняли все эти коды и разработали систему, как их дешифровать. После этого и был разработан скетч, реагирующий на эти коды и реализующий выполнение роботом определенных команд: движения прямо, назад, влево, вправо, останов. Больше этих пяти команд робот выполнить не в состоянии. А если необходимо передать 20, 30, 40 или более команд? Как это сделать? Вряд ли найдется в Интернете подходящее приложение.

4. Управлять такими мобильными объектами, как мобильные роботы, со смартфона с помощью сенсорных кнопок крайне неудобно. Оператор должен смотреть на робот и одновременно на экран, готовясь щелкнуть на нужную кнопку. А эту кнопку еще нужно мгновенно найти и сразу попасть в нее пальцем и она должна сразу сработать. Это практически нереально. В управлении будут происходить задержки, чреватые авариями и катастрофами. Поэтому такой способ можно применять лишь в проектах, не требующих быстрой реакции оператора на то или иное событие.

5. А как передать по Bluetooth на робот управляющую информацию с джойстика, потенциометра, кнопок, как в традиционных пультах управления моделями?

6. А если требуется передать информацию с одного робота на другой робот?

И так далее.

Здесь ответ один – необходимо использовать Bluetooth-модуль HC-05, работающий в режиме master, как это показано на рис. 3.3. Необходимо срочно приобретать модуль HC-05.

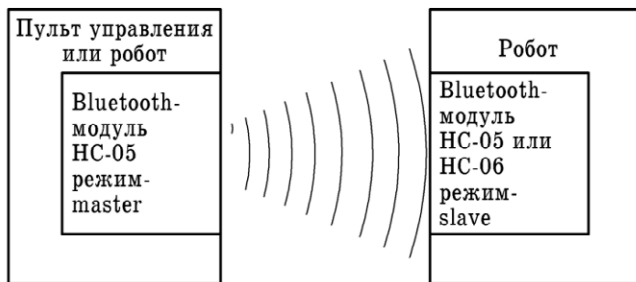


Рис. 3.2

3.2. Проблемы в приобретении модуля HC-05

Так, где приобрести? Естественно, на AliExpress. Для того, чтобы модуль корректно работал, его необходимо предварительно настроить с помощью, так называемых, АТ-команд. АТ-команда – это строка, начинающаяся с букв АТ (от английского attention – внимание). Модуль выполняет поступившую команду и отправляет обратно ответ – результат выполнения команды, который также является строкой. В Bluetooth-модулях HC-05 каждая АТ-команда и ответ на неё должна заканчиваться символами перевода строки «\r\n». АТ-команды задают режим работы модуля (master или slave), скорость обмена информацией, имя модуля, определяя его уникальный адрес, определяя версию прошивки программного обеспечения в модуле и прочее, прочее.

Был приобретен один модуль, но не удалось его настроить на режим работы master. Он отказывался понимать АТ-команды, характерные для режима master, но великолепно настраивался на режим slave. Причем продавец позиционировал данный модуль, как модуль, который может работать в обоих режимах. Затратив огромное количество времени и, списав неудачу на свою неопытность, был заказан такой же модуль, но у другого продавца. Модули недорогие, около 200 рублей, так, что можно потратиться. Картина повторилась. И так повторилось еще пять раз. Потеряно огромное количество времени. Бесполезно потрачены деньги.

Одновременно стало известно из блогов, что нормально работают модули с прошивкой второй версии, например 3.0-20100601. А покупаемые нами модули имели третью (например, 3.0-201700601) и даже четвертую версию прошивки. Они, хоть и новые, но не обеспечивают работу модуля в режиме master. Умные головы из интернета предлагали самостоятельно перепрошить модуль, но говорили и о возможных проблемах, появляющихся после перепрошивки.

Спор у последнего продавца был выигран, с указанием на то, что он, да и остальные продавцы, вводят покупателей в заблуждение ложной информацией о товаре. Деньги мгновенно вернули. Модуль, естественно, остался и отлично работает в режиме slave. Но что-то нужно было делать, и мы обратили внимание на магазин «Амперка» (amperka.ru). Там был заказан Bluetooth-модуль за 890 рублей, да еще доставка – 240 рублей. Официальное название этого модуля на сайте магазина – Bluetooth HC-05 (Тройка-модуль). Предварительно запросили у продавца версию прошивки. Он внятно ответил, что это 3.0-20100601, чего невозможно добиться ни у одного из китайских продавцов. Модуль великолепно настроился за несколько минут.

3.3. Подготовка модуля HC-05 к настройке

Нам для управления роботом или иным устройством необходимо два модуля HC-05. И оба необходимо настроить с помощью АТ-команд. Воспользуемся для настройки нашей средой программирования IDE. Но для этого

необходимо каким-то образом подключить модуль HC-05 к персональному компьютеру (ПК) или ноутбуку, который вы используете для написания скетчей для плат Arduino. Это делается с помощью специальных устройств, называемых преобразователями, конвертерами, адаптерами, программаторами. Вот некоторые из них:

- USB-UART преобразователь (Piranha CH340C);
- Atmel ISP переходник 10 пин – 6 пин;
- USB-UART преобразователь CP2102, в корпусе;
- USB-UART преобразователь CH340G, в корпусе;
- USBASP V3.0 программатор;
- USB ISP программатор;
- USB программатор UART CP2102;
- адаптер USB в RS485 (FT232);
- USB-UART преобразователь интерфейса PL2303.

Подключение модуля HC-05 при его настройке с помощью AT-команд к компьютеру (ноутбуку или ПК) с помощью преобразователя USB-UART показано на рис. 3.3.

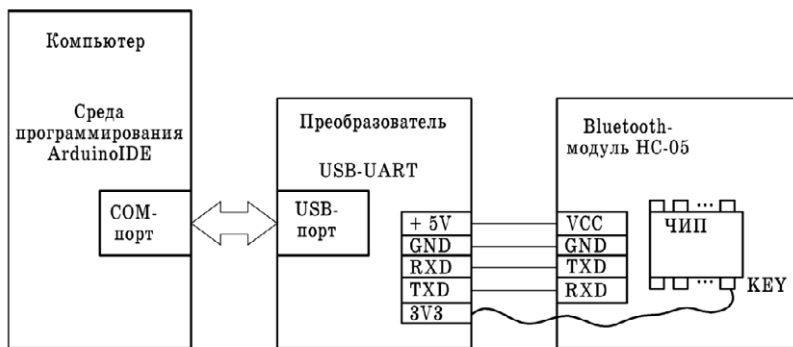


Рис. 3.3

Рассмотрим два типа преобразователя USB-UART. Они содержат два интерфейса:

- USB – для подключения к ноутбуку или ПК,
- UART – для подключения внешних устройств с UART-интерфейсом – контроллеров, модулей, жёстких дисков, роутеров, плат Arduino, мобильных телефонов и смартфонов, тюнеров, а также любых устройств с TTL-логикой.

USB программатор UART CP2102

Используется для программирования контроллера Arduino Pro Mini и других подобных контроллеров, имеющих поддержку шины UART с TTL логикой (рис. 3.4). Именно это устройство использовали мы для настройки Bluetooth-модулей. На этом рисунке слева – разъем USB, а справа – штыри разъема UART.

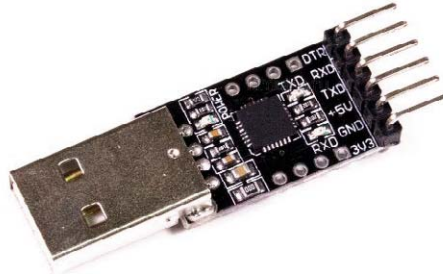


Рис. 3.4

Характеристики

- Чип CP2102;
- Скорость обмена данными по UART: 300 Бит/сек – 1 Мбит/сек;
- Питание: 5 В;
- Поддержка 5В/3.3В логики;
- Два уровня выходного питания: 3,3 В и 5 В;
- Буфер: чтения 576 байт / записи 640 байт;
- Встроенный стабилизатор питания 3.3В, 100 мА;
- Объем EEPROM: 1 Кб;
- Поддержка USB 3.0;
- Поддержка режима SUSPENDED USB;
- Поддерживаемые ОС: Windows 10/8/7/Vista/Server 2003/XP/2000/CE; Mac OS-X/OS-9, Linux, Android;
- Возможность настройки параметров платы и драйверов под свои проекты;
- Размеры: 26,5×15,6 мм;

USB программатор UART CP2102 отличается от других подобных устройств следующими функциями:

- Имеет дополнительный вывод DTR, который можно подключить к входу RESET на контроллерах, не имеющих USB на плате;
- Имеет дополнительные выводы на плате;
- Имеет возможность менять VID, PID и Имя, с которым опознается плата.

Программатор может использоваться для программирования жёстких дисков, роутеров, контроллеров Arduino, мобильных телефонов и смартфонов, тюнеров, а также любых устройств с TTL-логикой. Для работы устройства необходимо скачать драйвер и установить его на свой ПК. Драйвер находится в открытом доступе.

Преобразователь интерфейса USB-UART PL2303

Преобразователь интерфейса PL2303 (рис. 3.5) служит для подключения микроконтроллеров к ПК через порт UART и прошивки (записи программного обеспечения) контроллеров, поддерживающих TTL-логику.



Рис. 3.5

Характеристики

- Чип: PL-2303HX;
- Питание: 5 В (от USB порта компьютера);
- Интерфейс подключения к компьютеру: USB 3.0;
- Поддержка уровней сигналов TTL: 3,3 и 5 В;
- Поддерживаемые ОС: Win7/Vista/XP/2000/ME/98;
- Светодиодная индикация;
- Размеры: 50,7×15,2×7,6 мм.

Конвертер имеет два интерфейса для подключения к компьютеру и для подключения программируемых устройств:

- Для подключения устройства к компьютеру используется стандартный USB интерфейс;
- Для подключения программируемых устройств используется 5-контактный штыревой интерфейс.

Для подключения преобразователя к ПК вам необходимо установить драйвера с официального сайта. Для установки драйверов в ОС Windows 7/8/10 необходимо отключить проверку цифровой подписи драйверов.

3.4. Подключение преобразователя к ПК и модулю HC-05 при настройке модуля

Прежде всего, нам необходимо собрать схему (рис. 3.6), состоящую из преобразователя CP2102 (вверху) и Bluetooth-модуля HC-05 (внизу) в соответствии со схемой, приведенной на рис. 3.3. Bluetooth-соединение работает надежно, а при обрыве связи, восстанавливается за секунды.

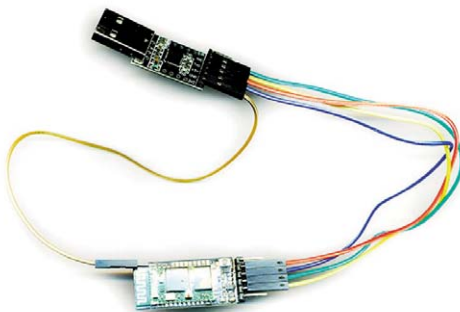


Рис. 3.6

Внешний вид используемых модулей в большом масштабе показан на рис. 3.7.



Рис. 3.7

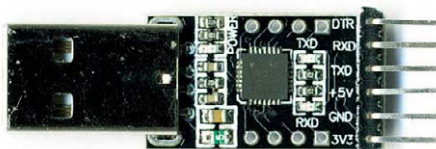


Рис. 3.8

За счет доступности, Bluetooth часто оказывается единственной возможностью беспроводной связи, которую предоставляют разработчики. Особенно это касается мобильных роботов.

В Bluetooth для передачи данных используется профиль последовательного порта (SPP) и топология «точка-точка». Когда соединение установлено, между двумя устройствами образуется «прозрачный» беспроводной канал связи, который выглядит так, как будто бы устройства связаны проводом. Точнее, двумя. Первый «провод» передает сигналы протокола последовательного обмена данными в одном направлении, второй – в обратном. Единственная существенная особенность – этот канал работает на какой-то конкретной скорости обмена. Информация передается в обоих направлениях, и оба устройства должны работать на одной допустимой скорости обмена: 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200, 230400 бод.

Нам потребуется ПО, с помощью которого мы будем отправлять AT-команды модулю. Тут подходит любая терминальная программа (например, Termite) или Arduino IDE, в состав которой входит монитор последовательного порта. Перед началом работы стоит вспомнить о том, что мы имеем дело с электронными компонентами, чувствительными к электростатическому напряжению и неправильному подключению. Поэтому делаем все аккуратно, как любят писать разработчики свободных аппаратных платформ и ПО – «на свой страх и риск».

3.5. Настройка slave-модуля Bluetooth

Подключаем модуль Bluetooth HC-05 проводами «мама-мама» к преобразователю USB-UART в соответствии с рис. 3.3 и 3.6 по следующей схеме:

- +5V (USB-UART) – VCC (Bluetooth);
- GND (USB-UART) – GND (Bluetooth);
- TXD (USB-UART) – RXD (Bluetooth);
- RXD (USB-UART) – TXD (Bluetooth);
- +3.3V (USB-UART) – KEY (Bluetooth).

Обратите внимание, контакт TXD одного устройства подключен к контакту RXD другого. Про пятый пункт требуется сказать отдельно. Для перехода в режим AT-команд необходимо подать логическую единицу (3.3 V) на контакт с названием «Кей». Скорее всего, такой «ноги» Вы не увидите. Но выход существует, надо использовать контакт 34 на плате модуля. Если смотреть сверху, расположив Bluetooth-модуль «ногами» к себе, это самый дальний контакт в правом ряду (рис. 3.9).

Заводим провод под прозрачную пластиковую изоляцию, чтобы он коснулся этого контакта. Если изоляции на модуле нет, фиксируем скотчем. Другой конец провода подключаем к контакту +3.3V преобразователя USB-UART (рис. 3.10).

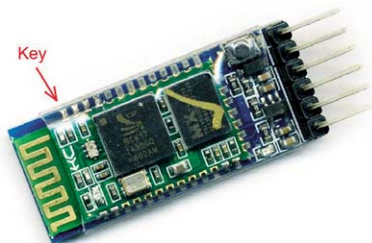


Рис. 3.9



Рис. 3.10

Подготовительные работы закончены и всё готово к настройке – «прошивке» slave-модуля HC-05. Соединяем преобразователь USB-UART и компьютер. Здесь возникает вопрос: коммуникационных портов COM в компьютере обычно несколько. К какому подключать? Ответ: к любому свободному. Но лучше не использовать порт, который используется при программировании плат Arduino (у нас это порт COM3), поскольку есть вероятность, что после настройки модулей HC-05, будут проблемы с работой этого порта с контроллерами Arduino. Придется перезапускать драйвер CH341SER.EXE, который находится в свободном доступе.

Подключаем преобразователь USB-UART к первому попавшемуся COM-порту. Открыв диспетчер устройств, обнаруживаем, что подключился к порту COM4 – запись Silicon Labs CP210x USB to UART Bridge (COM4) (рис. 3.11). Светодиод модуля HC-05 ведет себя следующим образом: две секунды горит, две секунды пауза. Это означает, что модуль находится в режиме AT-команд, и готов к общению на скорости 38400. Это важно. Если бы мы вначале подали питание, а лишь потом логическую единицу на контакт «Key», скорость модуля была бы иной, заданной его настройками, и светодиод мигал бы по-иному.

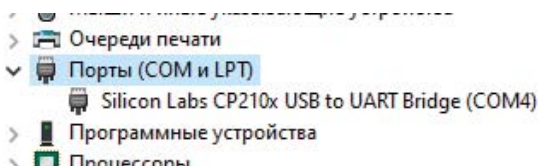


Рис. 3.11

Далее запускаем Arduino IDE и в разделе «Инструменты» в опции «Порт:» ставим галочку возле COM4 (рис. 3.12). Затем открываем монитор последовательного порта, щелкнув по опции «Монитор порта» (рис. 3.12). Открывается окно монитора последовательного порта (рис. 3.13). Выбираем скорость 38400 бод и режим «NL & CR». Набираем в верхнем окне команду AT и щелкаем по кнопке «Отправить». Модуль отвечает ОК (рис. 3.14). Все в порядке, можно продолжать.

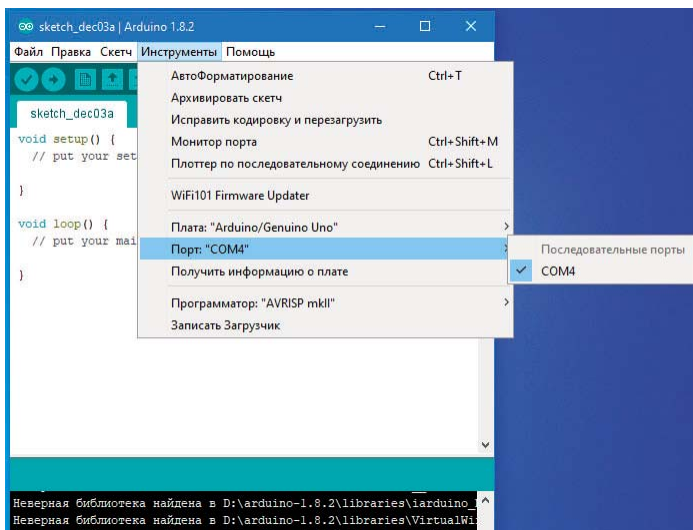


Рис. 3.12

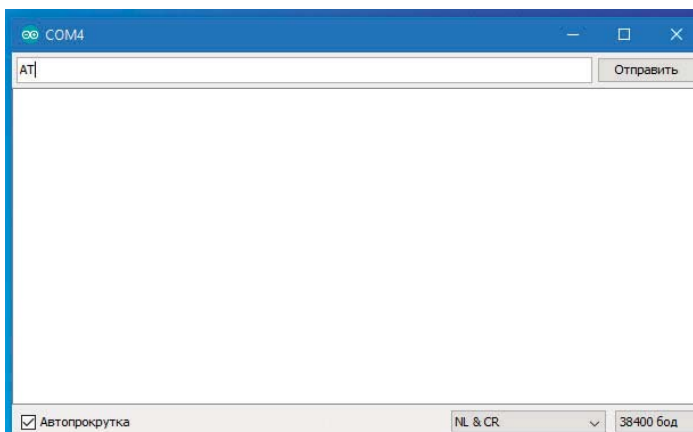


Рис. 3.13

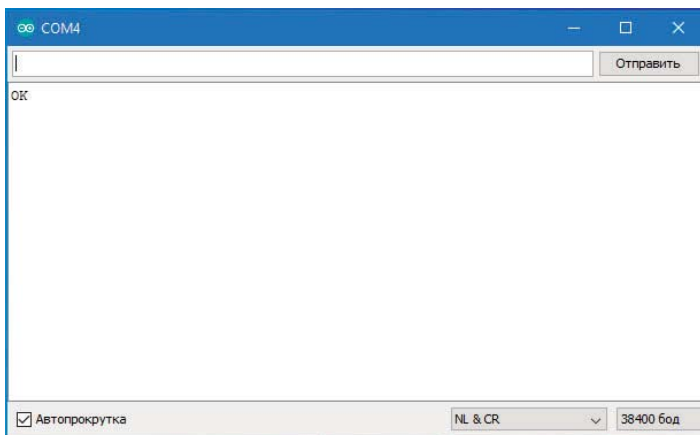


Рис. 3.14

Далее приведем команды, популярные при настройке slave-модуля, а полный набор AT-команд будет приведен в таблице далее. Конечные символы `\r\n` можно не записывать, если включен режим «NL & CR» (рис. 3.13, 3.14).

- `AT+DISC\r\n` – Разорвать соединение на случай, если модуль соединён.
- `AT+ORGL\r\n` – Сбросить пользовательские настройки в значения по умолчанию.
- `AT+RMAAD\r\n` – Очистить список пар авторизованных устройств, чтобы к модулю не подключился тот, кого отсоединили.
- `AT+NAME=имя\r\n` – Установить имя модуля (не более 32 символов).
- `AT+PSWD=пароль\r\n` – Установить PIN-код (пароль) для подключения к модулю (не более 16 символов).
- `AT+ROLE=0\r\n` – Установить модулю роль ведомого, если она не установилась при сбросе пользовательских настроек.
- `AT+RESET\r\n` – Перезагрузить модуль.

Нам можно поменять скорость обмена, PIN-код, имя, узнать адрес (MAC-адрес). В таблице 3.1 приведен листинг сеанса связи.

Чтобы убедиться, что все получилось, можно выдернуть преобразователь USB-UART из компьютера, а затем подключить и заново запросить все сведения. После проверки можно отсоединять все провода от модуля. Полезно будет как-то пометить подготовленный модуль. Например, подклеить скотчем кусочек бумаги и записать прямо на нем всю нужную информацию: адрес модуля и его имя, скорость работы и пароль.

Таблица 3.1

АТ-команда	Ответ из модуля	Назначение
АТ	OK	Проверили работоспособность модуля
АТ+ADDR?	+ADDR:FCA8:FF:392E OK	Узнали адрес модуля
АТ+NAME?	+NAME:HC-05 OK	Узнали имя модуля
АТ+UART?	+UART:9600,0,0 OK	Узнали скорость работы модуля
АТ+PSWD?	+PSWD:1234 OK	Узнали пароль модуля
АТ+VERSION?	+VERSION:3.0- 20170601 OK	Узнали версию программного обеспечения модуля
АТ+ROLE?	+ROLE:0 OK	Модуль настроен на роль ведомого-slave
АТ+NAME=RAB1	OK	Задали новое имя модулю: RAB1
АТ+UART=115200,0,0	OK	Задали новую скорость работы: 115200 бод
АТ+PSWD=5555	OK	Задали новый пароль модуля

Slave-модуль готов. Им можно уже пользоваться. Однако мы пока не достигли главного, не настроили master-модуль. Чтобы продолжить, нам необходимо подключить питание к slave-модулю, чтобы он оказался доступен для поиска. Подсоединяем его к плате Arduino, пока всего лишь к контактам +5 V и GND. Контакты RXD и TXD пока не задействуем. Плату запитываем от аккумуляторной батареи, сетевого адаптера или свободного USB-разъема компьютера. После включения питания светодиод на HC-05 начинает часто мигать, модуль ожидает соединения.

3.6. Настройка master-модуля Bluetooth

Начинаем настраивать master-модуль. Подключаем его к преобразователю USB-UART. Подключение абсолютно такое же, что и при настройке slave-модуля (рис. 3.3, 3.6). Снова внимательно проверяем, что провод +3.3V касается контакта «Key». Подключаем преобразователь USB-UART к компьютеру и видим, что светодиод модуля загорается на две секунды с двухсекундными паузами. Он готов к настройке с помощью АТ-команд. Открываем монитор последовательного порта и производим действия, аналогичные тем, что только что проделывали со slave-модулем. PIN-код и скорость обмена должны быть те же самые, иначе соединения не добиться.

Приведем наиболее часто используемые AT-команды при настройке master-модуля.

- T+DISC\r\n – Разорвать соединение на случай, если модуль соединён.
- AT+ORGL\r\n – Сбросить пользовательские настройки в значения по умолчанию.
- AT+RMAAD\r\n – Очистить список пар (авторизованных устройств) для того, чтобы модуль не пытался подключиться к тому от кого отсоединили.
- AT+BIND=АДРЕС\r\n – Установить фиксированный адрес для подключения (указываем адрес ведомого Bluetooth устройства)
- AT+CMODE=0\r\n – Указываем модулю подключаться только к фиксированному адресу
- AT+ROLE=1\r\n – Установить модулю роль ведущего устройства
- AT+PSWD=пароль\r\n – Запомнить пароль (PIN-код) ведомого Bluetooth устройства
- AT+PAIR=АДРЕС,10\r\n – Образовать пару с ведомым Bluetooth устройством, указав его адрес и таймаут 10 сек
- AT+RESET\r\n – Перезагрузить модуль.
- AT\r\n – Выполнить команду Тест для проверки наличия связи с модулем
- AT+LINC=АДРЕС\r\n – Подключиться к ведомому Bluetooth устройству, указав его адрес
- AT+STATE?\r\n – Получить текущее состояние модуля (на практике эта команда может ускорить процесс подключения)
- AT+VERSION?\r\n – Получить версию программного обеспечения модуля

Команды настройки master-модуля приведены в таблице 3.2.

Таблица 3.2

AT-команда	Ответ из модуля	Назначение
AT	OK	Проверили работоспособность модуля
AT+ADDR?	+ADDR:98d3:51:fdb890 OK	Узнали адрес модуля
AT+NAME?	+NAME:HC-05 OK	Узнали имя модуля
AT+UART?	+UART:9600,0,0 OK	Узнали скорость работы модуля
AT+PSWD?	+PSWD:1234 OK	Узнали пароль модуля
AT+ROLE?	+ROLE:0 OK	Модуль настроен на роль ведомого-slave

Окончание таблицы 3.2

AT+VERSION?	+VERSION:3.0-20100601 OK	Узнали версию программного обеспечения модуля
AT+UART=115200,0,0	OK	Задали новую скорость работы: 115200 бод
AT+PSWD=5555	OK	Задали новый пароль модуля
AT+RESET	OK	Перезагрузили модуль
AT+ROLE=1	OK	Определили модулю роль ведущего – master
AT+PAIR=FCA8,FF,392E,5	OK или ERROR:(16) или что-то подобное	Указали мастеру адрес ведомого модуля
AT+INQM=0,5,5	OK	Задаем: – режим поиска Bluetooth-модулей 0 – обычный поиск; – максимальное число найденных Bluetooth-модулей (5), – время в секундах поиска одного Bluetooth-модуля (5)
AT+INIT	OK	Инициализация профиля SPP. Профиль SPP эмулирует последовательный порт
AT+INQ	+INQ:14:3:5A694,1F00,7FFF +INQ: FCA8,FF,392E,7FFF OK	Показывает все найденные Bluetooth-устройства. Среди них и наш slave-модуль
AT+RNAME? FCA8,FF,392E	+RNAME:RAB1 OK	Проверим, правильно ли подключен наш модуль. Правильно.
AT+PAIR= FCA8,FF,392E,5	OK	Еще раз указали мастеру адрес ведомого модуля
AT+BIND= FCA8,FF,392E	OK	Работать только с одним slave-модулем. Адрес slave-модуля указан
AT+CMODE=0	OK	Работать только с фиксированным модулем

Готово! Осталось проверить работоспособность созданной пары Bluetooth. Ну, и как обещал, привожу полный перечень AT-команд Bluetooth-модуля HC-05 (таблица 3.3). Даже внешний вид этой таблицы показывает, насколько «мощным» является это устройство.

3.7. AT-команды модуля HC-05

Как уже говорилось выше, если в мониторе последовательного порта указано добавлять символы NL & CR то символы «\r\n» в командах ставить не нужно. Команды могут быть:

- обычная: AT+КОМАНДА\r\n,
- запрос: AT+КОМАНДА?\r\n,
- установка: AT+КОМАНДА=ПАРАМЕТР(Ы)\r\n.

Таблица 3.3

AT-команда	Запись	Ответ из модуля	Назначение
AT	AT\r\n	OK\r\n	Используется для проверки связи с модулем.
RESET	AT+RESET\r\n	OK\r\n	Программная перезагрузка модуля, как после кратковременного отключения питания.
VERSION	AT+VERSION?\r\n	+VERSION: ВЕРСИЯ\r\n OK\r\n	Запрос версии прошивки модуля. Модуль возвращает версию в виде строки до 32 байт. Пример ответа: +VERSION: V3.0-20100601 OK
AT+ORGL	AT+ORGL\r\n	OK\r\n	Сброс пользовательских настроек: Модуль сбрасывает следующие настройки: CLASS=0, IAC=9e8b33, ROLE=0, CMODE=0, UART=38400,0,0, PSWD=1234, NAME=hc01.com.
ADDR	AT+ADDR?\r\n	+ADDR: АДРЕС\r\n OK\r\n	Запрос адреса модуля: Модуль возвращает три части своего адреса NAP:UAP:LAP, разделённых двоеточием. Каждая часть состоит из шестнадцатеричных цифр. Пример ответа: +ADDR:1234:56:789ABC OK

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
NAME	AT+NAME?\r\n	+NAME:ИМЯ\r\nOK\r\n	Запрос / установка имени модуля: Имя модуля задается строкой до 32 байт. Пример ответа: +NAME:rab OK
	AT+NAME=ИМЯ\r\n	OK\r\n	Пример установки: AT+NAME=rab1 Некоторые модули реагируют на команду AT+NAME? только при нажатой кнопке модуля или наличии высокого уровня на входе K.
RNAME	AT+RNAME? АДРЕС\r\n	+RNAME:ИМЯ\r\nOK\r\n	Запрос имени найденного Bluetooth устройства: Адрес вводится после пробела, а части адреса (NAP,UAP,LAP) разделяются запятой. Модуль возвращает имя найденного Bluetooth устройства находящегося в зоне действия, адрес которого был в запросе. Пример запроса: AT+RNAME? 1234,56,789ABC Пример ответа: +RNAME:fedia OK
ROLE	AT+ROLE?\r\n	+ROLE:ПОЛЬ\r\nOK\r\n	Запрос / установка роли модуля: Роль модуля задается цифрой: 0 – ведомый, 1 – ведущий, 2 – ведомый в цикле*.
	AT+ROLE=ПОЛЬ\r\n	OK\r\n	Пример ответа: +ROLE:1 Пример установки: AT+ROLE=0
CLASS	AT+CLASS?\r\n	+CLASS:ТИП\r\nOK\r\n	Запрос / установка типа устройства: Тип устройства задается 32-битным числом, по которому можно определить назначение модуля: Bluetooth клавиатура, Bluetooth мышь, гарнитура ...
	AT+CLASS=ТИП\r\n	OK\r\n	Пример установки: AT+CLASS=0

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
IAC	AT+IAC?\r\n	+IAC:КОД\r\n OK\r\n	Запрос / Установка кода общего доступа GIAC: Код задается 32 битным числом и используется для обнаружения Bluetooth устройств. В роли ведущего, по данному коду этот модуль будет получать доступ к другим Bluetooth устройствам для их поиска (опроса). В роли ведомого по данному коду будет предоставляться доступ для опроса этого модуля другими ведущими. Пример ответа: +IAC:9e8b33 OK Пример установки: AT+IAC=9e8b33
	AT+IAC=КОД\r\n	OK\r\n или FAIL\r\n	
INQM	AT+INQM?\r\n	+INQM:РЕЖИМ, КОЛИЧЕСТВО, ВРЕМЯ\r\n OK\r\n	Запрос / Установка режима опроса модулей: Используемые параметры являются настройками для команды поиска (опроса) других Bluetooth устройств. РЕЖИМ поиска задается цифрой: 0 – стандартный, 1 – поиск по интенсивности сигнала. КОЛИЧЕСТВО задает предельное число найденных Bluetooth устройств, после которого требуется прекратить поиск. ВРЕМЯ поиска задаёт таймаут, после которого поиск прекращается. Реальное время поиска в секундах равно указанному числу, умноженному на 1,28. Пример ответа: +INQM:1,1,48 OK Пример установки: AT+INQM:1,1,48
	AT+INQM=РЕЖИМ, КОЛИЧЕСТВО, ВРЕМЯ\r\n	OK\r\n или FAIL\r\n	

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
PSWD	AT+PSWD?\r\n	+PSWD:КОД\r\nOK\r\n	Запрос / Установка PIN-кода: Код доступа (PIN-код) задается строкой до 16 байт. Код модуля в роли ведомого устройства является паролем доступа к этому модулю. Код модуля в роли ведущего устройства является паролем доступа к внешним Bluetooth устройствам.
	AT+PSWD=КОД\r\n	OK\r\n	Пример ответа: +PSWD:1234 OK Пример установки: AT+PSWD=erofei
UART	AT+UART?\r\n	+UART: СКОРОСТЬ, СТОП, ПРОВЕРКА\r\nOK\r\n	Запрос / установка скорости обмена информацией через интерфейс UART: СКОРОСТЬ задается числом бит/сек СТОП – число стоповых битов в передаваемом байте: 0 – один, 1 – два ПРОВЕРКА задает режим проверки передаваемой информации 0 – без проверки, 1 – проверка нечётности, 2 – проверка чётности.
	AT+UART= СКОРОСТЬ, СТОП, ПРОВЕРКА\r\n	OK\r\n	Пример ответа: +UART:38400,0,0 OK Пример установки: AT+UART=38400,0,0

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
CMODE	AT+CMODE?\r\n	+CMOD: РЕЖИМ\r\n OK\r\n	Запрос / установка режима подключения: Режим представлен цифрой: 0 – модуль в роли ведущего подключается только к тому Bluetooth устройству, адрес которого указан командой AT+BIND. 1 – модуль в роли ведущего подключается к любому ведомому Bluetooth устройству. 2 – модуль в роли ведомого работает в цикле* Пример ответа: +CMOD:0 OK Пример установки: AT+CMOD=1
	AT+CMODE= РЕЖИМ\r\n	OK\r\n	
BIND	AT+BIND?\r\n	+BIND:АДРЕС\r\n OK\r\n	Запрос / установка фиксированного адреса: Если модуль находится в роли ведущего (ROLE=1) и установлен режим подключения к фиксированному адресу (CMODE=0), то он будет подключаться только к тому Bluetooth устройству, адрес которого указан данной командой.
	AT+BIND= АДРЕС\r\n	OK\r\n	Части адреса вводятся: при установке – через запятую, а при ответе – через двоеточие. Пример ответа: +BIND:1234:56:789ABC OK Пример установки: AT+BIND=0,0,0

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
POLAR	AT+POLAR?\r\n	+POLAR: ЛОГ,ЛОГ\r\n OK\r\n	Запрос / установка активного логического уровня для включения светодиодов:
	AT+POLAR=ЛОГ, ЛОГ\r\n	OK\r\n	Полярность представлена цифрой 0 или 1 соответствующей активному логическому уровню. Первый параметр указывает логический уровень для включения светодиода, подключённого к выводу PIO8 (отображает режим работы). Второй – для светодиода подключённого к выводу PIO9 (отображает статус соединения). Пример ответа: +POLAR:1,1 OK Пример установки: AT+POLAR=1,1
PIO	AT+PIO=НОМЕР, УРОВЕНЬ\r\n	OK\r\n	Установка логического уровня PIO: Позволяет установить логический уровень на выводе PIO. Номер вывода представлен числом от 2 до 11, кроме 8 и 9. Уровень представлен цифрой 0 или 1. Пример установки: AT+PIO=11,0
MPIO	AT+MPIO?\r\n	+MPIO:ЧИСЛО\r\n OK\r\n	Запрос / установка логических уровней PIO:
	AT+MPIO= ЧИСЛО\r\n	OK\r\n	Позволяет узнать или установить логические уровни сразу на всех выводах PIO. Уровни представлены шестнадцатеричным числом, каждый бит которого соответствует уровню вывода PIO. Пример ответа: +MPIO:1F0 OK Пример установки: AT+MPIO:CFC

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
IPSCAN	AT+IPSCAN?\r\n	+IPSCAN:A,B,B,\r\n\r\nOK\r\n	Запрос / установка параметров IP сканирования: А – интервал сканирования Б – продолжительность сканирования В – интервал страниц Г – количество страниц
	AT+IPSCAN=A,B,B,\r\n\r\n	OK\r\n	Пример ответа: +IPSCAN:1024,512,1024,512 OK Пример установки: AT+IPSCAN:1024,512,1024,512
SNIFF	AT+SNIFF?\r\n	+SNIFF:A,B,B,\r\n\r\nOK\r\n	Запрос / установка параметров энергосберегающего режима: А – максимальное время Б – минимальное время В – период повторов Г – таймаут
	AT+SNIFF=A,B,B,\r\n\r\n	OK\r\n	Пример ответа: +SNIFF:0,0,0,0 OK Пример установки: AT+SNIFF=0,0,0,0
ENSNIFF	AT+ENSNIFF=AДРЕС\r\n	OK\r\n	Переход в энергосберегающий режим: Части адреса вводятся через запятую (NAP,UAP,LAP) Пример команды: AT+ENSNIFF=1234,56,789ABC
EXSNIFF	AT+EXSNIFF=AДРЕС\r\n	OK\r\n	Выход из энергосберегающего режима: Части адреса вводятся через запятую (NAP,UAP,LAP) Пример команды: AT+EXSNIFF=1234,56,789ABC

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
SENM	AT+SENM?\r\n	+SENM:СЕКРЕТ, ШИФР\r\n OK\r\n	Запрос / установка параметров безопасности: Режим секретности представлен цифрой: 0 – выключен 1 – незащищённое соединение 2 – защита на сервисном уровне 3 – защита на уровне соединения 4 – неизвестный режим
	AT+SENM=СЕКРЕТ,ШИФР\r\n	OK\r\n	Режим шифрования представлен цифрой: 0 – без шифрования 1 – шифруется только трафик РТР 2 – шифруется весь трафик Пример ответа: +SENM:0,0 OK Пример установки: AT+SENM:0,0
PMSAD	AT+PMSAD=АДРЕС\r\n	OK\r\n	Удаление устройства из списка пар: Удаление Bluetooth устройства из списка приведёт к необходимости заново образовывать пару для подключения к нему. Части адреса удаляемого устройства вводится через запятую (NAP,UAP,LAP) Пример команды: AT+PMSAD=1234,56,789ABC
RMAAD	AT+RMAAD\r\n	OK\r\n	Удаление всех устройств из списка пар: Очистка данного списка приведёт к необходимости заново образовывать пары с Bluetooth устройствами для подключения к ним.

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
FSAD	AT+FSAD=АДРЕС \r\n	OK\r\n или FAIL\r\n	Поиск устройства в списке пар: Если Bluetooth устройство с указанным адресом имеется в списке, то модуль вернёт OK иначе FAIL. Части адреса NAP,UAP,LAP вводятся через запятую Пример запроса: AT+FSAD=1234,56,789ABC
ADCN	AT+ADCN?\r\n	+ADCN: КОЛИЧЕСТВО\r\n OK\r\n	Запрос количества устройств в списке пар: При образовании пары ведущий-ведомый, данные о паре автоматически попадают в список пар и для последующих подключений (даже после отключения питания) не требуется повторно устанавливать пару. Пример ответа: +ADCN:10 OK
MRAD	AT+MRAD?\r\n	+MRAD:АДРЕС \r\n OK\r\n	Запрос адреса устройства из списка пар: Модуль вернёт адрес Bluetooth устройства из списка пар, с которым выполнялось последнее успешное соединение. Части адреса NAP:UAP:LAP выводятся через двоеточие Пример ответа: +MRAD:1234:56:789ABC OK
STATE	AT+STATE?\r\n	+STATE:СТАТУС \r\n OK\r\n	Запрос статуса модуля: Модуль вернёт свое текущее состояние в виде строки: INITIALIZED – инициализация READY – готов PAIRABLE – образование пары PAIRED – пара образована INQUIRING – запрос CONNECTING – подключение CONNECTED – подключён DISCONNECTED – отсоединён NUKNOW – неизвестное состояние Пример ответа: +STATE:CONNECTED OK

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
INIT	AT+INIT\r\n	OK\r\n или FAIL\r\n	Инициализация профиля SPP: Профиль SPP эмулирует последовательный порт.
INQ	AT+INQ\r\n	+INQ:АДРЕС, ТИП,СИГНАЛ\r\n +INQ:АДРЕС, ТИП,СИГНАЛ\r\n ... +INQ:АДРЕС, ТИП,СИГНАЛ\r\n	Поиск (опрос) данным модулем других Bluetooth устройств: Команда доступна модулю в роли ведущего. Модуль ищет Bluetooth устройства в радиусе действия и выводит каждый найденный модуль на новой строке. Режим поиска (опроса) устанавливается командой AT+INQM. Код опроса устанавливается командой AT+IAC. Тип искомых устройств указывается командой AT+CLASS. Поиск завершается по достижении предельного количества найденных Bluetooth устройств или по достижении таймаута, или командой AT+INQC. Пример ответа: +INQ:1234:56:789ABC,240404,7FFF
INQC	AT+INQC\r\n	OK\r\n	Завершить поиск (опрос) Bluetooth устройств: Досрочно завершает поиск Bluetooth устройств инициированный командой AT+INQ

Продолжение таблицы 3.3

АТ-команда	Запись	Ответ из модуля	Назначение
PAIR	AT+PAIR=АДРЕС, ТАЙМАУТ\r\n	OK\r\n или FAIL\r\n	Создать пару с Bluetooth устройством: Создание пары или сопряжение Bluetooth устройств инициируется ведущим устройством. Таймаут указывается десятичным числом в секундах. Если пара создана, то информация о ней автоматически запишется в список пар, модуль ответит OK\r\n после чего можно подключить Bluetooth устройство командой AT+LINK. Если пара не создана (например, не подошёл PIN-код или истек таймаут), то модуль ответит FAIL\r\n. Пример команды: AT+PAIR=1234,56,789ABC,10
LINK	AT+LINK=АДРЕС \r\n	OK\r\n или FAIL\r\n	Подключиться к Bluetooth устройству: После выполнения данной команды можно общаться с подключённым Bluetooth устройством. Команда доступна модулю в роли ведущего. Пример команды: AT+LINK=1234,56,789ABC

АТ-команда	Запись	Ответ из модуля	Назначение
DISC	AT+DISC\r\n	+DISC: РЕЗУЛЬТАТ\r\n OK\r\n	Отключиться от Bluetooth устройства: Команда указывает модулю отключиться от Bluetooth устройства, с которым установлено соединение. После отключения от Bluetooth устройства информация о нём сохраняется в списке пар. Если потребуется вновь подключиться к этому устройству, то создание пары будет обязательно (если Bluetooth устройство намеренно не удалить из списка пар). После выполнения команды модуль ответит результатом её выполнения: SUCCESS – успех LINK_LOSS – соединение потеряно NO_SLC – отсутствует SLC TIMEOUT – истекло время ожидания ERROR – ошибка Пример ответа: +DISC:SUCCESS OK

* Ведомый в цикле – модуль отправляет обратно всё, что получает от ведущего.

** На некоторые команды модуль реагирует только при нажатой кнопке модуля или наличии высокого уровня на выводе Key.

Описание ошибок выдаваемых модулем

Если отправить команду, которую модуль не знает, не может выполнить, или у команды неправильные аргументы, то модуль вернёт строку «ERROR:(НОМЕР)», где по указанному шестнадцатеричному номеру можно определить тип ошибки.

№ ошибки	Описание ошибки
0	Неправильная АТ команда (нет такой команды)
1	Результат по умолчанию
2	Ошибка сохранения пароля
3	Слишком длинное имя устройства (более 32 байт)
4	Имя устройства не указано
5	Часть адреса NAP слишком длинная (более 4 разрядов в шестнадцатеричной системе)
6	Часть адреса UAP слишком длинная (более 2 разрядов в шестнадцатеричной системе)
7	Часть адреса LAP слишком длинная (более 6 разрядов в шестнадцатеричной системе)
8	Не указана маска порта PIO
9	Не указан номер вывода PIO
A	Не указан тип (класс) устройства
B	Слишком длинный тип (класс) устройства
C	Не указан общий код доступа IAC (Inquire Access Code)
D	Слишком длинный общий код доступа IAC (Inquire Access Code)
E	Недопустимый общий код доступа IAC (Inquire Access Code)
F	Не указан пароль (или пароль пуст)
10	Слишком длинный пароль (более 16 байт)
11	Недопустимая роль модуля
12	Недопустимая скорость передачи данных
13	Недопустимый размер стоп-бита
14	Недопустимая настройка бита четности
15	Устройство отсутствует в списке пар (списке сопряжённых Bluetooth устройств)
16	Профиль последовательного порта (SPP, Serial Port Profile) не инициализирован
17	Повторная инициализация профиля SPP (SPP, Serial Port Profile)
18	Недопустимый режим опроса Bluetooth устройств
19	Слишком большое время опроса
1A	Не указан адрес Bluetooth устройства
1B	Недопустимый режим безопасности (секретности)
1C	Недопустимый режим шифрования

3.8. Пульт для передачи информации по Bluetooth

Мы собираемся передавать информацию с пульта управления на различные устройства: роботы, исполнительные механизмы различных типов и прочее. Задатчиками информации в пульте управления могут быть кнопки, тумблеры, потенциометры, энкодеры, комбинации этих устройств. Понятно, что основная часть пульта для всех этих случаев будет постоянной:

- контроллер Arduino,
- аккумулятор LI-PO,
- Bluetooth-модуль, работающий в режиме master.

В этом случае пульт управления можно оформить в виде конструкции, состоящей из двух частей:

- постоянная часть,
- сменяемая часть.

Сменяемая же часть пульта будет содержать соответствующие задатчики информации. А вообще для реализации любого проекта с использованием технологии Bluetooth нам потребуются:

- компьютер с установленной средой программирования Arduino IDE и драйверами для контроллера Arduino;
- два контроллера Arduino с USB кабелем для подключения к компьютеру при программировании; один контроллер для робота, второй для пульта управления;
- две двухбаночные (2S) аккумуляторные батареи LI-PO номиналом 7,4В...8,4В;
- два модуля HC-05 – один будет «master», другой «slave» (на роль «slave» допустимо взять и HC-06);
- преобразователь интерфейса USB-UART уже не нужен.

На рис. 3.15 приведен внешний вид пульта (со снятой крышкой) с потенциометром для управления поворотом башни робота-танка.

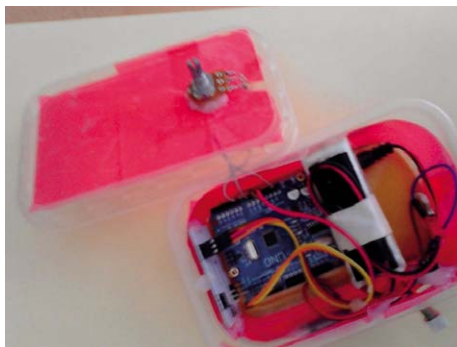


Рис. 3.15

Из рис. 3.15 видно, что сменяемая часть пульта управления оформлена в виде крышки, на которой и монтируется датчик информации – потенциометр. В проектах, рассмотренных ниже, мы будем демонстрировать сменную часть пульта для каждого случая.

3.9. Передача информации с двухосевого джойстика по Bluetooth

Схема пульта управления для этого случая приведена на рис. 3.16. Здесь использован стандартный модуль двухосевого джойстика KY-023 с кнопкой. Внешний вид сменяемой части пульта показан на рис. 3.17.

Пример 3.1. Требуется по Bluetooth-каналу передать информацию о положении двух осей (X и Y) и кнопки модуля KY-023. На приёмном конце может находиться любая плата Arduino Uno, к которой подключен Bluetooth-модуль HC-05 или HC-06, работающий в режиме slave (рис. 3.18).

Для просмотра принятой информации мы будем использовать монитор последовательного порта среды программирования Arduino IDE (рис. 3.14).

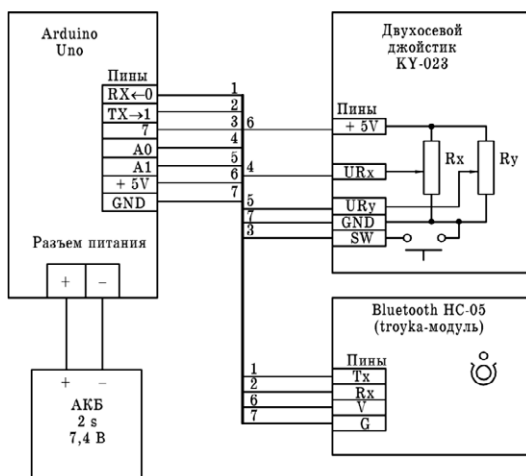


Рис. 3.16

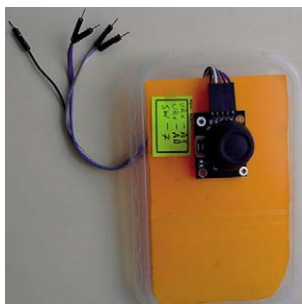


Рис. 3.17

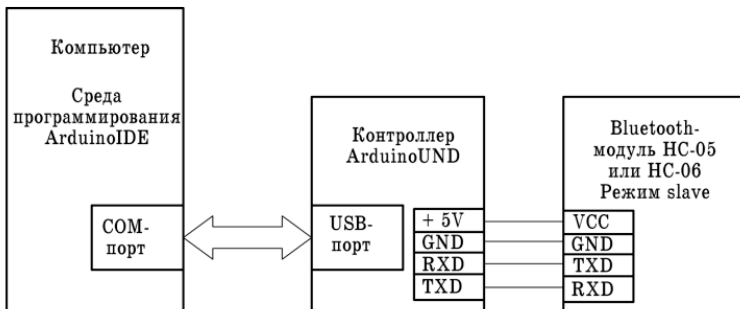


Рис. 3.18

Скетч (master3.ino) для пульта управления приведен ниже. При загрузке скетча в контроллер пульта пины RXD и TXD контроллера должны быть отсоединены от соответствующих пинов модуля HC-05. После загрузки скетча в контроллер соединения должны быть восстановлены.

```
//пины под оси джойстика и кнопку
int JoyStick_X = A0; // ось x
int JoyStick_Y = A1; // ось y
int JoyStick_Z = 7; // кнопка
int x, y, z, d;

void setup ()
{
    //настройка пинов на ввод информации
    pinMode (JoyStick_X, INPUT);
    pinMode (JoyStick_Y, INPUT);
    pinMode (JoyStick_Z, INPUT_PULLUP);
    //настройка скорости работы последовательного
    // порта и Bluetooth; самая низкая скорость 9600 bps,
    //можно и больше
    Serial.begin (9600);
}

void loop ()
{
    //принимаем информацию с осей и кнопки
    x = analogRead (JoyStick_X);
    y = analogRead (JoyStick_Y);
    z = digitalRead (JoyStick_Z);
    //переводим ось X в строку
    String s=String(x);
    //если это необходимо,
    //дополняем строку незначащими нулями
```

```

for(int i=1;i<=3;i++)
{
    d=s.length();
    if (d<4)
    {
        s='0'+s;
    }
}
//отправляем строку в Bluetooth-модуль
Serial.print(s);
delay(20);

//переводим ось Y в строку
String s1=String(y);
for(int i=1;i<=3;i++)
{
    d=s1.length();
    if (d<4)
    {
        s1='0'+s1;
    }
}
//отправляем строку в Bluetooth-модуль
Serial.print(s1);
delay(20);

//переводим кнопку Z в строку
String s2=String(z);
for(int i=1;i<=3;i++)
{
    d=s3.length();
    if (d<4)
    {
        s2='0'+s2;
    }
}
//отправляем строку в Bluetooth-модуль
Serial.print(s2);
delay (200);
}

```

Дадим несколько пояснений по этому скетчу. Эксперименты показали, что информация по Bluetooth передается в виде строк, а на приёмном конце эти строки нужно «вытаскивать» из приёмного буфера по одному символу – символ за символом. Но вся беда, что отправляемые строки имеют разную длину от 1-го символа цифры до 4-х символов цифр. Информация с каждой из осей X и Y джойстика может принимать значения от 0 до 1023. Например, зна-

чение 9 будет передаваться односимвольной строкой '9', значение 93 – двухсимвольной строкой '93', а значение 1011 – четырехсимвольной строкой '1011'. Скетч на приёмном конце Bluetooth-канала не «знает» длину отправляемой строки, что не прибавляет оптимизма при его программировании. Поэтому было принято простое решение – всю отправляемую информации приводить к четырехсимвольному формату и '9' станет '0009', '93' станет '0093', а '1011' останется '1011'. Это и сделано в скетче.

Ниже приведен скетч (slave3.ino), принимающий информацию из Bluetooth-канала с помощью Bluetooth-модуля HC-05, работающего в режиме slave, и выводящий её в монитор последовательного порта компьютера (рис. 3.18). Запись скетча в контроллер должна производиться при отключенных пинах RXD и TXD модуля или контроллера.

```
// символы для приема информации из буфера
char s,s1,s2;
// в эти строки будут добавляться символы из
//приемного буфера
String st=""; //ось X
String st1=""; //ось Y
String st2=""; //кнопка
//в эти переменные будут помещены цифровые результаты
int x,y,z;

void setup()
{
    //скорость работы последовательного порта
    //приемника должна совпадать со скоростью
    //передатчика
    Serial.begin(9600);
}

void loop()
{
    st="";
    st1="";
    st2="";
    if(Serial.available())
    {
        delay(10);
    }
    //если в порту что-то есть,
    //читаем четыре символа
    //и составляем строку st -
    //показания по оси X
    if(Serial.available())
```



```

{
    delay(10);
    //читаем первый символ
    s=Serial.read();
    delay(10);
    //добавляем его в строку
    st=st+s;
    //читаем второй символ
    s=Serial.read();
    delay(10);
    //добавляем его в строку
    st=st+s;
    //читаем третий символ
    s=Serial.read();
    delay(10);
    //добавляем его в строку
    st=st+s;
    //читаем четвертый символ
    s=Serial.read();
    //Serial.println(s);
    //добавляем его в строку;
    //результат готов
    st=st+s;
    delay(20);
    //преобразуем строку в целое число
    x=st.toInt();
    //выводим число в монитор порта
    Serial.print ("x=");
    Serial.println(x);
}

//читаем данные по оси Y джойстика;
//всё, как при чтении данных по оси X
if(Serial.available())
{
    delay(10);
    s1=Serial.read();
    delay(10);
    st1=st1+s1;
    s1=Serial.read();
    delay(10);
    st1=st1+s1;
    s1=Serial.read();
    delay(10);
    st1=st1+s1;
    s1=Serial.read();
    //результат готов

```

```

        st1=st1+s1;
        delay(20);
        y=st1.toInt();
        Serial.print ("y=");
        Serial.println(y);
    }
    //читаем данные по кнопке джойстика;
    //всё, как при чтении данных по оси X
    if(Serial.available())
    {
        delay(10);
        s2=Serial.read();
        delay(10);
        st2=st2+s2;
        s2=Serial.read();
        delay(10);
        st2=st2+s2;
        s2=Serial.read();
        delay(10);
        st2=st2+s2;
        s2=Serial.read();
        st2=st2+s2;
        delay(20);
        z=st3.toInt();
        Serial.print ("z=");
        Serial.println(z);
    }
    delay(25);
}

```

В этом скетче мы из приёмного буфера модуля HC-05 символ за символом «вынимаем» информацию в том порядке, в каком она отправлялась в Bluetooth-канал в предыдущем скетче. Сначала – четыре символа цифр – значения по оси X джойстика, затем – четыре символа цифр – значения по оси Y, и, наконец, – четыре символа цифр – состояние кнопки джойстика.

3.10. Управление роботом с кнопочного пульта по Bluetooth

Схема кнопочного пульта приведена на рис. 3.19. Внешний вид сменяемой части пульта для этого случая показан на рис. 3.20.

На рис. 3.19 не показан цифровой вольтметр, который предназначен для индикации напряжения аккумуляторной батареи Li-Po. Аккумуляторы этого типа весьма чувствительны к разряду и при значительном разряде выхо-

дят из строя. Если учитывать, что аккумулятор – самая дорогостоящая часть робота и пульта, то поставить копеечный цифровой вольтметр – дело весьма полезное.

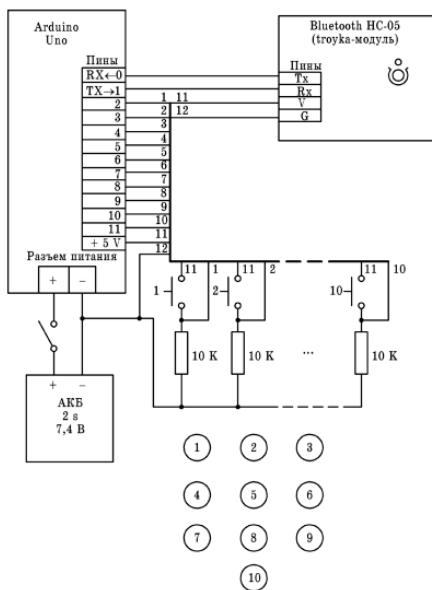


Рис. 3.19



Рис. 3.20

Пульт содержит десять основных кнопок и с номерами от 0 до 10 (рис. 3.19). Назначение этих кнопок такое же, что и в детских игрушках: управление направлением движения робота, управление поворотом башни с пушкой, стрельба из пушки.

Внешний вид робота приведен на рис. 3.21. Хорошо виден цифровой вольтметр для контроля напряжения аккумуляторной батареи. Робот содержит:

- контроллер Arduino Uno,
- шаговый двигатель 28BYJ-48 и драйвер ULN2003 для поворота башни робота,
- два модуля драйвера L293D для управления колесами робота и пушкой,
- инфракрасный дальномер GP2Y0A02YK0F (в проекте не используется),
- Bluetooth-модуль HC-05 или HC-06, работающий в режиме slave.

Схема робота приведена на рис. 3.22. На ней показано подключение всех модулей к контроллеру и источникам питания.



Рис. 3.21

Пример 3.3. Разработать скетч для пульта управления с кнопками и для робота, который бы выполнял команды, переданные с пульта. Скетч для пульта (master_3_1.ino) приведен ниже. Он хорошо закомментирован. Создается вечный цикл. Последовательно проверяется состояние кнопок пульта, начиная с 1-й и кончая 9-й. При обнаружении нажатой кнопки в эфир передается символ цифры нажатой кнопки. Например, это будет `Serial.print('1')`, если была нажата кнопка 1.

```
//пины
byte pin2 = 2; //к пину 2 подключена кнопка 1
byte pin3 = 3; //к пину 3 подключена кнопка 2
byte pin4 = 4; //к пину 4 подключена кнопка 3
byte pin5 = 5; //к пину 5 подключена кнопка 4
byte pin6 = 6; //к пину 6 подключена кнопка 5
```

```
byte pin7 = 7; //к пину 7 подключена кнопка 6
byte pin8 = 8; //к пину 8 подключена кнопка 7
byte pin9 = 9; //к пину 9 подключена кнопка 8
byte pin10 = 10; //к пину10 подключена кнопка 9
byte pin11 = 11; //к пину11 подключена кнопка 10 (не использ.)
```

```
//переменные
```

```
boolean r; // сюда принимаются уровни напряжения с кнопок
```

```
boolean s=true; //для организации вечного цикла
```

```
void setup()
```

```
{
  Serial.begin(9600); //скорость обмена с роботом
  pinMode(pin2, INPUT); // установить пин 2 на ввод
  pinMode(pin3, INPUT); // установить пин 3 на ввод
  pinMode(pin4, INPUT); // установить пин 4 на ввод
  pinMode(pin5, INPUT); // установить пин 5 на ввод
  pinMode(pin6, INPUT); // установить пин 6 на ввод
  pinMode(pin7, INPUT); // установить пин 7 на ввод
  pinMode(pin8, INPUT); // установить пин 8 на ввод
  pinMode(pin9, INPUT); // установить пин 9 на ввод
  pinMode(pin10, INPUT); // установить пин 10 на ввод
}
```

```
void loop()
```

```
{
  while(s==true) //вечный цикл
  {
    r=digitalRead(pin2); //читаем пин 2
    //проверяем нажатие кнопки 1
    if(r==HIGH)
    {
      Serial.print('1'); //нажата кнопка 1
      break; //выходим из цикла
    }

    r=digitalRead(pin3); //читаем пин 3

    //проверяем нажатие кнопки 2
    if(r==HIGH)
    {
      Serial.print('2'); //нажата кнопка 2
      break; //выходим из цикла
    }
  }
}
```

```
r=digitalRead(pin4);  
//проверяем нажатие кнопки 3  
if(r==HIGH)  
{  
    Serial.print('3');  
    break;  
}
```

```
r=digitalRead(pin5);  
//проверяем нажатие кнопки 4  
if(r==HIGH)  
{  
    Serial.print('4');  
    break;  
}
```

```
r=digitalRead(pin6);  
//проверяем нажатие кнопки 5  
if(r==HIGH)  
{  
    Serial.print('5');  
    break;  
}
```

```
r=digitalRead(pin7);  
//проверяем нажатие кнопки 6  
if(r==HIGH)  
{  
    Serial.print('6');  
    break;  
}
```

```
r=digitalRead(pin8);  
//проверяем нажатие кнопки 7  
if(r==HIGH)  
{  
    Serial.print('7');  
    break;  
}
```

```
r=digitalRead(pin9);  
//проверяем нажатие кнопки 8  
if(r==HIGH)
```

```

    {
        Serial.print('8');
        break;
    }

    r=digitalRead(pin10);
    //проверяем нажатие кнопки 9
    if(r==HIGH)
    {
        Serial.print('9');
        break;
    }
}
}

```

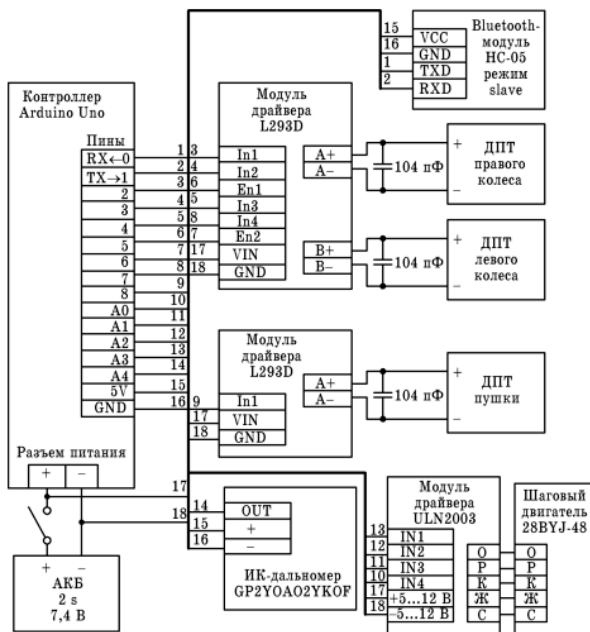


Рис. 3.22

Рассмотрим скетч robot_16-3.ino для робота, который выполняет команды, переданные с пульта.

```

//работаем с роботом 16
//робот управляется от пульта с блютуз
//1,2,3, – кнопки движения вперед
//5,7 – кнопки поворота башни влево – вправо
//4 – выстрел,
//6 – кнопка движения назад
//8, 9, 10 – кнопки задания скорости движения робота
//подключаем библиотеку для управления ШД башни
#include<AccelStepper.h>
#define HALFSTEP 8
// Определение пинов для управления двигателем
#define motorPin1 A0 // IN1 на 1-м драйвере ULN2003
#define motorPin2 A1 // IN2 на 1-м драйвере ULN2003
#define motorPin3 A2 // IN3 на 1-м драйвере ULN2003
#define motorPin4 A3 // IN4 на 1-м драйвере ULN2003

// инициализируем шаговый двигатель башни 28BYJ-48
// с последовательностью выводов IN1-IN3-IN2-IN4
// для использования библиотеки AccelStepper
AccelStepper stepper1(HALFSTEP, motorPin1, motorPin3,
motorPin2, motorPin4);

int a1 = 3; //правое колесо вперёд
int a2 = 2; // правое колесо назад
int b1 = 7; //левое колесо вперёд
int b2 = 4; // левое колесо назад
int enA =5; // правый скорость
int enB =6; // левый скорость
int c2=8; //пушка – выстрел
int k=250; //начальная скорость робота – максимум
char s; //принимаемый символ

void setup()
{
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(c2, OUTPUT);
    Serial.begin(9600);

    //настройки ШД
    stepper1.setMaxSpeed(1000.0); //максимальная скорость
    stepper1.setAcceleration(100.0); //ускорение
    stepper1.setSpeed(200); //начальная скорость

```



```

    delay(3000); //ждем
    analogWrite(enA, k);
    analogWrite(enB,k);

}

void loop()
{
    if(Serial.available())
    {
        s=Serial.read();
        Serial.println(s);
        switch (s)
        {
            //едем вперед влево
            case '1':
                analogWrite(enA, k);
                analogWrite(enB, 0);
                digitalWrite(b1, HIGH);
                digitalWrite(a1, HIGH);
                digitalWrite(b2, LOW );
                digitalWrite(a2, LOW);
                digitalWrite(c2, LOW);
                break;

            //едем вперед
            case '2':
                analogWrite(enA, k);
                analogWrite(enB, k);
                digitalWrite(b1, HIGH);
                digitalWrite(a1, HIGH);
                digitalWrite(b2, LOW );
                digitalWrite(a2, LOW);
                digitalWrite(c2, LOW);
                break;

            //едем вперед вправо
            case '3':
                analogWrite(enA, 0);
                analogWrite(enB, k);
                digitalWrite(b1, HIGH);
                digitalWrite(a1, HIGH);
                digitalWrite(b2, LOW );
                digitalWrite(a2, LOW);
                digitalWrite(c2, LOW);
                break;
        }
    }
}

```

```

//включаем мотор пушки, пушка стреляет
case '4':
    digitalWrite(c2, HIGH);
    break;

//едем строго назад
case '6':
    analogWrite(enA, k);
    analogWrite(enB, k);
    digitalWrite(b2, HIGH);
    digitalWrite(a2, HIGH);
    digitalWrite(b1, LOW );
    digitalWrite(a1, LOW);
    digitalWrite(c2, LOW);
    break;

//поворачиваем башню на 100 шагов влево
case '5':
    stepper1.move(100);
    stepper1.run();
    break;

//поворачиваем башню на 100 шагов вправо
case '7':
    stepper1.move(-100);
    stepper1.run();
    break;

//задаем низкую скорость робота
case '8':
    k=140;
    break;
//задаем среднюю скорость робота
case '9':
    k=200;
    break;
//задаем низкую скорость робота
case '0':
    k=255;
    break;

    }
}
else

```

```

    {
        //выключаем все двигатели
        digitalWrite(b1, LOW);
        digitalWrite(a1, LOW);
        digitalWrite(b2, LOW );
        digitalWrite(a2, LOW);
        digitalWrite(c2, LOW);
        digitalWrite( motorPin1, LOW );
        digitalWrite( motorPin2, LOW );
        digitalWrite( motorPin3, LOW );
        digitalWrite( motorPin4, LOW );

    }
}

```

Здесь получен эффект, когда та или иная функция выполняется роботом, пока нажата соответствующая кнопка на пульте управления. После отпущения кнопки робот перестает выполнять какие-либо действия. Это очень удобно для управления динамическими объектами. Это невозможно реализовать при использовании ИК-пультов и смартфонов.

3.11. Управление поворотом ротора шагового двигателя с помощью потенциометра по Bluetooth

Идея очень проста. На пульте управления имеется потенциометр. Поворот ручки потенциометра передается по каналу Bluetooth на робот-танк и преобразуется в синхронный поворот его башни. Внешний вид пульта управления приведен на рис. 3.15. Схема пульта приведена на рис. 3.23. Сменяемая часть пульта представлена потенциометром R.

Пример 3.3. Требуется разработать скетчи для пульта управления с потенциометром (рис. 3.23) и робота-танка (рис. 3.21, 3.22). Скетч для пульта управления (master4.ino) приведен ниже. Здесь все, как в примере 3.1, но гораздо проще, поскольку имеется лишь только один потенциометр – условно, ось X.

```

int JoyStick_X = A0; // пин для потенциометра
int x,d;

void setup ()
{
    pinMode (JoyStick_X, INPUT);

```

```

        Serial.begin (9600); // скорость обмена информацией по Bluetooth
    }
void loop ()
{
    //принимаем информацию с потенциометра
    x = analogRead (JoyStick_X);
    String s=String(x); // преобразуем число в строку
    //заполняем строку спереди незначащими нулями
    for(int i=1;i<=3;i++)
    {
        d=s.length();
        if (d<4)
        {
            s='0'+s;
        }
    }
    Serial.print(s); //отправляем строку в последовательный порт
    delay (200);
}

```

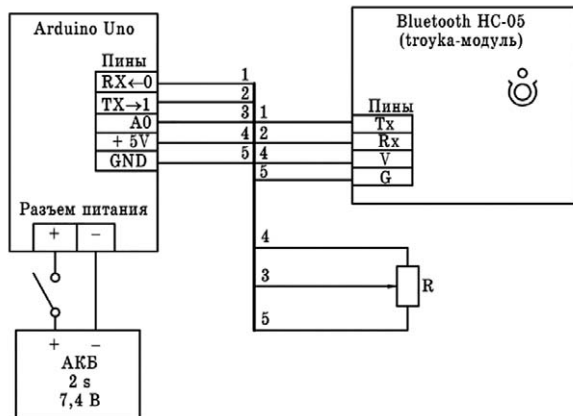


Рис. 3.23

Скетч для работа slave4.ino приведен ниже.

```

char s;
String st="";
int x;
#include<AccelStepper.h>
#define HALFSTEP 8

```

```

// пины для управления шаговым двигателем
#define motorPin1 A0 // IN1 на 1-м драйвере ULN2003
#define motorPin2 A1 // IN2 на 1-м драйвере ULN2003
#define motorPin3 A2 // IN3 на 1-м драйвере ULN2003
#define motorPin4 A3 // IN4 на 1-м драйвере ULN2003
// инициализируем ШД 28BYJ-48с выводами
// IN1-IN3-IN2-IN4 для использования библиотеки AccelStepper
AccelStepper stepper1(HALFSTEP, motorPin1, motorPin3, motorPin2, motorPin4);

void setup()
{
    Serial.begin(9600); // скорость обмена по послед. порту
    stepper1.setMaxSpeed(3000.0); // максимальная скорость ШД
    stepper1.setAcceleration(1000.0); // ускорение ШД
    stepper1.setSpeed(1000); // начальная скорость ШД
}

void loop()
{
    // принимаем четыре символа цифр
    // и формируем строку st
    st="";
    if(Serial.available())
    {
        delay(10);
        s=Serial.read();
        delay(10);
        st=st+s;
        s=Serial.read();
        delay(10);
        st=st+s;
        s=Serial.read();
        delay(10);
        st=st+s;
        s=Serial.read();
        delay(10);
        st=st+s;
        delay(10);
        x=st.toInt();
        Serial.print ("x=");
        Serial.println(x);
    }

    // вращаем вал мотора в заданном направлении
    if (x<=512)

```

```
        {  
            stepper1.moveTo(512-x);  
            stepper1.run();  
        }  
    if (x>512)  
    {  
        stepper1.moveTo(512-x);  
        stepper1.run();  
    }  
}
```

4. ГИРОСКОПЫ И АКСЕЛЕРОМЕТРЫ В СИСТЕМАХ УПРАВЛЕНИЯ МОБИЛЬНЫМИ РОБОТАМИ

4.1. Общие сведения

Гирископ

Трехосевой гироскоп – датчик поворота объекта, позволяющий вычислить углы поворотов по осям X, Y, Z: Roll (крен), Pitch (Тангаж) и Yaw (рыскание) (рис. 4.1).

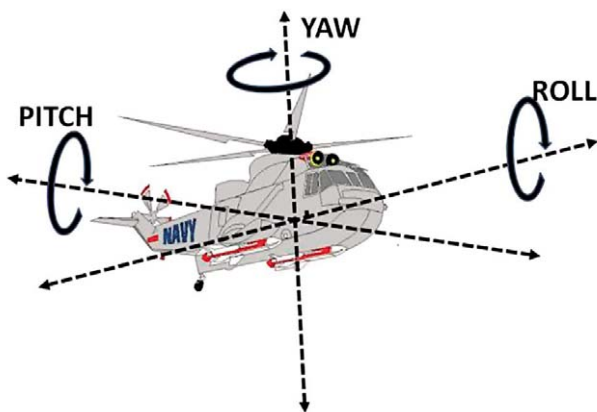


Рис. 4.1

Для правильной обработки параметров движения и верного распознавания динамических параметров применяют совместно акселерометр и гироскоп. При использовании гироскопа в рассмотренном случае он определит угол поворота и даст возможность правильно интерпретировать данные акселерометра. Использование гироскопа без акселерометра невозможно из-за особенностей математики гироскопа приводящих к накоплению погрешности. Специальная математика позволяет объединить обработку данных от обоих датчиков. Гироскопы и акселерометры широко применяются в авиации, ракетной, космической и военной технике. Например. Произошел захват цели прицелом танка во время движения. Перемещение по пересеченной местности вызывает мощные искажения угла подъема ствола орудия. Зафиксировать прицел на захваченной цели помогает акселерометр и гироскоп.

Акселерометр

Трехосевой акселерометр «чувствует» проекции ускорения на оси X, Y и Z (рис. 4.2).

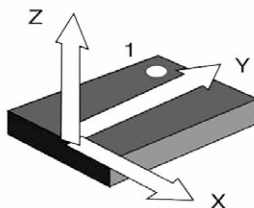


Рис. 4.2

Если объект размещен строго горизонтально и не движется, то проекции ускорения силы тяжести на оси X и Y равны нулю. Сила тяготения воспринимается только чувствительными элементами вертикальной оси Z. Время от времени в состоянии покоя производят проверку и калибровку акселерометра. Во время движения объект постоянно то ускоряется, то замедляется. Идеально равномерного движения не существует. Это и позволяет использовать акселерометр не только для определения положения объекта, но и для определения динамических параметров при движении. Акселерометр регистрирует сумму ускорения при движении и гравитацию.

4.2. Модуль GY-521 с гироскопом, акселерометром и термометром на базе микросхемы MPU-6050

GY-521 (рис. 4.3) – модуль с гироскопом, акселерометром и термометром на базе микросхемы MPU-6050 используется в робототехнике для определения положения в пространстве.

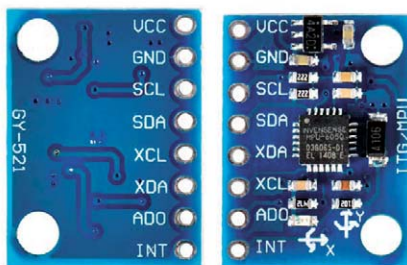


Рис. 4.3

Модуль GY-521 построен на базе микросхемы MPU6050. На плате модуля также расположена необходимая обвязка MPU6050, включая подтягивающие резисторы интерфейса I2C. Гироскоп используется для измерения линейных ускорений, а акселерометр – угловых скоростей. Совместное использование акселерометра и гироскопа позволяет определить движение тела в трехмерном пространстве.

Характеристики модуля GY-521 (MPU6050)

Питание: 3,5–6 В;

Ток потребления: 500 мкА;

Акселерометр, диапазон измерений: $\pm 2 \pm 4 \pm 8 \pm 16$ g;

Гироскоп, диапазон измерений: ± 250 500 1000 2000 °/s;

Интерфейс: I2C.

Назначение контактов

VCC – напряжение питания;

GND – общий провод;

SCL – тактовый сигнал I2C;

SDA – данные I2C;

XDA – данные шины I2C при работе в режиме мастера;

XCL – тактовый сигнал шины I2C при работе в режиме мастера;

AD0 – бит 0 адреса I2C;

INT – выход сигнала о готовности данных для использования, как внешнего прерывания микроконтроллера.

Подключение к плате Arduino

Подключение модуля к плате Arduino осуществляется по интерфейсу I2C. Схема подключения показана на рис. 4.4, 4.5.

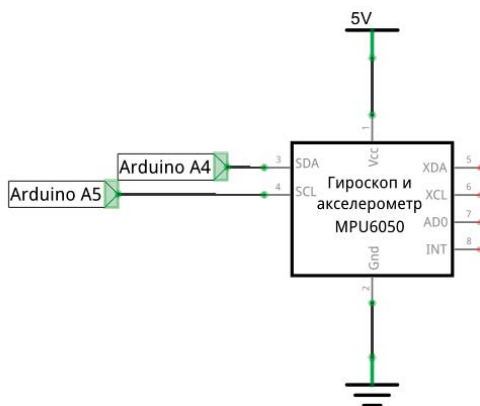


Рис. 4.4

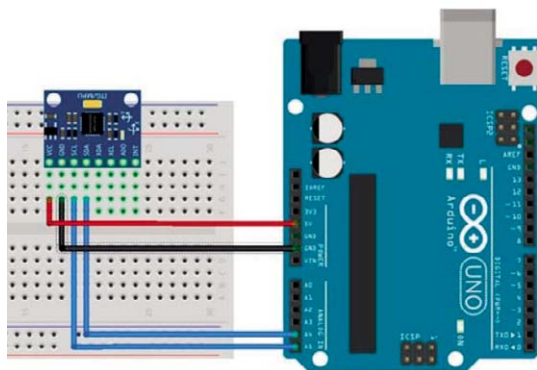


Рис. 4.5

Таблица подключения модуля GY-521 к контроллеру приведена ниже.

GY-521	GND	VCC	SDA	SCL
Arduino Uno	GND	+5V или +3,3V	A4	A5

В одних источниках напряжение питания модуля указывается строго +3,3В. В других: +5В. Здесь нужно проявлять осторожность. Ранее было отмечено, что модуль подключается к контроллеру с помощью интерфейса (шины) I2C. Этому интерфейсу посвящается следующий раздел.

4.3. Шина I2C

Общей особенностью электронных систем является необходимость обмена информацией между двумя и более отдельными компонентами – модулями, блоками, узлами, микросхемами. Разработан ряд интерфейсов и протоколов для них, которые помогают различным устройствам успешно общаться. Среди таких интерфейсов весьма популярными являются: UART, USART, SPI, I2C, CAN и ряд других. Каждый протокол имеет свои плюсы и минусы и важно немного знать о каждом из них, чтобы можно было принимать обоснованные решения при выборе компонентов или интерфейсов.

I2C (I²C, IIC) – (Inter-Integrated Circuit) – последовательная асимметричная шина для связи между интегральными схемами внутри электронных приборов. Использует две двунаправленные линии связи (SDA и SCL). Применяется для соединения низкоскоростных периферийных компонентов с процессорами и микроконтроллерами, например, на материнских платах, во встраиваемых системах, в мобильных телефонах, в системах управления роботами и т. д.

История создания

Разработана фирмой Philips Semiconductors в начале 1980-х как простая 8-битная шина внутренней связи для создания управляющих систем. Была рассчитана на частоту 100 кГц.

Стандартизована в 1992 году. В первой версии к стандартному режиму 100 кбит/с добавлен скоростной режим 400 кбит/с (Fast-mode, Fm). За счёт 10-битной адресации становится возможным подключение на одну шину более 1000 устройств, количество которых ограничивается максимально допустимой ёмкостью шины – 400 пФ.

В стандарте версии 2.0 (1998 год) представлены высокоскоростной режим работы 3,4 Мбит/с (High-speed mode, Hs) и требования пониженного энергопотребления. Незначительно доработан в версии 2.1 (2000 год).

В версии 3 (2007 год) добавлен режим 1 Мбит/с (Fast-mode plus, Fm+) и механизм идентификации устройств (ID).

В версии 4 (2012 год) появился однонаправленный режим 5 Мбит/с (Ultra Fast-mode, UFM) с использованием двухтактной логики без подтягивающих резисторов, добавлена таблица предустановленных идентификаторов.

В версии 5 (2012 год) исправлены ошибки.

В версии 6 (2014 год) пересчитаны графики, определяющие величину подтягивающих резисторов в зависимости от ёмкости шины и рабочего напряжения.

Принцип организации шины I2C

Принцип организации шины показан на рис. 4.6, 4.7.

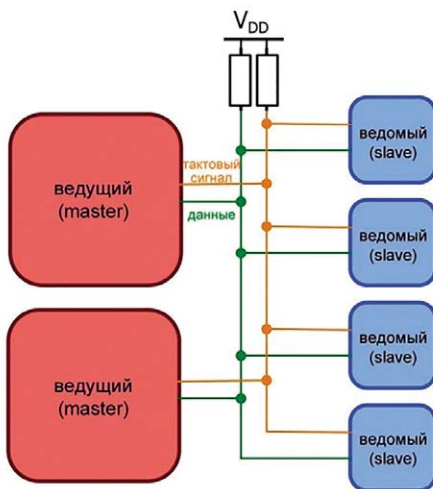


Рис. 4.6

На рис. 4.7 показаны один ведущий (master) и три ведомых (slave) устройства. Подтягивающие резисторы R_p обязательны.

Шина I2C синхронная, состоит из двух линий: данных (SDA) и тактов (SCL). Есть ведущий (master) и ведомые (slave) устройства. Инициатором обмена всегда выступает ведущий. Обмен между двумя ведомыми невозможен. Всего к шине можно подключить до 127 устройств.

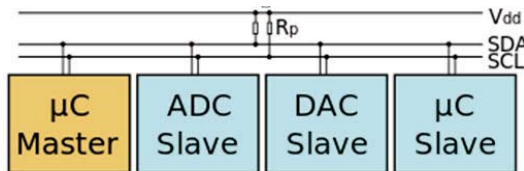


Рис. 4.7

Сигналы синхронизации – такты SCL на линии генерирует мастер. Линией SDA могут управлять как мастер так и ведомый в зависимости от направления передачи. Единицей обмена информации является пакет, обранный стартовым и стоповым условиями. Мастер в начале каждого пакета передает один байт, где указывает адрес ведомого и направление передачи последующих данных. Данные передаются 8-битными словами. После каждого слова передается один бит подтверждения приёма приёмной стороной.

Принцип работы

Тактировка последовательности передачи данных показана на рис. 4.8.

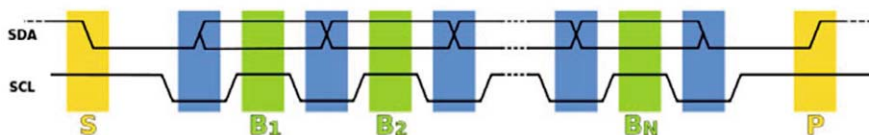


Рис. 4.8

На этом рисунке:

- S – старт обмена;
- B1, B2, ..., BN – передаваемые биты;
- P – стоп обмена.

I2C использует две двунаправленные линии, подтянутые к напряжению питания +5 В или +3,3 В, однако допускаются и другие.

Классическая адресация включает 7-битное адресное пространство с 16 зарезервированными адресами. Это означает, что разработчикам доступно до 112 свободных адресов для подключения модулей на одну шину.

Основной режим работы – 100 кбит/с и 10 кбит/с в режиме работы с пониженной скоростью. Также немаловажно, что стандарт допускает приостановку тактирования для работы с медленными устройствами.

Процесс передачи посылки. Состояние СТАРТ и СТОП

Процедура обмена начинается с того, что мастер формирует состояние СТАРТ: при ВЫСОКОМ уровне на линии SCL он генерирует переход сигнала линии SDA из ВЫСОКОГО состояния в НИЗКОЕ. Этот переход воспринимается всеми устройствами, подключенными к шине, как признак начала процедуры обмена. Генерация синхросигнала – это всегда обязанность ведущего. При передаче посылок по шине I²C каждый ведущий генерирует свой синхросигнал на линии SCL. После формирования состояния СТАРТ ведущий опускает состояние линии SCL в НИЗКОЕ состояние и выставляет на линию SDA старший бит первого байта сообщения. Количество байт в сообщении не ограничено. Спецификация шины I²C разрешает изменения на линии SDA только при НИЗКОМ уровне сигнала на линии SCL. Данные действительны и должны оставаться стабильными только во время ВЫСОКОГО состояния синхроимпульса.

Для подтверждения приёма байта от ведущего-передатчика ведомым-приёмником в спецификации протокола обмена по шине I²C вводится специальный бит подтверждения, выставляемый на шину SDA после приёма 8 бит данных.

Процедура обмена завершается тем, что ведущий формирует состояние СТОП – переход состояния линии SDA из НИЗКОГО состояния в ВЫСОКОЕ при ВЫСОКОМ состоянии линии SCL. Состояния СТАРТ и СТОП всегда вырабатываются ведущим. Считается, что шина занята после фиксации состояния СТАРТ. Шина считается освободившейся через некоторое время после фиксации состояния СТОП.

Подтверждение

Передача 8 бит данных от передатчика к приёмнику завершаются дополнительным циклом – формированием 9-го тактового импульса на линии SCL, при котором приёмник выставляет низкий уровень сигнала на линии SDA, как признак успешного приёма байта.

Подтверждение при передаче данных обязательно, кроме случаев окончания передачи ведомой стороной. Соответствующий импульс синхронизации генерируется ведущим. Передатчик отпускает (переводит в ВЫСОКОЕ состояние) линию SDA на время синхроимпульса подтверждения. Приёмник должен удерживать линию SDA в течение ВЫСОКОГО состояния синхроимпульса подтверждения в стабильном НИЗКОМ состоянии.

В том случае, когда ведомый-приёмник не может подтвердить свой адрес (например, когда он выполняет в данный момент какие-либо функции реального времени), линия данных должна быть оставлена в ВЫСОКОМ состоянии. После этого ведущий может выдать состояние СТОП для прерывания

пересылки данных. Если в пересылке участвует ведущий-приёмник, то он должен сообщить об окончании передачи ведомому-передатчику путём неподтверждения последнего байта. Ведомый-передатчик должен освободить линию данных для того, чтобы позволить ведущему выдать состояние СТОП или повторить состояние СТАРТ.

Синхронизация

Синхронизация выполняется с использованием подключения к линии SCL по правилу монтажного И. Это означает, что ведущий не имеет монопольного права на управление переходом линии SCL из НИЗКОГО состояния в ВЫСОКОЕ. В том случае, когда ведомому необходимо дополнительное время на обработку принятого бита, он имеет возможность удерживать линию SCL в низком состоянии до момента готовности к приёму следующего бита. Таким образом, линия SCL будет находиться в НИЗКОМ состоянии на протяжении самого длинного НИЗКОГО периода синхросигналов.

Устройства с более коротким НИЗКИМ периодом будут входить в состояние ожидания на время, пока не кончится длинный период. Когда у всех задействованных устройств кончится НИЗКИЙ период синхросигнала, линия SCL перейдет в ВЫСОКОЕ состояние. Все устройства начнут проходить ВЫСОКИЙ период своих синхросигналов. Первое устройство, у которого кончится этот период, снова установит линию SCL в НИЗКОЕ состояние. Таким образом, максимальный период синхролинии SCL определяется наидлиннейшим периодом синхронизации из всех задействованных устройств, а минимальный период определяется самым коротким периодом синхронизации устройств.

Механизм синхронизации может быть использован приёмниками как средство управления пересылкой данных на байтовом и битовом уровнях.

На уровне байта, если устройство может принимать байты данных с большой скоростью, но требует определенное время для сохранения принятого байта или подготовки к приёму следующего, то оно может удерживать линию SCL в НИЗКОМ состоянии после приёма и подтверждения байта, переводя таким образом передатчик в состояние ожидания.

На уровне битов устройство, такое, как микроконтроллер без встроенных аппаратных цепей I²C или с ограниченными цепями, может замедлить частоту синхроимпульсов путём продления их НИЗКОГО периода. Таким образом скорость передачи любого ведущего адаптируется к скорости медленного устройства.

Адресация в шине I2C

Каждое устройство, подключённое к шине, может быть программно адресовано по уникальному адресу. Для выбора приёмника сообщения ведущий использует уникальную адресную компоненту в формате посылки. В обычном режиме используется 7-битная адресация.

Процедура адресации на шине I2C заключается в том, что первый байт после сигнала СТАРТ определяет, какой ведомый адресуется ведущим для

проведения цикла обмена. Исключение составляет адрес «Общего вызова», который адресует все устройства на шине. Когда используется этот адрес, все устройства в теории должны послать сигнал подтверждения. Однако устройства, которые могут обрабатывать «общий вызов», на практике встречаются редко.

Первые семь битов первого байта образуют адрес ведомого. Восьмой, младший бит, определяет направление пересылки данных. «Ноль» означает, что ведущий будет записывать информацию в выбранного ведомого. «Единица» означает, что ведущий будет считывать информацию из ведомого.

После того, как адрес послан, каждое устройство в системе сравнивает первые семь бит после сигнала СТАРТ со своим адресом. При совпадении устройство полагает себя выбранным как ведомый-приёмник или как ведомый-передатчик, в зависимости от бита направления.

Адрес ведомого может состоять из фиксированной и программируемой части. Часто случается, что в системе будет несколько однотипных устройств (к примеру, ИМС памяти или драйверов светодиодных индикаторов), поэтому при помощи программируемой части адреса становится возможным подключить к шине максимально возможное количество таких устройств.

Все специализированные ИМС, поддерживающие работу в стандарте шины I²C, имеют набор фиксированных адресов, перечень которых указан производителем в описаниях контроллеров.

Комбинация бит 11110XX адреса зарезервирована для 10-битной адресации.

Как следует из спецификации шины, допускаются как простые форматы обмена, так и комбинированные, когда в промежутке от состояния СТАРТ до состояния СТОП ведущий и ведомый могут выступать и как приёмник, и как передатчик данных. Комбинированные форматы могут быть использованы, например, для управления последовательной памятью.

В любом случае по спецификации шины все разрабатываемые устройства должны сбрасывать логику шины при получении сигнала СТАРТ или повторный СТАРТ и подготавливаться к приёму адреса.

Тем не менее, основные проблемы с использованием I²C шины возникают именно из-за того, что разработчики, «начинающие» работать с I²C шиной, не учитывают того факта, что ведущий (часто – микропроцессор) не имеет монопольного права ни на одну из линий шины.

Возможные проблемы на шине

Для того, чтобы оценить технические приемы, которые делают I²C эффективной, нужно подумать о трудностях достижения надежной, но универсальной связи между несколькими независимыми компонентами. Ситуация достаточно проста, если у вас есть одна микросхема, которая всегда является ведущей (master), и одна микросхема, которая всегда является ведомой (slave). Но что, если у вас есть несколько ведомых? Что если ведомые не знают, кто ведущий? Что, если у вас есть несколько ведущих? Что произойдет, если

ведущий запросит данные у ведомого устройства, которое по какой-то причине перестало функционировать? Или что, если ведомый перестал функционировать в середине передачи? Что делать, если ведущий утверждает, что шина осуществляет передачу, а затем он выйдет из строя, прежде чем освободить шину? Это лишь небольшая часть коллизий, которые могут произойти на шине (рис. 4.4).

Применение

I2C находит применение в устройствах, предусматривающих простоту разработки и низкую себестоимость изготовления при относительно неплохой скорости работы. Список возможных применений:

- доступ к модулям памяти NVRAM;
- доступ к низкоскоростным ЦАП/АЦП;
- регулировка контрастности, насыщенности и цветового баланса мониторов;
- регулировка звука в динамиках;
- управление светодиодами, в том числе в мобильных телефонах;
- чтение информации с датчиков мониторинга и диагностики оборудования, например, термостат центрального процессора или скорость вращения вентилятора охлаждения;
- чтение информации с часов реального времени;
- информационный обмен между микроконтроллерами;
- информационный обмен между микроконтроллерами и модулями.

Преимущества

- необходим всего один микроконтроллер для управления набором устройств;
- используется всего два проводника для подключения многих устройств;
- возможна одновременная работа нескольких ведущих (master) устройств, подключенных к одной шине I2C;
- стандарт предусматривает «горячее» подключение и отключение устройств в процессе работы системы;
- встроенный в микросхемы фильтр подавляет всплески, обеспечивая целостность данных;
- адаптируется к потребностям разных ведомых устройств;
- включает в себя функционал ACK/NACK для улучшения обработки ошибок.

Недостатки

- ограничение на ёмкость линии – 400 пФ;
- несмотря на простоту протокола, программирование контроллера I2C затруднено из-за изобилия возможных нештатных ситуаций на

шине. По этой причине большинство систем используют I2C с единственным ведущим (Master) устройством, и распространённые драйверы поддерживают только монополярный режим обмена по I2C;

- трудность локализации неисправности, если одно из подключенных устройств ошибочно устанавливает на шине состояние низкого уровня.

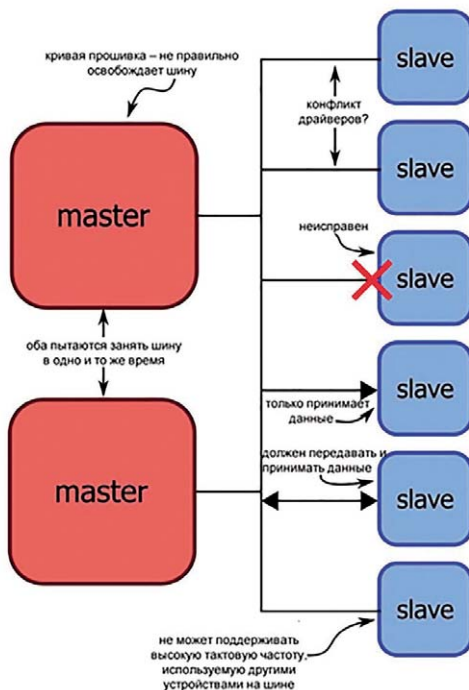


Рис. 4.9

Резюме

- Используется только два сигнала (тактовая синхронизация и данные) независимо от того, сколько устройство подключено к шине.
- Оба сигнала подтягиваются к положительному напряжению питания через резисторы, соответствующих номиналов.
- Каждое устройство взаимодействует с сигналами данных и тактовой синхронизации через драйверы вывода с открытым стоком или с открытым коллектором.
- Каждое ведомое устройство идентифицируется с помощью 7-битного адреса; устройство мастер должно знать эти адреса, чтобы общаться с конкретным ведомым устройством.

- Все передачи инициируются и прекращаются мастером; мастер может передавать данные одному или нескольким ведомым устройствам или запрашивать данные из ведомого устройства.
- Метки «ведущий-master» и «ведомый-slave» по своей сути непостоянны: любое устройство может функционировать и как ведущее, и как ведомое устройство, если оно содержит необходимое аппаратное и/или программное обеспечение. На практике, однако, встраиваемые системы часто используют архитектуру, в которой один мастер отправляет команды или собирает данные с нескольких ведомых устройств.
- Сигнал данных обновляется по заднему фронту тактового сигнала, а его выборка происходит по переднему фронту следующим образом: Временная диаграмма сигналов шины I2C.
- Данные передаются в однобайтовых секциях, причем каждый байт сопровождается однобитным сигналом подтверждения, называемым битом ACK/NACK (подтверждение или неподтверждение).

4.4. Библиотека Wire

Библиотека Wire используется для связи контроллера с устройствами и модулями через интерфейс I2C. Интерфейс I2C для передачи данных используют множество устройств.

Для связи по I2C используется всего два контакта: линия данных (SDA) и линия тактового сигнала (SCL). К соответствующим пинам Arduino можно подключить до 120 устройств, поддерживающих интерфейс I2C. Для обмена данными с такими устройствами и нужна Arduino библиотека Wire.

Расположение пинов SDA и SCL на разных платах Arduino может отличаться. Смотрите описание вашей платы контроллера, чтобы не допустить ошибку при подключении. В списке ниже указано расположение пинов I2C популярных плат Arduino.

- Arduino Nano: A4-SDA, A5-SCL
- Arduino Uno: A4-SDA, A5-SCL продублированы рядом с разъемом AREF (только версия R3)
- Arduino Mega: 20-SDA, 21-SCL продублированы рядом с разъемом AREF (только версия R3)
- Arduino Pro Mini: A4-SDA, A5-SCL
- Arduino Leonardo: 2-SDA, 3-SCL и пины рядом с разъемом AREF (второй канал)

Wire идет в комплекте стандартных библиотек и устанавливается вместе с Arduino IDE. Но ее можно скачать и отдельно.

Для установки библиотеки просто распакуйте zip архив в папку «C:\Program Files (x86)\Arduino\libraries» или в то место, где у вас установлена среда разработки Arduino IDE. Если у вас запущена программа Arduino IDE, то для работы с новой библиотекой её необходимо перезапустить. Рассмотрим функции библиотеки Wire.

begin()

Инициализирует библиотеку Wire и подключается к шине I2C как ведущий (мастер) или ведомый.

Синтаксис: `Wire.begin(address);`

Параметры:

- `address` – необязательный параметр: 7-битный адрес ведомого устройства; если не задан, плата подключается к шине, как мастер.

Возвращаемые значения: нет

Пример:

```
#include <Wire.h> // Подключаем библиотеку
void setup()
{
    Wire.begin(); // Подключаемся к шине i2c, как мастер
}

void loop()
{
}
```

requestFrom()

Отправляет запрос на определенное количество байтов от ведущего устройства к ведомому.

Синтаксис: `Wire.requestFrom(address, quantity, stop);`

Параметры:

- `address` – обязательный параметр: 7-битный адрес устройства к которому посылается запрос;
- `quantity` – обязательный параметр: количество запрашиваемых байт.
- `stop` – необязательный параметр типа `boolean`: `true` – после запроса отправляет STOP, освобождая шину I2C; `false` – после запроса отправляет RESTART; шина не освобождается и можно отправлять дополнительные запросы; по умолчанию – `true`.

Возвращаемое значение: количество байт, принятых от устройства.

Пример:

```
#include <Wire.h>
byte val = 0;
void setup()
{
    // подключиться к шине i2c (адрес для мастера не обязателен)
    Wire.begin();
    // настроить последовательный порт для вывода
    Serial.begin(9600);
}

void loop()
{
    // запросить 6 байтов от ведомого устройства #2
    Wire.requestFrom(2, 6);
    // ведомое устройство может послать меньше, чем запрошено
    while(Wire.available())
    {
        char c = Wire.read(); // принять байт как символ
        Serial.print(c);      // напечатать символ
    }

    delay(500);
}
```

beginTransmission()

Открывает канал связи по шине I2C с ведомым устройством.

Синтаксис: `Wire.beginTransmission(address);`

Параметры:

- `address` – обязательный параметр: 7-битный адрес устройства к которому посылается запрос.

Возвращаемые значения: нет.

endTransmission()

Отправляет данные, которые были поставлены в очередь методом `write()` и завершает передачу.

Синтаксис: `Wire.endTransmission(stop)`

Параметры:

- `stop` – необязательный параметр типа `boolean`: `true` – после запроса отправляет STOP, освобождая шину I2C; `false` – после запроса отправляет RESTART; шина не освобождается и можно отправлять дополнительные запросы; по умолчанию – `true`.

Возвращаемые значения:

- byte указывает на состояние передачи: 0 – успешная передача; 1 – объем данных для передачи слишком велик; 2 – принят NACK при передаче адреса; 3 – принят NACK при передаче данных; 4 – другие ошибки.

Пример:

```
#include <Wire.h>
byte val = 0;
void setup()
{
    Wire.begin(); // подключиться к шине i2c
}

void loop()
{
    // передача на устройство #44 (0x2c);
    //адрес устройства задан в техническом описании
    Wire.beginTransmission(44);
    Wire.write(val);      // отправить байт значения
    Wire.endTransmission(); // остановить передачу
}
```

write()

Ставит данные в очередь для передачи.

Синтаксис: Wire.write(data, length);

Параметры:

- data – обязательный параметр: байт или массив байтов для передачи.
- length – необязательный параметр: длина передаваемого массива данных, используется только при передаче массива в первом параметре.

Возвращаемые значения: количество записанных байт.

Пример:

```
#include <Wire.h>
byte val = 0;

void setup()
{
    Wire.begin(); // подключиться к шине i2c
}

void loop()
```

```

{
    // передача на устройство #44 (0x2c);
    //адрес устройства задан в техническом описании
    Wire.beginTransmission(44);
    Wire.write(val);      // отправить байт
    Wire.endTransmission(); // остановить передачу
}

```

available()

Возвращает количество байт, доступных для чтения.

Синтаксис: Wire.available();

Параметры: нет.

Возвращаемые значения: количество байт доступных для считывания.

Пример:

```

#include <Wire.h>
byte val = 0;

void setup()
{
    Wire.begin();      // подключиться к шине i2c
    Serial.begin(9600); //настроить последовательный порт для вывода
}

void loop()
{
    // запросить 6 байтов от ведомого устройства #2
    Wire.requestFrom(2, 6);

    // ведомое устройство может послать меньше, чем запрошено
    while(Wire.available())
    {
        char c = Wire.read(); // принять байт как символ
        Serial.print(c);      // вывести символ на экран
    }

    delay(500);
}

```

read()

Считывает байт переданной информации.

Синтаксис: Wire.read();

Параметры: нет.

Возвращаемые значения: принятый байт.

Пример:

```
#include <Wire.h>
byte val = 0;

void setup()
{
    // подключиться к шине i2c (адрес для мастера не обязателен)
    Wire.begin();
    // настроить последовательный порт для вывода
    Serial.begin(9600);
}

void loop()
{
    // запросить 6 байтов от ведомого устройства #2
    Wire.requestFrom(2, 6);
    // ведомое устройство может послать меньше, чем запрошено
    while(Wire.available())
    {
        char c = Wire.read(); // принять байт как символ
        Serial.print(c); // вывести символ на экран
    }

    delay(500);
}
```

setClock()

Устанавливает тактовую частоту обмена данными по интерфейсу I2C.

Синтаксис: `Wire.setClock(clockFrequency);`

Параметры:

- `clockFrequency` – обязательный параметр: новое значение частоты обмена данными в герцах; доступные значения: 10000 – медленный режим, 100000 – стандартное значение, 400000 – быстрый режим, 1000000 – быстрый режим плюс, 3400000 – высокоскоростной режим; необходимо убедиться, что выбранный режим поддерживается вашим процессором, обратившись к документации от производителя.

Возвращаемые значения: нет.

onReceive()

Добавляет функцию обработчик, которая будет выполняться при получении данных от ведущего устройства.

Синтаксис: `Wire.onReceive(handler);`

Параметры:

- `handler` – имя функции, которая будет выполняться когда ведомое устройство принимает данные; функция должна принимать один параметр `int` и ничего не возвращать.

Возвращаемые значения: нет.

Пример:

```
#include <Wire.h>
```

```
void setup()
{
    // подключиться к i2c шине с адресом #8
    Wire.begin(8);
    // зарегистрировать обработчик события
    Wire.onReceive(receiveEvent);
    // настроить последовательный порт для вывода
    Serial.begin(9600);
}
```

```
void loop()
{
    delay(100);
}
```

```
// функция, которая будет выполняться всякий раз,
// когда от мастера принимаются данные
// данная функция регистрируется как обработчик события,
// смотрите setup()
```

```
void receiveEvent(int howMany)
{
    while (Wire.available())
    {
        char c = Wire.read(); // принять байт как символ
        Serial.print(c);      // вывести символ на экран
    }
}
```


onRequest()

Добавляет функцию обработчик, которая будет выполняться при получении запроса от ведущего устройства.

Синтаксис: `Wire.onRequest(handler);`

Параметры:

- `handler` – имя функции, которая будет выполняться, когда ведомое устройство получает запрос от ведущего. Функция не принимает параметров и ничего не возвращает.

Возвращаемые значения: нет.

```
#include <Wire.h>

void setup()
{
    // подключиться к i2c шине с адресом #8
    Wire.begin(8);
    // зарегистрировать обработчик события
    Wire.onRequest(requestEvent);
}

void loop()
{
    delay(100);
}

//функция, которая будет выполняться всякий раз,
// когда мастером будут запрошены данные.
// данная функция регистрируется,
//как обработчик события, смотрите setup()
void requestEvent()
{
    Wire.write("hello "); // ответить сообщением
}
```

4.5. Библиотека Servo

Ряд примеров, приведенных ниже, будет использовать библиотеку Servo, поэтому рассмотрим особенности этой библиотеки. Библиотека Servo представляет собой набор функций для управления сервоприводами. Данная библиотека дает возможность управлять сразу двенадцатью сервоприводами с помощью большинства контроллеров Arduino. Некоторые платы Arduino позволяют подключать меньше сервоприводов (например, Arduino LilyPad), так

как у них меньше цифровых пинов. Другие платы дают возможность управлять сразу 48 сервоприводами (Arduino Mega).

Использование библиотеки Servo накладывает некоторые ограничения. На всех платах, кроме Arduino Mega, при работе с данной библиотекой, пропадает возможность использовать цифровые пины 9 и 10 в режиме ШИМ. На плате Arduino Mega режим ШИМ становится недоступен на пинах 11 и 12 только при подключении более 12 сервоприводов.

Далее в тексте – сервопривод и сервомашинка – одно и то же.

Данная библиотека автоматически устанавливается вместе с Arduino IDE. Но можно отдельно скачать библиотеку Servo для Arduino. Для установки библиотеки просто распакуйте zip архив в папку «C:\Program Files (x86)\Arduino\libraries» или в то место, где у вас установлена среда разработки Arduin IDE. Если у вас запущена программа Arduino IDE, то для работы с новой библиотекой её необходимо перезапустить.

После того, как вы скачали библиотеку Servo и установили ее, вы можете подключать библиотеку в свои скетчи и вам будут доступны примеры использования данной библиотеки. Для использования библиотеки Servo необходимо подключить ее в свой скетч и создать переменную типа servo. Сделать это очень просто:

```
#include <Servo.h>
```

```
Servo myservo;// myservo – имя сервопривода (сервомашинки)
```

```
void setup()
{
    программный код
}
```

```
void loop()
{
    программный код
}
```

Рассмотрим функции библиотеки Servo.

attach()

Указывает пин, к которому подключен сервопривод.

Синтаксис: servo.attach(pin, min, max);

Параметры:

- pin – обязательный параметр: цифровой пин, к которому подключен сигнальный провод сервопривода;

- `min` – необязательный параметр: ширина импульса в микросекундах, соответствующая минимальному (угол 0 градусов) положению сервопривода (по умолчанию 544);
- `max` – необязательный параметр: ширина импульса в микросекундах, соответствующая максимальному (угол 180 градусов) положению сервопривода.

Возвращаемые значения: нет.

Пример:

```
#include <Servo.h>

Servo myservo;

void setup()
{
    myservo.attach(9);
}

void loop()
{
    программный код
}
```

`write()`

Поворачивает сервопривод на заданный угол. Для сервоприводов постоянного вращения устанавливает скорость и направление вращения.

Синтаксис: `servo.write(angle);`

Параметры:

- `angle` – обязательный параметр: устанавливает угол от 0 до 180 градусов; при использовании сервопривода постоянного вращения значение 90 используется для неподвижного состояния, значение 0 – для максимальной скорости вращения в одну сторону, а 180 – для максимальной скорости вращения в другую сторону.

Возвращаемые значения: нет.

Пример:

```
#include <Servo.h>

Servo myservo;

void setup()
{
    myservo.attach(9);
}
```

```

        // Поворачиваем сервопривод в среднее положение
        myservo.write(90);
    }

void loop()
{
    программный код
}

```

writeMicroseconds()

Поворачивает сервопривод на угол заданный в микросекундах. С сервоприводами постоянного вращения работает по такому же принципу как и функция write().

Синтаксис servo.writeMicroseconds(ms);

Параметры:

- ms — обязательный параметр: значение в микросекундах

Возвращаемые значения: нет.

Пример:

```

#include <Servo.h>
Servo myservo;

void setup()
{
    myservo.attach(9);
    // Поворачивает сервопривод в среднее положение
    myservo.writeMicroseconds(1500);
}

void loop()
{
    программный код
}

```

read()

Возвращает текущее положение сервопривода.

Синтаксис: servo.read();

Параметры: нет.

Возвращаемые значения: целые числа от 0 до 180.

Пример:

```
#include <Servo.h>
Servo myservo;

void setup()
{
    // Открываем последовательный порт
    Serial.begin(9600);
    myservo.attach(9);
    // Считываем положение сервопривода
    int position = myservo.read();
    Serial.print("Текущее положение сервопривода: ");
    // Выводим значение угла на экран
    Serial.println(position);
}

void loop()
{
    программный код
}
```

attached()

Проверяет, указан ли управляющий пин для экземпляра класса Servo.

Синтаксис: `servo.attached()`;

Параметры: нет.

Возвращаемые значения: boolean; true – если пин был указан и false – если нет.

Пример:

```
#include <Servo.h>
Servo myservo;

void setup()
{
    // Открываем последовательный порт
    Serial.begin(9600);
    if(!myservo.attached())
    {
        // Указываем пин, если не указан был раньше
        myservo.attach(9);
    }
}
```

```
void loop()
{
    программный код
}
```

detach()

Отсоединяет экземпляр класса от пина. При отсоединения всех сервоприводов, заблокированные ШИМ выводу снова станут доступны.

Синтаксис: `servo.detach();`

Параметры: нет.

Возвращаемые значения: нет.

Пример:

```
#include <Servo.h>
Servo myservo;
```

```
void setup()
{
    Serial.begin(9600); // Открываем последовательный порт
    // Указываем пин сервопривода
    myservo.attach(9);
    // Считываем положение сервопривода
    int position = myservo.read();
    Serial.print("Текущее положение сервопривода: ");
    // Выводим значение угла на экран
    Serial.println(position);
    // Освобождаем пин, к которому был подключен сервопривод
    myservo.detach();
}

void loop()
{
    программный код
}
```

4.6. Примеры использования модуля GY-521

Ранее было показано, что за подключение модуля GY-521 к контроллеру Arduino через интерфейс I2C отвечают функции библиотеки Wire. За управление сервомашинками отвечают функции библиотеки Servo. А вот за считывание информации непосредственно с датчика (микросхемы) MPU6050

отвечает библиотека MPU6050. К сожалению, информация о функциях этой библиотеки в Интернете практически отсутствует, поэтому будем пользоваться информацией, которая имеется в немногочисленных работоспособных скетчах.

Первое разочарование: модуль GY-521 не дает информации об угле поворота модуля относительно оси Z – курс, рысканье. А ведь для роботов именно этот параметр является главным для стабилизации движения в каком-либо направлении по прямой.

Зато модуль великолепно вычисляет углы крена и тангажа, что важно для авиации. Угол тангажа можно использовать для стабилизации танковых орудий в вертикальной плоскости.

Второе разочарование: многочисленные интернет-форумы, споры на них, не решили проблемы определения угла рысканья, хотя сил на достижение этой цели было затрачено огромное количество.

Третье разочарование: многочисленные примеры, даже с использованием сложных библиотек Kalman и I2Cdev, не позволили добиться прорыва в определении угла рысканья. Поэтому здесь приводятся скетчи, которые хорошо себя зарекомендовали. Их немного и они используют библиотеку MPU6050.

Модуль GY-521 перед работой должен быть откалиброван, но в наших примерах калибровка не требуется. Видимо, она выполняется автоматически при выполнении функций библиотеки MPU6050.

Пример. Поворачивать в реальном времени вал сервомашинки на угол, равный углу поворота модуля вокруг оси Y. Скетч примера (gyr3.ino) приведен ниже.

```
//поворот вала сервомашинки на угол,  
// задаваемый поворотом датчика GY-521 вокруг оси Y
```

```
//подключаем три библиотеки  
#include <Wire.h>  
#include <MPU6050.h>  
#include <Servo.h>
```

```
Servo sg90;      // сервомашинка  
int servo_pin = 3; //пин для сервомашинки
```

```
MPU6050 sensor ; //датчик  
int16_t ax, ay, az ; //углы поворота вокруг осей  
int16_t gx, gy, gz ; //ускорения поворота вокруг осей
```

```
void setup ()  
{  
  sg90.attach ( servo_pin );  
  Wire.begin ( );  
  Serial.begin (9600);
```

```

Serial.println ( "Инициализация датчика" );
sensor.initialize ( );
Serial.println (sensor.testConnection ( ) ? "Соединение в норме" :
"Ошибка соединения");
delay (1000);
Serial.println ( "Считываем значения датчика" );
delay (1000);
}

void loop ()
{
    //считываем значения всех показаний с датчика
    sensor.getMotion6 (&ax, &ay, &az, &gx, &gy, &gz);
    //приводим значения ax, ay, az к диапазону 0°-180°
    ax= map (ax, -17000, 17000,180, 0) ;
    ay= map (ay, -17000, 17000,180, 0) ;
    az= map (az, -17000, 17000,180, 0) ;
    //Serial.print (ax); //используем при отладке
    //Serial.println (ay); //используем при отладке
    //Serial.println (ay); //используем при отладке
    //поворачиваем вал сервомашинки на угол ay
    sg90.write (ay);
    delay (100);
}

```

По аналогии напомним скетч, который поворачивает вал сервомашинки, на угол поворота модуля вокруг оси X.

Пример. Поворачивать в реальном времени вал сервомашинки на угол, равный углу поворота модуля вокруг оси X. Скетч примера (gyr4.ino) приведен ниже.

```

//поворот вала сервомашинки на угол,
// задаваемый поворотом датчика GY-521 вокруг оси X

//подключаем три библиотеки
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

Servo sg90;      // сервомашинка
int servo_pin = 3; //пин для сервомашинки

MPU6050 sensor ; //датчик
int16_t ax, ay, az ; //углы поворота вокруг осей
int16_t gx, gy, gz ; //ускорения поворота вокруг осей

```



```

void setup ()
{
    sg90.attach ( servo_pin );
    Wire.begin ( );
    Serial.begin (9600);
    Serial.println ( "Инициализация датчика" );
    sensor.initialize ( );
    Serial.println (sensor.testConnection ( ) ? "Соединение в норме" :
"Ошибка соединения");
    delay (1000);
    Serial.println ( "Считываем значения датчика" );
    delay (1000);
}

void loop ()
{
    //считываем значения всех показаний с датчика
    sensor.getMotion6 (&ax, &ay, &az, &gx, &gy, &gz);
    //приводим значения ax, ay, az к диапазону 0°-180°
    ax= map (ax, -17000, 17000,180, 0) ;
    ay= map (ay, -17000, 17000,180, 0) ;
    az= map (az, -17000, 17000,180, 0) ;
    //Serial.print (ax); //используем при отладке
    //Serial.println (ay); //используем при отладке
    //Serial.println (ay); //используем при отладке
    //поворачиваем вал сервомашинки на угол ax
    sg90.write (ax);
    delay (100);
}

```

А что же делать с курсом? Как его измерить? Для этого используется другой модуль GY-273 – магнитометр. Но многочисленные эксперименты и наблюдения автора показали, что не всё потеряно и модуль GY-521 может быть использован для стабилизации курса.

4.7. Модуль GY-521 в задачах стабилизации курса

Для того, чтобы стабилизировать курс присмотримся повнимательней к такому параметру, как gz. Для этого подключим гироскоп к контроллеру и загрузим скетч `giroskop_1.ino`, приведенный в следующем примере.

Пример. Вывести в монитор порта среднее значение параметра gz гироскопа, находящегося в состоянии покоя.

```

// подключение библиотек
#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"
MPU6050 accelgyro;
//переменные
int16_t ax, ay, az; //углы
int16_t gx, gy, gz; //ускорения
long s; //среднее значение параметра gz

void setup()
{
    Wire.begin();
    Serial.begin(38400);
    // инициализация гироскопа
    Serial.println("Initializing I2C devices...");
    accelgyro.initialize();
    delay(100);
    s=0;
}
void loop()
{
    // чтение значений гироскопа и акселерометра
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    s=0;
    for(int i=1;i<=100;i++)
    {
        accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        s=s+gz;
    }
    s=s/100;
    Serial.println(s); // вывод среднего значения gz в монитор
}

```

Здесь в цикле считываются и суммируются 100 значений параметра gz, а затем находится их среднее значение s. Результаты работы этого скетча приведены на рис. 4.10.

Из рис. 4.10 видно, что датчик ведет себя довольно «смирно», хотя иногда можно увидеть и очень большой разброс параметра gz. Кроме того, в разные дни он может давать и другие значения, например 2300 и т. д. Еще обнаруживается, что при повороте датчика вправо и влево относительно оси z, его показания изменяются, причем тем сильнее, чем быстрее происходит поворот. Прекращение поворота приводит к результатам, показанным на рис. 4.10. Результат поворота датчика вправо показан на рис. 4.11.

В момент поворота вправо параметр gz уменьшается. Даже появляются отрицательные значения.

Результат поворота датчика влево показан на рис. 4.12.

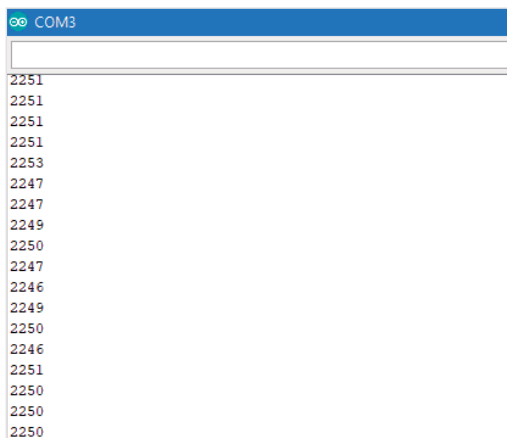


Рис. 4.10

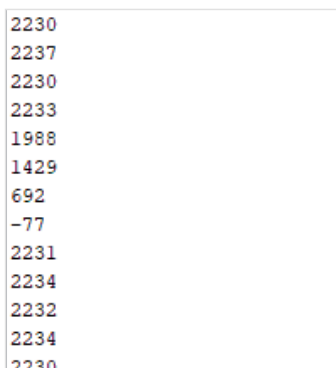


Рис. 4.11

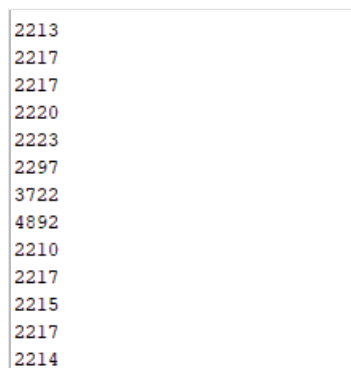


Рис. 4.12

При повороте датчика влево величина gz увеличивается. Обратите внимание на среднее значение gz в состоянии покоя. Оно различно в обоих случаях поворота (2230 и 2215), хотя измерения были проведены с разницей в несколько минут. Всё это говорит о том, что качество управления роботом будет страдать из-за такой нестабильности. На рис. 4.13 показан уход параметра gz при поворотах датчика вправо и влево. Если датчик поместить на мобильном роботе (желательно, вдоль продольной оси по центру), то его уход с заданного курса можно компенсировать поворотом робота в противоположном направлении (рис. 4.14).

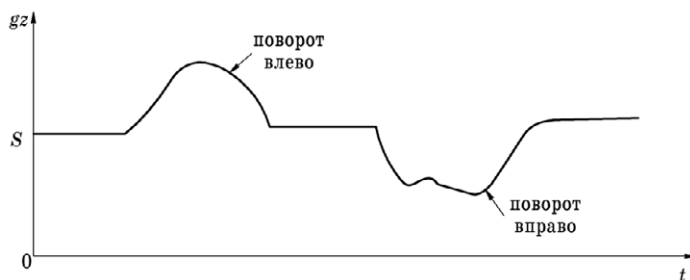


Рис. 4.13



Рис. 4.14

Идея такой компенсации заключается в следующем.

При уходе робота от заданного курса влево нужно найти площадь sl первой заштрихованной фигуры (интервалы времени $t1-t2$) и повернуть робот вправо, но ровно настолько, чтобы площадь sr второй заштрихованной фигуры (интервалы времени $t3-t4$) была равна площади sl .

При уходе робота от заданного курса вправо нужно найти площадь $sr2$ третьей заштрихованной фигуры (интервалы времени $t5-t6$) и повернуть робот влево, но ровно настолько чтобы площадь $sl2$ четвертой заштрихованной фигуры (интервалы времени $t7-t8$) была равна площади $sr2$.

Естественно, такой подход имеет недостаток. Предполагается, что внешние воздействия, вызывающие уходы робота с курса влево и вправо, обязательно заканчиваются, что видно из рис. 4.14. В реальности это может быть и не так.

Теперь возникает вопрос, как посчитать площади sl , sr , $sr2$, $sl2$? В обычном математическом смысле — очень трудно. Для этого нужно проинтегрировать сложные функции во времени. Есть очевидный способ интегрирования: нужно при вычислении величин sl , sr , $sr2$, $sl2$ суммировать все выбросы параметра gz относительно s от момента их начала до момента их окончания. Но при этом нужно знать величину s — среднее значение параметра gz . Ниже при-

веден скетч (giroskop_1.ino), который иллюстрирует нахождение среднего значения gz за 100 измерений.

```
// подключение библиотек
#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"
MPU6050 accelgyro; //имя гироскопа-акселерометра
int16_t ax, ay, az;
int16_t gx, gy, gz;
long s; // среднее значение gz

void setup()
{
    Wire.begin(); //инициализация библиотеки
    Serial.begin(38400); //скорость обмена с монитором порта
    // инициализация гироскопа
    Serial.println("Initializing I2C devices...");
    accelgyro.initialize(); //инициализация гироскопа
    delay(100);
    s=0; //начальное значение s
}

void loop()
{
    // чтение значений гироскопа и акселерометра
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    // вычисление и вывод значения s в монитор порта
    s=0;
    for(int i=1;i<=100;i++)
    {
        accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        s=s+gz;
    }
    s=s/101; //s – среднее значение gz
    Serial.println(s); //вывод s в монитор порта
}
```

На рис. 4.15 приведен алгоритм управления мобильным роботом с учетом всего сказанного выше.

В блоке 2 вычисляется значение s – среднего значения параметра gz. В блоке 3 выполняются измерение параметра gz и переменной s присваивается начальное значение, равное нулю.

В блоке 4 происходит сравнение параметра gz со средним значением s. Если $gz > s$, то это означает, что имеет место уход влево от курса и нужно вы-

числять величину sl . При этом к старому значению величины sl добавляется величина ухода, равная $gz-s$ (блоки 5 и 6) и блоки 4, 5 и 6 повторяются вновь. Это повторение продолжается до тех пор, пока в блоке 4 не перестанет выполняться условие $gz > s$.

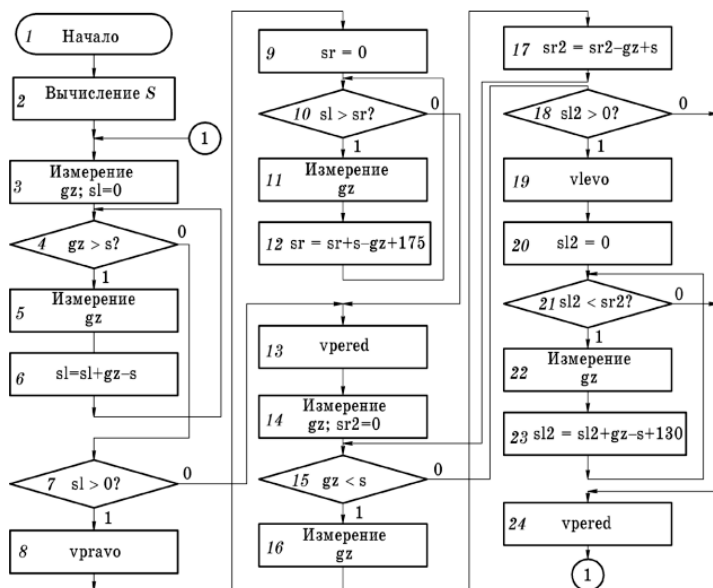


Рис. 4.15

Это означает, что уход влево от курса прекратился (момент времени t_2). Далее, в блоке 7 анализируется величина накопленного значения sl . Если оно равно или меньше нуля 0, то ухода влево не было, и робот должен двигаться прямо (блок 13) – функция $vpered$.

Если при выполнении блока 7 окажется, что величина $sl > 0$, то имел место уход влево от курса и его необходимо компенсировать поворотом робота вправо (блок 8) – функция $vpravo$ (момент времени t_3). Далее идет подготовка к расчету компенсирующего воздействия sr . При этом величина sr сначала обнуляется (блок 9), а затем вычисляется абсолютно также, как и величина sl (блоки 10, 11, 12). Только к значению sr добавляется величина отклонения $s-gz$ и еще небольшой «добавочек» в виде числа 175. Это число определяется экспериментально, когда происходит отладка робота. При отладке функция $vpered$ удаляется. Вручную робот поворачивается влево, и он самостоятельно должен повернуться в исходное положение. Если этого не происходит, то и появляется такой «добавочек». Связано это, конечно, с тем, что характеристика гироскопа несимметричная и нелинейная.

После того, как компенсирующее движение sr вправо достигло и превысило значение отклонения sl (блок 10), компенсация прекращается (момент времени $t4$) и робот выполняет движение вперед (блок 13) – функция `vpered`.

Нужно ли объяснять, что далее должны выполняться действия по анализу ситуации и принятию адекватных решений при уходе робота от курса вправо? Это выполняется в блоках 14...24. Здесь отклонение от курса вправо обозначается переменной $sr2$, а компенсирующее движение влево – переменной $sl2$. Также имеет место «добавочек», равный 130 (блок 23), полученный экспериментально. Этот фрагмент алгоритма «обслуживает» промежуток времени $t5...t8$ работы робота.

Затем весь процесс повторяется заново (соединитель 1).

Нет нужды говорить о том, что пытливый читатель может предложить свой, более эффективный, более точный, более совершенный алгоритм управления роботом. И, конечно, нужно повториться про недостаток приведенного алгоритма: он не может компенсировать отклонения от курса, которые имеют не случайный, а систематический, непрекращающийся характер. В качестве примера можно привести не идеальность колесной пары, когда одно колесо вращается чуть-чуть быстрее другого.

При случайных же воздействиях типа «пинок ботинком» виден замечательный результат – робот упорно становится на прежний курс.

И еще об одном важном нюансе. Контроллер при реализации этого алгоритма практически полностью занят и не может заниматься другой деятельностью: обслуживание датчиков и исполнительных механизмов, находящихся на борту робота. Всё это создает предпосылки по использованию на борту второго контроллера.

Скетч (`giroskop_6.ino`) этого примера приведен ниже.

```
// подключение библиотек
#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"

//переменные
MPU6050 accelgyro;
int16_t ax, ay, az;
int16_t gx, gy, gz;
long s, sr,sl,s2,sr2,sl2;

//пины
int a1 = 3; //правое колесо вперёд
int a2 = 2; // правое колесо назад
int b1 = 7; //левое колесо вперёд
int b2 = 4; // левое колесо назад
int enA =5; // правый скорость
int enB =6; // левый скорость
```

```

int lev =13; // указатель (светодиод) влево
int prav =A0; //указатель (светодиод)вправо

void setup()
{
    Wire.begin(); //инициализация библиотеки
    Serial.begin(38400); //настройка скорости работы монитора порта
    // инициализация гироскопа
    Serial.println("Initializing I2C devices...");
    accelgyro.initialize();

    sr=0;
    sl=0;

    //настройка пинов
    pinMode(b1, OUTPUT);
    pinMode(b2, OUTPUT);
    pinMode(a1, OUTPUT);
    pinMode(a2, OUTPUT);
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(lev, OUTPUT);
    pinMode(prav, OUTPUT);

    s=0; //среднее значение гироскопа в неподвижном состоянии
    for(int i=1;i<=100;i++)
    {
        accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        s=s+gz;
    }
    s=s/100;

    //робот – вперед!
    digitalWrite(b2, LOW);
    digitalWrite(a2, LOW);
    digitalWrite(a1, HIGH);
    digitalWrite(b1, HIGH);
    //Serial.println(s); используется при настройке

}

void loop()
{
    //если gz>s, ищем площадь sl

```



```

accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
sl=0;
while((gz>s))
{
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    sl=sl+gz-s;
}

//если sl>0, то был уход влево, компенсируем его поворотом вправо
if ((sl)>0)
{
    vpravo();
}

sr=0; //начальное значение площади sr
//подсчитываем площадь sr
while((sl)>(sr))
{
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    sr=sr+s+175-gz;
}

//компенсация вправо закончена, едем вперед
vpered();
//-----

//если gz<s, ищем площадь sr2
accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
sr2=0;
while((gz<s))
{
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    sr2=sr2-gz+s;
}

//если был уход вправо, компенсируем его поворотом влево
if ((sr2)>0)
{
    vlevo();
}

sl2=0; //начальное значение площади sl2
accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
//ищем значение sl2
while((sl2)<(sr2))
{
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
}

```

```

        sl2=sl2+gz+130-s;
    }
    // компенсация влево закончена, едем вперед
    vpered();
}

//поворот влево
void vlevo()
{
    analogWrite(enA, 170);
    analogWrite(enB, 0);
    digitalWrite(lev, HIGH); //включаем светодиод «влево»
    digitalWrite(prav, LOW); //выключаем светодиод «вправо»
}

//поворот вправо
void vpravo()
{
    analogWrite(enB, 170);
    analogWrite(enA, 0);
    digitalWrite(prav, HIGH); //включаем светодиод «вправо»
    digitalWrite(lev, LOW); //выключаем светодиод «влево»
}

//едем прямо
void vpered()
{
    analogWrite(enB, 200);
    analogWrite(enA, 180);
}

```

Пытливый читатель может придумать нечто более совершенное и эффективное.

5. ИНФРАКРАСНЫЕ ДАТЧИКИ В МОБИЛЬНЫХ РОБОТАХ

5.1. Общие сведения

Инфракрасное (ИК) излучение – это невидимое человеческим глазом электромагнитное излучение в диапазоне длин волн от 0,7 до 2000 мкм. Вокруг нас существуют огромное количество объектов, которые излучают в данном диапазоне. Его иногда называют «тепловое излучение», т. к. все тёплые предметы генерируют ИК-излучение. Длины волн для разных типов электромагнитного излучения приведены на рис. 5.1.

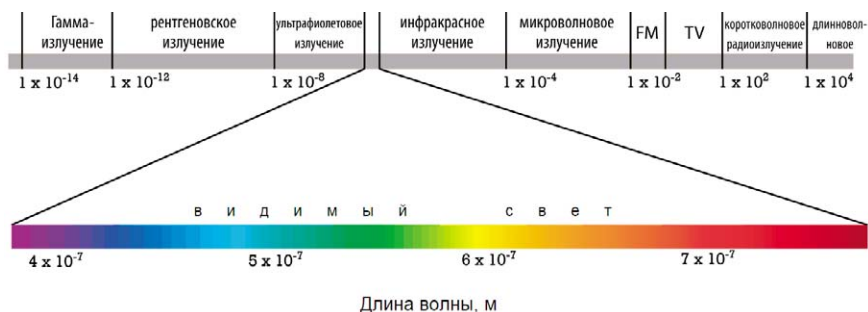


Рис. 5.1. Длины волн разных типов электромагнитного излучения

Модули на основе ИК-излучения используются, в основном, как детекторы препятствий для различного рода электронных устройств, начиная от роботов и заканчивая «умным домом». Они позволяют обнаруживать препятствия на расстоянии от нескольких сантиметров до десятков сантиметров.

Например, если оснастить мобильного робота (МР) несколькими такими ИК-модулями, то можно определять направление приближения препятствия и менять траекторию движения робота в нужном направлении.

Модуль ИК-сенсора обычно имеет излучатель (светодиод) и детектор (фотодиод) в инфракрасном диапазоне. Инфракрасный светодиод излучает в пространство ИК-излучение. Приёмник улавливает отражённое от препятствий излучение и по интенсивности отражённого излучения вычисляется расстояние до объекта. Для того, чтобы защититься от видимого излучения, фотодиод имеет светофильтр (он выглядит почти чёрным), который пропускает только волны в инфракрасном диапазоне. Разные поверхности по-разному отражают ИК-излучение, из-за чего дистанция срабатывания для разных препятствий будет отличаться.

5.2. Простейший ИК-модуль

Внешний вид ИК-модуля показан на рис. 5.2.



Рис. 5.2. Модуль с ИК-излучателем и ИК-приёмником

Когда перед сенсором нет препятствия, на выходе OUT такого модуля присутствует напряжение логической единицы. Когда сенсор детектирует отражённое от препятствия ИК-излучение, на выходе модуля напряжение становится равным логическому нулю. При этом на модуле загорается зелёный светодиод.

Помимо инфракрасного свето- и фотодиода важная часть модуля – это компаратор LM393. С помощью компаратора сенсор сравнивает интенсивность отражённого излучения с некоторым заданным порогом и устанавливает "1" или "0" на выходе. Потенциометр позволяет задать порог срабатывания ИК-датчика и, соответственно, дистанцию обнаружения препятствия.

Подключение ИК-модуля к контроллеру Arduino

Подключение ИК-модуля к контроллеру Arduino предельно простое:

- выводы VCC и GND модуля подключаем к пинам +5 V и GND соответственно платы Arduino,
- выход OUT модуля – к любому цифровому или аналоговому пину платы Arduino.

На рис. 5.3 показано подключение ИК-модуля к контроллеру Arduino Nano.

Пример скетча

Читаем показания с выхода модуля и выводим в монитор порта. Если ИК-модуль обнаружил препятствие, будем сообщать об этом.

```

//ИК-модуль подключен к аналоговому пину A5
const int datcik = A5;
void setup()
{
    Serial.begin(9600);
}

void loop()
{
    // считываем в переменную r в показания датчика;
    // они находятся в диапазоне от 0 до 1023
    int r = analogRead(datcik);
    Serial.println(r);
    if (r < 100)
    {
        //обнаружено препятствие
        Serial.println("Препятствие!");
    }
    delay(100);
}

```

В контроллере Arduino используется 10-разрядный АЦП, поэтому значение аналогового сигнала кодируются числами в диапазоне от 0 до 1023. В этом скетче пороговое значение равно 100.

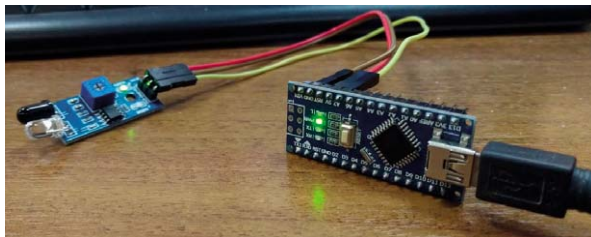


Рис. 5.3. ИК-модуль подключён к Arduino Nano

5.3. Инфракрасные дальномеры SHARP GP2Y0A21YK0F и SHARP GP2Y0A02YK0F

Инфракрасный дальномер GP2Y0A21YK0F позволяет определять расстояние до объекта в пределах 10–80 см, а GP2Y0A02YK0F – 20–150 см. Дальномер определяет расстояние по отражённому лучу света в инфракрасном спектре. Дальномер может использоваться для объезда препятствий и ориен-

тирования на местности. В зависимости от расстояния до объекта, аналоговый сигнал на выходе дальномера будет меняться. Таким образом, дальномер реагирует на дистанцию до объекта. Так, как принцип работы данных датчиков основан на излучении света, то инфракрасные дальномеры плохо подходят для определения расстояния до светопоглощающих объектов. В этом случае рекомендуется использовать ультразвуковой дальномер HC-SR04

Принцип работы инфракрасных дальномеров

Импульсы инфракрасного излучения испускаются излучателем. Это излучение отражается от объектов, находящихся в поле зрения датчика. Отраженное излучение возвращается на приёмник. Испускаемый и отраженный лучи образуют треугольник «излучатель – объект отражения – приёмник». Угол отражения напрямую зависит от расстояния до объекта. Полученные отраженные импульсы собираются высококачественной линзой и передаются на линейную CCD матрицу. По засветке определенного участка CCD матрицы определяется угол отражения и высчитывается расстояние до объекта.

Технические характеристики GP2Y0A21YK0F

- Диапазон измерения: 10–80 см
- Напряжение питания: 4.5–5.5 В
- Потребляемый ток: 30–40 мА
- Габариты: 29.5×13×13.5 мм

Технические характеристики GP2Y0A02YK0F

- Диапазон измерения: 20–150 см
- Напряжение питания: 4.5–5.5 В
- Потребляемый ток: 33–50 мА
- Габариты: 29.5×13×21.6 мм

Внешне дальномеры обоих типов очень похожи (рис. 5.4).

Дальномеры имеют три вывода: желтый – выходной сигнал, черный – земля, красный – +5 В. Выходной сигнал дальномера необходимо подать на один из аналоговых пинов контроллера.

Аналоговые пины контроллера Arduino работают в диапазоне 0–5 В. Аналого-цифровой преобразователь (АЦП) контроллера преобразует этот диапазон напряжений в диапазон целых чисел от 0 до 1023. Есть нюансы. Когда препятствие находится ближе нижней границы радиуса действия (10 см и 20 см), при приближении к объекту дальномер начинает показывать, что пре-

пятствие, наоборот, удаляется. И, наоборот. В связи с этим рекомендуется датчик устанавливать так, чтобы препятствие физически не могло оказаться ближе нижней границы радиуса действия.



Рис. 5.4

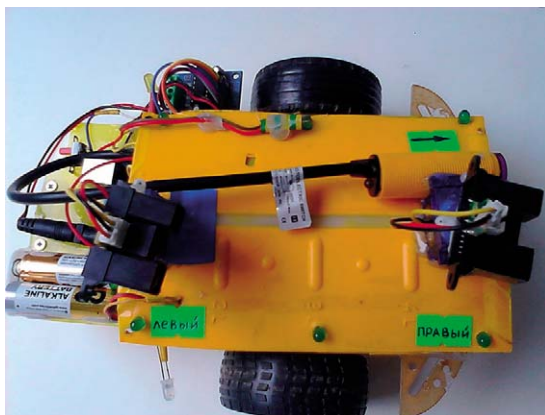
5.4. Описание мобильного робота и его схема

Будем использовать мобильный робот (МР), внешний вид которого приведен на рис. 5.5. На рис. 5.6 приведена схема электрическая принципиальная этого МР. Для простоты будем в дальнейшем называть все схемы электрические принципиальные просто схемой. На роботе установлены два ИК-дальномера GP2Y0A02YK0F – левый и правый. Направление движения МР вперед показано стрелкой.

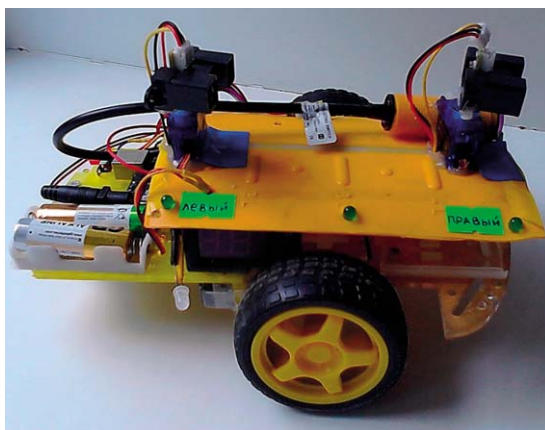
Рассмотрим сначала схему МР. Из нее видно, что на борту МР находится следующее оборудование:

- контроллер Arduino Uno,
- модуль драйвера L293D,
- два ИК-дальномера типа GP2Y0A02YK0F – левый и правый,
- один ИК-дальномер типа E18-D80NK, смотрящий вперед,
- один ИК-приёмник пульта управления,
- два светодиода с гасящими сопротивлениями в 220 Ом,
- сервомашинка для управления поворотом заднего колеса МР,
- два двигателя постоянного тока (ДПТ) левого и правого колеса,
- аккумуляторная батарея (АКБ) – две банки (2S) с напряжением +7,4...+8,4 В.

Сервомашинка, светодиоды и ДПТ колес относятся к исполнительным механизмам (ИМ) робота. Все три ИК-дальномера и ИК-приёмник сигналов пульта управления относятся к датчикам. Контроллер и модуль драйвера получают питание непосредственно от АКБ. Все датчики получают питание +5 В от контроллера.



а)



б)

Рис. 5.5. Внешний вид мобильного робота
а – вид сверху; б – вид сбоку

Толстой линией на рис. 5.6 изображена шина. Её можно рассматривать, как пучок проводов. Каждый провод имеет свою нумерацию, начало и конец. Если провод приходит в несколько мест, то это означает, что используется несколько проводов, имеющих начало в одном месте. Особенно это хорошо видно на проводах 12 и 11 питания и заземления для датчиков. Номера проводов в шине не нужно путать с номерами пинов контроллера.

Дальномеры установлены на сервомашинках [1]. Так их очень легко поворачивать в нужное положение в диапазоне углов $0...180$ градусов. Для этого можно использовать даже неисправные сервомашинки. Если же требуется программное управление положением вала сервомашинки, то они, ко-

нечно, должны быть исправными. В наших примерах мы не будем поворачивать сервомашинки, поэтому они могут быть неисправными. Кроме этого на МР расположен ещё один ИК-дальномер типа E18-D80NK цилиндрической формы в передней части МР. О нем речь пойдет ниже. Для управления двигателями постоянного тока (ДПТ) колес робота используется модуль драйвера L293D, работа которого подробно описана в [1].

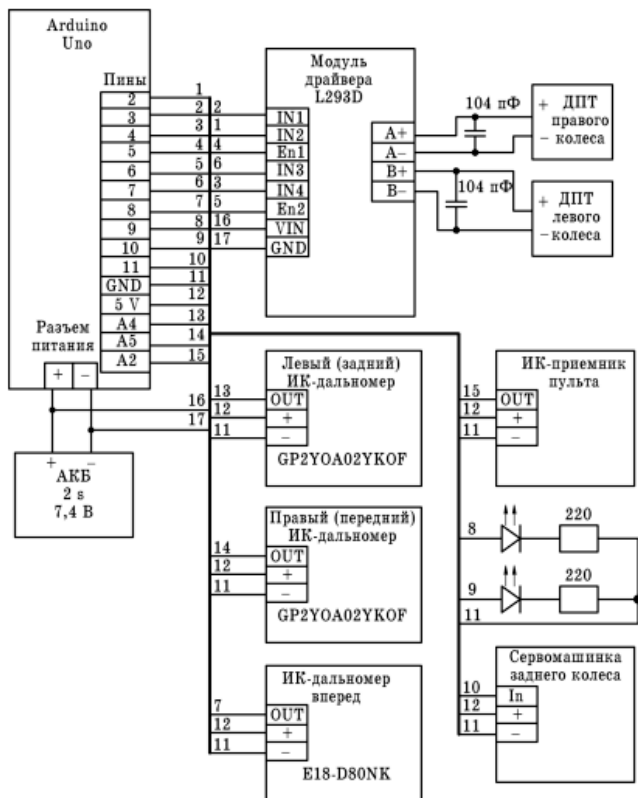


Рис. 5.6. Схема электрическая принципиальная МР

В примерах, рассмотренных далее, может быть использовано не все оборудование, установленное на МР. Это касается сервомашинки заднего колеса, ИК-приёмника пульта и светодиодов. Их применение оговаривается отдельно.

5.5. Движение робота вдоль стенки. Примеры

Пример 5.1. Робот стоит на месте. С помощью двух ИК-дальномеров GP2Y0A02YK0F MP определяет расстояние до цели или преграды и выводит их в монитор порта. Контроллер Arduino должен быть подключен к компьютеру со средой программирования Arduino IDE. Скетч примера (robot_41_1.ino) приведен ниже.

```
// желтый – выходной сигнал, черный – земля, красный – +5В
// робот 41 стоит на месте,
//с помощью двух ИК-дальномеров
//определяет расстояние до цели или преграды
//пины для колес
int in1 = 2;
int in2 = 3;
int in3 = 4;
int in4 = 7;
//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int l; //данные с левого дальномера
unsigned int p; //данные с правого дальномера

void setup()
{
    // настраиваем пины на ввод или вывод
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    Serial.begin(9600); // инициализация монитора порта
    pinMode(A4, INPUT); //левый ИК-дальномер
    pinMode(A5, INPUT); // правый ИК-дальномер
}

void loop()
{
    l = analogRead(A4); // читаем левый дальномер
    Serial.print("Левый дальномер->");
    Serial.println(l); // выводим показания в монитор порта
    delay(1500);
    p = analogRead(A5); // читаем правый дальномер
    Serial.print("Правый дальномер->");
```

```
Serial.println(p); // выводим показания в монитор порта
delay(1500);
```

```
}
```

Пример 5.2. Часто перед работой МР необходимо убедиться в правильности подключения приводов постоянного тока (ППТ) колес или гусениц, направления их вращения. Кроме этого, полезно узнать, синхронно ли они вращаются, т. е. прямо ли движется робот. Для этого желательно использовать простой скетч такой проверки. Скетч примера (robot_41_2.ino) приведен ниже.

```
// желтый – A0, черный – земля, красный – +5В
```

```
// робот 41: обкатка
```

```
//пины для колес
```

```
int in1 = 3;
```

```
int in2 = 2;
```

```
int in3 = 7;
```

```
int in4 = 4;
```

```
//пины для скорости колёс
```

```
int en1 = 5;
```

```
int en2 = 6;
```

```
void setup()
```

```
{
```

```
    //настройка режима работы пинов
```

```
    pinMode(in1, OUTPUT);
```

```
    pinMode(in2, OUTPUT);
```

```
    pinMode(in3, OUTPUT);
```

```
    pinMode(in4, OUTPUT);
```

```
    pinMode(en1, OUTPUT);
```

```
    pinMode(en2, OUTPUT);
```

```
}
```

```
void loop()
```

```
{
```

```
    //задаем движение вперед
```

```
    digitalWrite(in2, LOW);
```

```
    digitalWrite(in1, HIGH);
```

```
    digitalWrite(in3, HIGH);
```

```
    digitalWrite(in4, LOW);
```

```
    //робот постепенно разгоняется
```

```
    for (int i=0; i<=255;i++)
```

```
    {
```

```
        analogWrite(en1, i);
```

```
        analogWrite(en2, i);
```

```
        delay(10);
```

```
    }
```

```
}
```

Пример 5.3. Робот должен двигаться вперед вдоль длинной стенки, стараясь не удаляться от неё и не приближаться к ней. Дальномёры должны быть развернуты по направлению к стенке под небольшим углом к поперечной оси робота (рис. 5.7). Кроме этого, на роботе установлены два светодиода. Один из них загорается, когда робот движется влево, а другой загорается, когда робот движется вправо. Это очень удобно при отладке скетча. Очень хорошо видна динамика эволюций робота.

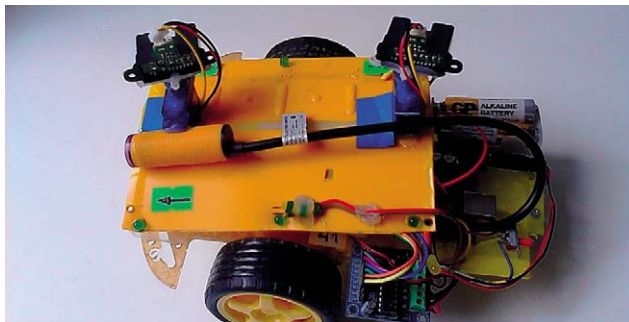


Рис. 5.7. Положение дальномёров при движении робота вдоль стенки

Скетч примера (robot_41_3.ino) имеет вид. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```
// желтый – A0, черный – земля, красный – +5V
// робот 41 едет правым бортом вдоль стенки;

//в этом ему помогают два ИК-дальномёра;
//дальномёры смотрят под углом друг к другу
//скетч работает хорошо
//пины для колёс
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;
//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int l; //данные с левого дальномёра
unsigned int p; //данные с правого дальномёра
int led=9;      //светодиод влево
int ledp=10;    //светодиод вправо
//усредненные данные с левого и правого дальномёров
float lev, prav;
```

```

void setup()
{
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
  pinMode(en1, OUTPUT);
  pinMode(en2, OUTPUT);
  pinMode(led, OUTPUT);
  pinMode(ledp, OUTPUT);
  //Serial.begin(9600); // инициализация монитора порта
  pinMode(A4, INPUT); //левый ИК-дальномер
  pinMode(A5, INPUT); // правый ИК-дальномер
  //начальная скорость равна нулю
  analogWrite(en1, 0);
  analogWrite(en2, 0);
  //робот будет ехать вперед
  digitalWrite(in2, LOW);
  digitalWrite(in1, HIGH);
  digitalWrite(in3, HIGH);
  digitalWrite(in4, LOW);
}

void loop()
{
  //обнуляем усредненные значения дальномеров
  lev=0;
  prav=0;
  //делаем 500 измерений
  for(int i=1;i<=500;i++)
  {
    l = analogRead(A4);
    lev=lev+l;
    //Serial.print("Левый дальномер->");
    //Serial.println(l);
    //delay(1500);
    p = analogRead(A5);
    prav=prav+p;
    //Serial.print("Правый дальномер->");
    //Serial.println(p);
    //delay(1500);
  }
  //усредненные показания левого датчика
  lev=lev/500;
  //Serial.print("Левый дальномер->");
  //Serial.println(lev);
  //усредненные показания правого датчика

```

```

prav=prav/500;
//Serial.print("Правый дальномер->");
//Serial.println(prav);

//аналитика
//если робот слишком прижался к стенке
//или двигается к стенке, то нужно двигаться влево;
// в противном случае – вправо
if((lev>400)||(prav>400)||(prav>lev))
{
    vlevo();
    delay(100);
}
else
{
    vpravo();
    delay(100);
}
} //конец loop

//функция, реализующая движение робота влево
void vlevo()
{
    //добавим оборотов правому мотору
    analogWrite(en1, 140);
    analogWrite(en2, 80);
    //зажжем левый светодиод
    digitalWrite(led, HIGH);
    //погасим правый светодиод
    digitalWrite(ledp, LOW);
}

//функция, реализующая движение робота вправо
void vpravo()
{
    //добавим оборотов левому мотору
    analogWrite(en1, 80);
    analogWrite(en2, 120);
    //погасим левый светодиод
    digitalWrite(led, LOW);
    //зажжем правый светодиод
    digitalWrite(ledp, HIGH);
}

```

Дадим несколько пояснений к этому скетчу. Для этого воспользуемся рисунком 5.8. На нем показаны различные варианты расположения робота относительно стены, вдоль которой он двигается.

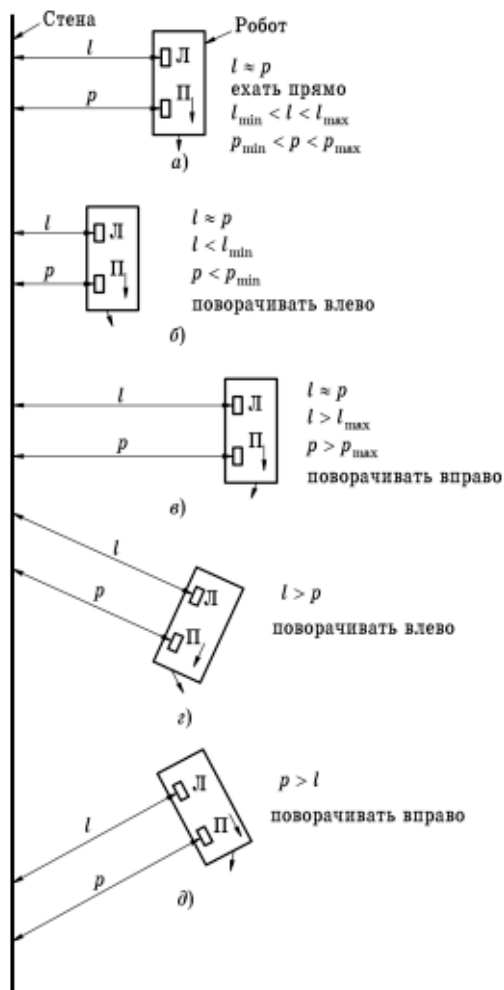


Рис. 5.8. Различные комбинации положения МР относительно стены

Робот оснащен двумя ИК-дальномерами: левым — Л и правым — П. Стрелка на роботе направлена к передней части робота. Стрелки впереди робота показывают, куда он должен двигаться в той или иной ситуации. Величинами l и p обозначены расстояния от левого и правого датчиков до стенки соответственно.

Можно выделить пять вариантов развития ситуации.

Вариант 1. Робот движется вдоль стенки, параллельно ей и на допустимом от неё расстоянии: $l_{\min} < l < l_{\max}$ и $p_{\min} < p < p_{\max}$ и l приблизительно равна p (рис. 5.8, а). В этом случае робот должен двигаться вперед.

Вариант 2. Робот недопустимо приблизился к стенке: $l < l_{\min}$ или $p < p_{\min}$ (рис. 5.8, б). В этом случае робот должен поворачивать влево.

Вариант 3. Робот недопустимо отдалился от стенки: $l_{\max} < l$ и $p_{\max} < p$ (рис. 5.8, в). В этом случае робот должен поворачивать вправо.

Вариант 4. Робот движется к стенке: $l > p$ (рис. 5.8, г). В этом случае робот должен поворачивать влево.

Вариант 5. Робот движется от стенки: $l < p$ (рис. 5.8, д). В этом случае робот должен поворачивать вправо.

Нюанс. У этих дальномеров имеется одна особенность. Чем ближе дальномер к препятствию, тем выше его показания. И это справедливо, если дальномер не приблизился к препятствию на недопустимо близкое расстояние (10 и 20 см). Здесь нужно постараться так разместить дальномер на роботе, чтобы он никогда не мог приблизиться к препятствию на недопустимо близкое расстояние.

В рассматриваемом скетче реализованы четыре из пяти возможных вариантов развития событий, что дает неплохие результаты. Анализ положения робота выполняется в операторе

```
if(((lev>400))||((prav>400))||((prav>lev))
{
    vlevo();
    delay(100);
}
else
{
    vpravo();
    delay(100);
}
```

Здесь в проверочном выражении $((lev > 400)) || ((prav > 400)) || (prav > lev)$ как раз учитывается, что, чем ближе дальномер к стенке, тем выше его показания. Для повышения помехоустойчивости делается 500 измерений расстояния и находится среднее. Это делается для каждого дальмера.

5.6. Движение робота внутри круглого вольера. Примеры

Пример 5.4. Робот должен двигаться внутри круглого вольера. Для ориентирования в пространстве используем передний (правый) дальномер (рис. 5.9, 5.10). Ось дальмера (длинная стрелка) немного смещена вправо относительно продольной оси робота (короткая стрелка).

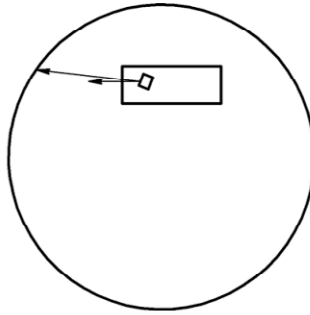


Рис. 5.9. Робот с ИК-дальномером внутри круглого вольера (схема)



Рис. 5.10. Робот с ИК-дальномером внутри круглого вольера

Скетч примера (robot_41_4.ino) приведен ниже. Некоторые операторы скетча закомментированы. Они использовались при отладке. Для индикации направления движения робота используются два светодиода, которые подключаются к пинам 9 и 10 и через гасящие сопротивления в 220 Ом соединяются с землей. Управление ДПТ колес осуществляется через модуль драйвера L293D.

```
// желтый – A0, черный – земля, красный – +5В
// робот 41 ездит против часовой стрелки
// внутри круглого вольера;
// используется один правый ИК-дальномер
//скетч работает хорошо
```

```

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

//пины для светодиодов
int led=9; //светодиод 1 движения влево
int ledp=10; //светодиод 2 движения прямо

//переменные
unsigned int p; //данные с правого дальномера
float prav; //усредненные данные правого дальномера

void setup()
{
    //настройка пинов на вывод
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600);    //инициализация послед. порта

    //настройка пинов на ввод
    pinMode(A4, INPUT); //левый ИК-дальномер
    pinMode(A5, INPUT); //правый ИК-дальномер

    analogWrite(en1, 0); //начальная скорость 0
    analogWrite(en2, 0); // начальная скорость 0
    //задаем движение вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void loop()
{
    prav=0;
    //накапливаем данные с правого дальномера

```

```

for(int i=1;i<=500;i++)
{
    p = analogRead(A5);
    prav=prav+p;
}
//усредненные показания правого датчика
prav=prav/500;

//аналитика
if((prav>450))
{
    vlevo();//поворачиваем влево
}
else
{
    прямо();//едем прямо
}
}

//функция, реализующая движение робота прямо
void прямо()
{
    //скорости колес одинаковы
    analogWrite(en1, 80);
    analogWrite(en2, 80);
    digitalWrite(led, LOW); //гасим светодиод 1
    digitalWrite(ledp, HIGH); //зажигаем светодиод 2
}

//функция, реализующая поворот робота влево
void vlevo()
{
    //скорость правого колеса больше скорости левого
    analogWrite(en1, 90);
    analogWrite(en2, 00);
    digitalWrite(led, HIGH); //зажигаем светодиод 1
    digitalWrite(ledp, LOW); //гасим светодиод 2
}
}

```

В этом скетче выполняются следующие действия:

- Многократно (500 раз) принимается информация с правого датчика и суммируется. Это делается для повышения помехоустойчивости устройства;
- Находится усредненное значение prav показаний датчика;

- Если усредненное значение превышает значение 450, выполняется поворот робота влево – отворот от стенки. В противном случае робот едет прямо.

Перемещение робота представляет собой движение по ломаной траектории с поворотом влево в точках излома. Данный пример иллюстрируется прилагаемым видеороликом.

Недостатком этого скетча является то, что робот может двигаться лишь на небольшой скорости. Если он наткнется на стенку, то уже не может сдать назад. Не нужно забывать, что у ИК-дальномера имеется «мертвая зона» – 10 и 20 см для разных датчиков. В ней вообще приближение робота к стенке оценивается дальномером, как удаление от нее и наоборот.

Попытаемся устранить недостатки данного скетча.

Пример 5.5. Робот должен двигаться внутри круглого вольера. Для ориентирования в пространстве используем передний (правый) дальномер 1 и задний (левый дальномер) (рис. 5.11). Оси дальномеров (длинные стрелки) немного смещены вправо относительно продольной оси робота (короткая стрелка). Для индикации направления движения робота используются два светодиода, которые подключаются к пинам 9 и 10 и через гасящие сопротивления в 220 Ом соединяются с землей. Управление ДПТ колес осуществляется через модуль драйвера L293D (6 пинов). На рис. 5.11 цифрами 1 и 2 обозначены правый и левый дальномеры соответственно, p – текущее расстояние от правого дальномера до стенки вольера, l – текущее расстояние от левого дальномера до стенки вольера.

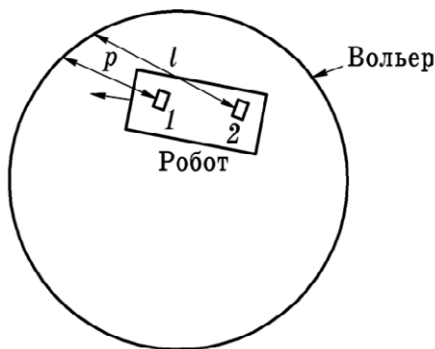


Рис. 5.11. Робот с двумя ИК-дальномерами внутри круглого вольера

Алгоритм управления роботом приведен на рис. 5.12.

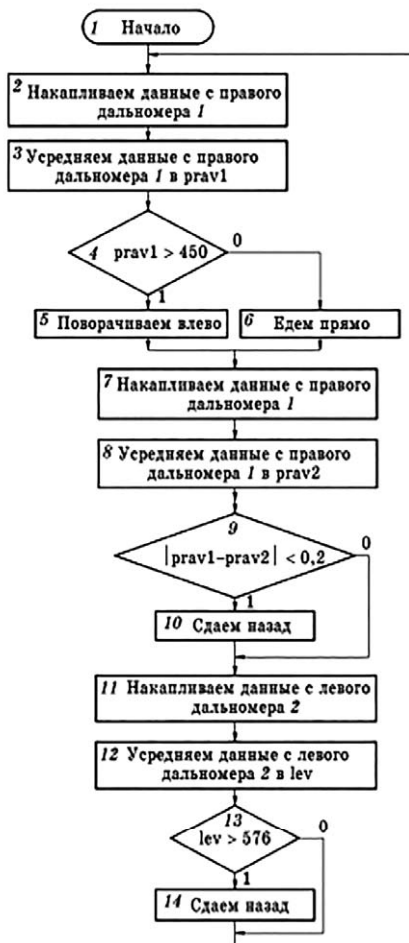


Рис. 5.12. Алгоритм управления роботом

Алгоритм реализует циклическое выполнение следующих действий, реализуемых в функции loop:

1. Накапливаются и усредняются в переменной `prav1` данные, поступившие с правого датчика (блоки 2 и 3);
2. Значение переменной `prav1` сравнивается с числом 450 (блок 4);
3. Если `prav1 > 450`, то робот слишком близко подошел к стенке, и делается его отворот влево (блок 5). В противном случае выполняется движение прямо (блок 6);
4. Далее выполняются повторные действия по накоплению и усреднению в переменной `prav2` данных с правого датчика (блоки 7 и 8);

5. Выполняется сравнение переменных `prav1` и `prav2` (блок 9);
6. Если абсолютная величина разности значений `prav1` и `prav2` мала, то робот, получив команды на движение прямо или влево (функции `primo()` и `vlevo()`), не двинулся с места ($\text{abs}(\text{prav1}-\text{prav2}) < 0.2$). В этом случае выполняется функция `nazad()`, реализующая движение робота назад и вправо в течении 400 мс (блок 10). В противном случае никаких действий не выполняется.
7. Накапливаются и усредняются в переменной `lev` данные, поступившие с левого датчика (блоки 11 и 12);
8. В блоке 13 выполняется анализ переменной `lev`. Если её значение превышает число 576, то выполняется функция `nazad()` (блок 14), поскольку робот недопустимо близко приблизился к стенке. Понятно, что данные с правого датчика мы уже использовать не можем, поскольку стенка находится в его «мертвой зоне». В противном случае никаких действий не выполняется.
9. Выполняется переход (соединитель 1) к блоку 2 и все рассмотренные выше действия повторяются снова.

Скетч примера (`robot_41_5`) имеет вид. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```
// желтый – A0, черный – земля, красный – +5В;
// робот 41 ездит против часовой стрелки
// внутри круглого вольера;
//используются оба ИК-дальномера;
//скетч работает хорошо

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

//пины для светодиодов
int led=9; //светодиод 1 движения влево
int ledp=10; //светодиод 2 движения прямо

// переменные
unsigned int p1,p2,l; //данные с датчиков
float prav1, prav2, lev; //усредненные данные датчиков

void setup()
{
```

```

//настраиваем пины на вывод
pinMode(in1, OUTPUT);
pinMode(in2, OUTPUT);
pinMode(in3, OUTPUT);
pinMode(in4, OUTPUT);
pinMode(en1, OUTPUT);
pinMode(en2, OUTPUT);
pinMode(led, OUTPUT);
pinMode(ledp, OUTPUT);
Serial.begin(9600); // инициализация послед. порта
//настраиваем пины на ввод информации
pinMode(A4, INPUT); //левый ИК-дальномер
pinMode(A5, INPUT); // правый ИК-дальномер
pinMode(datc, INPUT); //дальномер E18-D80NK
//начальная скорость – 0, движение – вперед
analogWrite(en1, 0);
analogWrite(en2, 0);
digitalWrite(in2, LOW);
digitalWrite(in1, HIGH);
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
}

```

```

void loop()
{
    // обнуляем все показания дальномеров
    prav1=0;
    prav2=0;
    lev=0;
    p1=0;
    p2=0;
    l=0;
    //настраиваемся на движение вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);

    //накапливаем данные с правого дальномера
    for(int i=1;i<=500;i++)
    {
        p1 = analogRead(A5);
        prav1=prav1+p1;
    }

    //усредненные показания правого дальномера
    prav1=prav1/500;
}

```

```

//аналитика
if((prav1>450))
{
    vlevo();//поворачиваем влево
    //delay(20);
}
else
{
    priamo();//едем прямо
    //delay(20);
}

//снова накапливаем данные с правого датчика
for(int j=1;j<=500;j++)
{
    p2 = analogRead(A5);
    prav2=prav2+p2;
}

//усредненные показания правого датчика
prav2=prav2/500;
if (abs(prav1-prav2)<0.2)
{
    nazad();
    //delay(30);
}

//накапливаем данные с левого датчика
for(int k=1;k<=500;k++)
{
    l = analogRead(A4);
    lev=lev+l;
}

//усредненные показания левого датчика
lev=lev/500;
if ((lev>576))
{
    nazad();
    //delay(30);
}
}

//функция, реализующая движение робота прямо
void priamo()
{
    //скорости колёс одинаковы

```



```

    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
}

```

//функция, реализующая поворот робота влево

```

void vlevo()
{
    // скорость правого колеса больше скорости левого
    analogWrite(en1, 130);
    analogWrite(en2, 00);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

```

//функция, реализующая движение робота назад

```

void nazad()
{
    //отъезжаем назад вправо
    analogWrite(en1, 90);
    analogWrite(en2, 130);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    delay(400);
}

```

5.7. Инфракрасный дальномер (датчик препятствия) E18-D80NK

Назначение

E18-D80NK – инфракрасный датчик препятствий, состоящий из излучателя и приёмника (рис. 5.13). Служит для обнаружения объектов на расстоянии от 3 до 80 сантиметров. Может использоваться в мобильных роботах, системах умного дома, охранных сигнализациях, станках с ЧПУ, интерактивных медиаустройствах, в системах промышленной автоматике, в автомобилях и т. д.

Характеристики

- Питание: +3...+5,5 В постоянного тока;
- Выходной рабочий ток: 100 мА;
- Дистанция срабатывания: 60–800 мм (регулируется с помощью винта подстроечного резистора в задней части корпуса);
- Время реакции: 2 мс;
- Угол обзора: $<15^\circ$;
- Имеется световая индикация срабатывания;
- Тип выхода: цифровой нормально открытый NPN-NO, либо цифровой нормально закрытый NPN-NC (зависит от партии);
- Материал корпуса: пластик;
- Длина провода: 0,45 м;
- Рабочая температура: $-25...+55\text{ }^\circ\text{C}$;
- Габаритные размеры: 45×18 мм;
- Масса брутто: 36 г.

Контакты

- коричневый – питание +5 В;
- черный – цифровой выходной сигнал;
- синий – «земля», GND.

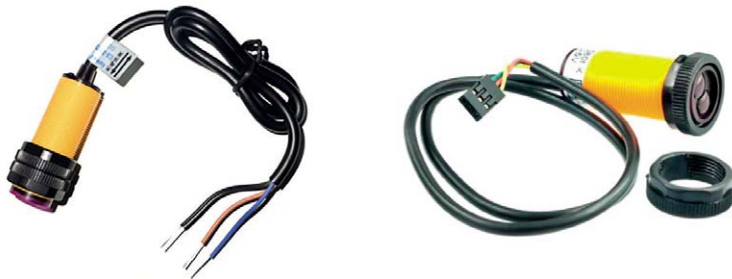


Рис. 5.13. Внешний вид датчика

Описание устройства

Датчик (датчик препятствий) состоит из ИК-излучателя и приемника, собранных в одном корпусе. Он позволяет определить наличие препятствия на расстоянии от 30 до 800 мм. Расстояние срабатывания датчика зависит от свойств отражающей поверхности материала препятствия. Принцип работы датчика заключается в следующем. Датчик испускает инфракрасные лучи и, если впереди находится какой-либо предмет (препятствие), то инфра-

красный луч отражается от поверхности предмета. Приёмник датчика фиксирует этот отраженный луч. Таким образом, датчик определяет, что в заданной зоне находится препятствие. Если же в заданной зоне предмет отсутствует, то посланный инфракрасный луч не находит поверхности отражения. Таким образом, датчик фиксирует, что в заданной зоне нет предмета.

Структурная схема датчика

На рис. 5.14 приведена структурная схема датчика. Она содержит ИК-излучатель и высокочувствительный ИК-приёмник. Излученный ИК-сигнал отражается от поверхности предмета и принимается приёмником. После этого осуществляется расчет расстояния до объекта. Устройство также оснащено подстроечным резистором для быстрой и легкой настройки рабочего диапазона.

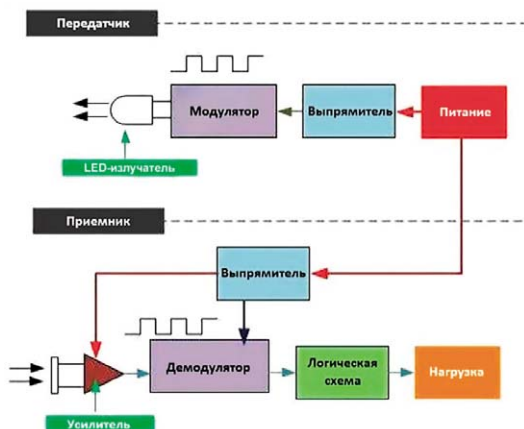


Рис. 5.14. Структурная схема датчика

Из схемы видно, что для измерения расстояния используется модулированный ИК-сигнал. Приёмник настроен на приём только таких модулированных сигналов, что исключает его ложное срабатывание. При такой реализации приёмник менее чувствителен к сторонним излучениям по сравнению с обычными ИК-приёмниками.

Подключение дальномера к контроллеру

Вариант подключения дальномера к контроллеру Arduino Uno приведен на рис. 5.15. Выходной сигнал дальномера можно подключать к любому цифровому или аналоговому пину контроллера, предварительно настроив последний на ввод информации.

Вариант подключения дальномера к контроллеру Arduino Nano приведен на рис. 5.16. Выходной сигнал дальномера можно подключать к любому цифровому или аналоговому пину контроллера, предварительно настроив последний на ввод информации.

На рис. 5.17 показано два типа цветовой кодировки выводов дальномера: Тип 1 и Тип 2. Тип 2 требует использования дополнительного внешнего резистора.

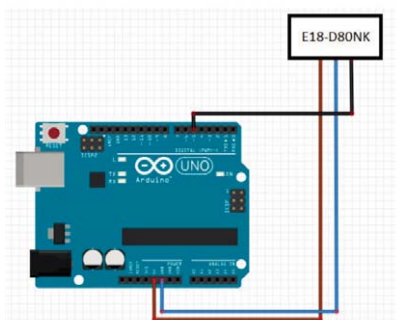


Рис. 5.15. Вариант подключения дальномера к Arduino Uno

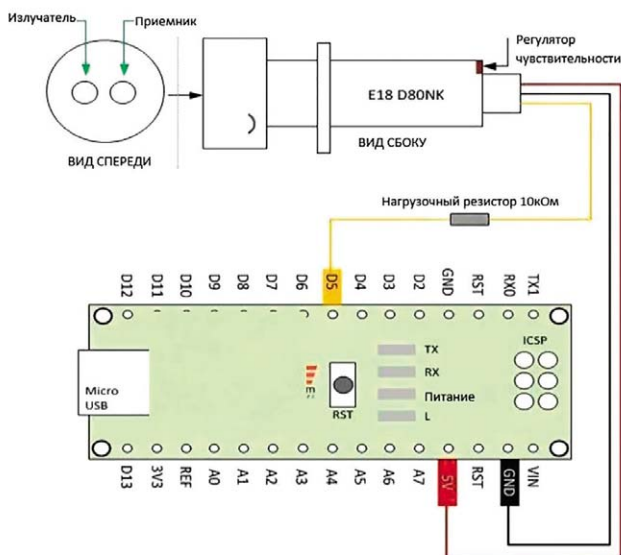


Рис. 5.16. Вариант подключения дальномера к Arduino Nano

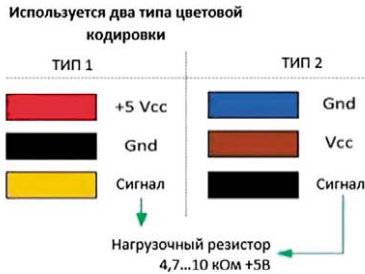


Рис. 5.17. Два типа цветовой кодировки выводов дальномера: Тип 1 и Тип 2

Пример 1. Подключим дальномер к пину 5 контроллера. Если используем второй тип дальномера, то не забываем про гасящее сопротивление. Выдадим в монитор порта сообщение "Detected.", если датчик обнаружил препятствие и "No Detected.", когда препятствие не обнаружено. Скетч примера приведен ниже.

```
/* E18-D80NK ИК – датчик препятствий */
void setup()
{
  Serial.begin(9600); //скорость обмена данными 9600
  pinMode(5,INPUT); пин 5 настраиваем на ввод
}
void loop()
{
  while(1) //бесконечный цикл
  {
    delay(500);
    if(digitalRead(5)==LOW)
    { // если на выходе 0, выдать сообщение
      Serial.println("Detected.");
    }
    else
    { //Если на выходе 1, выдать сообщение
      Serial.println("No Detected.");
    }
  }
}
```

Пример 2. Подключим дальномер к пину 10 контроллера. Если используем второй тип дальномера, то не забываем про гасящее сопротивление. Выдадим в монитор порта сообщение "+", если датчик обнаружил препятствие и "-", когда препятствие не обнаружено. Скетч примера приведен ниже.

```
/* E18-D80NK ИК – датчик препятствий */
void setup()
```

```

{
  Serial.begin(9600);
  pinMode(10,INPUT); //используем 10-й пин, настройка на ввод

}

void loop()
{
  delay(500);    //задержка 0,5 сек
  if(digitalRead(10)==LOW)
  { //если с датчика пришел сигнал низкого уровня
    Serial.println("+");    //срабатывание
  }
  else
  { //в противном случае – нет срабатывания
    Serial.println("-");
  }
}
}

```

Рассмотрим еще несколько примеров.

Пример 5.6. Робот движется внутри круглого (овального) вольера. Используются дальномеры двух типов:

- 1) E18-D80NK. Он стоит впереди робота и смотрит вперед;
- 2) SHARP GP2Y0A02YK0F. Он стоит сзади (левый). Ось его немного смещена относительно продольной оси робота.

Передний датчик контролирует траекторию движение робота. Задний датчик контролирует опасные ситуации – недопустимые приближения робота к стенке или даже соприкосновения с ней. Скетч примера (robot_41_6.ino) приведен ниже. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```

// желтый – A0, черный – земля, красный – +5V
// робот 41 ездит против часовой стрелки
//внутри круглого вольера;
//используются два датчика:
//1. ИК-дальномер E18 – D80NK стоит впереди
//2. ИК-дальномер SHARP GP2Y0A02YK0F стоит сзади,
//но смотрит вперед под небольшим углом
//скетч работает хорошо

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

```

```

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

//пины для светодиодов
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения прямо

// пин для ИК-датчика E18 – D80NK
int datc=8;

//переменные
//данные с дальномера SHARP GP2Y0A02YK0F
unsigned int l;
//усредненные данные дальномера SHARP GP2Y0A02YK0F
float lev;

void setup()
{
    //настраиваем пины на вывод
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта

    //настраиваем пины на ввод информации
    pinMode(A4, INPUT); //левый (задний) ИК-дальномер
    pinMode(datc, INPUT); //передний ИК-дальномер

    //начальная скорость равна 0
    analogWrite(en1, 0);
    analogWrite(en2, 0);

    //задаем движение вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

```

```

void loop()
{
    //обнуляем данные с датчика
    lev=0;
    l=0;

    //задаем движение вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);

    //аналитика
    if(digitalRead(datsc)==LOW)
    {
        // передний датчик сработал
        vlevo(); //поворачиваем влево
        //delay(20);
    }
    else
    {
        // передний датчик не сработал
        priamo();//едем прямо
        //delay(20);
    }

    //накапливаем данные с левого датчика
    for(int k=1;k<=500;k++)
    {
        l = analogRead(A4);
        lev=lev+l;
    }
    //усредняем показания левого датчика
    lev=lev/500;
    if ((lev>576))
    {
        //стенка очень близко
        nazad(); //сдаем назад
        //delay(30);
    }
} //конец loop

//функция, реализующая движение робота прямо
void priamo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(led, LOW);
}

```



```

        digitalWrite(ledp, HIGH);
    }

//функция, реализующая поворот робота влево
void vlevo()
{
    analogWrite(en1, 250);
    analogWrite(en2, 00);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

//функция, реализующая движение робота назад
void nazad()
{
    analogWrite(en1, 90);
    analogWrite(en2, 130);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    delay(300);
}

```

Пример 5.7. Робот движется внутри круглого (овального) вольера. Используется только один дальномер SHARP GP2Y0A02YK0F. Он стоит сзади (левый). Ось его немного смещена относительно продольной оси робота.

Он контролирует и траекторию движение робота и опасные ситуации – недопустимые приближения робота к стенке или даже соприкосновения с ней. Скетч примера (robot_41_9.ino) приведен ниже. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```

// желтый – A0, черный – земля, красный – +5V
// робот 41 ездит против часовой стрелки
//внутри круглого вольера;
//используется задний (левый) ИК-дальномер
//скетч работает хорошо

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

```

```

//пины для светодиодов
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения прямо

//переменные
unsigned int l; //данные с датчика
float lev1, lev2; //усредненные данные датчика

void setup()
{
    //настраиваем пины на вывод
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта

    //настраиваем пины на ввод
    pinMode(A4, INPUT); //левый ИК-датчик

    //начальная скорость равна 0
    analogWrite(en1, 0);
    analogWrite(en2, 0);

    //направление движения – вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void loop()
{
    lev1=0;
    lev2=0;
    l=0;

    //направление движения – вперед
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

```

```

//накапливаем данные с левого датчика
for(int i=1;i<=500;i++)
{
    l = analogRead(A4);
    lev1=lev1+l;
}
//усредненные показания левого датчика
lev1=lev1/500;

//аналитика
if((lev1>450))
{
    vlevo();//поворачиваем влево
}
else
{
    pramo();//едем прямо
}

//снова накапливаем данные с левого датчика
for(int j=1;j<=500;j++)
{
    l = analogRead(A4);
    lev2=lev2+l;
}
//усредненные показания правого датчика
lev2=lev2/500;

//проверка упирания в стенку
if (abs(lev1-lev2)<0.2)
{
    nazad1();
}

//проверка слишком близкого приближения к стенке
if ((lev2>540))
{
    nazad2();
}
}конец loop

//функция, реализующая движение робота прямо
void pramo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(led, LOW);
}

```

```

        digitalWrite(ledp, HIGH);
    }

//функция, реализующая поворот робота влево
void vlevo()
{
    analogWrite(en1, 130);
    analogWrite(en2, 00);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

//функция 1, реализующая движение робота назад
void nazad1()
{
    analogWrite(en1, 90);
    analogWrite(en2, 130);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    digitalWrite(led, LOW);
    digitalWrite(ledp, LOW);
    delay(400);
}

//функция 2, реализующая движение робота назад
void nazad2()
{
    analogWrite(en1, 90);
    analogWrite(en2, 130);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, HIGH);
    delay(200);
}

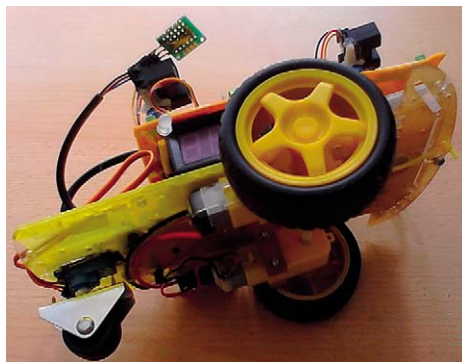
```

В этом скетче используются две функции nazad1() и nazad2(), реализующие перемещение робота назад и вправо. Эти функции абсолютно одинаковы за исключением времени выполнения. Функция nazad1() более радикальна и используется, когда робот уже уперся в стену. Функция nazad2() выполняется меньше во времени, когда робот недопустимо приблизился к стенке вольера.

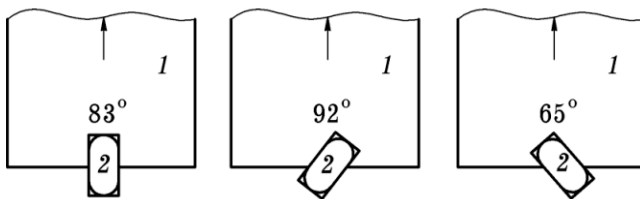
5.8. Колесный робот с поворачивающимся задним колесом

До сих пор мы управляли поворотами робота, изменяя скорость вращения правого или левого колеса. Однако можно и не менять скорость колёс, а, например, сделать заднее колесо поворачивающимся. Этот вариант является осуществимым только для роботов с колесным шасси. Для роботов с гусеничным шасси этот вариант невозможно реализовать.

На рис. 5.18 показан колесный робот с поворачивающимся задним колесом. Заднее колесо закреплено на сервомашинке, ось которой можно поворачивать в диапазоне $0^\circ \dots 180^\circ$ программным образом. Для того, чтобы заднее колесо эффективно работало, необходимо поместить в место его положения какой-либо тяжелый груз, что повышает сцепление колеса с поверхностью, по которой движется робот. В качестве привода поворота заднего колеса подойдет и любой шаговый двигатель (ШД), что, конечно, дороже и труднее при программировании движения робота. По этой причине займемся управлением робота с задним колесом на сервомашинке.



а)



б)

Рис. 5.18. Колесный робот с поворачивающимся задним колесом:
а – внешний вид; б – положение заднего колеса
при различных углах его поворота

На рис. 5.18, б показан вид сверху задней части 1 робота, 2 – колесо. Эксперименты показали, что колесо находится в среднем положении при угле поворота вала сервомашинки, равном 83° . Конечно, эта величина определяется экспериментально для каждой реализации робота. Если угол поворота вала сервомашинки будет больше 83° (например, 93°), то колесо будет повернуто влево, и робот будет поворачивать влево. И, наоборот, если угол поворота вала сервомашинки будет меньше 83° (например, 65°), то колесо будет повернуто вправо, и робот будет поворачивать вправо. Конечно, эти величины тоже выбираются из экспериментальных наблюдений. Если сервомашинка установлена как-то по-другому, то все углы будут другими и должны определяться экспериментально. Использование поворотного колеса позволяет поворачивать робот по-автомобильному, что позволяет не сбавлять скорость движения при поворотах и быстрее реагировать на команды контроллера.

Пример 5.8. Робот движется внутри круглого (овального) вольера. Используется только один дальномер SHARP GP2Y0A02YK0F. Он стоит сзади (левый). Ось его немного смещена относительно продольной оси робота. Он контролирует и траекторию движение робота и опасные ситуации – недопустимые приближения робота к стенке или даже соприкосновения с ней. Поворот робота вперед влево и назад вправо будем выполнять с помощью поворачивающегося заднего колеса на сервомашинке. Скетч примера (robot_41_10.ino) приведен ниже. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```
// желтый – A0, черный – земля, красный – +5V
// робот 41 ездит против часовой стрелки
//внутри круглого вольера;
//используется задний (левый) ИК-дальномер
//скетч работает хорошо

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

//пины для светодиодов
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения прямо
```

```

//переменные
unsigned int l;//данные с датчика
float lev1, lev2;//усредненные данные датчика

void setup()
{
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта
    pinMode(A4, INPUT);//левый ИК-датчик
    servo.attach(11); //привязываем сервомашинку к пину 11
    servo.write(77); //ставим вал по центру
    delay(5000); //ждем
    analogWrite(en1, 0);
    analogWrite(en2, 0);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void loop()
{
    lev1=0;
    lev2=0;
    l=0;

    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);

    //накапливаем данные с левого датчика
    for(int i=1;i<=500;i++)
    {
        l = analogRead(A4);
        lev1=lev1+l;
    }
    //усредненные показания левого датчика
    lev1=lev1/500;

```

```

//аналитика
if((lev1>450))
{
    vlevo();    //поворачиваем влево
}
else
{
    priamo();    //едем прямо
}

//снова накапливаем данные с левого датчика
for(int j=1;j<=500;j++)
{
    l = analogRead(A4);
    lev2=lev2+l;
}
//усредненные показания левого датчика
lev2=lev2/500;
if (abs(lev1-lev2)<0.1)
{
    nazad1();
}
if ((lev2>540))
{
    nazad2();
}
}

//функция, реализующая движение робота прямо
void priamo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    servo.write(77); //ставим вал по центру
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
}

//функция, реализующая поворот робота влево
void vlevo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    servo.write(97); //ставим вал влево
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

```



```

//функция 1, реализующая движение робота назад – вправо
void nazad1()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    digitalWrite(led, LOW);
    digitalWrite(ledp, LOW);
    servo.write(57); //ставим вал вправо
    delay(300);
}

//функция, реализующая движение робота назад – вправо
void nazad2()
{
    analogWrite(en1, 90);
    analogWrite(en2, 130);
    digitalWrite(in2, HIGH);
    digitalWrite(in1, LOW);
    digitalWrite(in3, LOW);
    digitalWrite(in4, HIGH);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, HIGH);
    servo.write(57); //ставим вал вправо
    delay(400);
}

```

Прилагается несколько видеороликов, иллюстрирующих движение робота с поворачивающимся задним колесом внутри круглого вольера. На них хорошо видны два двигателя постоянного тока, расположенных в задней части робота, исключительно для её утяжеления. Это необходимо для улучшения сцепления заднего колеса с поверхностью, по которой он движется. Таким образом достигается необходимая эффективность заднего колеса при поворотах.

5.9. Движение робота вокруг круглого вольера. Примеры

Рассмотрим несколько примеров движения робота с задним поворачивающимся колесом вокруг круглого вольера.

Пример 5.9. Робот движется вокруг круглого (овального) вольера против часовой стрелки. Используется только один дальномер SHARP GP2Y0A02YK0F. Он стоит сзади (левый) (рис. 5.19).

На рис. 5.19 введены следующие обозначения: 1 – вольтер, 2 – робот, 3 – ИК-дальномер, 4 – ось дальномера, 5 – продольная ось робота. Ось 4 дальномера немного смещена влево относительно продольной оси робота. Дальномер контролирует расстояние робота от поверхности вольтера. Поворот робота влево и вправо будем выполнять с помощью поворачивающегося заднего колеса на сервомашинке. На рис. 5.20 приведены показания дальномера при различных удалениях робота от поверхности вольтера.

На этом рисунке: 1 – вольтер; 2, 3, 4 – траектории с различным удалением. Из рис. 5.20 хорошо видно, что, чем ближе робот к поверхности вольтера, тем выше показания дальномера. Естественно, всё это справедливо, пока робот, а с ним – и дальномер, не оказались недопустимо близко к поверхности вольтера. Такая ситуация наверняка приведет к срыву управления и допускать её нельзя.

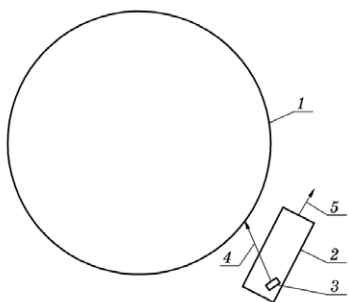


Рис. 5.19. Взаимное расположение робота и вольтера

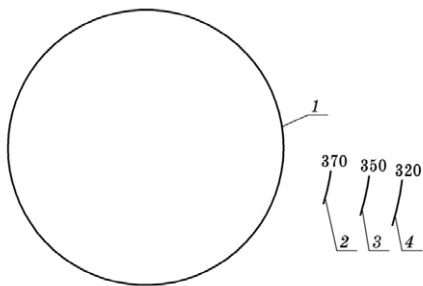


Рис. 5.20. Показания дальномера при различном удалении робота от вольтера

На рис. 5.21 показан простейший алгоритм управления роботом. Несмотря на простоту, он работает достаточно эффективно. Авторы оставляют за читателем возможность реализовать более сложные и качественные алгоритмы.

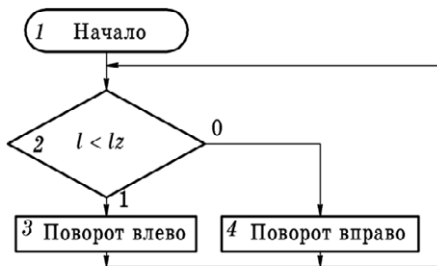


Рис. 5.21. Алгоритм управления роботом при его движении вокруг вольтера

В алгоритме не отражены действия по приему и усреднению информации, принятой с дальномера. Действия в алгоритме выполняются циклически и бесконечно. В блоке 2 показания l дальномера сравниваются с предельным значением lz. Если показания l меньше предельного значения lz, то робот отъехал слишком далеко от вольера и его необходимо срочно возвращать поворотом влево (блок 3). В противном случае робот необходимо удалять от поверхности вольера поворотом вправо (блок 4). Вот и всё.

Скетч примера (robot_41_11.ino) приведен ниже. Некоторые операторы скетча закомментированы. Они использовались при отладке.

```
// желтый – A0, черный – земля, красный – +5В
// робот 41 ездит против часовой стрелки
// вокруг круглого вольера;
//используется задний (левый) ИК-дальномер
//скетч работает хорошо

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;
//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int l; //данные с дальномера
int led=9;      //светодиод движения влево
int ledp=10;    //светодиод движения вправо
float lev;      //усредненные данные дальномера

void setup()
{
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта
    pinMode(A4, INPUT); //левый ИК-дальномер
    servo.attach(11);    //привязываем сервопривод к пину 11
```

```

servo.write(83);      //ставим колесо по центру
delay(5000);          //ждем

//начальная скорость равна нулю
analogWrite(en1, 0);
analogWrite(en2, 0);
//моторы – вперед!
digitalWrite(in2, LOW);
digitalWrite(in1, HIGH);
digitalWrite(in3, HIGH);
digitalWrite(in4, LOW);
}

void loop()
{
    lev=0;
    l=0;

    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
    //накапливаем данные с левого датчика
    for(int i=1;i<=500;i++)
    {
        l = analogRead(A4);
        lev=lev+l;
    }
    //усредненные показания левого датчика
    lev=lev/500;
    //Serial.println(lev);
    //аналитика
    if((lev<300))
    {
        vlevo();//поворачиваем влево
        //delay(20);
    }

    else
    {
        vpravo();//поворачиваем вправо
        //delay(20);
    }
}

```

```

//функция, реализующая поворот робота влево
void vleft()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    servo.write(99); //ставим вал влево
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

//функция, реализующая движение робота вправо
void vpravo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
    servo.write(65); //ставим вал вправо
}

```

Величина l_z для этого скетча равна 300. Меняя ее, можно регулировать приближение робота к поверхности вольера. Эта величина также зависит от скорости движения робота. Чем сильнее робот приближен к вольеру, тем меньше должна быть скорость его движения. Для поворота робота влево ставим заднее колесо влево на угол 99 – команда `servo.write(99)`. Для поворота робота вправо ставим заднее колесо влево на угол 65 – команда `servo.write(65)`. Алгоритм управления роботом в этом примере довольно прост – заднее колесо, в зависимости от обстановки, устанавливается на поворот влево или вправо.

Усложним алгоритм – угол поворота заднего колеса относительно среднего положения (83°) будет изменяться не скачкообразно, как в предыдущем примере, а плавно с шагом в 5° .

Пример 5.10. Робот движется вокруг круглого (овального) вольера против часовой стрелки. Используется только один дальномер SHARP GP2Y0A02YK0F. Алгоритм управления роботом для данного примера приведен на рис. 5.22.

Работа скетча выполняется циклически. В блоке 2 выполняется прием и усреднение данных с левого дальномера – переменная `lev`.

Далее, в блоке 3 выполняется сравнение `lev` с константой 300. Эта константа задает расстояние движения МР от поверхности вольера. Если `lev` $\leq$ 300, то МР слишком удалился от поверхности вольера. Это означает, что заднее колесо нужно повернуть влево и выполняется переход к блоку 4. Здесь выполняется проверка угла поворота `ugol` заднего колеса с предельным значением угла, равным в этом примере 92° градусам. Если текущее значение угла поворота колеса не превышает 92° , то выполняется увеличение этого угла

на 5 градусов (блок 5) с последующим поворотом вала сервомашинки на новый угол (блок 6). Если угол поворота колеса превышает предельное значение в 92° , то никаких действий не выполняется.

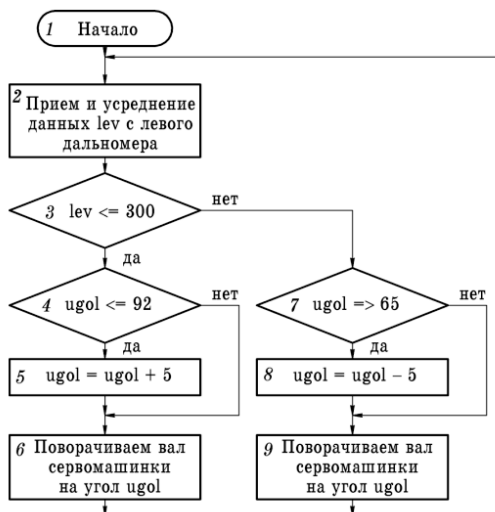


Рис. 5.22. Алгоритм управления роботом

Аналогичные действия выполняются, когда показания дальномера $lev > 300$. Это означает, что МР слишком приблизился к поверхности вольера и его необходимо поворачивать вправо, уменьшая текущий угол поворота заднего колеса на 5 (блок 8). При этом, конечно, выполняется проверка угла на предельное значение в 65° (блок 7).

Таким образом, поворот колеса, в зависимости от показаний дальномера, выполняется постепенно на 5 градусов влево или вправо. В идеале при движении МР поворот заднего колеса должен происходить в пределах 10° . Скетч этого примера (robot_41_12.ino) приведен ниже.

```

// желтый – A0, черный – земля, красный – +5В
// робот 41 ездит против часовой стрелки
//вокруг круглого вольера;
//используется задний (левый) ИК-дальномер
//скетч работает хорошо
  
```

```

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo
  
```

```

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;
//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int l;//данные с датчика
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения прямо
float lev; //усредненные данные датчика
int ugo=83; //среднее положение заднего колеса
void setup()
{
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); //инициализация послед. порта
    pinMode(A4, INPUT); //левый ИК-датчик
    servo.attach(11); //привязываем привод к пину 11
    servo.write(ugo); //ставим колесо по центру
    delay(5000); //ждем
    analogWrite(en1, 0);
    analogWrite(en2, 0);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void loop()
{
    lev=0;
    l=0;
    //накапливаем данные с левого датчика
    for(int i=1;i<=500;i++)
    {
        l = analogRead(A4);
        lev=lev+l;
    }
}

```

```

        //усредняем показания левого дальномера
        lev=lev/500;
        //Serial.println(lev);
        //аналитика
        if((lev<=300))
        {
            vlevo();//поворачиваем влево
            //delay(20);
        }

        else
        {
            vpravo();//поворачиваем вправо
            //delay(20);
        }
    }

//функция, реализующая поворот робота влево
void vlevo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    if (ugol <= 92)
    {
        ugol = ugol+5;
    }
    servo.write(ugol); //ставим колесо влево
    //delay(10);
    //зажигаем светодиод поворота влево
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

//функция, реализующая движение робота вправо
void vpravo()
{
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    if (ugol >= 65)
    {
        ugol = ugol-5;
    }
    //delay(10);
    //зажигаем светодиод поворота вправо
    digitalWrite(led, LOW);
}

```



```
digitalWrite(ledp, HIGH);
servo.write(ugol); //ставим колесо вправо

}
```

Пример 5.11. Робот движется вокруг круглого (овального) вольера против часовой стрелки. Используется только один дальномер SHARP GP2Y0A02YK0F. Скетч примера (robot_41_13.ino) приведен ниже.

```
// желтый – A0, черный – земля, красный – +5V
// робот 41 ездит против часовой стрелки
//вокруг круглого вольера;
//используется задний (левый) ИК-дальномер
//скетч работает хорошо

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

//пины для скорости колёс
int en1 = 5;
int en2 = 6;

//переменные
unsigned int l; //данные с дальномера
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения прямо
float lev; //усредненные данные дальномера
int ugol=83; //начальный угол поворота заднего колеса

void setup()
{
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта
    pinMode(A4, INPUT); //левый ИК-дальномер
    servo.attach(11); //привязываем привод к пину A3
    servo.write(ugol); //ставим вал по центру
```

```

        delay(5000); //ждем
        analogWrite(en1, 110);
        analogWrite(en2, 110);
        digitalWrite(in2, LOW);
        digitalWrite(in1, HIGH);
        digitalWrite(in3, HIGH);
        digitalWrite(in4, LOW);
    }

    void loop()
    {
        lev=0;
        l=0;
        //накапливаем данные с левого датчика
        for(int i=1;i<=500;i++)
        {
            l = analogRead(A4);
            lev=lev+l;
        }
        //усредненные показания левого датчика
        lev=lev/500;
        //Serial.println(lev);
        //аналитика
        if((lev<=300))
        {
            vlevo1();//поворачиваем влево
            //delay(20);
        }
        if((lev>=350))
        {
            vlevo2();//едем прямо
            //delay(20);
        }

        if((lev>=370))
        {
            vlevo3();//поворачиваем вправо
            //delay(20);
        }
    }

    //функция, реализующая поворот робота влево
    void vlevo1()
    {
        servo.write(100); //ставим колесо влево
        //delay(10);
        digitalWrite(led, HIGH);
    }

```

```

        digitalWrite(ledp, LOW);
    }

    //функция, реализующая движение робота прямо
    void vlevo2()
    {
        servo.write(82); //ставим колесо прямо
        //delay(10);
        digitalWrite(led, HIGH);
        digitalWrite(ledp, HIGH);
    }

    //функция, реализующая поворот робота вправо
    void vlevo3()
    {
        servo.write(72); //ставим колесо вправо
        //delay(10);
        digitalWrite(led, LOW);
        digitalWrite(ledp, HIGH);
    }
}

```

Пример 5.12. До сих пор мы рассматривали случаи, когда робот двигался вокруг круглого (овального) вольера против часовой стрелки. Сейчас мы рассмотрим случай, когда робот должен двигаться вокруг вольера по часовой стрелке. Что изменится в алгоритме управления роботом? Немного. При удалении от вольера робот должен поворачивать не влево, как раньше, а вправо. При приближении к вольеру робот должен поворачивать не вправо, как раньше, а влево. Используется только один датчик SHARP GP2Y0A02YK0F. Скетч для этого примера (robot_41_14.ino) приведен ниже.

```

// желтый – A0, черный – земля, красный – +5В
// робот 41 ездит по часовой стрелке
//вокруг круглого вольера;
//используется передний (правый) ИК-датчик
//скетч работает хорошо

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;

```

```

//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int p;//данные с датчика
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения вправо
float prav;//усредненные данные датчика
int ugol=83;

void setup()
{
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
    pinMode(in4, OUTPUT);
    pinMode(en1, OUTPUT);
    pinMode(en2, OUTPUT);
    pinMode(led, OUTPUT);
    pinMode(ledp, OUTPUT);
    Serial.begin(9600); // инициализация послед. порта
    pinMode(A4, INPUT); //левый ИК-датчик
    pinMode(A5, INPUT); //правый ИК-датчик
    servo.attach(11); //привязываем привод к пину A3
    servo.write(ugol); //ставим вал по центру
    delay(5000); //ждем
    analogWrite(en1, 110);
    analogWrite(en2, 110);
    digitalWrite(in2, LOW);
    digitalWrite(in1, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);
}

void loop()
{
    prav=0;
    p=0;
    //накапливаем данные с правого датчика
    for(int i=1;i<=500;i++)
    {
        p = analogRead(A5);
        prav=prav+p;
    }
    //усредненные показания правого датчика
    prav=prav/500;
    //Serial.println(prav);
    //аналитика

```

```

    if((prav<=300))
    {
        vpravo();//поворачиваем вправо
        //delay(20);
    }
    if((prav>=350))
    {
        priamo();//двигаемся прямо
        //delay(20);
    }

    if((prav>=370))
    {
        vlevo();//поворачиваем влево
        //delay(20);
    }
}

```

//функция, реализующая поворот робота вправо

```

void vpravo()
{
    servo.write(62); //ставим колесо вправо
    //delay(10);
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
}

```

//функция, реализующая движение робота прямо

```

void priamo()
{
    servo.write(83); //ставим колесо прямо
    //delay(10);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, HIGH);
}

```

//функция, реализующая поворот робота влево

```

void vlevo()
{
    servo.write(90); //ставим колесо влево
    //delay(10);
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

```

5.10. Робот, объезжающий предметы

Пример 5.13. Требуется разработать МР, который бы объезжал предметы, как это показано на рис. 5.23. Будем использовать МР с задним поворачивающимся колесом. Предметы с нечетными номерами 1, 3, 5... и т. д. он должен объезжать справа. Предметы же четными номерами 2, 4, 6... и т. д. он должен объезжать слева. Признаком четности или нечетности предмета будет являться переменная $priz$. При объезде предмета с нечетным номером $priz=1$. При объезде предмета с четным номером $priz=2$.

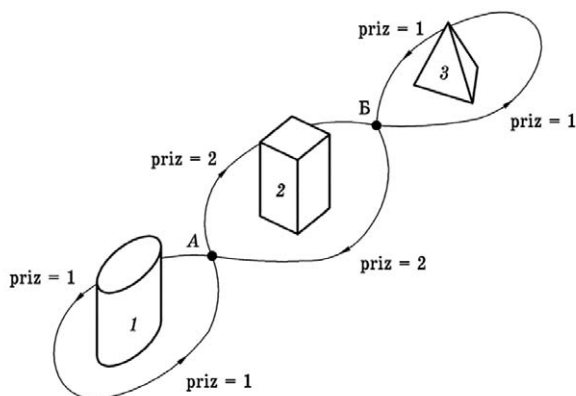


Рис. 5.23

В системе управления будем использовать два дальномера типа SHARP GP2Y0A02YK0F (рис. 5.24).

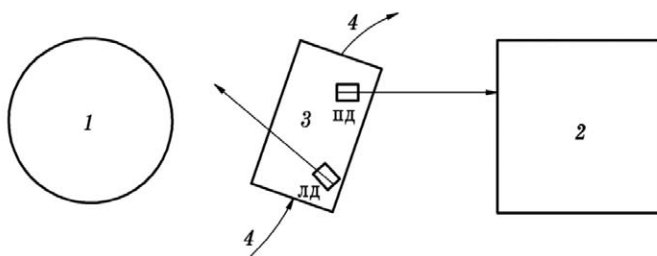


Рис. 5.24

На этом рисунке:

- 1, 2 – предметы, объезжаемые роботом,
- 3 – робот,
- 4 – траектория движения робота,

- ЛД – левый дальномер,
- ПД – правый дальномер.

Из рис. 5.24 видно, что левый дальномер развернут влево и «видит» предметы с нечетными номерами. Правый же дальномер развернут вправо и «видит» предметы с четными номерами.

Из рис. 5.23 также следует, что в точке А траектории правый дальномер начинает «видеть» предмет с четным номером 2, что является сигналом к кардинальному изменению траектории робота. Он должен двигаться с поворотом вправо по часовой стрелке, объезжая предмет 2 слева. До этого он, объезжая предмет 1, двигался с поворотом влево против часовой стрелки.

Это же касается и точки Б траектории. В ней левый дальномер начинает «видеть» предмет 3, что является сигналом к изменению его траектории. Он должен начинать двигаться с поворотом влево против часовой стрелки, объезжая предмет 3 справа.

Вот, собственно, и весь алгоритм движения робота. Важно не забыть при начале движения поставить робот справа от предмета 1. Да, и еще один нюанс. Как вы догадываетесь, количество предметов может быть произвольным: 1, 2... 10, 11 и т. д. Ну и, конечно, предметы должны быть расположены в пределах дальности работы дальномеров.

Скетч для этого примера с необходимыми комментариями (robot_41_15.ino) приведен ниже.

```
// желтый – А0, черный – земля, красный – +5В
// робот 41 объезжает предметы
//поочередно: первый – против часовой стрелки,
//второй – по часовой стрелке и т. д.
//перед запуском робот устанавливается справа от первого объекта

//подключаем библиотеку Servo.h для работы с сервоприводом
#include <Servo.h>
Servo servo; //объявляем переменную servo типа Servo

//пины для колес
int in1 = 3;
int in2 = 2;
int in3 = 7;
int in4 = 4;
//пины для скорости колёс
int en1 = 5;
int en2 = 6;
//переменные
unsigned int p,l;//данные с дальномеров
int led=9; //светодиод движения влево
int ledp=10; //светодиод движения вправо
float prav, lev;//усредненные данные дальномеров
int ugol=83; // начальный угол поворота заднего колеса
```

```

//priz – признак направления объезда предмета:
//1 – объезд против часовой стрелки
//2 – объезд по часовой стрелке
int priz;
void setup()
{
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT);
  pinMode(in4, OUTPUT);
  pinMode(en1, OUTPUT);
  pinMode(en2, OUTPUT);
  pinMode(led, OUTPUT);
  pinMode(ledp, OUTPUT);
  Serial.begin(9600); // инициализация послед. порта
  pinMode(A4, INPUT); //левый ИК-дальномер
  pinMode(A5, INPUT); //правый ИК-дальномер
  servo.attach(11); //привязываем сервопривод к пину A3
  servo.write(ugol); //ставим заднее колесо по центру
  delay(5000); //ждем
  analogWrite(en1, 110); //скорость правого колеса
  analogWrite(en2, 110); //скорость левого колеса
  digitalWrite(in2, LOW);
  digitalWrite(in1, HIGH); //правое колесо – вперед
  digitalWrite(in3, HIGH); //левое колесо – вперед
  digitalWrite(in4, LOW);
  priz=1; // объезжаем первый объект против часовой стрелки
}

void loop()
{
  prav=0;
  lev=0;
  l=0;
  p=0;
  //накапливаем данные с правого дальномера
  for(int i=1;i<=500;i++)
  {
    p = analogRead(A5);
    prav=prav+p;
  }
  //усредненные показания правого дальномера
  prav=prav/500;
  //Serial.println(prav);

  //накапливаем данные с левого дальномера
  for(int i=1;i<=500;i++)

```



```

        {
            I = analogRead(A4);
            lev=lev+I;
        }
//усредненные показания левого датчика
lev=lev/500;
//Serial.println(lev);

//аналитика
if (priz==1)
{
    if(prav>300)
    {
        //при объезде предмета
        // против часовой стрелки
        //обнаружен следующий предмет,
        //который нужно объезжать
        // по часовой стрелке
        priz=2;
    }
    else
    {
        //объезд предмета против
        //часовой стрелки
        protiv();
    }
}
else
{
    if(lev>300)
    {
        //при объезде предмета
        //по часовой стрелке
        //обнаружен следующий предмет,
        //который нужно объезжать
        //против часовой стрелки
        priz=1;
    }
    else
    {
        //объезд предмета
        //по часовой стрелке
        po();
    }
}
}

```

```

//функция, организующая объезд предмета
//по часовой стрелке
void po()
{
    //аналитика
    if((prav<=300))
    {
        vpravo1();//поворачиваем вправо
    }
    if((prav>=350))
    {
        priamo1();//двигаемся прямо
    }

    if((prav>=370))
    {
        vlevo1();//поворачиваем влево
    }
}

void protiv()
{
    if((lev<=300))
    {
        vlevo ();//поворачиваем влево
    }
    if((lev>=350))
    {
        priamo();//двигаемся прямо
    }

    if((lev>=370))
    {
        vlpravo();//поворачиваем вправо
    }
}

//функция, реализующая поворот робота влево
void vlevo()
{
    servo.write(92); //ставим колесо влево
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

//функция, реализующая движение робота прямо
void priam()

```

```

    {
        servo.write(83); //ставим колесо прямо
        digitalWrite(led, HIGH);
        digitalWrite(ledp, HIGH);
    }

//функция, реализующая поворот робота вправо
void vpravo()
{
    servo.write(65); //ставим колесо вправо
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
}

//функция, реализующая поворот робота вправо
void vpravo1()
{
    servo.write(65); //ставим колесо вправо
    digitalWrite(led, LOW);
    digitalWrite(ledp, HIGH);
}

//функция, реализующая движение робота прямо
void priamo1()
{
    servo.write(83); //ставим колесо прямо
    digitalWrite(led, HIGH);
    digitalWrite(ledp, HIGH);
}

//функция, реализующая поворот /робота влево
void vlevo1()
{
    servo.write(92); //ставим колесо влево
    digitalWrite(led, HIGH);
    digitalWrite(ledp, LOW);
}

```

6. СИСТЕМА УПРАВЛЕНИЯ ДРОНОМ САМОЛЕТНОГО ТИПА НА БАЗЕ КОНТРОЛЛЕРА ARDUINO

6.1. Общие сведения

В настоящее время наблюдается значительный интерес к конструированию различных летающих механизмов – дронов, планеров, глайдеров, вертолетов и т. д.

Беспилотный летательный аппарат (БПЛА) – летательный аппарат без экипажа на борту. В разговорной речи его также называют беспилотником или дроном (от англ. drone «трутень»).

БПЛА могут обладать разной степенью автономности – от управляемых дистанционно до полностью автоматических. БПЛА также различаются по конструкции, назначению и множеству других параметров.

Управление БПЛА может осуществляться эпизодической подачей команд или непрерывно. В последнем случае БПЛА называют дистанционно-пилотируемым летательным аппаратом (ДПЛА).

БПЛА могут решать разведывательные задачи (на сегодня это основное их предназначение), применяться для нанесения ударов по наземным и морским целям, перехвата воздушных целей, осуществлять постановку радиопомех, управления огнём и целеуказания, ретрансляции сообщений и данных, доставки грузов.

Основным преимуществом БПЛА/ДПЛА является существенно меньшая стоимость их создания и эксплуатации (при условии равной эффективности выполнения поставленных задач). По экспертным оценкам боевые БПЛА верхнего диапазона сложности стоят от 5–6 млн долл., в то время как стоимость пилотируемого истребителя-бомбардировщика F-35 составляет около 100 млн долл. (плюс существенные затраты на обучение пилота). Важным фактором является то, что оператор боевого БПЛА не рискует своей жизнью, в отличие от пилота боевого самолёта. Недостатком БПЛА является уязвимость систем дистанционного управления, что особенно важно для БПЛА военного назначения.

Согласно Правилам использования воздушного пространства Российской Федерации, БПЛА определяется как «летательный аппарат, выполняющий полёт без пилота (экипажа) на борту и управляемый в полёте автоматически, оператором с пункта управления или сочетанием указанных способов». Министерство обороны США использует схожее определение, где единственным признаком БПЛА является отсутствие пилота.

Международная организация гражданской авиации (ИКАО) разделяет радиоуправляемые модели и БПЛА, указывая, что первые предназначены

прежде всего для развлечения и должны регулироваться местными – а не международными правилами использования воздушного пространства.

Беспилотная авиационная система (БПАС)

Вместо термина БПЛА может использоваться более широкое понятие Беспилотная авиационная система, которая включает в себя:

- собственно БПЛА,
- пункт управления (пульт оператора, приёмопередающая аппаратура),
- систему связи с БПЛА: например, прямая спутниковая радиосвязь,
- дополнительное оборудование, необходимое для перевозки или обслуживания БПЛА.

Конструкция БПЛА

Отсутствие пилота на борту снимает с БПЛА ряд ограничений, характерных для пилотируемой авиации, что может сильно отразиться на их конструкции.

- Беспилотный летательный аппарат можно выполнить сколь угодно малых размеров, в то время как пилотируемый невозможно сделать меньше габаритов человека.
- БПЛА не имеет физиологических ограничений на перегрузки при выполнении манёвров, что также может отражаться на конструкции.
- Для БПЛА могут быть снижены требования к надёжности, так как это не влечёт прямой угрозы жизни человека.
- Время полёта беспилотных аппаратов не ограничено ресурсом систем жизнеобеспечения лётчика. В настоящее время вполне реальны проекты беспосадочных БПЛА, вырабатывающих ресурс в течение одного полёта, который может продолжаться до нескольких лет.
- По конструкции дроны бывают:
 - самолетного типа;
 - квадрокоптеры (4 двигателя);
 - октокоптеры (8 двигателей);
 - вертолётного типа.

Фюзеляж

Фюзеляж крупных БПЛА в основном идентичен пилотируемому самолёту или вертолёту, за исключением отсутствия кабины.

Система питания и двигатели

В небольших БПЛА применяют бесколлекторные двигатели, которые используют литий-полимерные аккумуляторы, солнечные батареи, водородные топливные элементы и т. д.

Для БПЛА с большим радиусом действия используются двигатели внутреннего сгорания или воздушно-реактивные.

Система связи и бортовая аппаратура управления

В качестве бортовой аппаратуры управления, как правило, используются специализированные вычислители на базе цифровых сигнальных процессоров или компьютеры формата PC/104, MicroPC под управлением операционных систем реального времени (QNX, VME, VxWorks, XOberon). Программное обеспечение пишется обычно на языках высокого уровня, таких как Си, Си++, Модула-2, Оберон SA или Ада95.

Для передачи на пункт управления данных, полученных с бортовых сенсоров, в составе БПЛА имеется радиопередатчик, обеспечивающий радиосвязь с наземным приёмным оборудованием. В зависимости от формата изображений и степени их сжатия пропускная способность цифровых радиолиний передачи данных с борта БПЛА может составлять единицы-сотни Мбит/с.

По типу управления

- Управляемые автоматически.
- Управляемые оператором с пункта управления (ДПЛА).
- Гибридные.

По максимальной взлётной массе

Воздушный кодекс РФ требует обязательной постановки на учёт БПЛА взлётной массой от 0,15 до 30 кг через портал Госуслуг, а также обязательной регистрации БПЛА весом более 30 кг (для управления БПЛА весом более 30 кг необходимо также получить сертификат лётной годности и свидетельство внешнего пилота).

По назначению

- Разведывательные.
- Ударные (способные вести огонь по противнику самостоятельно).
- Мишени (например, Ла-17М, «Адъютант»).

Применение

Военное

БПЛА могут решать следующие задачи:

- авиаразведка (на сегодня это основное их предназначение),
- нанесение ударов по наземным и морским целям,
- перехват воздушных целей,
- постановка радиопомех,
- управление огнём и целеуказания,
- ретрансляция сообщений и данных,
- доставка грузов подразделениям,
- барражирующие боеприпасы.

Барражирующие боеприпасы – дроны-камикадзе. При стоимости менее миллиона долларов они могут уничтожить новейший танк или ракетный комплекс гораздо большей стоимости. Барражирующие боеприпасы особенно эффективны против средств ПВО противника, так как малый размер, тихоходность, композитные материалы дрона-камикадзе затрудняют его обнаружение средствами ПВО, рассчитанными на гораздо более крупные боевые самолёты или крылатые ракеты. Барражирующие боеприпасы дешевле крылатой или противорадиолокационной ракеты, при той же эффективности.

Логистика и производство

- инвентаризации складских помещений;
- доставка грузов.

Строительство

- планирование и мониторинг строительных работ;
- определение границ участка;
- контроль за безопасностью;
- инспектирование строений;
- картографирование.

Сельское хозяйство

- распыление удобрений и средств защиты растений;
- получение актуальной и точной информации о площади, рельефе, специфике грунта полей, состоянии растений и почв;
- инвентаризация сельхозугодий;
- оценка всхожести сельскохозяйственных культур;
- прогнозирование урожайности сельскохозяйственных культур.

Электроэнергетика

- обследование электростанций, линий электропередач и теплотрасс.

Нефтегазовый сектор

- получение информации из труднодоступных мест;
- обследования нефтяной инфраструктуры, утечек и нарушений;
- определение районов аварий и их снижение;
- обнаружение несанкционированных работ.

Экологический мониторинг

- борьба с браконьерами и незаконными рубками;
- мониторинг состояния лесов, обнаружение пожаров;
- мониторинг таяния ледников.

Безопасность

- анализ дорожно-транспортных происшествий;
- мониторинг крупных мероприятий;
- отслеживание преступников;
- поисково-спасательные операции;
- обнаружение чрезвычайной ситуации.

Гражданский рынок

- доставка еды, медикаментов и других товаров;
- для тренировок амурских тигров – охотясь за летательными аппаратами, тигры поддерживают свою физическую форму.

Спортивное направление

- гонки на дистанционно пилотируемых аппаратах (ДПЛА) или дрон-рейсинг. Как правило, в соревнованиях участвуют небольшие (до 25 см в поперечнике) квадрокоптеры, развивающие скорость до 150 км/ч.
- Управление ДПЛА с помощью FPV (вида от первого лица), пилоты должны пройти трёхмерную трассу, образованную ландшафтом и искусственными препятствиями, на время или на скорость.

Аэротакси

В Дубае на международном саммите «World Government Summit» в 2017 году была представлена первая модель беспилотного летающего такси. БПЛА, в котором может разместиться один пассажир, способен находиться в воздухе около получаса за один полёт. Он оборудован четырьмя «ногами», на каждой из них установлено по два пропеллера. При посадке пассажир указывает пункт назначения на сенсорном экране. Полёт такого такси проходит под наблюдением наземного диспетчерского центра.

С 2021 года беспилотники вертолётного типа используются Почтой России в Чукотском автономном округе.

В июне 2021 года компанией «ENhang» в Японии был проведён впервые испытательный полёт беспилотного аэротакси. В феврале 2023 года был проведён первый в Японии полёт беспилотного аэротакси с пассажирами на борту.

Космические исследования

Беспилотные автономные летательные аппараты начинают находить применение и в исследованиях планет и их спутников, имеющих атмосферу. Так, в 2020 году в рамках космической программы «Марс-2020» на Марс был отправлен БПЛА в виде соосного вертолёта «Ingenuity». В 2026 году планируется отправить на Титан БПЛА в виде октокоптера (8 моторов) «Dragonfly» в рамках космической программы «Новые рубежи». Возможность применения БПЛА рассматривается и в рамках программы «Венера-Д».

6.2. Подключение бесколлекторного электродвигателя постоянного тока к контроллеру ARDUINO

Все «электрические» дроны используют для своего движения бесколлекторные (бесщёточные) электродвигатели (БД) постоянного тока (BLDC – Brushless DC Motor). В этой главе мы рассмотрим управление скоростью вращения БД постоянного тока A2212/13T с помощью электронного контроллера скорости ESC (Electronic Speed Controller – электронный контроллер скорости) и контроллера Arduino. Можно, конечно использовать любой другой БД, число типов которых огромно. БД подбираются под весовые характеристики. Чем больше вес модели, тем более мощным должен быть БД и тем на больший ток должен быть рассчитан ESC.

6.2.1. Принцип действия БД

БД постоянного тока в настоящее время кроме авиационных моделей часто используются в потолочных вентиляторах и электрических движущихся транспортных средствах благодаря их плавному вращению. В отличие от других электродвигателей постоянного тока БД подключаются с помощью трех выводов – фаз. Соответственно, БД имеет три катушки-обмотки, разнесенные в пространстве на 120°. Различают Inrunner и OutRunner BLDC двигатели (рис. 6.1).

В Inrunner двигателях магниты ротора находятся внутри статора с обмотками, а в OutRunner двигателях магниты расположены снаружи и вращаются вокруг неподвижного статора с обмотками. То есть в Inrunner (по этому принципу конструируется большинство двигателей постоянного тока) ось внутри двигателя вращается, а оболочка остается неподвижной.

В OutRunner сам двигатель вращается вокруг оси с катушкой, которая остается неподвижной. OutRunner двигатели особенно удобны для примене-

ния в электрических велосипедах, поскольку внешняя оболочка двигателя непосредственно приводит в движение колесо велосипеда, что позволяет обойтись без механизма сцепления. К тому же OutRunner двигатели обеспечивают больший крутящий момент, что делает их также идеальным выбором для применения в электрических движущихся средствах и дронах.



Рис. 6.1

Существует еще бесстержневой (coreless) тип БД, который находит применение в «карманных» дронах. Эти двигатели работают по несколько иным принципам, но рассмотрение принципов их работы выходит за рамки книги.

БД бывают с датчиками (Sensor) и без датчиков (Sensorless). Для БД, которые обеспечивают плавность вращения, без рывков, необходима обратная связь. Поэтому контроллер ESC должен знать позиции и полюса магнитов ротора, чтобы правильно запитывать катушки статора. Эту информацию можно получить двумя способами: первый из них заключается в размещении датчика Холла внутри двигателя. Датчик Холла будет обнаруживать магнит и передавать информацию об этом в контроллер ESC. Этот тип двигателей называется Sensor BLDC (с датчиком) и он находит применение в электрических движущихся транспортных средствах. Второй метод обнаружения позиции магнитов заключается в использовании обратной ЭДС (электродвижущей силы), генерируемой катушками в то время, когда магниты пересекают их. Достоинством этого метода является то, что он не требует использования каких-либо дополнительных устройств (датчик Холла) – фазовый провод самостоятельно используется в качестве обратной связи благодаря наличию обратной ЭДС. Этот тип чаще всего применяется в дронах и других летательных аппаратах (ЛА).

6.2.2. Преимущества БД

Сейчас существует множество различных типов дронов – с двумя лопастями, с четырьмя лопастями и т. д. Но все они используют именно БД. Почему именно их, ведь БД стоят дороже, чем обычные электродвигатели постоянного тока (ДПТ)? Существует несколько причин для этого:

- большой крутящий момент, который очень важен для того чтобы оторвать ЛА от земли;
- эти двигатели доступны в формате OutRunner, что позволяет обойтись без сцепления в конструкции ЛА;
- маленький уровень вибраций во время работы, что очень важно для неподвижного зависания ЛА в воздухе;
- хорошее соотношение мощности к весу двигателя. Это очень важно для использования в ЛА. Обычный ДПТ, обеспечивающий такой же крутящий момент, что и БД, будет, как минимум, в два раза тяжелее него.

6.2.3. Контроллер скорости ESC

Хотя БД относятся к двигателям постоянного тока, они управляются с помощью последовательности импульсов. Для преобразования напряжения постоянного тока источника питания в последовательность импульсов и распределения их по трем фазам БД используется контроллер ESC.

В любой момент времени напряжение с ESC подается только на две фазы БД. При этом электрический ток входит в одну катушку через одну фазу, и покидает его через другую. Это приводит к тому, что магниты ротора выравниваются по отношению к запитанным катушкам.

Затем контроллер ESC подает питание на две другие фазы. Этот процесс смены фаз, на которые подается питание, продолжается непрерывно, что заставляет ротор двигателя вращаться. Скорость вращения ротора зависит от того как быстро меняются пары фаз, на которые подается напряжение. Направление вращения зависит от порядка смены фаз, на которые поочередно подается напряжение.

Внешний вид одного из вариантов контроллера ESC приведен на рис. 6.2.

На рис. 6.2 можно выделить три группы выводов.

1. Три вывода справа подключаются непосредственно к фазам БД. Для смены направления вращения ротора БД достаточно поменять подключение двух любых выводов к БД.
2. Два толстых провода слева – это провода источника питания, в роли которого выступает аккумулятор, как правило, это литий – полимерный (Li-Po) аккумулятор. Красный провод идет к плюсу аккумулятора, а черный – к минусу аккумулятора.

3. Три провода малого диаметра слева обеспечивают подключение контроллера ESC к контроллеру Arduino. Красный и черный проводники этой триады подключаются к пинам 5V и GND контроллера Arduino. Они обеспечивают питание электроники контроллера ESC. Желтый провод обеспечивают передачу широтно-импульсного сигнала из контроллера Arduino в контроллер ESC.

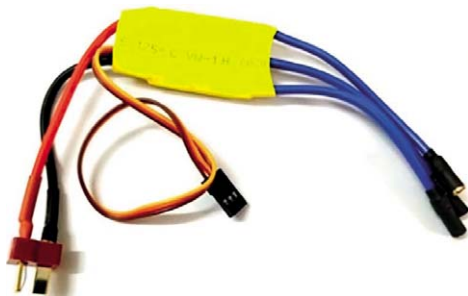


Рис. 6.2

Управление скоростью вращения ротора выполняется на основе широтно-импульсной модуляции (ШИМ, в англ. PWM). Контроллер ESC получает ШИМ сигнал и преобразует его в скорость вращения ротора БД. Принцип управления им очень похож на управление сервомоторами. Сигнал ШИМ, подаваемый на контроллер ESC, должен иметь период 20ms, а коэффициент заполнения этого ШИМ сигнала будет определять скорость вращения BLDC двигателя. Поскольку точно такой же принцип используется для управления углом поворота сервомотора, то для управления скоростью ротора БД можно использовать библиотеку Servo.h.

Есть один очень интересный момент. Почти все контроллеры ESC поставляются со схемой BEC (Battery Eliminator Circuit) – цепь, исключаяющая батарею. Как следует из ее названия, данная схема устраняет потребность в использовании отдельной батареи для питания контроллера Arduino. В данном случае не понадобится отдельный источник питания для платы Arduino. Контроллер ESC сам обеспечит плату Arduino регулируемым напряжением питания +5 V. В различных контроллерах ESC используются различные схемы регулировки данного напряжения, но, в большинстве случаев, распространена схема с линейной регулировкой.

Каждый контроллер ESC содержит в своем ПЗУ встроенную прикладную программу, написанную производителем контроллера. Эта программа во многом определяет логику функционирования контроллера. Наиболее популярными встроенными программами для контроллеров ESC являются Traditional, Simon-K и BL-Heli. Эта программа может изменяться пользователем, однако здесь мы не будем рассматривать данный вопрос.

6.2.4. Схема подключения БД к контроллеру Arduino

Варианты схем подключения БД через контроллер ESC к плате Arduino показаны на рис. 6.3 и 6.4. На рис. 6.3 показано простейшее подключение ESC контроллеру Arduino. На рис. 6.4 показан вариант, когда с помощью потенциометра RV1 можно регулировать скорость вращения ротора БД. Потенциометр подключен к пину A0 платы Arduino.

Особенностью, которая сразу бросается в глаза (рис. 6.3, 6.4), является то, что контроллер Arduino при таком включении не требует отдельного питания от аккумуляторной батареи (АКБ). Он получает питание непосредственно через пины 5V и GND с контроллера ESC. Это, конечно, плюс. Вместо пина 5V можно использовать пин VIN (внешнее напряжение).

Наиболее популярными в настоящее время являются литий-полимерные (Li-Po) АКБ. Величина напряжения на выходе АКБ типа Li-Po зависит от количества банок (s) в ней. Каждая банка дает напряжение 3,7В...4,2В. АКБ выбирается в зависимости от величины рабочего напряжения, требуемого для нормальной работы БД. Как правило, БД работоспособен при разной величине подаваемого напряжения, например: 1s, 2s, 3s (одна, две и три банки соответственно). Величина рабочего напряжения в банках иногда указывается на корпусе БД и обязательно в технической документации на двигатель.

Чем больше банок имеет АКБ, тем выше амплитуда импульсов, подаваемых на БД и тем выше мощность на его валу. Тем больших размеров пропеллер мы можем надевать на ротор БД. А это приводит к увеличению скорости ЛА, его грузоподъемности. Мы обычно «летаем» на двух-трех банках. Естественно, при трех банках, модель обладает большим «могуществом». Есть такой термин в авиации.

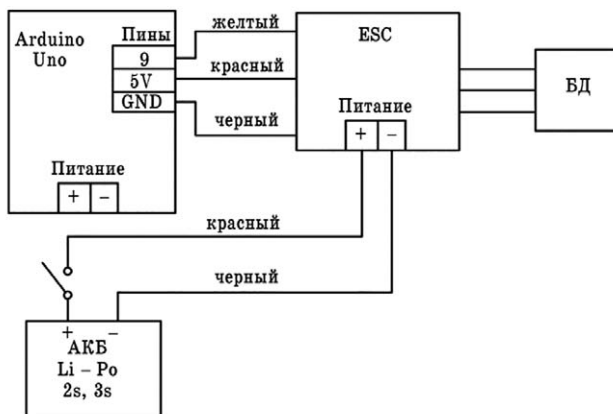


Рис. 6.3

Двух банок вполне хватает для питания контроллера, а вот одной банки уже не хватит, хотя есть небольшие легкие модели, которые летают и на одной банке. Контроллер Arduino Uno уже не подключишь, но в линейке Arduino имеются модели с меньшим питающим напряжением: Arduino Pro Mini, Arduino LilyPad.

И еще. Максимальный рабочий ток ESC должен превышать ток, потребляемый БД. На корпусе ESC имеются соответствующие маркировки. Например: 2-4S LIPO 25A или 20A LIPO 2-4S. Лаконично, но все понятно.

Три фазы (провода) БД необходимо подсоединить к трем выходным проводам контроллера ESC в произвольном порядке. При неправильном направлении вращения ротора БД можно поменять местами две фазы.

У некоторых контроллеров ESC нет выходных проводов. В этом случае необходимо припаивать провода от БД к контактам контроллера ESC. Обязательно нужно изолировать места паяк, т. к. через эти провода возможно протекание достаточно больших токов и любое короткое замыкание может привести к повреждению БД, ESC, контроллера Arduino и АКБ. АКБ – самая дорогая вещь из всех названных. И еще самая пожароопасная.

Внешний вид собранной конструкции показан на рис. 6.5.

Пример 6.1 Разработать скетч для регулирования скорости ротора БД с помощью потенциометра (рис. 6.4, 6.5). Скетч для этого примера приведен ниже. Контроллер ESC подключен к пину 9 платы Arduino, а потенциометр – к пину A0.

```
#include <Servo.h> //используем библиотеку Servo.h для
//формирования ШИМ сигнала, задающего скорость вращ. ротора БД
// к пину 9 подключен ESC
// к пину A0 подключен движок потенциометра
Servo ESC; //задаем servo – объекту имя ESC
void setup()
{
    ESC.attach(9); //подключаем контроллер ESC к пину 9 Arduino
}
void loop()
{
    //считываем напряжение с выхода потенциометра
    int pot = analogRead(A0);

    //конвертируем значения со входа A0 (0-1023)
    // в диапазон 0-180, т. к. сервомоторы могут работать
    //только в диапазоне 0-180
    pot = map(pot, 0, 1023, 0, 180);
    //выводим ШИМ сигнал в ESC
    ESC.write(pot);
}
```

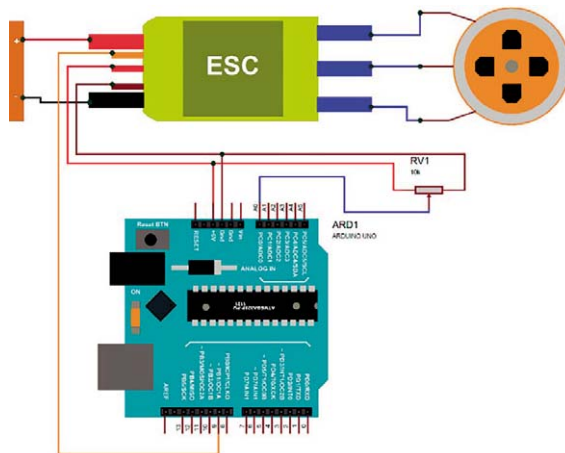


Рис. 6.4

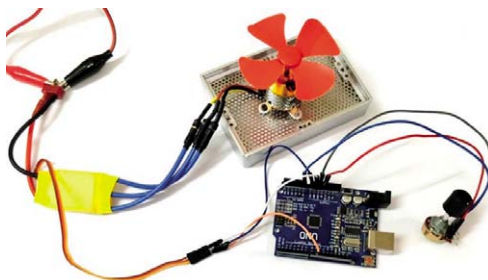


Рис. 6.5

Напомним, что для управления сервомашинками (сервоприводами) необходимо использовать ШИМ сигнал. ШИМ сигнал можно генерировать только на тех цифровых пинах платы Arduino, которые обозначены символом ~ (тильда). В контроллере Arduino Uno это пины 3, 5, 6, 9, 10, 11.

Затем мы должны конвертировать полученное с пина A0 значение (от 0 до 1023) в диапазон от 0 до 180. В дальнейшем значение 0 у нас будет означать 0 % коэффициент заполнения ШИМ, а значение 180 – 100 % коэффициент заполнения ШИМ. Конвертация значения из диапазона 0–1023 в диапазон 0–180 будет осуществляться с помощью функции: $\text{pot} = \text{map}(\text{pot}, 0, 1023, 0, 180);$

Тестирование работы схемы

Убедитесь в том, что БД надежно закреплен, иначе он будет прыгать во время вращения. При подаче питания БД пропишит «приветствие» и станет ожидать поступления Управляющих сигналов на фазы. При вращении ручки

потенциометра двигатель начнет медленно вращаться. При дальнейшем повороте ручки потенциометра и увеличении напряжения на его выходе скорость вращения двигателя будет увеличиваться. Когда напряжение достигнет верхней допустимой границы, двигатель остановится. В дальнейшем вы можете повторить весь этот процесс заново.

Пример 6.2. Разработать скетч, когда производится постепенный разгон двигателя (рис. 6.3). Скетч для этого примера приведен ниже. Контроллер ESC подключен к пину 9 платы Arduino.

```
#include <Servo.h> //используем библиотеку сервомотора для
                  //формирования нужного нам ШИМ сигнала
Servo ESC; //даем имя нашему servo-объекту,
           //в нашем случае это будет имя ESC
void setup()
{
    ESC.attach(9); // подключим ESC к пину 9 Arduino
    delay(3000);
}
void loop()
{
    for (int i=0; i<=180; i++)
    {
        //генерируем ШИМ сигнал с необходимым
        //коэффициентом заполнения
        ESC.write(i);
        delay(100);
    }
}
```

6.3. Разработка передатчика для дрона

Поскольку в этой книге мы уже изучали организацию связи между пультом и роботом по Bluetooth, то было бы очень удобно воспользоваться полученными результатами. В качестве пульта мы использовали старенький пульт передатчика E-FLY (рис. 6.6), приёмники для которого практически сейчас не достать (раритет), а старые уже давно вышли из строя из-за полетов в неблагоприятных условиях.

Вынимаем из передатчика плату. В корпусе остаются лишь четыре потенциометра, управляемые двумя джойстиками. Устанавливаем в корпусе передатчика плату Arduino Uno, модуль Bluetooth HC-05, настроенный на режим работы master в паре с модулем Bluetooth HC-05 (можно HC-06), который будет установлен на дроне. Модуль Bluetooth HC-05 дрона должен быть настроен

на режим работы slave. Производим необходимые соединения в соответствии с рис. 6.7.



Рис. 6.6

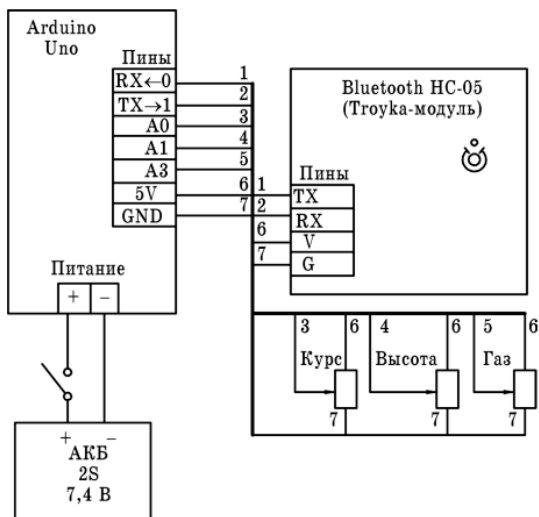


Рис. 6.7

Скетч для передатчика master2.ino приведен ниже.

```
int JoyStick_X = A1; // влево-вправо (курс)
int JoyStick_Y = A0; // вверх-вниз (высота)
//int JoyStick_Z = A2; //резерв
int gaz=A3; // газ
int x, y, z,d,v;
```

```

void setup ()
{
    pinMode (JoyStick_X, INPUT);
    pinMode (JoyStick_Y, INPUT);
    pinMode (gaz, INPUT);
    //pinMode (JoyStick_Z, INPUT);
    Serial.begin (9600); // 9600 bps
}

void loop ()
{
    //читаем информацию с потенциометров пульта
    x = analogRead (JoyStick_X); // курс
    y = analogRead (JoyStick_Y); // высота
    //z = analogRead (JoyStick_Z); // резерв
    v = analogRead (gaz); // газ

    //переводим число x в строку sx
    String sx=String(x);
    // при необходимости доводим строку sx до 4 символов
    for(int i=1;i<=3;i++)
    {
        d=sx.length();
        if (d<4)
        {
            sx='0'+sx;
        }
    }
    Serial.print(sx); //отправляем строку sx (курс) в эфир
    delay(20);

    //с числом y делаем всё то же самое, что и с числом x
    String sy=String(y);
    for(int i=1;i<=3;i++)
    {
        d=sy.length();
        if (d<4)
        {
            sy='0'+sy;
        }
    }
    Serial.print(sy); //отправляем строку sy (высота) в эфир
    delay(20);

    //с числом v делаем всё то же самое, что и с числом x
    String sv=String(v);
    for(int i=1;i<=3;i++)

```

```

    {
        d=sv.length();
        if (d<4)
        {
            sv='0'+sv;
        }
    }
    Serial.print(sv); //отправляем строку sv (газ) в эфир
    delay (200);
}

```

6.4. Разработка аппаратуры для дрона (минимальная комплектация)

На борту дрона в минимальной комплектации должны находиться:

- контроллер Arduino;
- модуль Bluetooth HC-05 (HC-06), работающий в режиме slave;
- две сервомашинки (9 грамм);
- БД;
- контроллер ESC, который по току и напряжению подходит для выбранного БД;
- аккумулятор в 2–3 банки.

Схема аппаратуры дрона приведена на рис. 6.8. Каких-либо дополнительных объяснений она не требует. Понятно, что Bluetooth обеспечивает лишь небольшую дальность передачи информации (40–50 метров), поэтому этот дрон можно запускать в зале или на улице, не допуская значительных разлётов.

Скетч `samolet.ino` для данного проекта приведен ниже. Если внимательно посмотреть на скетч для передатчика, то видно, что в эфир мы отправляем три строки по 4 символа в каждой. В первой строке указано положение руля направления (курса) на пульте управления, во второй строке – положение руля высоты на пульте управления, в третьей – положение ручки газа на пульте управления. Считывать эту информацию и интерпретировать её на дроне нужно тоже в этой последовательности.

```

//работаем с самолетом,
//управляется от пульта с блютуз:
//курс, высота, газ
char sx,sy,sv;
String stx=""; // строка – сюда примем курс
String sty=""; // строка – сюда примем высоту
String stv=""; // строка – сюда примем газ

```

```

//String stz="";//резерв
int x,y,v;
#include <Servo.h> //используем библиотеку для сервомашинки
Servo ESCX; //курс -даем имя нашему servo-объекту,
//в нашем случае это будет имя ESCX
Servo ESCY; //высота
Servo ESCV; //раз

void setup()
{
    ESCX.attach(9); // пин для подключения сервомашинки
                     // управления по курсу
    ESCY.attach(10); //пин для подключения сервомашинки
                     // управления по высоте
    ESCV.attach(11); // пин для подключения ESC
    //Serial.begin(9600);//используется при настройке
}

void loop()
{
    //обнуляем принимаемую информацию
    stx="";
    sty="";
    //stz="";
    stv="";

    if(Serial.available())
    {
        delay(10);
    }

    if(Serial.available())
    { //в буфере что-то есть, извлечем курс
        delay(10);
        //считываем из буфера 4 символа
        //символ за символом
        sx=Serial.read();
        delay(10);
        stx=stx+sx;
        sx=Serial.read();
        delay(10);
        stx=stx+sx;
        sx=Serial.read();
        delay(10);
        stx=stx+sx;
    }
}

```

```

        sx=Serial.read();
        //Serial.println(s);
        stx=stx+sx;//строка сформирована
        delay(20);
        //преобразуем строку в целое число
        x=stx.toInt();
        //используем при отладке
        //Serial.print ("x=");
        //Serial.println(x);
    }

    if(Serial.available())
    {
        // извлекаем высоту также, как и курс
        delay(10);
        sy=Serial.read();
        delay(10);
        sty=sty+sy;
        sy=Serial.read();
        delay(10);
        sty=sty+sy;
        sy=Serial.read();
        delay(10);
        sty=sty+sy;
        sy=Serial.read();
        sty=sty+sy;
        delay(20);
        y=sty.toInt();
        //Serial.print ("y=");
        //Serial.println(y);
    }

    if(Serial.available())
    {
        //извлекаем газ также, как и курс
        delay(10);
        sv=Serial.read();
        delay(10);
        stv=stv+sv;
        sv=Serial.read();
        delay(10);
        stv=stv+sv;
        sv=Serial.read();
        delay(10);
        stv=stv+sv;
        sv=Serial.read();
        stv=stv+sv;
        delay(20);
        v=stv.toInt();
    }

```

```

        //Serial.print ("v=");
        //Serial.println(v);
    }

    /* if(Serial.available())
        {//это для четвертого канала, если его использовать
            delay(10);
            sz=Serial.read();
            delay(10);
            stz=stz+sz;
            sz=Serial.read();
            delay(10);
            stz=stz+sz;
            sz=Serial.read();
            delay(10);
            stz=stz+sz;
            sz=Serial.read();
            stz=stz+sz;
            delay(20);
            z=stz.toInt();
            //Serial.print ("z=");
            //Serial.println(z);
        }*/

    if (x<1000) //боремся с помехами
    {
        int x1= map(x, 142, 991, 0, 180); //конвертируем
        //значения из диапазона 142-991 с выхода
        //потенциометра управления по курсу в диапазон
        // 0-180
        ESCX.write(x1); //отправляем число на сервомотор
    }

    if (y<1000) //боремся с помехами
    {
        int y1= map(y, 36, 889, 180, 0); //конвертируем
        //значения из диапазона 36-889 с выхода
        //потенциометра управления по высоте в диапазон
        //0-180
        ESCY.write(y1); // отправляем число на сервомотор
    }

    if (v<1000) //боремся с помехами
    {
        int v1= map(v, 73, 908, 0, 180); //конвертируем
        //значения из диапазона 73-908 с выхода

```

```

//потенциометра управления скоростью в диапазон
// 0- 180
ESCV.write(v1);//отправляем число в ESC
    }
    delay(25);
}

```

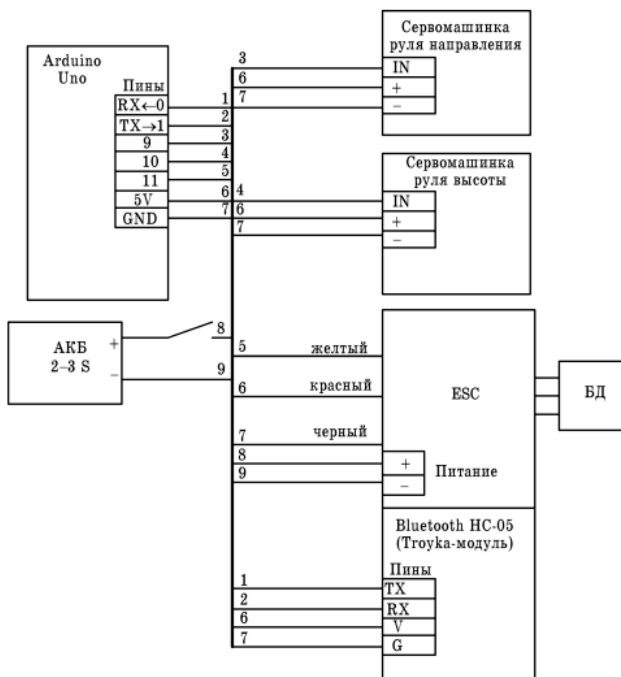


Рис. 6.8

Как только в буфере модуля HC-05 на дроне что-то появляется, выполняется посимвольное чтение из него двенадцати символов. Первые 4 символа соответствуют положению ручки руля курса, вторые четыре – положению ручки руля высоты, последние четыре – положению ручки газа на пульте.

Есть еще один тонкий момент. Потенциометры ручек управления на пульте «выдают» не положенные числа в диапазоне 0–1023, а немного другие. Для ручки управления курсом это числа в диапазоне 142–991, для ручки управления высотой это числа в диапазоне 36–889, для ручки управления газом это числа в диапазоне 73–908. Не факт, что на всех пультах управления будут подобные диапазоны. Нужно в обязательном порядке их замерить и учитывать при программировании дрона.

На рис. 6.10 приведен внешний вид дрона со всей аппаратурой. На этом фото хорошо видны:

- две сервомашинки (сервомотора) руля направления (курса) и руля высоты с тягами – в задней части дрона,
- контроллер Arduino с модулем HC-05 – на крыле,
- БД с винтом – в передней части дрона,
- контроллер ESC плохо виден, он расположен по правому борту в передней части дрона,
- аккумулятор (2S-3S) расположен в корпусе дрона в передней части и не виден.

При расположении аппаратуры на ЛА необходимо выполнять правило центровки: центр тяжести ЛА должен находиться ближе к первой трети крыла.

6.5. Что дальше?

Ответ на этот вопрос прост и иллюстрируется на рис. 6.10. Мы можем оснастить передатчик еще и клавиатурой, с помощью которой можем выполнять любые действия на дроне: включать-выключать ту или иную аппаратуру, сбрасывать грузы, задавать режимы полета, задавать профили полета, выполнять сложные выражи и т. д.



Рис. 6.9



Рис. 6.10

Такая клавиатура позволяет выполнять 16 функций, используя всего 8 пинов контроллера.

Кроме того, имеется реальная возможность сильно увеличить радиус действия дрона, используя, например, радиомодуль беспроводной передачи данных NRF24L01 + PA + LNA, который передает информацию более чем на 1000 метров. Авторы надеются провести необходимые исследования в этой области и опубликовать их, но уже в другой книге.

ЗАКЛЮЧЕНИЕ

Вот и перевернута последняя страница книги. В ней вы нашли много новых технических решений, алгоритмов, скетчей. Авторы далеки от мысли, что все представленные материалы – верх совершенства и истина в последней инстанции. Отнюдь нет. Да и не ставилась такая задача. Задача была простая – проверить ту или иную идею, выяснить особенности работы того или иного устройства, датчика, модуля. Получить конкретные результаты, на базе которых можно строить новые системы с более совершенными характеристиками и параметрами. Если вы, прочитав эту книгу, загорелись идеей что-то улучшить, усовершенствовать, спроектировать, то авторы будут считать, что они свою задачу выполнили: подвигли вас к творчеству.

СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСОВ

1. Зубарев А. А. Ассемблер для микроконтроллеров AVR: Учебное пособие. – Омск: Изд-во СибАДИ, 2007. – 112 с.
2. Соммер У. Программирование микроконтроллерных плат Arduino/ Freeduino. – СПб. : БХВ-Петербург, 2012. – 256 с.
3. Ревич Ю. В. Занимательная электроника. – 3-е изд., перераб. и доп. – СПб. : БХВ-Петербург, 2015. – 576 с.
4. Петин В. А. Проекты с использованием контроллера Arduino. – 2-е изд., перераб. и доп. – СПб. : БХВ-Петербург, 2015. – 464 с.
5. Карвинен, Теро, Карвинен, Киммо, Валтоари, Вилле. Делаем сенсоры: проекты сенсорных устройств на базе Arduino и Raspberry Pi. : Пер. с англ. – М. : ООО «И. Д. Вильямс», 2015. – 432 с.
6. Кангин В. В. Контроллеры Arduino в системах автоматизации, управления и контроля. Вводный курс [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2019. – 276 с. : ил.
7. Кангин В. В. Контроллеры Arduino в мобильных роботах [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2021. – 396 с. : ил.
8. Кангин В. В., Козлов В. Н. Аппаратные и программные средства систем управления. Промышленные сети и контроллеры [Текст]: учебное пособие / В. В. Кангин. – М. : БИНОМ. Лаборатория знаний, 2010. – 418 с., ил.
9. Кангин В. В., Кангин М. В., Богов А. Н., Ямолдинов Д. Н. Управление техническими системами. Контроллер ADAM-5510 и его программирование в UltraLogik [Текст]: учебное пособие / В. В. Кангин. – Н. Новгород: НГТУ, 2008. – 396 с., ил.
10. Кангин В. В., Кангин М. В., Ямолдинов Д. Н. Проектирование SCADA-систем. [Текст]: учебное пособие / В. В. Кангин. – Н. Новгород: НГТУ, 2010. – 568 с., ил.
11. Кангин В. В., Кангин М. В., Ямолдинов Д. Н. Программные аспекты проектирования SCADA-систем. [Текст]: учебное пособие / В. В. Кангин. – Н. Новгород: НГТУ, 2007. – 259 с., ил.
12. Кангин В. В., Кангин М. В., Ямолдинов Д. Н. Компьютеры в системах управления технологическими процессами. [Текст]: учебное пособие / В. В. Кангин. – Н. Новгород: НГТУ, 2005. – 246 с., ил.
13. Кангин В. В., Кангин М. В., Богов А. Н., Меретюк В. Н., Ямолдинов Д. Н. Промышленные сети, контроллеры и модули УСО. [Текст]: учебное пособие / В. В. Кангин. – Н. Новгород: НГТУ, 2006. – 418 с., ил.
14. Кангин В. В., Ложкин Л. Д., Ямолдинов Д. Н. Обмен информацией в промышленной сети PlcNet // ИКТ, Т. 8, № 3, 2010. С. 49–54.
15. Кангин В. В., Ложкин Л. Д., Ямолдинов Д. Н. Программная реализация межсетевого шлюза сетей ETHERNET и PLCNET// ИКТ, Т. 9, № 2, 2011. С. 36–41.
16. Кангин В. В., Кангин М. В., Ямолдинов Д. Н. Разработка SCADA-систем [Текст]: учебное пособие / В. В. Кангин. – Москва; Вологда: Инфра-Инженерия, 2019. – 564 с., ил., табл.
17. Кангин В. В. Средства автоматизации и управления. Аппаратные и программные аспекты [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2014. – 520 с.
18. Кангин В. В. Промышленные контроллеры в системах автоматизации технологических процессов [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2013. – 408 с.

19. Кангин В. В., Ложкин Л. Д. Организация обмена информацией в промышленной сети на базе модулей ADAM-4000 // ИКТ, Т. 12, № 3, 2014. С. 41–46.
20. Кангин В. В. Персональные компьютеры в системах управления и автоматизации технологических процессов [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2018. – 304 с.
21. Кангин В. В. Мобильные роботы и дроны на базе контроллеров Arduino [Текст]: учебное пособие / В. В. Кангин. – Старый Оскол: ТНТ, 2024. – 336 с. : ил.
22. URL: <http://arduino.ru>.
23. URL: <http://wiki.amperka.ru>.
24. URL: <http://cxem.net>.
25. URL: <http://zi-zi.ru>.
26. URL: <http://kit.ru>.
27. URL: <http://lesson.iarduino.ru>.
28. URL: <http://soltau.ru>.
29. URL: <http://arthurphdent.livejournal.com>.
30. URL: <http://kakprosto.ru>.
31. URL: <http://arduino-kit.ru>.
32. URL: <http://amperka.ru>.
33. URL: <http://mypractic.ru>.
34. URL: <http://maxpark.ru>.
35. URL: <http://iarduino.ru>.



Инфра-Инженерия

Издательство технической литературы

Издательство предлагает учебную литературу по следующим направлениям:

- IT. Информатика
- Авиационная и ракетно-космическая техника
- Автоматизация
- Автомобильная техника
- Арматура. Трубопроводы
- Бетоны
- Бурение нефтяных и газовых скважин
- Вооружение. Радиоэлектронные системы.
- Газовое хозяйство
- Геодезия, картография и маркшейдерское дело
- Геология. Геофизика. Геохимия
- Горное дело
- Железнодорожный транспорт
- Лесная промышленность
- Логистика
- Машиностроение
- Медицина и биоинженерия
- Metallургия
- Нефтегазовая промышленность
- Педагогика. Психология
- Пищевая промышленность
- Промышленная безопасность. Охрана труда
- Разработка и эксплуатация нефтяных и газовых месторождений
- Сварочное дело
- Словари
- Сети и коммуникации. Волоконно-оптическая техника
- Строительство
- Судостроение
- Транспортное строительство. Дороги. Мосты. Тоннели
- Физико-математические науки
- Химия. Химические технологии
- Экология. Безопасность жизнедеятельности
- Экономика. Управление. Электронная коммерция
- Электроника
- Электро- и теплоэнергетика

Интернет-магазин

infra-e.ru



WhatsApp



VK



Telegram



Телефон

8-800-250-66-01
8-911-512-48-48

Более 2000 выпущенных книг
по 30 темам и направлениям

Доставляем наши книги по
всей России от Калининграда
до Камчатки, а также в
страны СНГ

Нам доверяют более 90
учебных заведений в России
и за рубежом



Учебное издание

Кангин Владимир Венедиктович
Кангин Егор Михайлович

КОНТРОЛЛЕРЫ ARDUINO

В МОБИЛЬНЫХ РОБОТАХ И ДРОНАХ

Учебное пособие

ISBN 978-5-9729-2451-6



Подписано в печать 23.12.2024
Формат 60×84/16. Усл. печ. л. 18,83. Печать по требованию.
Бумага офсетная. Гарнитура «Times New Roman».

Издательство «Инфра-Инженерия»
160011, г. Вологда, ул. Козленская, д. 63
Тел.: 8 (800) 250-66-01
E-mail: booking@infra-e.ru
<https://infra-e.ru>

Издательство приглашает к сотрудничеству
авторов научно-технической литературы